

grian

NEM Electricity Price Forecasting and Battery Dispatch

From raw market data to probabilistic forecasts
and forecast-driven battery revenue optimisation

10 chapters • 24 figures • 200+ definitions • zero assumed knowledge

Contents

01. Data Ingestion and the NEM ... 12

What is the NEM? ... 12

The Five Regions ... 12

How Electricity Is Priced ... 14

The Gross Pool ... 14

The Price Range ... 14

Timestamps: A Critical Detail ... 15

Interval-ending vs interval-start ... 15

The Target Transform: Taming Wild Prices ... 16

The problem with raw prices ... 16

The inverse hyperbolic sine (arcsinh) ... 16

Data Quality ... 17

Missing intervals ... 17

The INTERVENTION flag ... 17

5-minute to 30-minute aggregation ... 17

Data Storage: Parquet Format ... 17

Reproducibility: Why It Matters ... 17

02. Price Distribution and Stylised Facts ... 19

What Are "Stylised Facts"? ... 19

The Daily Price Cycle: The Duck Curve ... 19

Why does the duck curve happen? ... 19

Heavy Tails: Extreme Events Are Not Rare ... 20

Why heavy tails matter ... 20

Autocorrelation: Today Looks Like Yesterday ... 20

What the ACF tells us ... 21

Volatility Clustering: Calm Breeds Calm, Chaos Breeds Chaos ... 21

Why does volatility cluster? ... 21

The Price-Duration Curve ... 22

Seasonality: Annual and Weekly Rhythms ... 22

Weekly seasonality ... 22

Annual seasonality ... 23

Summary of Stylised Facts ... 23

03. Price Formation ... 24

The Merit-Order Supply Stack ... 24

The generators in the stack ... 24

How demand position determines price ... 25

The Hockey Stick: Net Load vs Price ... 25

Why the hockey stick is so important ... 25

Anatomy of a Price Spike ... 26

The three phases of a spike ... 26

What causes spikes ... 26

Generator Bidding Behaviour ... 27

Strategic bidding ... 27

Why bidding matters for forecasting ... 27

Supply Mix Transition ... 27

Market Institutions and Rules ... 27

Who runs the market ... 28

The market price cap and floor ... 28

Administered pricing and the cumulative price threshold ... 28

Scarcity signals: lack of reserve and the reserve trader ... 28

Beyond the spot market ... 28

04. Weather and Renewables ... 30

Why Weather Is the Master Variable ... 30

ERA5: A Complete Picture of the Atmosphere ... 30

What Is a Reanalysis? ... 30

ERA5: The Gold Standard ... 30

Key Variables for Electricity Prices ... 31

Why 100m Wind, Not Surface Wind? ... 31

From ERA5 Grid to Regional Aggregates ... 32

Solar Radiation and the Clear-Sky Index ... 32

How Sunlight Reaches the Ground ... 32

Components of Solar Radiation ... 32

Clear-Sky Models and pvlib ... 33

The Clear-Sky Index: Isolating the Weather Signal ... 33

CSI Edge Cases and Quality Control ... 34

Persistence of Cloudiness ... 34

Wind Speed and Wind Power ... 34

The Physics of Wind Energy ... 34

The Wind Power Curve ... 35

Calculating Wind Speed from ERA5 ... 36

Why Wind Is Harder to Forecast Than Solar ... 36

Capacity Factor and Its Implications ... 37

Temperature and Electricity Demand ... 37

The Comfort Zone and the U-Shaped Response ... 37

Temperature Deviation as a Feature ... 37

Weather as a Price Driver: The Causal Chain ... 38

Feature Engineering: From Raw Weather to Model Inputs ... 38

Transforming Weather Variables ... 38

Lagged vs. Contemporaneous Weather Features ... 38

AEMO's Own Forecasts: NEMSEER Pre-Dispatch ... 39

Persistence Forecasts as Weather Baselines ... 39

What Is a Persistence Forecast? ... 39

05. Forecasting Framework and Baselines ... 41

The Day-Ahead Forecasting Problem ... 41

What Exactly Are We Predicting? ... 41

Why Day-Ahead? ... 41

The Target Transform in Practice ... 42

The Feature Matrix ... 42

What Goes Into a Price Forecast? ... 42

Feature Engineering Principles ... 43

Data Leakage: The Silent Model Killer ... 43

What Is Data Leakage? ... 43

Common Sources of Leakage in Electricity Price Forecasting ... 43

The Rolling-Origin Backtest ... 43

Why Not a Simple Train/Test Split? ... 43

The Rolling-Origin Procedure ... 44

The Embargo: Why It Exists ... 44

Expanding vs. Sliding Windows ... 45

Baseline Models: The Bar Your Model Must Clear ... 45

Why Baselines Matter ... 45

The Similar-Day Naïve Baseline ... 45

The Autoregressive Baseline ... 46

Calendar-Enhanced AR ... 46

Forecast Error Metrics ... 46

Mean Absolute Error (MAE) ... 46

Root Mean Square Error (RMSE) ... 46

Relative MAE (rMAE) ... 47

The Diebold-Mariano Test ... 47

Battery Dispatch: Converting Forecasts to Revenue ... 47

The Arbitrage Problem ... 47

The Linear Program (LP) ... 48

Why an LP, Not Heuristic Rules? ... 49

Solving the LP with cvxpy ... 49

Model Predictive Control (MPC) ... 49

From One-Shot to Rolling Dispatch ... 49

Why Only Execute the First Period? ... 50

MPC Limitations with Point Forecasts ... 50

The Capture Ratio ... 50

Definition and Interpretation ... 50

Why Capture Ratio, Not MAE? ... 51

AEMO Pre-Dispatch as Benchmark ... 52

06. Classical Models: LEAR ... 54

From Baselines to Real Models ... 54

Linear Regression: The Foundation ... 54

What Is Linear Regression? ... 54

Ordinary Least Squares (OLS) ... 54

The Problem with Too Many Features ... 54

Regularisation: Controlling Model Complexity ... 55

The Bias-Variance Tradeoff ... 55

Ridge Regression (L2 Penalty) ... 55

LASSO Regression (L1 Penalty) ... 55

Why L1 Creates Sparsity and L2 Does Not ... 56

The Regularisation Parameter λ ... 56

LassoCV: Automatic λ Selection ... 57

Cross-Validation for λ ... 57

Cross-Validation for Time Series ... 57

The LEAR Model ... 57

Background and Motivation ... 57

The LEAR Formulation ... 58

Per-Hour Models: Why Separate Is Better ... 58

Feature Selection via the L1 Penalty ... 59

How LASSO Selects Features ... 59

The Regularisation Path and Feature Importance ... 59

LASSO vs. Ridge for Electricity Prices ... 59

Time Series Fundamentals for LEAR ... 60

Stationarity ... 60

The Training Window Length ... 60

Forecast Horizons and Error Growth ... 60

Model Evaluation and Comparison ... 60

Evaluating LEAR's Performance ... 60

The Diebold-Mariano Test for Model Comparison ... 61

Recalibration and Concept Drift ... 61

Why Models Degrade Over Time ... 61

Refit Frequency ... 61

Monitoring for Degradation ... 61

07. Machine Learning ... 64

Why Move Beyond Linear Models? ... 64

Decision Trees: The Building Block ... 64

How a Decision Tree Learns ... 64

Why Trees Are Natural for Price Forecasting ... 65

The Problem with Single Trees ... 65

Gradient Boosting: From Weak to Strong	... 65
The Ensemble Idea	... 65
How Gradient Boosting Works	... 65
The "Gradient" in Gradient Boosting	... 66
LightGBM: The Industry Standard	... 66
Why LightGBM?	... 66
Key Innovations	... 66
Hyperparameters That Matter	... 67
Feature Importance	... 67
Quantile Regression: From Point to Probabilistic	... 68
Why a Single Number Is Not Enough	... 68
Quantile Levels and Prediction Intervals	... 69
The Pinball Loss	... 69
GBT Quantile Regression	... 69
The Quantile Crossing Problem	... 70
Neural Networks for Price Forecasting	... 71
What Is a Neural Network?	... 71
Architecture Choices	... 71
When Neural Networks Add Value (and When They Don't)	... 72
Training Neural Networks for Quantile Forecasting	... 72
Learning Curves and Model Selection	... 73
The Training Size Effect	... 73
Overfitting: The Central Risk	... 73
Regularisation Across Model Families	... 74
GBT vs Neural Network: A Practical Decision Framework	... 74

08. Probabilistic Forecasting ... 77

Why Probabilistic Forecasts Matter	... 77
Forms of Probabilistic Forecasts	... 77
Quantile Regression Averaging (QRA)	... 78
The Forecast Combination Principle	... 78
The QRA Method	... 78
Why Not Simple Averaging?	... 78
Evaluating Probabilistic Forecasts	... 79
Calibration (Reliability)	... 79
Sharpness	... 80
The CRPS: A Single-Number Metric	... 80
Spike Coverage: The Critical Edge Case	... 81
Conformal Prediction	... 81
The Calibration Problem in Practice	... 81
Split Conformal Prediction	... 81
Adaptive Conformal Prediction	... 82
Practical Considerations for Conformal Prediction	... 82
The Calibration-Sharpness Tradeoff in Practice	... 83
Putting It All Together	... 83
Calibration for Battery Dispatch: Asymmetric Costs	... 83
Copula Scenario Generation	... 83
Marginals versus dependence	... 84
Sklar's theorem and the copula idea	... 84
The Gaussian copula	... 84
When tails matter: vine copulas	... 84
Cross-market dependence: energy and FCAS	... 85
Sampling and scenario count	... 85

09. From Forecast to Money ... 87

The Dispatch Decision Under Uncertainty	... 87
Stochastic Programming: Scenario MPC	... 87

The Idea ...	89
Generating Scenarios from Quantile Forecasts ...	87
Solving the Scenario LP ...	88
How Many Scenarios? ...	88
Robust Optimisation: Chance-Constrained MPC ...	89
The Idea ...	89
The Chance-Constrained LP ...	90
The Conservatism Tradeoff ...	90
Comparing Scenario MPC and Chance-Constrained MPC ...	90
Sensitivity Analysis: Battery Parameters ...	91
Duration (Hours of Storage) ...	91
Round-Trip Efficiency ...	91
Cycle Constraints and Degradation ...	92
The Value of Information ...	92
How Much Does Forecast Quality Matter? ...	92
The Economic Assessment ...	94
Revenue Decomposition ...	94
From Capture Ratio to Annual Revenue ...	94
Regional Basis and Inter-regional Trade ...	95
Why regional prices separate ...	95
Basis risk for battery operators ...	95
Settlement residues ...	95

10. Capstone: Sunlight to Revenue ... 97

The Complete Pipeline ...	97
Pipeline Components and Their Error Sources ...	97
Grey-Box Models ...	98
The Philosophy: Combining Physics and Data ...	98
The Merit-Order Stage: Isotonic Regression ...	98
The Residual Model ...	99
Why Grey-Box Outperforms Either Alone ...	99
Ablation Studies ...	100
Why Remove Components? ...	100
Interpreting Ablation Results ...	100
Ablation vs Feature Importance ...	101
Fair Model Comparison ...	101
The Problem of Unfair Comparisons ...	101
Principles for Fair Comparison ...	101
The Naive Baseline as Anchor ...	102
The Value Chain: Where to Invest ...	102
Marginal Value of Each Pipeline Stage ...	102
The 80/20 Rule in Practice ...	102
Limitations and Honest Assessment ...	103
What This Project Does Not Address ...	103
Sources of Forecast Error ...	103
What a Production System Would Add ...	104
The Broader Context ...	104
Energy Storage Economics ...	104
The Forecasting Industry ...	104
What Makes This Problem Interesting ...	105

11. FCAS and Co-optimised Dispatch ... 107

Why 50 Hz Matters ...	107
The Inertia Problem ...	107
Regulation vs Contingency Services ...	108
Regulation FCAS ...	108
Contingency FCAS ...	108

The Key Difference ...	109
The Ten FCAS Markets ...	109
The 1-Second Markets ...	109
Price Behaviour Across FCAS Markets ...	109
FCAS Revenue as a Share of Total Battery Revenue ...	110
Co-optimisation in One Dispatch ...	110
How NEMDE Works ...	110
Using nempy to See Co-optimisation ...	111
Price Interaction Effects ...	111
The Joint Capacity Constraint: The Enablement Trapezium ...	112
The Physical Constraint ...	112
The SoC Constraint on FCAS ...	112
How the Trapezium Affects Battery Strategy ...	113
Co-optimised Dispatch: Energy-Only vs Joint ...	113
Setting Up the Comparison ...	113
The LP Extension for Co-optimisation ...	113
Worked Example: Evening Peak with Mannum ...	114
The nempy View of One Interval ...	115
Causer-Pays: The Cost of Regulation ...	115
How Causer-Pays Works ...	115
Why Batteries Are Good (and Bad) at Causer-Pays ...	115
Forecasting FCAS Prices ...	116
The Challenge ...	116
What Can Be Forecast ...	116
A Practical Forecasting Approach ...	116
Integrating FCAS into the MPC ...	117
The Capture Ratio with FCAS ...	117
Redefining the Benchmark ...	117
Benchmarking Against AEMO Pre-dispatch ...	118
Worked Example: Clearing One Interval with nempy ...	118
Setup ...	118
Sensitivity Analysis with nempy ...	119
Practical FCAS Bidding for a Battery ...	119
Offer Structure ...	119
Stacking FCAS Services ...	119
The Regulation Decision ...	120
Exercises ...	120
Exercise 1: Sweep the Value of Reserving Headroom for FCAS ...	120
Exercise 2: Find the Break-Even FCAS Price ...	121
Exercise 3: Estimate the Annual FCAS Revenue from Spike Capture ...	121

12. Bidding and Market Participation ... 124

The Missing Link: From Schedule to Offer ...	124
The NEM Offer Structure ...	124
Price-Quantity Bands ...	124
How Dispatch Works ...	125
The Bidirectional Battery ...	125
Rebidding: Updating Offers in Real Time ...	125
The Rebidding Mechanism ...	125
Rebid Reasons and Compliance ...	126
Rebidding Strategy for Batteries ...	126
The Emulator: Two Modes of Simulation ...	126
Why Simulate the Market? ...	126
Price-Taker Mode ...	127
Price-Maker Mode ...	127
What nempy Does Under the Hood ...	127

Price-Taker vs Price-Maker: When Market Impact Matters	... 128
The Residual Demand Curve	... 128
The Market Impact Factor	... 128
Revenue Loss from Market Impact	... 129
The Self-Cannibalization Problem	... 129
Offer Construction from Scenarios	... 129
From Probabilistic Forecast to Offer Bands	... 129
The Quantile-to-Band Mapping	... 129
The Stochastic Dispatch Target	... 130
Worked Example: One Day's Offer Construction	... 130
The Full Price-Maker Backtest	... 131
Rolling-Origin Simulation	... 131
Computational Considerations	... 132
Revenue Assessment and Benchmarks	... 132
Capture Ratio in Price-Maker Mode	... 132
Revenue Decomposition by Source	... 133
Strategy Search (Advanced)	... 133
The Methods Placement Principle	... 133
Reinforcement Learning for Bidding	... 133
Genetic Search for Offer Strategies	... 134
Putting It Together: The Complete Bidding Pipeline	... 135
FCAS Co-Optimisation in Bidding	... 135
Energy and FCAS Offers	... 135
Loss Factors and Regional Considerations	... 135
Exercises	... 136
Exercise 12.1: Offer Aggressiveness and the Revenue-Risk Tradeoff	... 136
Exercise 12.2: Finding the Price-Taker Threshold	... 136
Exercise 12.3 (Optional Advanced): Strategy Search vs Convex Baseline	... 137

13. Realised Revenue ... 140

The Gap Between Model and Reality	... 140
Marginal Loss Factors	... 140
What They Are	... 140
How AEMO Computes MLFs	... 140
The Mannum Example	... 140
Why Remote Renewables Have Falling MLFs	... 141
Modifying the LP for MLFs	... 141
Network Charges and the True Cost of Connection	... 141
Transmission Use of System (TUOS) Charges	... 141
Connection Agreement Costs	... 141
Degradation as a Variable Cost	... 142
From Hard Constraint to Economic Cost	... 142
Degradation Mechanisms for LFP	... 142
Computing the Degradation Cost per MWh	... 142
Incorporating Degradation into the LP	... 143
Augmentation	... 143
Operational Constraints	... 143
Ramp Limits	... 143
Minimum Up/Down Times	... 144
Auxiliary Load	... 144
Capacity Fade Over the Asset Life	... 144
The Revenue Waterfall	... 144
Building the Waterfall Step by Step	... 144
Sensitivity of the Waterfall	... 145
Using nempy to Validate the LP	... 145
Why Validate?	... 145

- What to Compare ... 145
- Running the Validation ... 145
- The FCAS Trapezium ... 146

Exercises ... 146

- Exercise 1: Degradation Breakeven Spread ... 146
- Exercise 2: Loss Factor and Siting ... 146

14. Failure Analysis and Explainability ... 148

Decision Regret, Not Forecast Error ... 148

- Computing Decision Regret ... 148
- Regret by Regime ... 148

The Asymmetric Cost of Error ... 149

- Missed Spikes: The Expensive Failure ... 149
- False Alarms: The Hidden Cost ... 149
- Quantifying the Asymmetry ... 149

Regime-Conditional Evaluation ... 150

- Why Overall Metrics Lie ... 150
- Regime-Conditional MAE and CRPS ... 150
- Regime-Conditional Calibration ... 150

Attribution on the Model ... 151

- SHAP: Feature Attribution for Gradient-Boosted Models ... 151
- Partial Dependence: Recovering the Hockey Stick ... 151
- What SHAP Cannot Tell You ... 152

Error Decomposition ... 152

- The Decomposition Framework ... 152
- Interpreting the Residual ... 152
- The Counterfactual: What If We Had Known? ... 153

Comparing Against Mannum's Real Dispatch ... 153

- The Anchor Asset ... 153
- Pulling Mannum's Dispatch Data ... 153
- The Comparison: Course Model vs Habitat ... 153
- Divergence Analysis ... 154
- What Habitat Knows That the Model Does Not ... 154

The Incident Report ... 154

- Purpose and Structure ... 154
- Worked Example: The Five Worst Days ... 155
- Patterns Across Incident Reports ... 156

The Forecast Error vs Decision Regret Scatter ... 156

Benchmarking Regret ... 156

- Absolute and Relative Regret ... 156
- Benchmarks for the Mannum BESS ... 157

Exercises ... 157

- Exercise 1: Identify the Input Whose Forecast Error Most Often Precedes a Missed Spike ... 157
- Exercise 2: Test Whether a Simple Rule on That Input Would Have Improved Dispatch ... 157
- Exercise 3: Hypothesise What Additional Information Habitat Used ... 158

15. Live Operations and the Intra-day Loop ... 160

Why Operations Is a Separate Discipline ... 160

Intra-day Re-optimisation ... 166

- The Cost of a Stale Plan ... 160
- From LP Re-solve to Emulator Re-optimisation ... 160
- Re-optimisation Frequency ... 161

The Operational Loop ... 161

- The Market's Clock ... 161
- Data Latency as a Real Constraint ... 161
- The Forecast-Bid-Settle Cycle ... 162

Monitoring ... 162

- What to Monitor ... 162
- Retraining vs Recalibration ... 163

Risk and Oversight ... 164

- Position and Exposure Limits ... 164
- Daily P&L Tracking ... 164
- Human-in-the-Loop ... 165

Worked Example: Intra-day Re-optimisation on a Volatile Day ... 165

- Setup ... 165
- Price Profile ... 165
- Day-Ahead Plan ... 166
- Intra-day Re-optimisation ... 166
- Perfect Foresight ... 166
- Comparison ... 166
- Real-World Comparison ... 166

Exercises ... 166

- Exercise 1: Re-optimisation Value vs Volatility ... 167
- Exercise 2: Drawdown Limit Design ... 169

16. Decision-Focused Learning ... 169

The Gap Between Accuracy and Value ... 169

The Two-Stage Baseline and Its Failure Mode ... 169

- Predict, Then Optimise ... 169
- A Concrete Example of the Failure Mode ... 170

Decision Regret as the Loss Function ... 170

- Definition ... 170
- From Accuracy-Based Losses to Decision Regret ... 171

Differentiable Optimisation Layers ... 171

- The Challenge: Backpropagating Through an Argmin ... 171
- The Implicit Function Theorem ... 171
- Available Tools ... 172

The Vanishing-Gradient Problem ... 172

- Why Linear Objectives Make Gradients Vanish ... 172
- Fix 1: Quadratic Regularisation ... 173
- Fix 2: The SPO+ Surrogate Loss ... 174

The Training Loop ... 174

- Computation Graph ... 174
- Step-by-Step Training Procedure ... 174
- Practical Considerations ... 175

Connection to the Methods Hierarchy ... 175

- Where Decision-Focused Learning Sits ... 175
- Decision-Focused Learning and FCAS ... 175

Worked Example: Decision-Focused Training for the Mannum BESS ... 175

- Setup ... 175
- Results ... 176
- Where the Improvement Comes From ... 176
- Interpretation ... 176

Benchmark: Decision-Focused vs. Two-Stage on the Held-Out Period ... 176

- Experimental Protocol ... 176
- Results by Day Type ... 176
- Statistical Significance ... 176
- Comparison with AEMO Pre-Dispatch ... 177

Exercises ... 177

- Exercise 1: The Regularisation Trade-Off ... 177
- Exercise 2: Where the Model Chooses to Be Wrong ... 178
- Exercise 3: SPO+ vs. QP Regularisation

How to use this guide: Each chapter corresponds to a hands-on notebook. Read the chapter first to understand the concepts, then work through the notebook to apply them. Every technical term is defined in blue definition boxes. Real-world examples appear in orange boxes. Key insights are highlighted in green boxes. Each chapter ends with a glossary of all terms introduced.

1. Data Ingestion and the NEM

What is the NEM?

Imagine a giant marketplace where electricity is bought and sold every five minutes, stretching over 5,000 kilometres along Australia's eastern coast. This is the **National Electricity Market (NEM)** — one of the world's longest interconnected power systems, serving roughly 10 million homes and businesses.

Unlike buying petrol (where the price changes once a day), electricity is traded in a **wholesale market** where the price changes every five minutes. These prices can swing wildly — from *negative* values (generators paying you to take their electricity) to over \$17,000 per megawatt-hour in extreme events. Understanding this market is the foundation of everything in this course.

Wholesale electricity market: A centralised marketplace where generators (power plants, wind farms, solar farms) sell electricity to retailers and large consumers. The NEM is Australia's wholesale market for the eastern states.

Megawatt-hour (MWh): The standard unit of electrical energy in wholesale markets. One MWh powers roughly 300–500 average Australian homes for one hour. When we say the price is "\$80/MWh", it means a generator earns \$80 for producing enough electricity to run ~400 homes for an hour.

The Five Regions

The NEM is not one big grid with one price — it is divided into **five pricing regions**, each corresponding roughly to a state. Each region has its own electricity price at any given moment, determined by the generators operating in that region.

The National Electricity Market (NEM)

Five interconnected pricing regions

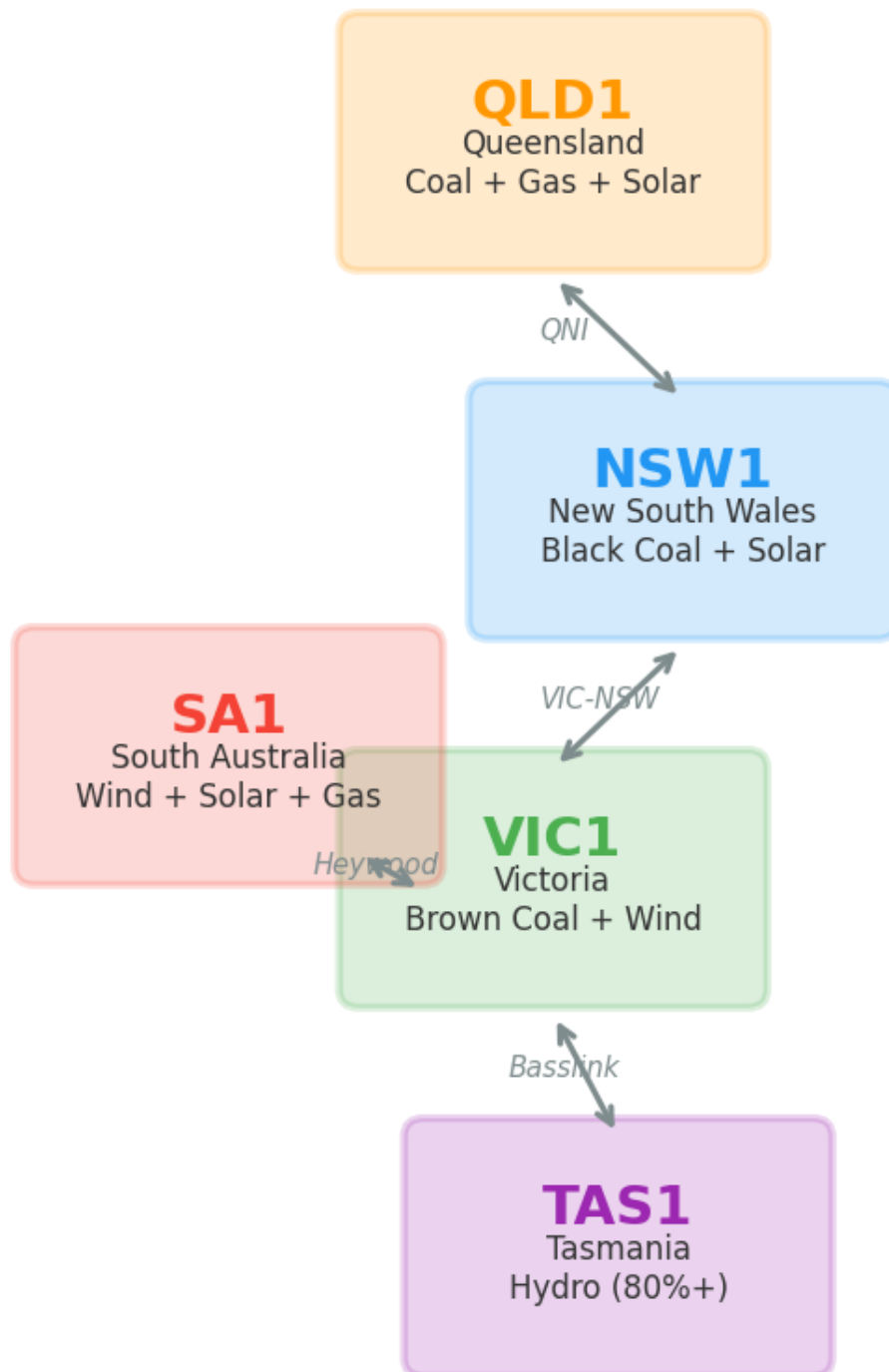


Figure 1.1 — The five NEM regions and their dominant generation types. Each region has its own price, but they are connected by transmission lines (interconnectors) that allow electricity to flow between regions.

Region	State	Dominant generation	Character
NSW1	New South Wales	Black coal, growing solar	Largest market, moderate volatility
QLD1	Queensland	Coal + gas + large-scale solar	High solar penetration, summer spikes
VIC1	Victoria	Brown coal (declining), wind	Transitioning rapidly to renewables
SA1	South Australia	Wind + solar + gas peakers	Most volatile, highest renewable share
TAS1	Tasmania	Hydro (80%+)	Structurally different, hydro-buffered

Interconnector: A high-voltage transmission line that connects two regions. Interconnectors allow electricity to flow from a region with cheap power to one with expensive power. But they have limited capacity — when an interconnector "binds" (hits its transfer limit), the connected regions decouple and can have very different prices. For example, SA1 might spike to \$5,000/MWh while VIC1 sits at \$50, simply because the wire connecting them is full.

Real-world example: South Australia (SA1) is the most interesting region for forecasting because it has the highest share of wind and solar in the NEM. On a windy, sunny day, SA1 can have negative prices (too much renewable energy, not enough demand). On a hot summer evening when the wind drops and everyone turns on their air conditioning, SA1 can spike to \$15,000/MWh within minutes. This extreme volatility is what makes SA1 both challenging and lucrative for battery operators.

How Electricity Is Priced

The Gross Pool

Every single megawatt-hour of electricity in the NEM goes through a central auction, run by the **Australian Energy Market Operator (AEMO)**. This is called a **gross pool** — unlike some markets where buyers and sellers negotiate privately, all electricity must pass through the pool.

Here is how it works, step by step:

Generators submit bids. Every five minutes, each generator tells AEMO: "I can produce X megawatts at \$Y per MWh." Generators can submit up to 10 price-quantity bands, from their minimum price to their maximum.

AEMO stacks the bids. AEMO arranges all bids from cheapest to most expensive. This creates the **merit-order supply stack** — a staircase of available generation, sorted by price.

AEMO dispatches from the bottom. Starting with the cheapest generator, AEMO dispatches generation until total supply equals total demand. The most expensive generator needed to balance supply and demand is the **marginal generator**.

The marginal generator sets the price. Every generator dispatched in that interval receives the marginal generator's bid price — not their own bid price. This is called **marginal pricing** (or uniform pricing).

Marginal pricing: A pricing rule where all sellers receive the price of the most expensive unit needed to meet demand. This means cheap generators (like solar and wind) earn far more than their own costs when an expensive gas plant sets the price. It's the same principle as an auction where all winners pay the price of the last accepted bid.

Dispatch price: The 5-minute price set by the marginal generator in each region. Six consecutive dispatch prices are averaged to produce the 30-minute **trading price**, which is the financially settled price — and our forecasting target.

Why this matters for forecasting: Because the price is set by the *last* generator dispatched (not the average), a small change in demand at the margin can cause a huge price change. If demand rises just slightly and pushes the market from a \$40/MWh coal unit to a \$300/MWh gas peaker, the price jumps 7.5× even though demand only changed by 1%. This is the fundamental driver of electricity price volatility.

The Price Range

NEM price range (\$/MWh) — most prices cluster in the green zone

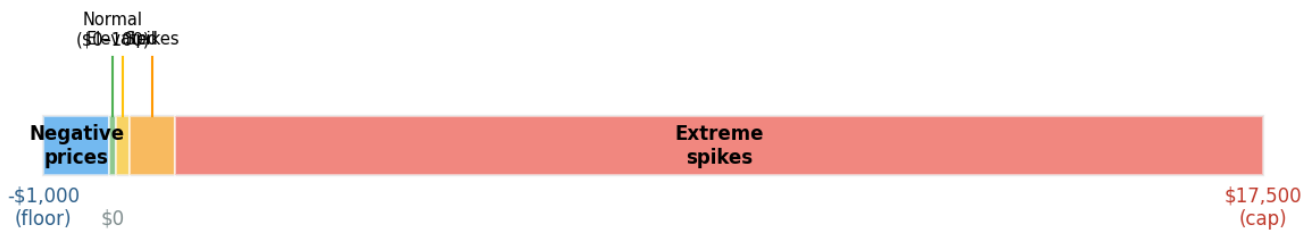


Figure 1.2 — The NEM price range spans from $-\$1,000$ to $\$17,500/\text{MWh}$, but most prices cluster in the $\$0$ – 100 zone. The extreme range is what makes electricity prices so challenging to forecast.

The NEM has administered price limits:

- **Market Price Cap (MPC):** $\$17,500/\text{MWh}$. The maximum price a generator can bid. This cap exists to prevent unlimited market power exploitation.
- **Market Floor Price:** $-\$1,000/\text{MWh}$. Generators bid negative to *avoid shutting down* (coal plants take hours to restart and cost millions to cycle) or to earn Renewable Energy Certificates.
- **Cumulative Price Threshold (CPT):** If prices stay extremely high for an extended period (roughly 7 days), an administered cap of $\sim\$300/\text{MWh}$ kicks in to protect consumers.

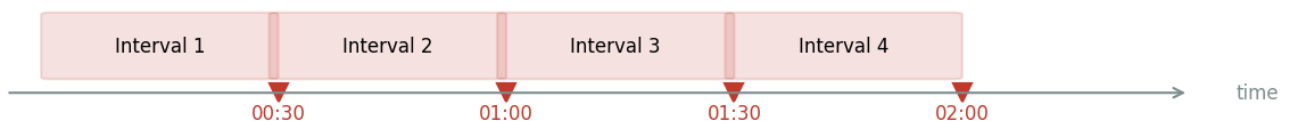
Real-world example — negative prices: On a sunny, windy Sunday in spring, SA1 might have 2,000 MW of wind and solar generation but only 1,200 MW of demand. The excess renewable energy pushes prices negative. Coal generators, which cannot easily shut down and restart, bid at $-\$1,000/\text{MWh}$ to stay running — they would rather *pay* $\$1,000$ per MWh than endure the $\$5\text{M}+$ cost of shutting down and restarting. This is a massive opportunity for batteries: charge for free (or get paid to charge) during negative prices, then discharge when prices recover.

Timestamps: A Critical Detail

Interval-ending vs interval-start

AEMO records data using **interval-ending** timestamps. This is a crucial but easy-to-miss convention that can silently corrupt your analysis if handled incorrectly.

AEMO convention: interval-ENDING



Our convention: interval-START

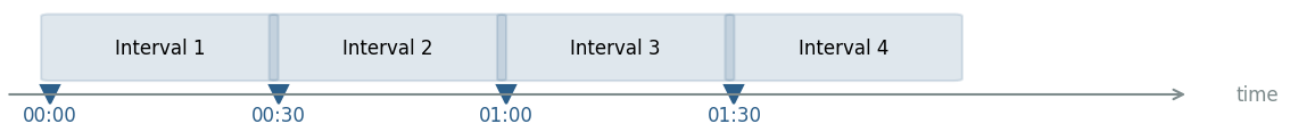


Figure 1.3 — Interval-ending timestamps label each period with its **END** time, not its **START** time. We shift to interval-start convention on load to make data joins safe.

Interval-ending timestamp: A convention where each data point is labelled with the time at the **END** of the measurement period. The row timestamped "14:30" actually contains data measured from 14:00 to 14:30. AEMO uses this convention for all its data products.

Interval-start timestamp: A convention where each data point is labelled with the time at the START of the measurement period. The same data would be labelled "14:00" instead. This is more intuitive and makes joining different datasets (price, weather, demand) straightforward.

Why this matters: Suppose you want to join a price observation with a weather observation. The AEMO row labelled "14:30" covers the period 14:00–14:30. A weather observation at 14:30 was measured at the *end* of that period — it might contain information that was not yet available at the start. If you join on matching timestamps without shifting, you accidentally give your model future information — a form of **data leakage** that inflates performance.

The rule: Shift AEMO timestamps back by one interval on load (14:30 → 14:00), converting to interval-start convention. Do this **exactly once**, at the data loading boundary. Applying it twice creates a systematic 30-minute offset. Forgetting it creates look-ahead leakage. Both errors produce no obvious error message — they silently corrupt every downstream analysis.

The Target Transform: Taming Wild Prices

The problem with raw prices

Electricity prices have statistical properties that break most forecasting models:

- **Enormous range:** −\$1,000 to \$17,500. A model trained on this range will be dominated by extreme observations.
- **Heavy tails:** Extreme prices (>\$1,000) occur rarely but are far more extreme than a normal distribution would predict. Statisticians call this **leptokurtosis**.
- **Asymmetry:** The right tail (price spikes) is much heavier than the left tail (negative prices).
- **Negative values:** Prices can be negative, which rules out the common logarithmic transform (you cannot take the log of a negative number).

The inverse hyperbolic sine (arcsinh)

The **arcsinh transform** solves all of these problems at once.

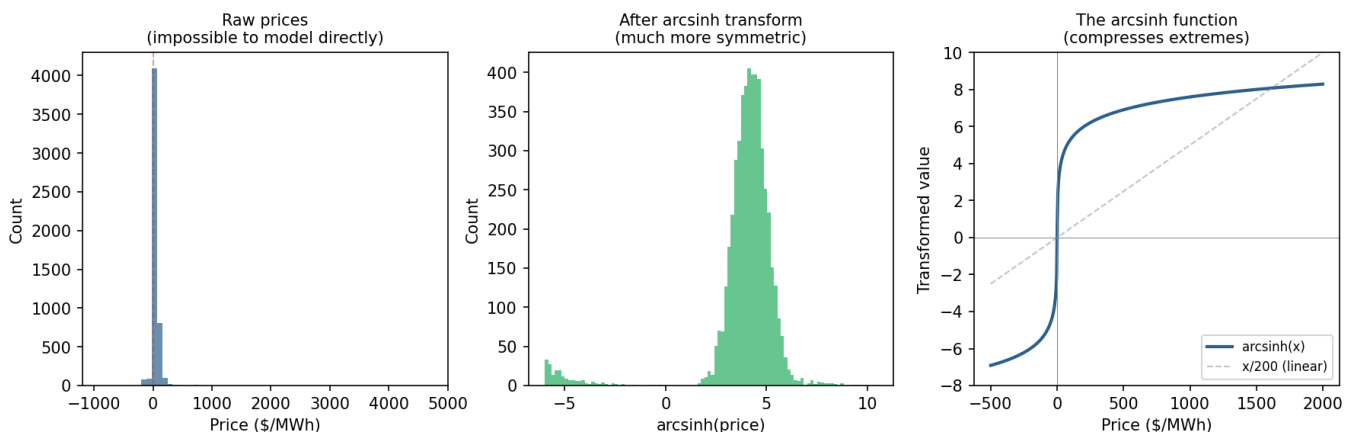


Figure 1.4 — Left: raw prices are extremely right-skewed, making modelling nearly impossible. Centre: after arcsinh transformation, the distribution is much more symmetric. Right: the arcsinh function compresses extreme values while being nearly linear near zero.

Inverse hyperbolic sine (arcsinh): A mathematical function defined as $\text{arcsinh}(x) = \ln(x + \sqrt{x^2 + 1})$. It behaves like a logarithm for large values (compressing them) but is defined for all real numbers, including negatives. It is the standard target transform for electricity price forecasting.

$$y = \text{arcsinh}(x) = \ln(x + \sqrt{x^2 + 1})$$

Why arcsinh is perfect for electricity prices:

Property	Why it helps
Defined for negatives	Handles −\$1,000 floor prices without special cases
Compresses large values	A \$15,000 spike becomes ~10 instead of dominating the model
Nearly linear near zero	Doesn't distort the bulk of prices (\$20–\$80 range)
Invertible	We can convert predictions back to dollars using $\sinh(y)$

The golden rule: Always *model* in arcsinh space (train on arcsinh-transformed prices), but always *evaluate* in dollar space. If you measure forecast error in arcsinh space, you will hide poor performance on spikes — exactly the events that matter most for battery revenue.

Real-world example — why the transform matters: Without arcsinh , a single \$15,000 spike in the training data would dominate the mean squared error, causing the model to over-predict prices to avoid ever missing a spike. With arcsinh , that spike becomes ~ 10 in transformed space — still the largest value, but not $1,000\times$ larger than a normal \$50 price (which becomes ~ 4.6). The model can now learn patterns from both normal and extreme prices without being overwhelmed.

Data Quality

Missing intervals

Even AEMO's data feed is not perfect. Over a 4.5-year history at 5-minute resolution ($\sim 470,000$ intervals), you will encounter gaps — missing intervals due to system outages, delayed publications, or corrupted files.

Standard approaches for handling missing data:

Gap length	Strategy	Rationale
1–3 intervals (5–15 min)	Forward fill	Short gaps; price likely unchanged
4–6 intervals (15–30 min)	Forward fill or flag	Borderline; depends on context
7+ intervals (>30 min)	Flag and exclude	Too long to impute reliably

Forward fill: Replace each missing value with the most recent known value. For electricity prices, this assumes the price stays constant during short data outages — a reasonable assumption since prices typically don't change by much in 5–15 minutes during normal conditions.

The INTERVENTION flag

During **market interventions** (when AEMO directs generators to change output for grid reliability), the dispatch engine runs twice: once with the intervention, once without. Both results appear in the data. Always filter to `INTERVENTION = 0` — this is the "what would have happened under normal market operation" price, which is the correct modelling target.

5-minute to 30-minute aggregation

The 30-minute trading price is the arithmetic mean of six consecutive 5-minute dispatch prices. This matches AEMO's settlement calculation exactly. Using a different aggregation method (median, weighted average) would not match the actual financial settlement and would introduce systematic error.

Data Storage: Parquet Format

Parquet: A columnar data storage format, standard in data science. It stores data by column rather than by row, which enables excellent compression and fast partial reads. A 200 MB CSV file of NEM price data compresses to roughly 15 MB in parquet format — with no information loss.

Advantages of parquet for this project:

- **Compression:** $\sim 13\times$ smaller than CSV. Saves disk space and speeds up loading.
- **Type preservation:** Dates remain dates, floats remain floats. No "is this column a string or a number?" parsing issues.
- **Partial reads:** You can load just the columns you need without reading the whole file.
- **Universal:** pandas, polars, duckdb, and spark all read parquet natively.

Reproducibility: Why It Matters

Reproducibility: The ability for anyone to re-run your analysis and get exactly the same results. In scientific research and industry, this is essential for trust, debugging, and regulatory compliance.

Energy market research has a reproducibility problem. Download the same data a month later and you might get slightly different results — AEMO revises settlement data, library versions change, or different random seeds produce different train/test splits.

This project enforces reproducibility through four mechanisms:

1. **Cached downloads:** Every data download is saved locally. Once cached, the data never changes.
2. **Pinned random seeds:** A single seed (42) controls all random operations — train/test splits, cross-validation folds, model initialisation.
3. **Fixed date windows:** The train and test periods are specified in the config file, not computed from the data.
4. **Version pinning:** All Python package versions are locked in `pyproject.toml`.

Real-world example — why reproducibility matters: Imagine you build a model that shows 15% improvement over the baseline. Your colleague re-runs it a week later and gets 8%. Did the model change? Did the data change? With full reproducibility, you can guarantee the same inputs produce the same outputs — and any difference must be due to a genuine change you made, not random variation or data drift.

Glossary

Term	Definition
NEM	National Electricity Market — Australia's wholesale electricity market for the eastern states
AEMO	Australian Energy Market Operator — runs the NEM dispatch and settlement
MWh	Megawatt-hour — standard unit of energy in wholesale markets
Gross pool	Market design where all electricity must be traded through the central exchange
Marginal pricing	All dispatched generators receive the price of the most expensive unit needed
Dispatch price	5-minute price set by the marginal generator
Trading price	30-minute average of six dispatch prices; the financially settled price
Interconnector	Transmission line connecting two NEM regions
MPC	Market Price Cap — maximum bid price (\$17,500/MWh)
Interval-ending	Timestamp convention where rows are labelled with the period's END time
arcsinh	Inverse hyperbolic sine — the target transform for electricity prices
Forward fill	Imputation method that carries the last known value forward through gaps
Parquet	Columnar data storage format; standard for analytical workloads

Summary

The NEM is a marginal-pricing gross pool where the price is set by the most expensive generator dispatched every five minutes. Prices range from $-\$1,000$ to $\$17,500/\text{MWh}$ with extreme volatility, driven by the interaction of variable renewables, inflexible coal, and weather-driven demand. Data arrives in interval-ending timestamps and must be shifted to interval-start convention exactly once on load. The arcsinh transform tames the wild price distribution for modelling, but we always evaluate in dollar space. Parquet provides efficient storage, and strict reproducibility measures ensure every result can be recreated exactly.

2. Price Distribution and Stylised Facts

What Are "Stylised Facts"?

Before building any forecasting model, we need to understand the patterns that electricity prices consistently exhibit. In economics and finance, these recurring patterns are called **stylised facts** — statistical properties that appear reliably across different time periods and markets.

Stylised facts: Empirical regularities that are observed consistently across multiple datasets, time periods, or markets. They describe "what always seems to be true" about a system, without necessarily explaining why. A good forecasting model should be consistent with these facts.

Think of stylised facts as the "personality traits" of electricity prices. Just as a person might always be early to meetings and prefer coffee over tea, electricity prices have their own persistent habits: they follow daily rhythms, cluster their volatility, and occasionally explode into extreme spikes. Any model that contradicts these traits is probably wrong.

The Daily Price Cycle: The Duck Curve

The most visible pattern in electricity prices is the **diurnal cycle** — prices follow a predictable daily rhythm driven by when people use electricity and when the sun shines.

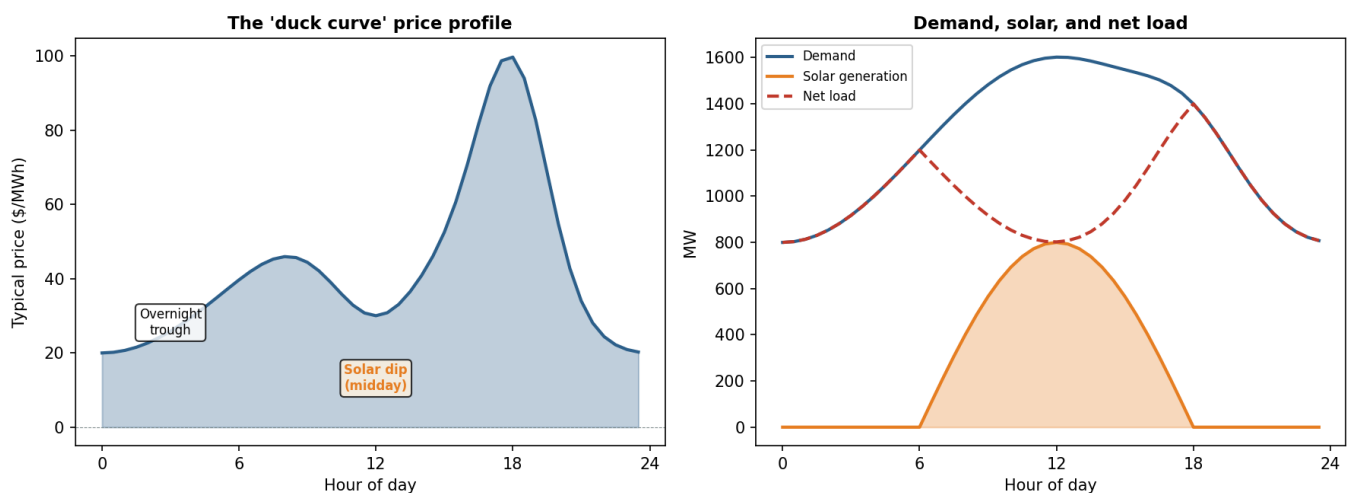


Figure 2.1 — Left: the characteristic "duck curve" daily price profile showing the overnight trough, midday solar dip, and evening spike. Right: demand, solar generation, and net load through the day.

Why does the duck curve happen?

The shape is driven by the interaction of three forces:

Demand follows human activity. People wake up, turn on lights and appliances (morning ramp). They go to work, return home, cook dinner, run heating/cooling (evening peak). They go to sleep (overnight trough).

Solar generation peaks at midday. When the sun is strongest, solar farms flood the grid with cheap electricity, pushing the price down — even to negative values on sunny days.

The evening collision. Just as solar generation drops rapidly at sunset, demand peaks as people return home. This creates a sharp supply shortage, forcing expensive gas peakers online and causing the price spike that defines the right side of the duck curve.

Duck curve: The characteristic shape of net electrical load (demand minus solar generation) through the day. It resembles a duck in profile: a belly dip during midday solar, a neck ramp in the late afternoon, and a head peak in the evening. First identified by California's grid operator (CAISO), it has become a defining feature of grids with high solar penetration.

Net load: Total electricity demand minus variable renewable generation (wind + solar). Net load represents the amount of electricity that must be supplied by dispatchable generators (coal, gas, hydro, batteries). When net load is high, prices are high; when net load is low or negative, prices are low or negative. It is the single most important driver of electricity prices.

Real-world example: In South Australia on a sunny spring day, solar generation might peak at 2,000 MW around noon while demand is only 1,400 MW. Net load goes *negative* — there is more solar than the state needs. The price drops below zero. Then at 6pm, the sun sets, solar drops to zero, and demand stays high. Net load jumps from -600 MW to +1,400 MW in just two hours. The price rockets from -\$20 to \$200+. This daily drama is the duck curve in action.

Heavy Tails: Extreme Events Are Not Rare

In a **normal (Gaussian) distribution**, extreme events are vanishingly rare. A value more than 3 standard deviations from the mean occurs only 0.3% of the time. But electricity prices do not follow a normal distribution — they have **heavy tails**, meaning extreme events occur far more often than a normal distribution would predict.

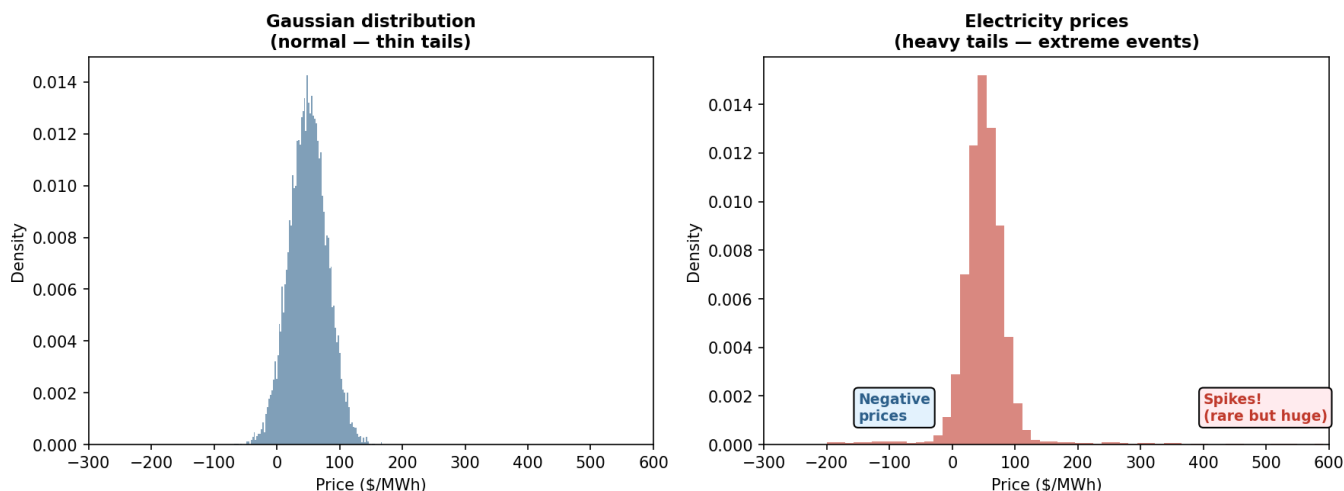


Figure 2.2 — Left: a normal distribution with thin tails — extreme values are extremely rare. Right: electricity prices have heavy tails, with both extreme spikes (right) and negative prices (left) occurring much more frequently than a normal distribution would predict.

Heavy tails (fat tails): A statistical property where extreme values occur much more frequently than a normal distribution predicts. If prices were normally distributed with mean \$50 and standard deviation \$30, a price above \$200 would be a "five-sigma event" — expected roughly once every 3.5 million observations. In reality, NEM prices exceed \$200 roughly once every 100–200 observations.

Kurtosis: A measure of how "heavy" the tails of a distribution are. A normal distribution has kurtosis of 3. Electricity prices typically have kurtosis of 30–100+ (called **leptokurtic**), meaning the tails are enormously heavier than normal. This is why standard statistical methods that assume normality fail badly for electricity prices.

Why heavy tails matter

Heavy tails have profound consequences for modelling and trading:

Mean and variance are unreliable. A few extreme spikes dominate the average, making sample statistics unstable. The average price over a month can shift dramatically depending on whether a single \$10,000 spike occurred.

Normal-distribution-based risk measures fail. Value-at-Risk calculated under a normality assumption will severely underestimate the risk of extreme price events.

Model training is distorted. Standard least-squares regression will be pulled toward fitting the extreme observations, distorting predictions for the 99% of "normal" prices. This is one reason we use the arcsinh transform (Chapter 1).

Real-world example — the Tomago smelter: In January 2019, a heatwave caused NEM prices to spike above \$14,000/MWh for several hours. The Tomago aluminium smelter in NSW — one of the world's largest electricity consumers — voluntarily reduced its load to avoid the extreme prices. The cost of running at full production during those few hours would have exceeded the profit from weeks of aluminium production. These tail events are not theoretical curiosities — they drive real business decisions worth millions of dollars.

Autocorrelation: Today Looks Like Yesterday

Electricity prices exhibit strong **autocorrelation** — the price at any given time is strongly related to the price at the same time yesterday, the day before, and even the same time last week.

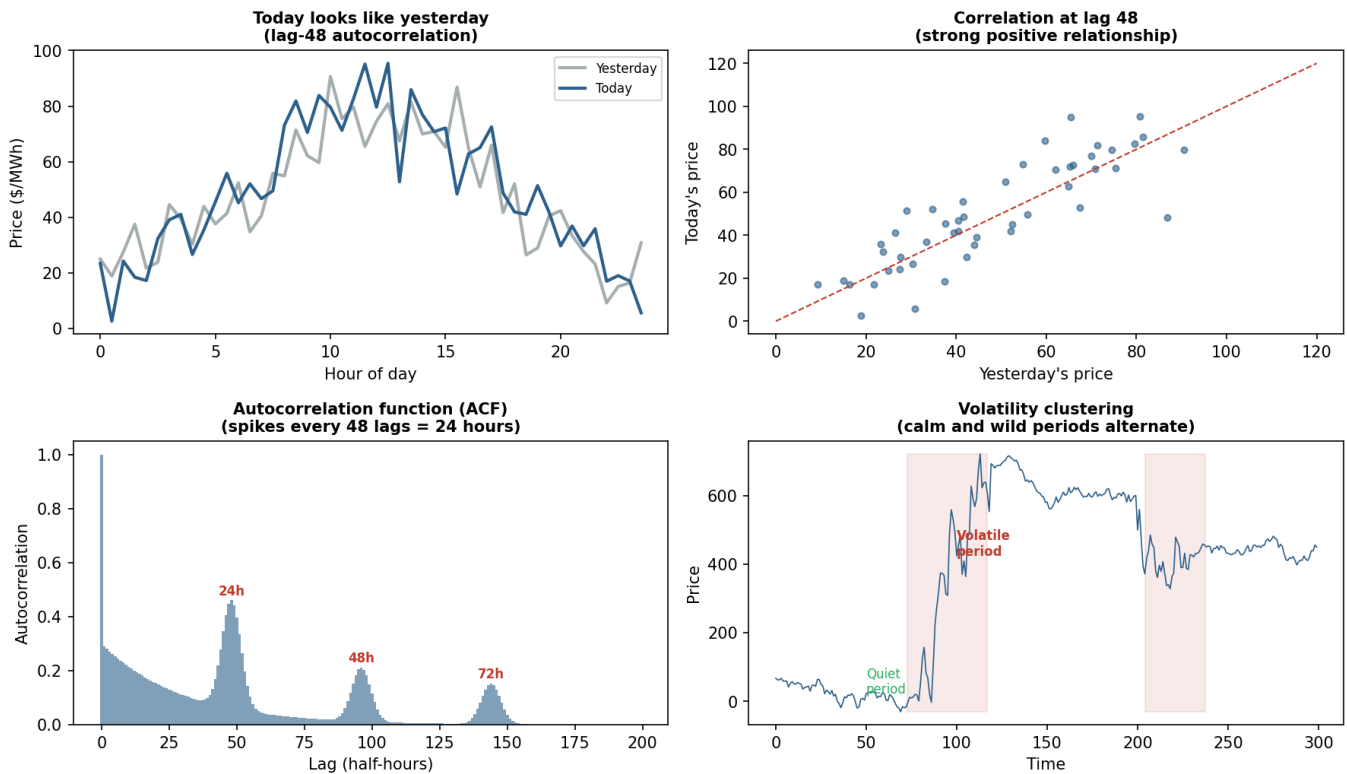


Figure 2.3 — Top left: today's prices closely track yesterday's. Top right: scatter plot showing the strong positive correlation at lag 48 (24 hours). Bottom left: the autocorrelation function (ACF) showing spikes at 24h, 48h, and 72h lags. Bottom right: volatility clustering — calm and wild periods tend to persist.

Autocorrelation: The correlation of a time series with a lagged copy of itself. If today's 2pm price is strongly correlated with yesterday's 2pm price, we say there is strong autocorrelation at lag 48 (48 half-hours = 24 hours). Autocorrelation tells us how "predictable" a time series is from its own past values.

Autocorrelation function (ACF): A plot showing the autocorrelation at every possible lag. For electricity prices, the ACF shows peaks at lags of 48 (24 hours), 96 (48 hours), and 336 (one week) — revealing that prices repeat on daily and weekly cycles.

What the ACF tells us

The ACF for NEM prices reveals several key patterns:

- **Lag 48 (24 hours):** The strongest peak. Today's 2pm price is strongly related to yesterday's 2pm price. This is the diurnal cycle — the daily rhythm of human activity repeats.
- **Lag 96 (48 hours):** Still significant but weaker. Prices two days ago are informative but less so than yesterday.
- **Lag 336 (1 week):** A smaller but real peak. The weekly cycle matters because weekdays have higher demand than weekends.
- **Slow decay:** Autocorrelation decreases with lag but never reaches zero quickly, suggesting long-memory effects (persistent regimes of high or low prices).

Key insight: The lag-48 autocorrelation is the single most powerful forecasting feature in electricity prices. "What was the price at this time yesterday?" is the most useful piece of information for predicting the price right now. This is why price lags dominate feature importance in every model we build.

Volatility Clustering: Calm Breeds Calm, Chaos Breeds Chaos

Electricity prices exhibit **volatility clustering** — periods of high volatility (wild price swings) tend to be followed by more high volatility, and calm periods tend to be followed by more calm periods.

Volatility: The degree of variation in a time series. High volatility means prices are swinging wildly; low volatility means prices are stable. In finance, volatility is usually measured as the standard deviation of returns over a recent window.

Volatility clustering: The empirical observation that large price changes tend to be followed by more large price changes (of either sign), and small changes tend to be followed by small changes. In technical terms, the absolute returns are autocorrelated even when the returns themselves are not. This is a hallmark of financial and commodity time series.

Why does volatility cluster?

Several mechanisms drive volatility clustering in the NEM:

Weather persistence. A heatwave doesn't last one afternoon — it persists for days. During the heatwave, high demand keeps prices volatile. When the heatwave breaks, prices calm down.

Generator outages. When a large coal plant trips (unexpectedly shuts down), the supply shortage persists until the plant is repaired — days or weeks. The reduced supply margin keeps prices volatile throughout.

Seasonal renewables. Spring brings consistently windy, sunny conditions that keep net load low and prices calm. Summer heatwaves bring consistently high demand that keeps prices volatile.

The Price-Duration Curve

A powerful way to visualise the price distribution is the **price-duration curve**, which sorts all prices from highest to lowest and plots them against the percentage of time each price level is exceeded.

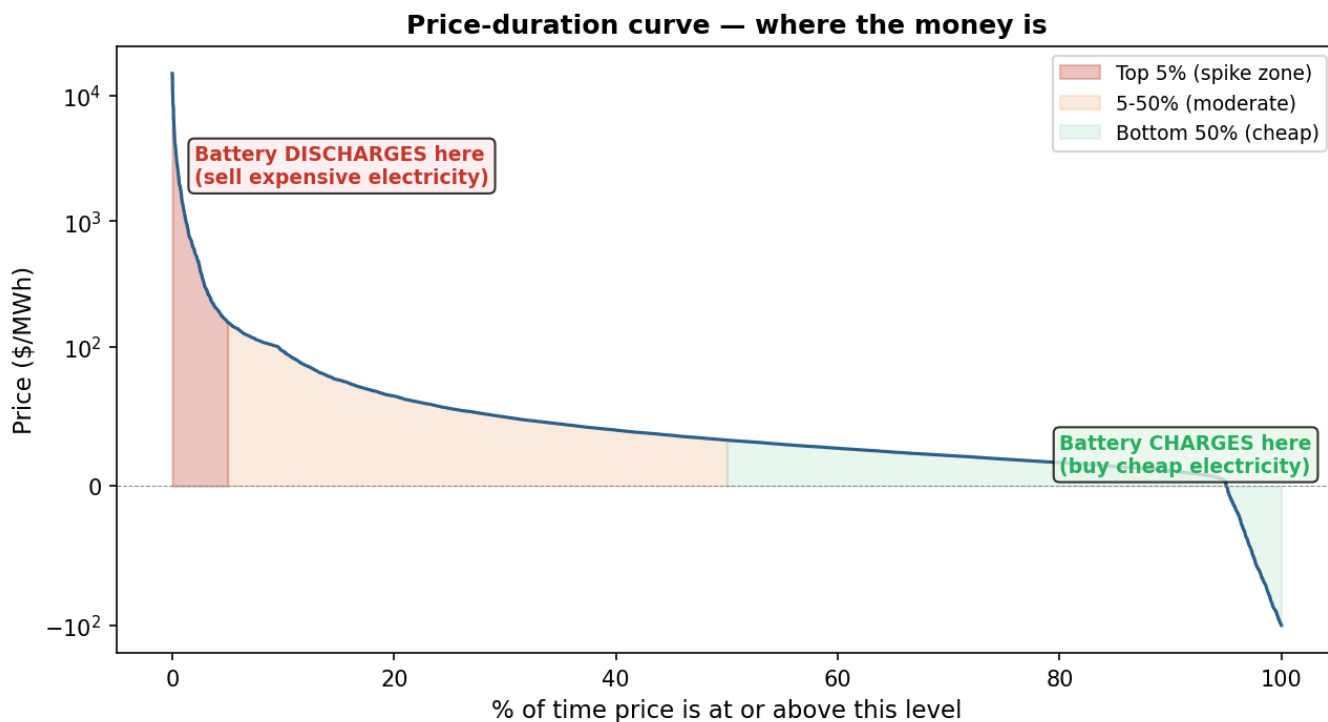


Figure 2.4 — The price-duration curve. The top 5% of hours (spike zone) contribute a disproportionate share of total wholesale cost. Batteries aim to discharge in the red zone and charge in the green zone.

Price-duration curve: A plot where prices are sorted from highest to lowest and plotted against the fraction of time they are exceeded. It shows at a glance what percentage of the time prices are above any given level. For batteries, it reveals the "spread" — the gap between charge prices (bottom of the curve) and discharge prices (top).

The price-duration curve reveals a fundamental fact about electricity markets: **a tiny fraction of hours accounts for a huge share of total cost.** In the NEM, the most expensive 5% of hours can account for 30–50% of total annual wholesale cost. This concentration is what makes battery arbitrage possible — buy during the cheap 50% at the bottom, sell during the expensive 5% at the top.

Real-world example — the Mannum BESS: The Mannum BESS — a 100 MW / 200 MWh lithium iron phosphate battery in SA1, owned by Epic Energy and optimised by Habitat Energy — might earn \$10 million per year in perfect-foresight arbitrage. Of that \$10M, roughly \$6–7M comes from the most expensive 5% of intervals — perhaps 450 half-hours out of 17,520 in a year. Getting those 450 intervals right (knowing when they will occur in advance) is the core challenge of price forecasting for battery dispatch.

Seasonality: Annual and Weekly Rhythms

Beyond the daily cycle, electricity prices show seasonal patterns at multiple time scales:

Weekly seasonality

- **Weekdays vs weekends:** Demand is 10–20% lower on weekends because commercial and industrial loads drop. Prices follow.
- **Monday mornings:** The transition from weekend to weekday creates a demand ramp, often with higher prices.
- **Friday evenings:** The transition to weekend relaxes demand.

Annual seasonality

- **Summer (Dec–Feb in Australia):** Extreme heat drives air conditioning load, creating the highest demand and the most price spikes. This is the most volatile season.
- **Winter (Jun–Aug):** Heating load creates a secondary demand peak, but generally less extreme than summer.
- **Spring/Autumn (Mar–May, Sep–Nov):** Mild weather reduces demand. High solar and wind penetration can push prices consistently low.

Seasonality: A regular, predictable pattern that repeats at a fixed frequency. Daily seasonality repeats every 24 hours (the duck curve). Weekly seasonality repeats every 7 days (weekday vs weekend). Annual seasonality repeats every 12 months (summer vs winter). Seasonal patterns can be removed ("deseasonalised") to reveal the underlying trend and random variation.

Summary of Stylised Facts

Stylised fact	What it means	Modelling implication
Diurnal cycle	Prices follow a daily rhythm	Include hour-of-day features
Heavy tails	Extreme events far more common than normal	Use arcsinh transform; don't assume normality
Autocorrelation	Today resembles yesterday	Include lagged price features (especially lag 48)
Volatility clustering	Volatile periods persist	Include recent volatility as a feature
Seasonality	Weekly and annual patterns	Include day-of-week and month features
Mean reversion	Spikes are short-lived, prices return to a base	Don't extrapolate spikes; model reversion

Glossary

Term	Definition
Stylised facts	Consistent empirical patterns in a dataset
Diurnal cycle	Daily repeating pattern in prices
Duck curve	Daily net load shape: midday dip, evening spike
Net load	Demand minus wind and solar generation
Heavy tails	Extreme values occur far more often than a normal distribution predicts
Kurtosis	Measure of tail heaviness; electricity prices have very high kurtosis
Autocorrelation	Correlation of a time series with its own past values
ACF	Autocorrelation function — autocorrelation plotted at every lag
Volatility	Degree of price variation over time
Volatility clustering	Tendency for volatile periods to persist
Price-duration curve	Prices sorted from high to low vs % of time exceeded
Seasonality	Regular patterns repeating at fixed intervals
Mean reversion	Tendency of extreme prices to return to a long-run average

Summary

Electricity prices exhibit a consistent set of stylised facts that any forecasting model must respect. The daily duck curve — with its midday solar dip and evening spike — is the dominant pattern. Heavy tails mean extreme events are not rare; they are a defining feature of the market. Strong autocorrelation at lag 48 (24 hours) makes yesterday's price the single best predictor of today's. Volatility clusters in time, driven by persistent weather and supply conditions. And the price-duration curve reveals that a tiny fraction of extreme-price hours drives a disproportionate share of market value — making accurate spike forecasting the key to battery profitability.

3. Price Formation: How Prices Are Actually Set

The Merit-Order Supply Stack

In Chapter 1, we described how AEMO dispatches generators from cheapest to most expensive. Now we examine this mechanism in detail — the **merit-order supply stack** — because understanding how prices form is essential for understanding what drives price changes.

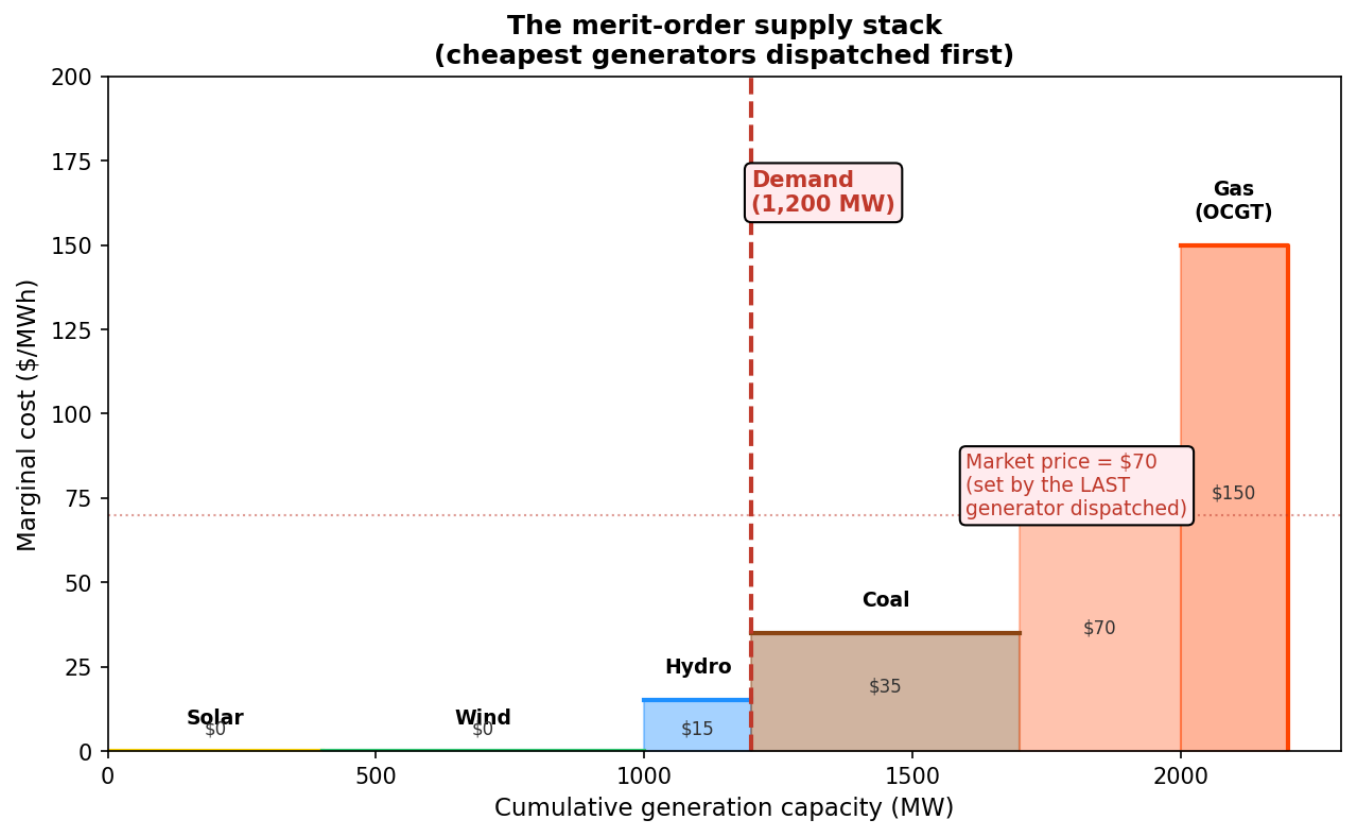


Figure 3.1 — The merit-order supply stack. Generators are arranged from cheapest (left) to most expensive (right). The price is set by the marginal generator — the last one dispatched to meet demand.

Merit order: The ranking of available generators from lowest to highest marginal cost. AEMO dispatches generators in merit order — the cheapest first — until total generation meets demand. The "merit" is purely economic: the cheapest generator has the highest merit.

Supply stack: A graphical representation of the merit order, showing cumulative generation capacity (x-axis) against bid price (y-axis). It forms a staircase shape, stepping up as more expensive generators are added. Where the demand line intersects the supply stack determines the market price.

The generators in the stack

Each fuel type occupies a characteristic position in the merit order:

Fuel type	Typical marginal cost	Position in stack	Why
Solar	~\$0/MWh	Bottom	No fuel cost — sunlight is free
Wind	~\$0/MWh	Bottom	No fuel cost — wind is free
Hydro	\$10–30/MWh	Low-middle	Water has opportunity cost but no fuel cost
Coal	\$25–50/MWh	Middle	Cheap fuel but inflexible
Gas (CCGT)	\$60–100/MWh	Upper-middle	Moderate fuel cost, efficient
Gas (OCGT)	\$100–300/MWh	Top	Expensive fuel, inefficient, but fast to start

Marginal cost: The cost of producing one additional unit of output. For a gas plant, this is primarily the cost of the gas burned. For solar and wind, the marginal cost is essentially zero — once the farm is built, producing one more MWh costs nothing (the fuel is free). Note: marginal cost is NOT the same as total cost. A solar farm has high construction costs but zero marginal costs.

CCGT (Combined Cycle Gas Turbine): A gas plant that uses waste heat from the gas turbine to drive a steam turbine, achieving higher efficiency (50–60%). More efficient but slower to start than OCGT.

OCGT (Open Cycle Gas Turbine): A simple gas turbine that discards waste heat. Less efficient (30–40%) but can start from cold in 10–15 minutes. Used as "peaker" plants — they only run during high-demand periods when prices justify their high fuel cost.

How demand position determines price

The critical insight: **the price depends not on the total generation mix, but on which generator is at the margin.**

Example — low demand (1 AM): Only 1,000 MW of demand. Solar and wind supply 600 MW, coal supplies the remaining 400 MW. The marginal generator is a coal plant bidding \$35/MWh. All generators — solar, wind, and coal — receive \$35/MWh. Price: **\$35**.

Example — moderate demand (11 AM): 1,500 MW of demand. Solar and wind supply 900 MW, coal supplies 500 MW, and a CCGT gas plant supplies the final 100 MW. The marginal generator is the gas plant bidding \$75/MWh. All generators receive \$75/MWh — even the solar farms with zero fuel cost. Price: **\$75**.

Example — extreme demand (6 PM heatwave): 2,200 MW of demand. Solar is gone (sunset). Wind supplies 300 MW, coal 500 MW, CCGT 300 MW, and OCGT peakers supply the final 100 MW. The marginal OCGT bids \$250/MWh. Price: **\$250**. A 43% increase in demand (from 1,500 to 2,200 MW) caused a 233% increase in price — this is the nonlinear relationship between demand and price.

The Hockey Stick: Net Load vs Price

When we plot net load against price across thousands of observations, a striking pattern emerges — the **hockey stick**.

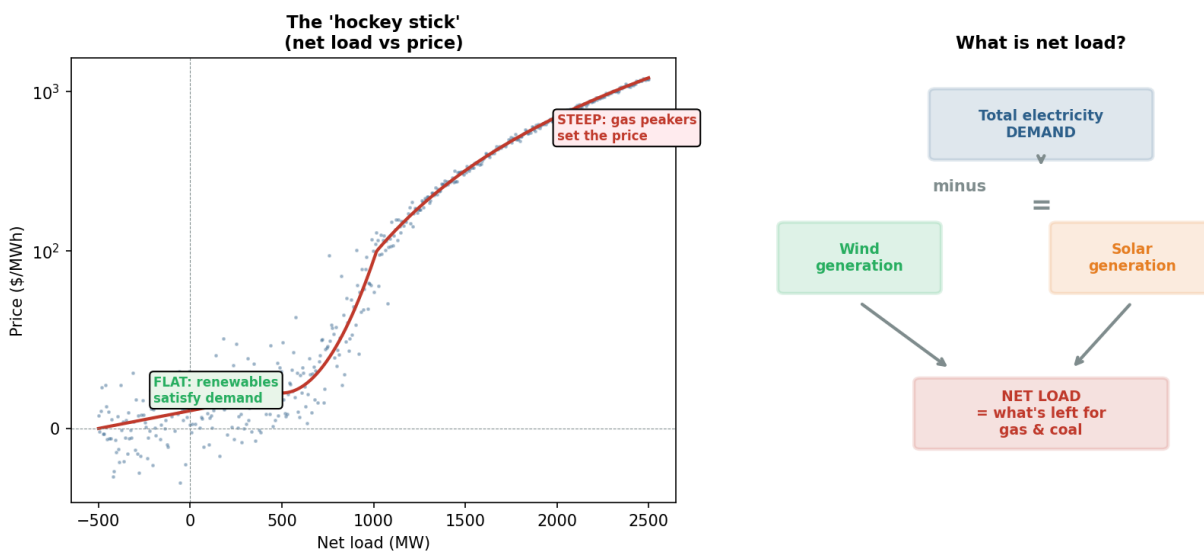


Figure 3.2 — Left: the hockey stick relationship between net load and price. Prices are roughly flat at low net load (renewables satisfy demand) and rise exponentially at high net load (expensive peakers set the price). Right: net load decomposition.

Hockey stick (in electricity pricing): The characteristic nonlinear relationship between net load and price. At low net load, prices are flat (cheap generators have spare capacity). At high net load, prices rise exponentially as expensive peakers are dispatched. The plot resembles a hockey stick lying flat with its blade curving upward.

Why the hockey stick is so important

The hockey stick is arguably the most important empirical relationship in electricity price forecasting:

It explains why renewable energy depresses prices. More solar/wind → lower net load → we stay in the flat part of the stick → lower prices. This is called the **merit-order effect** of renewables.

It explains why small demand changes cause huge price changes. On the steep part of the stick, a 5% increase in net load can double or triple the price.

It explains why price spikes are hard to predict. The transition from the flat part to the steep part is abrupt and depends on the exact position of generators in the stack, which changes day to day.

Merit-order effect: The tendency of zero-marginal-cost renewables (solar, wind) to push down wholesale electricity prices by displacing expensive generators from the merit order. When wind and solar are abundant, the demand that would otherwise be met by gas peakers is met by renewables, and the marginal generator is a cheaper coal plant. The more renewables in the system, the stronger this effect.

Key modelling insight: Net load is the single most powerful predictor of electricity prices — more powerful than demand alone, because it captures the offsetting effect of renewables. A model that includes net load can capture the hockey stick shape and the merit-order effect simultaneously.

Anatomy of a Price Spike

Price spikes are the most dramatic feature of electricity markets. A spike can take the price from \$50 to \$15,000 in a single 5-minute interval. Understanding the mechanism helps us forecast them — or at least understand when they are more likely.

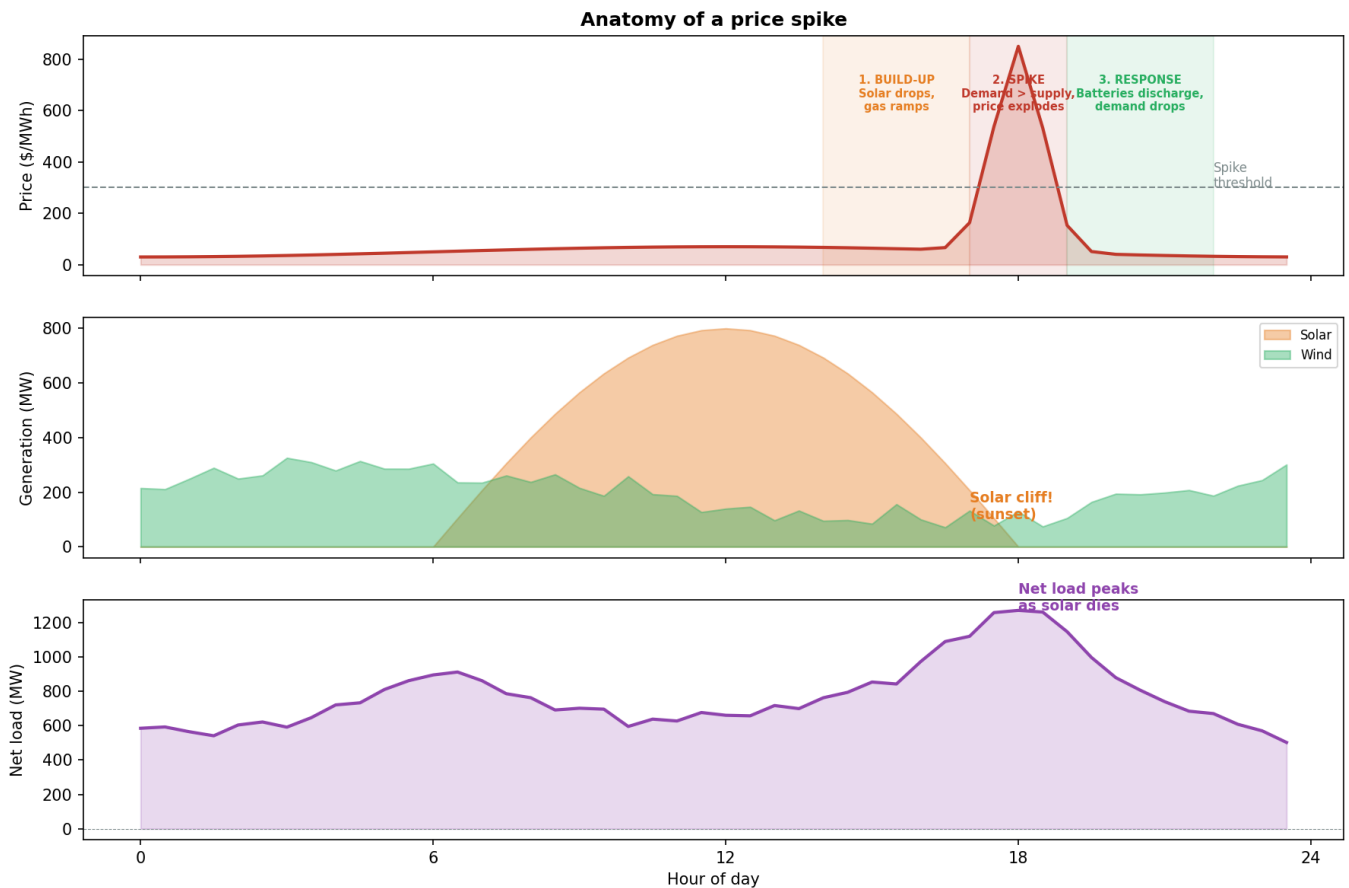


Figure 3.3 — The three phases of a price spike: build-up (solar drops, gas ramps), the spike itself (demand exceeds available supply), and the response (batteries discharge, demand drops).

The three phases of a spike

Phase 1 — Build-up (1–3 hours before): - Solar generation ramps down as the sun sets - Demand stays high or rises (evening cooking, heating/cooling) - Net load climbs steadily - Gas plants start ramping up - Prices rise from \$50 to \$100–200

Phase 2 — The spike (minutes): - Net load exceeds comfortable supply margin - The market clears on the most expensive generators - Some generators bid strategically at the market price cap - Prices jump to \$1,000–\$15,000+ in a single interval

Phase 3 — Response (30–60 minutes): - Batteries discharge, injecting stored energy - Price-responsive loads reduce consumption - AEMO may direct interconnector flows or issue reliability interventions - Prices fall back below \$200

Price spike: An abrupt, temporary, extreme increase in the wholesale electricity price, typically lasting one to a few trading intervals. There is no universal threshold, but common definitions include prices above \$300/MWh, \$1,000/MWh, or the 99th percentile of historical prices. Spikes account for a tiny fraction of all intervals but contribute disproportionately to annual wholesale cost.

What causes spikes

Cause	Mechanism	Predictability
Heatwaves	High A/C demand, low wind	Moderate — weather forecasts give 2–5 day warning

Generator trip	Large unit unexpectedly fails	Low — can happen any time
Sunset ramp	Solar drops faster than gas can ramp	High — happens predictably at sunset
Interconnector failure	Region isolated from cheap imports	Low — equipment failure
Strategic bidding	Generators bid high when they have market power	Low — depends on competitor behaviour

Real-world example — the February 2017 SA heatwave: Temperatures exceeded 40°C for three consecutive days. Air conditioning load pushed demand to record levels. Wind generation was below average. SA1 prices exceeded \$1,000/MWh for 18 intervals (9 hours) over the three days, with peaks above \$14,000/MWh. The total excess cost to retailers was estimated at over \$50 million. A battery operator who had forecast even half of these spikes correctly could have earned several million dollars in those three days alone.

Generator Bidding Behaviour

The supply stack is not fixed — generators can change their bids strategically, shifting the price-setting dynamics.

Bidding behaviour: The strategy generators use when submitting price-quantity offers to AEMO. While the merit order assumes generators bid at their marginal cost, in practice generators consider market conditions, competitor behaviour, and their portfolio position when setting bids.

Strategic bidding

Generators with **market power** (large share of capacity in a region) can influence prices by:

Withholding capacity: Bidding some capacity at the market price cap so it won't be dispatched, reducing supply and raising the price for the remaining capacity.

Rebidding: Changing bids intra-day in response to changing conditions. A generator might bid cheaply in the morning (to be dispatched) then rebid expensive in the afternoon (when demand is higher and competition is thinner).

Market power: The ability of a single market participant to influence the market price. In electricity markets, market power arises when a single generator or portfolio controls enough capacity that withholding some of it would materially raise the price. Market power is more prevalent in smaller, more concentrated regions like SA1.

Why bidding matters for forecasting

Strategic bidding introduces an element of **game-theoretic uncertainty** into price formation. Even with perfect weather forecasts and exact demand predictions, the price depends on *how generators choose to bid* — which is itself a strategic decision based on their expectations of market conditions.

This is a fundamental limit on forecasting accuracy: the price is partly determined by human decisions that respond to expectations, creating a feedback loop. No model can perfectly predict strategic behaviour.

Supply Mix Transition

The NEM is in the middle of a generational transition from fossil fuels to renewables. This transition is reshaping the price formation dynamics:

Trend	Effect on prices	Time scale
Coal plant retirements	Higher prices when coal was the marginal unit	Years
More solar	Lower midday prices, deeper duck curve belly	Ongoing
More wind	Lower prices generally, but intermittent	Ongoing
More batteries	Price smoothing, faster spike response	Emerging
Electrification (EVs, heat pumps)	Higher demand, new load shapes	5–15 years

Implication for forecasting: The supply stack is not static — it shifts over months and years as generators enter and exit. A model trained on 2020 data may not perform well on 2025 data because the generation mix has changed. This is why rolling retraining (updating the model with recent data) is essential.

Market Institutions and Rules

Understanding who runs the NEM and what guardrails exist is essential vocabulary for working on a battery trading desk. This section provides the institutional context — the rules of the game that shape every price the forecasting models try to predict.

Who runs the market

Three bodies govern the NEM:

Institution	Role
AEMO (Australian Energy Market Operator)	Operates the market and the power system in real time — dispatches generators, publishes prices, manages system security
AER (Australian Energy Regulator)	Monitors compliance, investigates market manipulation, enforces the rules
AEMC (Australian Energy Market Commission)	Makes and amends the rules that AEMO and participants operate under

AEMO (Australian Energy Market Operator): The independent operator responsible for running the NEM's dispatch engine (NEMDE) every five minutes, publishing spot prices, managing system security, and producing demand and price forecasts (pre-dispatch). AEMO does not own generators or set prices — it runs the auction that determines them.

The market price cap and floor

The NEM spot price is bounded:

- **Market Price Cap (MPC):** The maximum price any generator can bid. As of 2024–25, the MPC is \$17,500/MWh, indexed annually to CPI. This cap exists to prevent extreme exploitation of market power during supply shortages. The cap has risen over time — it was \$12,500/MWh before 2010.
- **Market Floor Price (MFP):** The minimum price, set at $-\$1,000/\text{MWh}$. Negative prices occur when generators (typically renewables with contracts that pay them regardless of the spot price) bid below zero to avoid being constrained off. A battery charging at $-\$200/\text{MWh}$ is being *paid* to take electricity.

Why the cap matters for forecasting: The market price cap creates a hard ceiling on price spikes. A model that predicts \$25,000/MWh is wrong by construction — the price cannot exceed \$17,500/MWh. More subtly, the cap compresses the upper tail of the price distribution: events that would "naturally" produce different extreme prices are all capped at the same value, making the tail harder to model statistically. When reporting capture ratios, the cap also limits the upside — even perfect foresight cannot earn more than the capped price.

Check the current MPC: The MPC is indexed annually. Before asserting a specific value, verify the current cap at AEMO's website. The value stated here (\$17,500/MWh for 2024–25) is indicative.

Administered pricing and the cumulative price threshold

When prices spike persistently, AEMO can intervene:

- **Cumulative Price Threshold (CPT):** If the rolling sum of spot prices over a seven-day window exceeds a threshold (currently \$1,612,600, indexed annually), AEMO declares an **Administered Price Period** and caps the price at the **Administered Price Cap (APC)** of \$600/MWh. This prevents sustained extreme pricing from causing systemic financial distress among retailers and large consumers.

Administered Price Period: A period triggered when cumulative spot prices over a rolling seven-day window exceed the Cumulative Price Threshold. During this period, AEMO caps the spot price at the Administered Price Cap (\$600/MWh), dramatically reducing generator revenue. For a battery, an administered price period truncates the arbitrage opportunity — a forecasting model should flag when the CPT is approaching.

Scarcity signals: lack of reserve and the reserve trader

AEMO publishes real-time indicators of system stress that affect prices:

Lack of Reserve (LOR): AEMO declares LOR conditions when available generation reserves fall below specified thresholds. LOR1 is an early warning; LOR2 indicates insufficient reserves to cover the largest single contingency; LOR3 means involuntary load shedding is imminent. LOR declarations are strong signals of imminent price spikes — generators know reserves are scarce and bid accordingly.

Reliability and Emergency Reserve Trader (RERT): When the market alone cannot ensure adequate reserves, AEMO can contract emergency reserves outside the normal market — paying generators or large consumers to be available. The cost of RERT is recovered from market participants. RERT activation signals extreme scarcity and near-certain elevated prices.

LOR as a forecasting feature: Incorporating LOR declarations as features in a price forecasting model is straightforward — they are published in real time via AEMO's market notices. An LOR2 declaration during an evening peak is one of the strongest predictors of an imminent price spike, because it signals that the system is operating on the steep part of the supply stack with minimal margin.

Beyond the spot market

The spot market is where our forecasting and dispatch operate, but battery operators also interact with adjacent markets:

- **Financial hedging:** Contracts for difference (CFDs), swaps, and caps that lock in future revenue at agreed prices, reducing exposure to spot price volatility. These are managed by the trading team, not the data science team, but they create incentives that shape dispatch strategy.
- **Capacity mechanisms:** The NEM does not currently have a formal capacity market (unlike the UK or US PJM), but capacity-like mechanisms are evolving. The Capacity Investment Scheme (CIS) provides revenue certainty for new renewable and storage investment.

These instruments sit with trading and risk management, not the forecasting role, but understanding that they exist explains why a desk sometimes dispatches differently from what the spot forecast alone would suggest — the battery may be managing a hedge position.

Glossary

Term	Definition
Merit order	Ranking of generators from cheapest to most expensive
Supply stack	Visual representation of cumulative generation vs bid price
Marginal cost	Cost of producing one additional unit of output
CCGT	Combined Cycle Gas Turbine — efficient gas plant
OCGT	Open Cycle Gas Turbine — fast-start but expensive peaker
Hockey stick	Nonlinear net-load-to-price relationship
Merit-order effect	Price depression caused by zero-cost renewables
Price spike	Abrupt, extreme, temporary price increase
Market power	Ability of one participant to influence the price
Strategic bidding	Setting bids based on market conditions rather than cost
AEMO	Australian Energy Market Operator — runs the dispatch and publishes prices
AER	Australian Energy Regulator — monitors compliance and enforces rules
AEMC	Australian Energy Market Commission — makes and amends market rules
Market Price Cap (MPC)	Maximum allowable spot price (\$17,500/MWh in 2024–25, indexed to CPI)
Market Floor Price	Minimum spot price (–\$1,000/MWh)
Administered Price Period	AEMO-imposed price cap triggered by sustained extreme pricing
Cumulative Price Threshold	Rolling seven-day price sum that triggers administered pricing
Lack of Reserve (LOR)	AEMO declaration indicating insufficient generation reserves
RERT	Reliability and Emergency Reserve Trader — AEMO's emergency reserve mechanism

Summary

Electricity prices are set by the merit-order supply stack — the cheapest generators dispatched first, with the most expensive one setting the price for all. The hockey stick relationship between net load and price is the most important pattern: prices are flat when renewables dominate and rise exponentially when expensive peakers are needed. Price spikes occur when demand exceeds comfortable supply margins, driven by heatwaves, sunset ramps, generator trips, or strategic bidding. The NEM's generation mix is transitioning rapidly from coal to renewables, continuously reshaping the supply stack and the price formation dynamics.

4. Weather and Renewable Generation

Why Weather Is the Master Variable

In the old world of electricity — grids dominated by coal and gas — weather was a secondary consideration. It affected demand (air conditioning in summer, heating in winter), but the supply side was stable. A coal plant produces the same output whether it is sunny or cloudy, windy or calm. The operator controls the throttle, not the atmosphere.

In the new world — grids where wind and solar provide 50% or more of generation — weather has become the **master variable**. It now drives *both sides* of the electricity equation simultaneously:

- **Demand side:** Temperature determines how much electricity consumers use. A 42°C day in Adelaide triggers massive air conditioning load. A 5°C morning in Melbourne increases heating demand.
- **Supply side:** Solar output depends on sunlight reaching panels. Wind output depends on wind speed at the height of turbine blades. Both are entirely determined by weather.

The combination creates a "double whammy" effect that makes renewable-heavy grids far more volatile than fossil-fuel grids. A hot, calm, hazy day is the worst case: demand is extreme (everyone running air conditioners), wind generation is zero (no wind), and solar generation is reduced (haze and high temperatures lower panel efficiency). An expensive gas peaker must fill the gap, and the price spikes.

Dispatchable generation: Power plants that can be turned on and off, or ramped up and down, at the operator's command. Coal, gas, and hydro plants are dispatchable. The operator decides how much electricity to produce.

Variable renewable energy (VRE): Generation sources whose output depends on weather conditions and cannot be directly controlled by the operator. Wind and solar are VRE — a wind farm produces whatever the wind allows, not what the operator wants. This fundamental difference from dispatchable generation is why weather forecasting has become central to electricity price forecasting.

Real-world example: On 24 January 2019, South Australia experienced temperatures exceeding 46°C — among the highest recorded in an Australian capital city. Demand surged as every air conditioner in the state ran at full capacity. Meanwhile, wind generation dropped below 200 MW (out of ~2,000 MW installed capacity) as a heat dome brought still conditions. With solar panels also underperforming due to extreme heat (panel efficiency drops above ~25°C), the NEM price in SA1 exceeded \$14,000/MWh for multiple intervals. The combination of extreme demand and collapsed renewable supply — both driven by the same weather system — created a "perfect storm" for price spikes.

The double-edged sword: In a renewable-heavy grid, weather simultaneously creates the problem (high demand) and removes the solution (low renewable supply). This coupling means that weather forecast errors affect price forecasts twice — once through the demand channel and once through the supply channel. A model that ignores weather is flying blind.

ERA5: A Complete Picture of the Atmosphere

What Is a Reanalysis?

Weather stations are scattered unevenly across the landscape. Some regions have dense networks of instruments; others — oceans, deserts, remote areas — have almost none. Satellites provide global coverage but measure the top of the atmosphere, not conditions at the surface. Radiosondes (weather balloons) give vertical profiles but are launched only twice daily from a limited number of sites. No single observation system provides a complete, consistent picture of the atmosphere.

A **reanalysis** solves this problem by combining *all* available observations with a physics-based weather model to produce a gridded, gap-free reconstruction of the atmosphere's historical state. The model fills in where observations are missing, and the observations correct where the model is wrong. The result is far more reliable than either source alone.

Reanalysis: A systematic method of producing a comprehensive record of how the atmosphere has changed over time. A weather forecasting model is run over historical periods, continuously correcting itself by "assimilating" (incorporating) all available observations — surface stations, satellites, aircraft reports, ocean buoys, radiosondes. The output is a physically consistent, spatially complete, temporally regular dataset that represents our best estimate of past atmospheric conditions.

Data assimilation: The mathematical process of combining a model's prediction with real-world observations to produce an improved estimate of the atmospheric state. It uses statistical techniques (typically variants of Kalman filtering or variational methods) to weight the model and observations according to their respective uncertainties. Where observations are dense and reliable, the assimilated state closely follows them; where observations are sparse, the model's physics fills the gaps.

ERA5: The Gold Standard

ERA5 (ECMWF Reanalysis version 5) is produced by the **European Centre for Medium-Range Weather Forecasts (ECMWF)**, widely regarded as the world's leading weather modelling institution. ERA5 is the fifth generation of ECMWF's global reanalysis products and is considered the current gold standard for historical atmospheric data.

ERA5 provides:

Property	Value	Significance
Spatial resolution	0.25° × 0.25° (~31 km at equator, ~25 km at Australian latitudes)	Fine enough to capture regional weather patterns
Temporal resolution	Hourly	Matches the timescale of electricity market operations
Temporal coverage	1940 to near-present (updated with ~5 day lag)	Covers the entire history of the modern NEM (1998–present)
Vertical levels	137 model levels from surface to 80 km altitude	Provides wind at multiple heights, including turbine hub height
Variables	Hundreds, covering temperature, wind, radiation, precipitation, pressure, humidity, and more	All weather drivers of electricity demand and supply

ECMWF (European Centre for Medium-Range Weather Forecasts): An intergovernmental organisation headquartered in Reading, England, that operates some of the world's most advanced weather forecasting models. Despite being European, ECMWF's global models and reanalysis products are used worldwide, including extensively in Australian energy research. ERA5 is freely available for research and commercial use via the Copernicus Climate Data Store.

Grid point: One location in a regular spatial grid. ERA5's 0.25° grid means there is a data point every 0.25 degrees of latitude and longitude. Over South Australia, this translates to a grid point roughly every 25 km — close enough to capture the broad weather patterns that drive regional generation, though not fine enough to capture microclimates at individual wind farm sites.

Key Variables for Electricity Prices

Of ERA5's hundreds of variables, four are critical for electricity price forecasting:

Variable	ERA5 short name	Unit	What it measures	Why it matters
Surface solar radiation downwards	ssrd	J/m ² (accumulated)	Total solar energy reaching the ground	Drives solar panel output
2-metre temperature	t2m	Kelvin	Air temperature at 2m above the surface	Drives heating/cooling demand
100m u-component of wind	u100	m/s	East–west wind speed at 100m height	Used to calculate wind speed at turbine hub height
100m v-component of wind	v100	m/s	North–south wind speed at 100m height	Used to calculate wind speed at turbine hub height

u-component and v-component of wind: Wind is a vector — it has both speed and direction. Rather than storing speed and direction directly, ERA5 decomposes wind into two perpendicular components: the **u-component** (east–west, with positive values meaning wind blowing from west to east) and the **v-component** (north–south, with positive values meaning wind blowing from south to north). The actual wind speed is calculated from these components using the Pythagorean theorem: $speed = \sqrt{u^2 + v^2}$. This decomposition is standard in meteorology because it avoids the mathematical difficulties of working with circular (directional) data.

Accumulated variable: Some ERA5 variables, including surface solar radiation (ssrd), are stored as *accumulated* values — the total energy received since the start of each forecast period, not the instantaneous rate. To convert to a rate (watts per square metre), you must compute the difference between consecutive time steps and divide by the time interval (3,600 seconds for hourly data). Forgetting this conversion and using the raw accumulated values directly is a common error that produces nonsensical results.

Why 100m Wind, Not Surface Wind?

Modern wind turbines are enormous structures. A typical utility-scale turbine installed in the 2020s has a hub height (the height of the central axis where the blades connect) of **80–120 metres** above the ground. The rotor diameter can exceed 150 metres, meaning blade tips sweep from 30m to over 180m altitude.

Wind speed increases with height above the ground — a phenomenon called **wind shear**. Near the surface, friction from terrain (trees, buildings, hills) slows the wind. Higher up, the air moves more freely. The rate of increase depends on surface roughness: over flat, open farmland, wind speed at 100m might be 30% higher than at 10m; over forested or urban terrain, it can be 50–100% higher.

Wind shear: The increase in wind speed with altitude, caused by surface friction. Wind shear is typically modelled using a power law: $v(h) = v(h_{ref}) \times (h / h_{ref})^\alpha$, where α is the wind shear exponent (typically 0.10–0.25 depending on terrain roughness). This means a surface wind measurement at 10m must be "extrapolated" upward to estimate conditions at turbine hub height — a process that introduces significant uncertainty.

ERA5's 100m wind data (u_{100} , v_{100}) is invaluable because it provides wind information at approximately the right height for turbine operation, without needing the uncertain extrapolation from surface observations. Surface (10m) wind from weather stations would need to be scaled up using wind profile models — adding a layer of approximation and error.

From ERA5 Grid to Regional Aggregates

ERA5 provides data at every grid point, but the NEM prices are set at the *regional* level — one price for all of SA1, one for all of VIC1, and so on. We need to convert the gridded weather data into a single representative value per variable per region per time step.

The standard approach is **spatial averaging**: compute the arithmetic mean of all ERA5 grid points that fall within the region's geographic boundary. For South Australia, this might involve averaging across 50–100 grid points.

This is a simplification. In reality, solar farms and wind farms are not uniformly distributed across a region — they cluster in areas with good resources (sunny plains for solar, ridgelines and coastal areas for wind). A more sophisticated approach would weight each grid point by the installed generation capacity nearby. However, for price forecasting at the regional level, the simple spatial average captures the broad weather patterns that drive aggregate generation and is usually sufficient.

Real-world example: South Australia's major wind farms are concentrated along the coast and in the mid-north ranges, where wind speeds are highest. The interior is calmer. A simple spatial average over the entire state understates the wind conditions at actual wind farm locations. However, the day-to-day and hour-to-hour variation — the signal that matters for forecasting — is strongly correlated across the region. When a cold front sweeps through SA, it brings strong winds to both the coast and the ranges. The spatial average captures this frontal signal even if it slightly underestimates the absolute wind speed at farm locations.

Solar Radiation and the Clear-Sky Index

How Sunlight Reaches the Ground

The sun delivers approximately **1,361 watts per square metre** (W/m^2) at the top of Earth's atmosphere — a value called the **solar constant**. But by the time this radiation reaches the Earth's surface, it has been significantly reduced by its journey through the atmosphere. Several factors determine how much arrives at any given location and time:

Solar constant: The total solar power per unit area received at the top of Earth's atmosphere on a surface perpendicular to the sun's rays, approximately $1,361 \text{ W/m}^2$. Despite its name, it varies slightly ($\sim 0.1\%$) over the 11-year solar cycle, but this variation is negligible for energy applications.

1. Sun angle and atmospheric path length. When the sun is directly overhead (noon in the tropics), its rays pass through the minimum amount of atmosphere. At low sun angles (morning, evening, winter at high latitudes), the same rays must traverse a much longer path through the atmosphere, increasing absorption and scattering. This path length is quantified by the **air mass (AM)**.

Air mass (AM): The ratio of the atmospheric path length to the shortest possible path (sun directly overhead). $\text{AM} = 1$ means the sun is directly overhead; $\text{AM} = 2$ means the path is twice as long (sun at $\sim 60^\circ$ from zenith). At sunrise and sunset, AM can exceed 10. More air mass means more absorption and scattering, reducing the radiation that reaches the ground. The standard test condition for solar panels is AM 1.5, representing typical mid-latitude conditions.

2. Clouds. Clouds are by far the largest source of short-term variability in solar radiation. A thick cumulonimbus cloud can block more than 80% of incoming radiation. Thin cirrus clouds may reduce it by only 10–20%. The type, thickness, altitude, and spatial extent of clouds all matter.

3. Aerosols. Tiny particles suspended in the atmosphere — dust, smoke, pollution, sea salt — scatter and absorb radiation. In Australia, bushfire smoke can significantly reduce solar output across entire regions for days. The "Black Summer" bushfires of 2019–2020 reduced solar irradiance across eastern Australia by 20–40% during heavy smoke events.

4. Water vapour. Atmospheric moisture absorbs radiation at specific infrared wavelengths. Humid tropical air absorbs more than dry desert air, even under clear skies.

Components of Solar Radiation

The total solar radiation reaching a horizontal surface — called **Global Horizontal Irradiance (GHI)** — consists of two components:

Global Horizontal Irradiance (GHI): The total solar power per unit area received on a horizontal surface at the Earth's surface. It is the sum of the direct beam component and the diffuse (scattered) component. GHI is the primary driver of solar panel electricity output and is measured in watts per square metre (W/m^2). Typical peak values in Australia range from 800–1,100 W/m^2 on clear days.

Direct Normal Irradiance (DNI): The component of solar radiation arriving in a straight line from the sun. It creates shadows and can be focused by mirrors or lenses. Under clear skies, DNI is the dominant component of GHI. Under overcast skies, DNI drops to near zero because the direct beam is blocked by clouds.

Diffuse Horizontal Irradiance (DHI): The component of solar radiation that has been scattered by the atmosphere (by molecules, aerosols, and clouds) and arrives at the surface from all directions across the sky, not directly from the sun. On a heavily overcast day, nearly all radiation is diffuse. Even under clear skies, 10–20% of GHI is typically diffuse due to molecular (Rayleigh) scattering.

The relationship between these components is:

$$\text{GHI} = \text{DNI} \times \cos(\theta_z) + \text{DHI}$$

where θ_z is the **solar zenith angle** — the angle between the sun's position and directly overhead (the zenith). When the sun is directly overhead, $\theta_z = 0^\circ$ and $\cos(\theta_z) = 1$. At sunrise/sunset, $\theta_z \approx 90^\circ$ and $\cos(\theta_z) \approx 0$, so GHI consists almost entirely of diffuse radiation.

Solar zenith angle (θ_z): The angle between the sun's position in the sky and the point directly overhead (the zenith). $\theta_z = 0^\circ$ means the sun is directly overhead; $\theta_z = 90^\circ$ means the sun is on the horizon. The zenith angle depends on latitude, time of day, and day of year, and can be calculated precisely using astronomical formulae. It is the complement of the more familiar "solar elevation angle" (elevation = $90^\circ - \text{zenith}$).

Clear-Sky Models and pvlib

A **clear-sky model** predicts the solar irradiance that *would* arrive at the surface if there were no clouds. It depends on factors that are known precisely — geographic location, time (which determines the sun's position), and atmospheric composition — but not on cloud cover, which is the main source of uncertainty.

Clear-sky model: A mathematical model that calculates the theoretical maximum solar irradiance at a given location and time under cloudless conditions. It accounts for the sun's position (astronomy), atmospheric path length (air mass), and atmospheric attenuation (aerosols, water vapour, ozone). Several implementations exist; the **Ineichen/Perez model** is widely used and accounts for altitude and atmospheric turbidity.

pvlib: An open-source Python library for simulating photovoltaic (solar panel) energy systems. It includes functions for calculating sun position, clear-sky irradiance, panel temperature effects, and system output. Developed by Sandia National Laboratories, it is the standard tool for solar energy research and is used in this project to compute clear-sky reference values.

Linke turbidity coefficient: A dimensionless number that quantifies how much the atmosphere attenuates solar radiation compared to a perfectly clean, dry atmosphere. A value of 1 means a pristine atmosphere (rare in practice); typical values range from 2 (clean, dry air) to 5+ (polluted or very humid). The Ineichen clear-sky model uses this coefficient to adjust the clear-sky irradiance for local atmospheric conditions.

The **pvlib** library implements several clear-sky models. The calculation proceeds in two steps:

Solar position calculation. Given latitude, longitude, and time, compute the sun's zenith angle, azimuth, and extraterrestrial radiation. This is pure astronomy — deterministic and precise.

Atmospheric attenuation. Given the sun's position, altitude, and atmospheric turbidity, calculate how much radiation survives its passage through the atmosphere. The Ineichen model applies empirical correction factors derived from thousands of clear-sky measurements worldwide.

The result is a smooth curve of GHI through the day — rising from zero at sunrise, peaking at solar noon, and returning to zero at sunset — that represents the maximum possible irradiance under the given atmospheric conditions.

The Clear-Sky Index: Isolating the Weather Signal

The **clear-sky index (CSI)** is the ratio of actual measured (or reanalysis) irradiance to the clear-sky model's prediction:

$$\text{CSI} = \text{GHI}_{\text{actual}} / \text{GHI}_{\text{clearsky}}$$

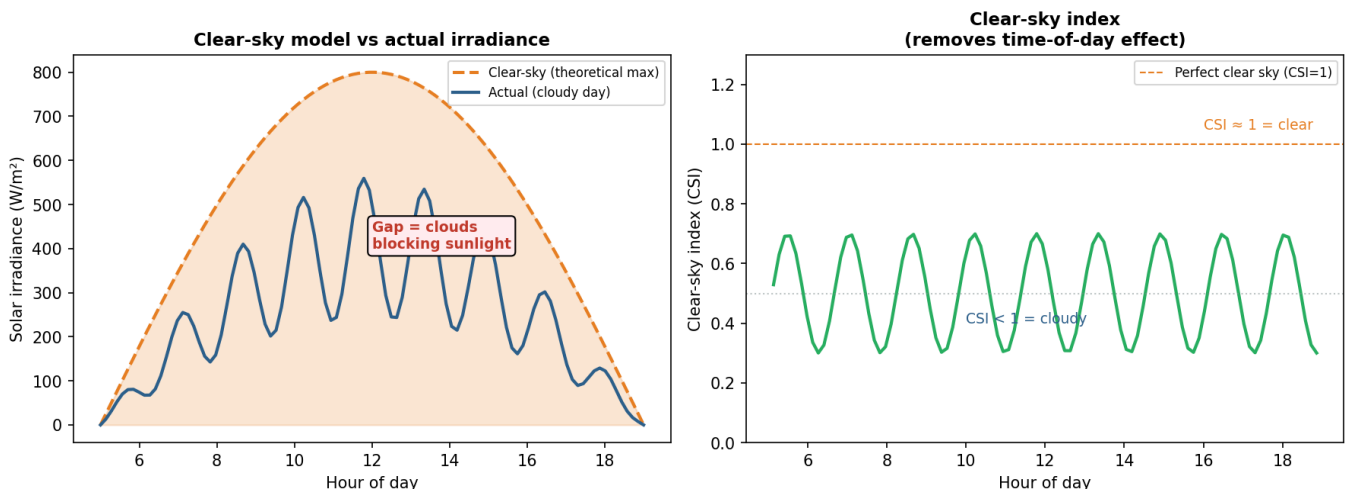


Figure 4.1 — The clear-sky index normalises actual irradiance against the astronomical expectation. CSI near 1.0 indicates clear skies; values below 1.0 indicate cloud cover. The CSI removes the predictable diurnal and seasonal cycles, isolating the weather-driven variability that matters for forecasting.

Clear-sky index (CSI): The ratio of observed Global Horizontal Irradiance to the clear-sky (theoretical cloud-free) irradiance at the same location and time. CSI = 1.0 means the sky is clear and solar output matches the theoretical maximum. CSI = 0.5 means clouds are blocking half the

expected radiation. CSI is only defined when $\text{GHI_clearsky} > 0$ (daytime); during nighttime, it is undefined and set to NaN (not a number).

The CSI is a far more useful forecasting feature than raw GHI because it **separates the weather signal from the astronomical signal**:

Situation	Raw GHI (W/m^2)	CSI	Interpretation
Clear noon, summer	1,050	1.0	Clear sky
Cloudy noon, summer	300	0.29	Very cloudy
Clear noon, winter	650	1.0	Clear sky
Clear 4pm, summer	400	1.0	Clear sky, low sun angle
Cloudy 4pm, summer	150	0.38	Moderately cloudy

A raw GHI of 400 W/m^2 could mean "clear sky at 4pm" or "moderately cloudy at noon." Without context, the number is ambiguous. A CSI of 0.38 unambiguously means "clouds are blocking 62% of the expected radiation, regardless of time of day or season." This context-independence makes CSI a much better input for machine learning models.

Why normalise? The raw GHI signal is dominated by the predictable astronomical cycle (sunrise, solar noon, sunset, seasonal tilt). Any forecasting model could learn this cycle from calendar features alone. The *interesting* part — the part that differs between a clear day and a cloudy day, and thus affects the electricity price — is captured by the CSI. By normalising away the predictable component, we give the model a feature that contains only the information it cannot get elsewhere.

CSI Edge Cases and Quality Control

The clear-sky index has several special cases that require careful handling:

- **CSI ≈ 1.0 :** Clear sky. Actual irradiance matches the theoretical expectation.
- **CSI < 1.0 :** Clouds are present, reducing irradiance below the clear-sky value. Values of 0.2–0.4 indicate heavy overcast; 0.6–0.8 indicates thin or broken cloud.
- **CSI > 1.0 :** This can genuinely occur due to **cloud enhancement** — sunlight reflecting off the edges of clouds can briefly boost irradiance above the clear-sky level, sometimes by up to 40% (CSI ≈ 1.4). This happens when the sun is not blocked but nearby clouds act as additional reflectors.
- **CSI > 1.5 :** Almost certainly a measurement or data error. These values should be clipped to 1.5 to prevent them from corrupting downstream analysis.
- **CSI undefined (nighttime):** When $\text{GHI_clearsky} = 0$ (or very close to zero, as at the low-sun-angle edges of the day), the CSI division produces infinity or numerically unstable values. These should be replaced with NaN.

Cloud enhancement: A phenomenon where solar irradiance at the surface briefly exceeds the clear-sky value due to reflections from the edges or undersides of nearby clouds. The direct beam from the sun is unobstructed, and additional diffuse radiation reflected from cloud surfaces augments the total. Cloud enhancement events typically last seconds to minutes and can produce GHI values up to $\sim 1,400 \text{ W/m}^2$, exceeding the solar constant at the top of the atmosphere.

Persistence of Cloudiness

Cloud cover — and therefore the CSI — is not random from one hour to the next. It has characteristic **persistence timescales** driven by the weather systems that produce it:

- **Seconds to minutes:** Individual cumulus clouds passing over a solar farm create rapid fluctuations. These "ramp events" matter for grid stability but are too fast for day-ahead forecasting.
- **Hours:** Synoptic weather features — fronts, troughs, cloud bands — create multi-hour periods of consistent cloudiness or clearness.
- **Days:** Large-scale circulation patterns — blocking high-pressure systems, cut-off low-pressure systems — can maintain clear or cloudy conditions for three to seven days.
- **Seasonal:** Some regions have systematically more cloud in certain seasons (e.g., winter frontal systems in southern Australia).

Synoptic scale: Weather features that span hundreds to thousands of kilometres and persist for days — fronts, low-pressure systems, high-pressure ridges. These are the scales captured by ERA5's $\sim 25 \text{ km}$ grid and are the primary drivers of day-to-day variation in cloudiness, wind, and temperature.

For day-ahead price forecasting, the hourly-to-daily persistence is the relevant timescale. Today's CSI at a given hour is a reasonable (though imperfect) predictor of tomorrow's CSI at the same hour, because the weather systems that determine cloudiness typically persist for several days. This persistence is what makes weather a useful forecasting feature even without access to a numerical weather prediction forecast.

Wind Speed and Wind Power

The Physics of Wind Energy

Wind turbines extract kinetic energy from moving air. The fundamental physics dictates that the power available in the wind is proportional to the **cube of the wind speed**:

$P_{\text{wind}} = \frac{1}{2} \times \rho \times A \times v^3$

where: - P_{wind} is the power available in the wind (watts) - ρ is the air density (approximately 1.225 kg/m³ at sea level) - A is the swept area of the rotor (m²) — for a rotor diameter D , $A = \pi(D/2)^2$ - v is the wind speed (m/s)

Betz limit: A theoretical maximum on the fraction of wind energy that a turbine can extract, derived by German physicist Albert Betz in 1919. The limit is $16/27 \approx 59.3\%$. No wind turbine, regardless of design, can convert more than 59.3% of the kinetic energy in the wind into mechanical energy. Modern turbines achieve 35–45% efficiency (called the **power coefficient**, C_p), which is 60–75% of the Betz limit.

The cubic relationship has profound implications for forecasting. Consider a wind speed error of just 1 m/s:

Actual wind speed	Forecast wind speed	Power ratio (forecast/actual)	Error
8 m/s	7 m/s	$7^3/8^3 = 0.67$	−33%
8 m/s	9 m/s	$9^3/8^3 = 1.42$	+42%
12 m/s	11 m/s	$11^3/12^3 = 0.77$	−23%

A 1 m/s error in wind speed translates to a 23–42% error in power output. This amplification is why wind power forecasting is inherently harder than solar power forecasting — small atmospheric uncertainties create large generation uncertainties.

The Wind Power Curve

The theoretical cubic relationship is modified by the engineering design of actual turbines, producing the characteristic **wind power curve**:

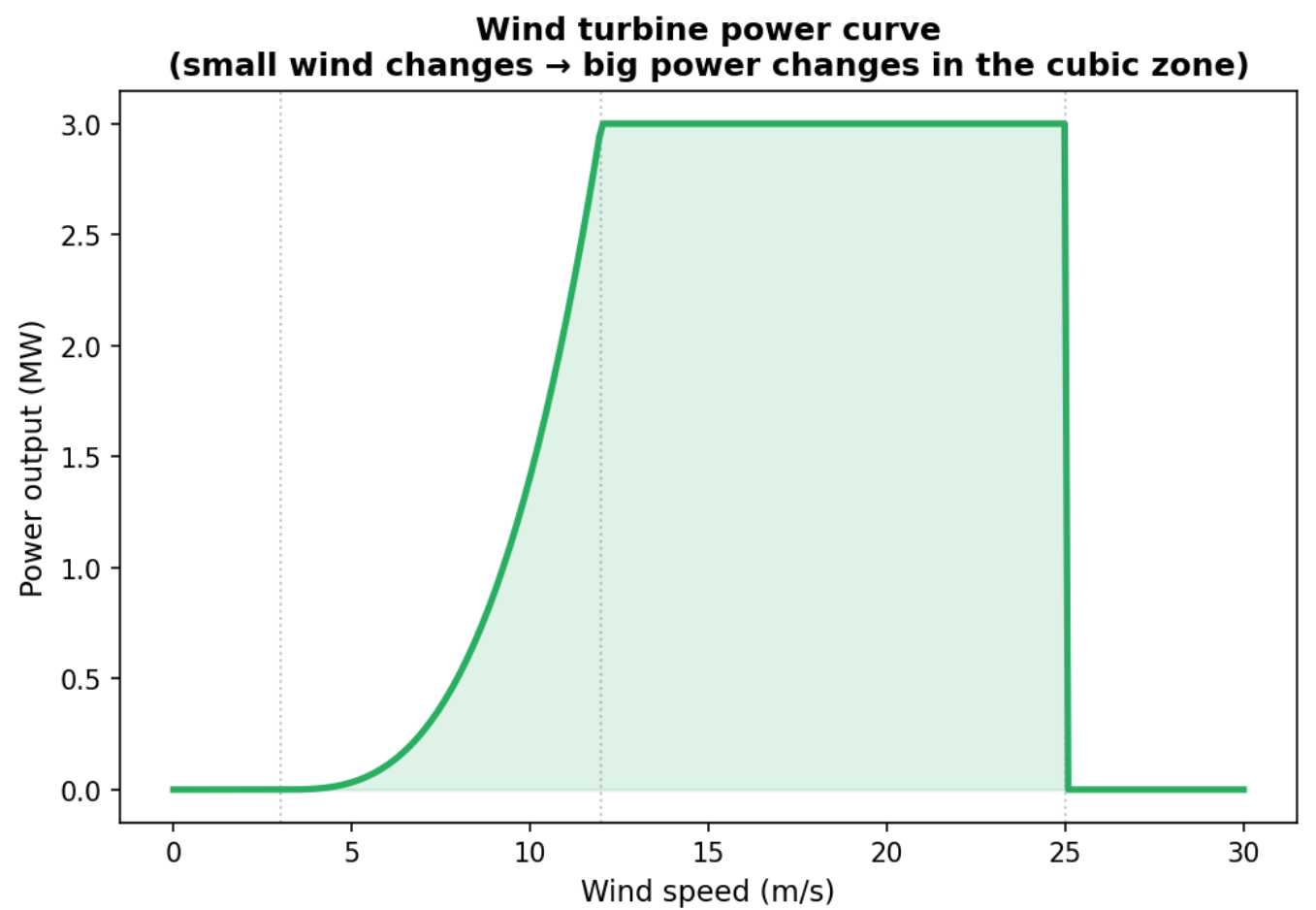


Figure 4.2 — The wind power curve showing the relationship between wind speed and electrical output. The curve has four distinct regions: below cut-in (no output), the cubic ramp, rated power (flat), and cut-out (shutdown). Small changes in wind speed near the steep part of the curve produce large changes in output.

Wind power curve: A graph showing the relationship between wind speed and the electrical power output of a wind turbine. It is the fundamental characteristic of a turbine model and encodes the engineering limits (cut-in speed, rated speed, cut-out speed) as well as the aerodynamic performance in between. Real power curves differ slightly from the idealised version due to turbulence, air density variations, and control system behaviour.

The power curve has four distinct regions:

Region 1 — Below cut-in speed (0 to ~3 m/s): No power output. The wind is too weak to overcome the friction and inertia of the rotor and drivetrain. The turbine sits idle.

Cut-in speed: The minimum wind speed at which a turbine begins generating electricity, typically 2.5–4 m/s (~9–14 km/h). Below this speed, the aerodynamic torque on the blades is insufficient to overcome mechanical losses and start the generator.

Region 2 — Cut-in to rated speed (~3 to ~12 m/s): Power output increases approximately as the cube of wind speed. The turbine operates with its blades pitched to extract maximum energy from the wind. This is the steepest part of the power curve and the region where wind speed forecast errors create the largest generation forecast errors.

Region 3 — Rated speed to cut-out speed (~12 to ~25 m/s): Power output is constant at the turbine's **rated capacity** (the maximum sustained output it is designed for, typically 2–8 MW for modern utility-scale turbines). The blades are **pitched** (rotated along their long axis) to shed excess wind energy and prevent mechanical overload. Increasing wind speed beyond rated produces no additional power — the control system deliberately wastes the excess.

Rated capacity (nameplate capacity): The maximum sustained power output a turbine is designed to produce, reached at its rated wind speed. A "3 MW turbine" can produce at most 3 MW of electricity, regardless of how strong the wind blows. Rated capacity is used to calculate **capacity factor** — the ratio of actual output to theoretical maximum output.

Blade pitch control: A mechanism that rotates each turbine blade along its longitudinal axis to change the angle at which it meets the incoming wind. At low wind speeds, the blade is pitched to maximise energy capture. At high wind speeds (above rated), the blade is pitched to reduce the aerodynamic force, shedding excess energy to keep the output at rated capacity and prevent structural damage.

Region 4 — Above cut-out speed (>~25 m/s): The turbine shuts down entirely for safety. The blades are feathered (pitched to 90° so they present minimum area to the wind), and the rotor is braked to a stop. Output drops from full rated capacity to zero. This is rare but creates dramatic and sudden drops in wind generation across a wind farm or region.

Cut-out speed: The maximum wind speed at which a turbine can operate safely, typically 25 m/s (~90 km/h). Above this speed, the turbine shuts down to prevent structural damage to the blades, tower, and nacelle. Modern turbines increasingly use "storm ride-through" modes that reduce output gradually rather than cutting off abruptly, but the basic principle remains: extremely high winds produce *less* generation, not more.

Real-world example — cut-out events: During severe storm events in South Australia, wind speeds can exceed 25 m/s across large areas. When this happens, wind farms shut down en masse, removing hundreds or thousands of megawatts of generation from the system within minutes. This is exactly when the grid is under maximum stress from storm-related outages and high demand. The September 2016 SA blackout was partly triggered by multiple wind farm shutdowns due to a severe storm — a tragic illustration of the cut-out problem.

Calculating Wind Speed from ERA5

ERA5 provides wind as two perpendicular components at 100m height. The wind speed — which is what determines turbine output — is calculated using the Pythagorean theorem:

$$\text{wind_speed} = \sqrt{(u_{100}^2 + v_{100}^2)}$$

For example, if the east–west component $u_{100} = 6$ m/s and the north–south component $v_{100} = -8$ m/s, then:

$$\text{wind_speed} = \sqrt{(6^2 + (-8)^2)} = \sqrt{(36 + 64)} = \sqrt{100} = 10 \text{ m/s}$$

The negative sign on v_{100} means the wind is blowing from north to south, but the speed calculation only cares about the magnitude, not the direction. This is appropriate because a turbine's power output depends on wind speed, not wind direction (turbines rotate on their yaw axis to face the wind from any direction).

Yaw control: A mechanism that rotates the entire nacelle (the housing containing the generator and gearbox) atop the tower so that the rotor faces directly into the wind. Modern turbines continuously adjust their yaw angle based on wind direction measurements. Because of yaw control, the power output depends on wind *speed* but not wind *direction*.

Why Wind Is Harder to Forecast Than Solar

Wind generation is fundamentally harder to predict than solar generation:

Factor	Solar	Wind
Deterministic component	Strong — sunrise, noon, sunset are precisely predictable	None — wind can be strong or weak at any time of day
Dominant timescale	Diurnal (daily cycle, always present)	Synoptic (weather systems, 2–7 days, irregular)
Spatial coherence	High — cloud systems span 100+ km	Lower — wind varies significantly over 10–50 km due to terrain
Response to forecast error	Linear — 10% GHI error $\approx$ 10% power error	Cubic — 10% speed error $\approx$ 30% power error

Nighttime generation	Zero (no forecast needed)	Non-zero (must forecast 24 hours, not just daytime)
----------------------	---------------------------	---

The cubic amplification problem: Because wind power depends on the cube of wind speed, small errors in wind speed forecasts are amplified into large errors in power forecasts. This is the single biggest challenge in wind power forecasting. A weather model that predicts wind speed within ± 1 m/s — considered a good forecast — can still produce wind power errors of 30% or more. This error propagation is why purely physical wind forecasting is insufficient for electricity price prediction and why statistical methods that learn the speed-to-power mapping from historical data are essential.

Capacity Factor and Its Implications

Capacity factor: The ratio of a generator's actual output over a period to its theoretical maximum output (rated capacity $\times$ time). A wind farm with a capacity factor of 35% means it produces, on average, 35% of what it would produce if it ran at full rated output 24 hours a day, 365 days a year. Capacity factor reflects both the intermittency of the resource (wind does not always blow) and practical limitations (maintenance downtime, curtailment).

South Australian wind farms typically achieve capacity factors of **30–40%**. But this average hides a strongly **bimodal distribution** — output is frequently either near zero (low wind) or near full rated capacity (strong wind), with relatively little time spent at intermediate levels. This bimodality occurs because the steep part of the power curve (region 2) covers a relatively narrow band of wind speeds. Much of the time, the wind is either below the effective range (low output) or above rated speed (full output).

This bimodality matters for price forecasting because it means wind generation acts more like a binary switch (on/off) than a continuous dial. A weather system that brings strong winds "switches on" the wind fleet; a calm period "switches it off." This creates step changes in the supply stack that produce corresponding step changes in price.

Temperature and Electricity Demand

The Comfort Zone and the U-Shaped Response

The relationship between temperature and electricity demand follows a characteristic **U-shape** (or V-shape):

- **Below $\sim 18^\circ\text{C}$:** Demand increases as temperature falls, driven by electric heating. Each degree below the comfort zone adds roughly 1–3% to demand.
- **$18\text{--}22^\circ\text{C}$ (the comfort zone):** Demand is at its minimum. Comfortable temperatures require neither heating nor cooling.
- **Above $\sim 22^\circ\text{C}$:** Demand increases as temperature rises, driven by air conditioning. The response is steeper than heating because air conditioning is energy-intensive. Each degree above the comfort zone can add 2–5% to demand.

Temperature-demand relationship: The empirical relationship between ambient temperature and electricity demand. It is approximately U-shaped: demand is lowest at comfortable temperatures ($\sim 18\text{--}22^\circ\text{C}$) and increases for both warmer and colder temperatures. The heating branch (cold) and cooling branch (hot) have different slopes, reflecting the different energy intensities of heating and cooling systems.

Heating degree hours (HDH) and cooling degree hours (CDH): Metrics that quantify accumulated thermal discomfort. $\text{HDH} = \max(0, T_{\text{base}} - T_{\text{actual}})$ summed over each hour, where T_{base} is typically 18°C . $\text{CDH} = \max(0, T_{\text{actual}} - T_{\text{base}})$ summed similarly. These capture the *duration* and *intensity* of heating or cooling stress. A single hour at 40°C (CDH contribution: 22) creates more cooling demand than two hours at 29°C (CDH contribution: $2 \times 11 = 22$) — but the integrated metric captures both cases equally.

The temperature–demand response is **nonlinear** and **asymmetric**:

- The cooling side is steeper because air conditioners consume more electricity per degree of cooling than heaters consume per degree of heating (in the NEM, where gas heating is common, electric heating load is actually modest).
- The response is not instantaneous — buildings have thermal mass, so demand responds with a lag of 1–4 hours to temperature changes.
- The response depends on humidity. A 35°C day with 70% humidity creates more air conditioning load than a 35°C day with 20% humidity, because air conditioners must remove moisture as well as heat.

Real-world example — temperature and demand in SA1: On a mild autumn day (22°C , low humidity), SA1 might have demand of 1,200 MW. On a 42°C summer day, demand can exceed 3,000 MW — a 150% increase driven almost entirely by air conditioning. This demand surge, combined with reduced renewable output (wind often drops during extreme heat events), creates the conditions for price spikes exceeding $\$10,000/\text{MWh}$.

Temperature Deviation as a Feature

Rather than using raw temperature as a forecasting feature, it is more useful to use **temperature deviation** — the difference between actual temperature and the comfort baseline:

$$T_{\text{dev}} = T_{\text{actual}} - T_{\text{base}}$$

where T_{base} is typically 18°C . Positive values indicate cooling demand; negative values indicate heating demand. This transformation captures the nonlinear demand response more naturally than raw temperature:

- $T_{dev} = +20$ (i.e., $T = 38^{\circ}\text{C}$): Extreme cooling demand
- $T_{dev} = 0$ (i.e., $T = 18^{\circ}\text{C}$): Minimal demand
- $T_{dev} = -10$ (i.e., $T = 8^{\circ}\text{C}$): Moderate heating demand

Some models further split this into separate heating and cooling components to allow different slopes:

$$\text{HDD} = \max(0, T_{base} - T_{actual}) \quad \text{CDD} = \max(0, T_{actual} - T_{base})$$

Heating Degree Days (HDD) and Cooling Degree Days (CDD): Cumulative measures of how cold or hot a period was relative to a base temperature (typically 18°C). HDD sums the degrees below base; CDD sums the degrees above. Originally defined on a daily basis, the concept extends to hourly resolution as heating degree hours and cooling degree hours. These are standard features in energy demand modelling because they directly proxy the thermal load on HVAC systems.

Weather as a Price Driver: The Causal Chain

The relationship between weather and price is not direct — it flows through a chain of physical and market mechanisms:

The causal chain: Weather → Renewable generation + Demand → Net load → Position on supply stack → Marginal generator → Price. Weather does not "cause" prices directly. It causes changes in supply (renewable generation) and demand (heating/cooling load), which together determine net load, which determines which generator sets the price. Understanding this chain is essential for building good features — we want features that are as close to the price-setting mechanism as possible.

Each link in the chain introduces uncertainty:

1. **Weather → Solar generation:** Moderated by panel efficiency, inverter capacity, curtailment
2. **Weather → Wind generation:** Amplified by the cubic power curve, moderated by turbine availability
3. **Weather → Demand:** Moderated by building thermal mass, behavioral responses, time-of-day effects
4. **Supply + Demand → Net load:** Direct subtraction
5. **Net load → Price:** The highly nonlinear supply stack mapping (the "hockey stick" from Chapter 3)

The nonlinearity at step 5 is crucial. A 100 MW change in net load might change the price by \$2 in the middle of the supply stack, or by \$5,000 at the steep end. This means the *same* weather forecast error can produce vastly different price forecast errors depending on where the system is operating on the supply stack.

Real-world example — the same weather, different price impact: Suppose wind generation is forecast to be 500 MW but actually comes in at 400 MW — a 100 MW miss. On a mild day with 1,500 MW demand, this pushes net load from 1,000 MW to 1,100 MW, moving the system along the flat part of the supply stack. Price might increase from \$45 to \$55 — a \$10 error. On a 42°C day with 3,000 MW demand, the same 100 MW miss pushes net load from 2,500 MW to 2,600 MW, hitting the steep part of the supply stack. Price might increase from \$300 to \$2,000 — a \$1,700 error. Same weather forecast error, 170× different price impact.

Feature Engineering: From Raw Weather to Model Inputs

Transforming Weather Variables

Raw ERA5 variables are not directly suitable as forecasting features. They need to be transformed into quantities that have clear relationships with electricity market outcomes:

Raw variable	Transformation	Feature	Rationale
ssrd (accumulated J/m^2)	Differentiate, convert to W/m^2 , divide by clear-sky	Clear-sky index	Removes astronomical cycle, isolates weather signal
u100, v100 (m/s)	Pythagorean combination	Wind speed (m/s)	Directly related to turbine power output
t2m (Kelvin)	Convert to $^{\circ}\text{C}$, subtract base	Temperature deviation ($^{\circ}\text{C}$)	Captures demand response to thermal stress
t2m (Kelvin)	CDD/HDD calculation	Cooling/heating degree hours	Time-integrated demand proxy

Lagged vs. Contemporaneous Weather Features

For day-ahead forecasting — where the forecast is issued before noon for the following trading day — we face a fundamental problem: we do not know tomorrow's weather. There are three approaches:

Approach 1: Use weather forecasts as features. Feed the output of a numerical weather prediction (NWP) model into the price forecasting model. This is operationally correct — it uses the same weather information that would be available to a real-time forecaster. The drawback is that it requires access to historical NWP forecasts for training, which are large, expensive, and not always available.

Numerical Weather Prediction (NWP): The practice of forecasting weather by running physics-based computer models that simulate the atmosphere's evolution. NWP models solve the equations of fluid dynamics, thermodynamics, and radiative transfer on a three-dimensional grid, starting from an observed initial state. Modern NWP models can produce useful forecasts up to ~10 days ahead. For energy applications, the most relevant products are forecasts of temperature, wind, and solar radiation at 1–72 hour horizons.

Approach 2: Use lagged actual weather. Today's weather is a reasonable predictor of tomorrow's weather, because weather systems change slowly (synoptic-scale persistence). Use today's CSI as a predictor of tomorrow's CSI; use today's wind speed as a predictor of tomorrow's wind speed. This is simple and does not require NWP data, but it misses weather changes (e.g., a front arriving tomorrow that was not present today).

Approach 3: Use reanalysis as "perfect weather" (the approach in this project). Use ERA5 reanalysis for both historical weather and "forecast" weather. This provides an *upper bound* on how well a weather-informed model can perform — it tells us how much value weather information adds when the weather is known perfectly. In production, using actual NWP forecasts would reduce performance, and the gap between "perfect weather" and "NWP forecast" quantifies the value of improving weather forecasts.

Upper-bound thinking: Using ERA5 reanalysis as if it were a perfect weather forecast establishes a performance ceiling. If a model using perfect weather achieves a capture ratio of 0.75, then no weather-informed model can do better than 0.75 regardless of how good the NWP forecast becomes. If a model *without* weather achieves 0.65, we know the maximum value of weather information is a 10 percentage-point improvement. This upper-bound reasoning is invaluable for deciding whether to invest in better weather data.

AEMO's Own Forecasts: NEMSEER Pre-Dispatch

AEMO publishes its own forecasts of wind and solar generation through the **NEMSEER** (NEM System Event and Emergency Reserve) pre-dispatch system. These are updated every 30 minutes with a horizon of up to 40 hours.

Pre-dispatch: AEMO's process of projecting electricity dispatch outcomes (generation schedules, prices, interconnector flows) for the upcoming 24–40 hours. Pre-dispatch runs every 30 minutes and incorporates generator bids, demand forecasts, network constraints, and renewable generation forecasts. The pre-dispatch price is AEMO's best estimate of future prices — and serves as a natural benchmark for price forecasting models.

Using AEMO's generation forecasts as features (rather than raw weather) has advantages:

- They already account for **installed capacity** — a weather model does not know how many panels are in SA1, but AEMO does
- They already account for **curtailment** — AEMO knows if a wind farm is constrained by network limits
- They already account for **turbine power curves** — AEMO's forecast converts wind speed to megawatts
- They represent the **market's expectation** — other participants react to these forecasts, so they partly determine the price

The disadvantage is that if AEMO's forecast has a systematic bias (consistently over- or under-forecasting wind, for example), our price model inherits that bias.

Persistence Forecasts as Weather Baselines

What Is a Persistence Forecast?

The simplest weather forecast is **persistence**: assume tomorrow's weather will be the same as today's weather at the same time.

Persistence forecast: A forecasting method that predicts the future value of a variable will equal its current (or most recent past) value. For weather variables, persistence at the daily timescale means: tomorrow's temperature at 2pm = today's temperature at 2pm. Persistence is the simplest forecast that captures the autocorrelation (serial correlation) in weather data. It is the standard baseline — any model that cannot beat persistence has added complexity without adding skill.

- **Solar persistence:** Tomorrow's CSI at hour H = today's CSI at hour H
- **Wind persistence:** Tomorrow's wind speed at hour H = today's wind speed at hour H
- **Temperature persistence:** Tomorrow's temperature at hour H = today's temperature at hour H

Persistence works well when weather changes slowly — a multi-day high-pressure system bringing clear skies, or a sustained period of strong westerly winds. It fails when the weather changes rapidly — a cold front arriving overnight, a sea breeze onset in the afternoon, or a thunderstorm developing.

For solar forecasting, it is critical to persist the *clear-sky index*, not raw GHI. Persisting raw GHI would systematically fail because of the seasonal change in day length and solar angle — tomorrow's noon GHI in winter is lower than today's noon GHI if the season is progressing toward winter, even if the weather is identical.

The persistence test: Persistence is the "you must be this tall to ride" test for weather and renewable forecasting models. If a sophisticated machine learning model cannot beat "tomorrow equals today," the model is either badly designed or the weather is genuinely unpredictable at the relevant horizon. Always evaluate models against persistence before drawing conclusions about their skill.

Glossary

Term	Definition
VRE	Variable Renewable Energy — generation whose output depends on weather (wind, solar)
Dispatchable	Generation that can be controlled by the operator (coal, gas, hydro)
ERA5	ECMWF's fifth-generation global reanalysis dataset
Reanalysis	Physics-model reconstruction of historical atmosphere using all available observations
Data assimilation	Mathematical fusion of model predictions and real observations
ECMWF	European Centre for Medium-Range Weather Forecasts
GHI	Global Horizontal Irradiance — total solar radiation on a horizontal surface
DNI	Direct Normal Irradiance — beam component of solar radiation
DHI	Diffuse Horizontal Irradiance — scattered component of solar radiation
Solar zenith angle	Angle between the sun and directly overhead
Air mass	Ratio of atmospheric path length to the minimum path
Clear-sky model	Model predicting irradiance under cloud-free conditions
CSI	Clear-Sky Index — ratio of actual to clear-sky irradiance
pvlb	Python library for solar energy system simulation
Linke turbidity	Coefficient quantifying atmospheric clarity
Cloud enhancement	Irradiance exceeding clear-sky due to cloud reflections
Wind shear	Increase in wind speed with altitude
Cut-in speed	Minimum wind speed for turbine generation (~3 m/s)
Rated capacity	Maximum sustained turbine output
Cut-out speed	Maximum safe operating wind speed (~25 m/s)
Blade pitch control	Rotating blades to regulate power capture
Capacity factor	Ratio of actual output to theoretical maximum
Betz limit	Theoretical maximum wind energy extraction efficiency (~59.3%)
HDD / CDD	Heating / Cooling Degree Days — thermal stress metrics
NWP	Numerical Weather Prediction — physics-based weather forecasting
Pre-dispatch	AEMO's projection of future dispatch outcomes
Persistence forecast	Predicting tomorrow = today
Synoptic scale	Weather features spanning hundreds to thousands of km

Summary

Weather is the master variable in renewable-dominated electricity grids, driving both supply (wind and solar generation) and demand (heating and cooling load) simultaneously. ERA5 reanalysis provides the gold-standard historical weather dataset — globally complete, hourly, physically consistent — with key variables including surface solar radiation, 100m wind components, and 2m temperature. Solar irradiance is best represented by the clear-sky index, which normalises against the predictable astronomical cycle to isolate the weather-driven variability. Wind power follows a cubic relationship with wind speed in the operating range, amplifying small forecast errors into large generation errors, and the wind power curve introduces additional nonlinearity through cut-in, rated, and cut-out thresholds. Temperature drives electricity demand through a U-shaped response centred on a comfort zone of ~18–22°C. The causal chain from weather to price passes through multiple nonlinear transformations — most critically the supply-stack mapping — meaning identical weather forecast errors can produce vastly different price forecast errors depending on system conditions. Feature engineering transforms raw weather into forecasting-ready quantities (CSI, wind speed, temperature deviation), and persistence forecasts provide the essential baseline that any model must beat.

5. Forecasting Framework and Baselines

The Day-Ahead Forecasting Problem

What Exactly Are We Predicting?

Before building any model, we must define the forecasting problem with absolute precision. Vagueness at this stage — "predict electricity prices" — leads to models that look good on paper but are useless in practice. Here is the exact specification:

- **Target variable:** The 30-minute **trading price** (\$/MWh) for a single NEM region (SA1 in this project), transformed via arcsinh (see Chapter 1).
- **Forecast horizon:** 48 half-hour periods, covering one full trading day (from 04:00 to 04:00 AEST the following day).
- **Gate closure:** 12:00 noon today. The forecast must be issued using **only** information available before noon. No data from 12:01 onwards is allowed.
- **Available information at gate closure:** All historical prices, demand, weather, and AEMO pre-dispatch data up to (but not including) noon.
- **Output format:** Either a **point forecast** (a single predicted price for each half-hour) or a **probabilistic forecast** (predicted quantiles or a full distribution for each half-hour).

Gate closure: The deadline by which a forecast must be issued. In our day-ahead setup, gate closure is noon — the forecast for the next trading day must be finalised by 12:00. After gate closure, no additional information can be incorporated into the forecast. This mirrors the operational reality of a battery operator who must commit to a dispatch plan before the trading day begins.

Forecast horizon: The time span into the future that a forecast covers. A 48-step horizon at 30-minute resolution covers 24 hours. The difficulty of forecasting generally increases with horizon — predicting the price one hour from now is much easier than predicting the price 20 hours from now, because more can change in a longer period.

Point forecast: A single predicted value for each future time step — one number per half-hour. A point forecast says "the price at 6pm will be \$85." It gives the model's best guess but conveys no information about how confident the model is.

Probabilistic forecast: A prediction that describes a range of possible outcomes and their likelihoods — for example, quantiles (10th, 50th, 90th percentile) or a full probability distribution. A probabilistic forecast says "the price at 6pm has a 10% chance of being below \$40, a 50% chance of being below \$85, and a 10% chance of exceeding \$250." This additional information is critical for optimal battery dispatch under uncertainty.

This is a **multi-step** forecasting problem: we need 48 values, not one. Each of the 48 predictions has a different effective horizon — the first prediction is for ~16 hours ahead (noon to 4am), and the last is for ~40 hours ahead (noon to 4am the following day). Error generally grows with horizon, so later predictions are less reliable.

Why Day-Ahead?

The day-ahead horizon is not an academic choice — it matches the **decision cycle** of a grid-scale battery operator:

1. Before noon, the operator receives a price forecast for the next 24-hour trading day.
2. Based on this forecast, they solve an optimisation problem: when should the battery charge? When should it discharge? When should it sit idle?
3. The resulting dispatch schedule is executed against actual spot prices.
4. Revenue = $\sum (\text{price} \times \text{net discharge})$ over all half-hours.

Trading day: The NEM's trading day runs from 04:00 AEST to 04:00 AEST the following day — not midnight to midnight. This convention exists because the overnight period (midnight to 4am) has consistent low-demand, low-price characteristics that belong logically with the preceding day. All 48 half-hours of a trading day are settled together.

Real-world example — the battery operator's morning: A battery operator at the Hornsdale Power Reserve (the "Tesla Big Battery" in South Australia) might receive a price forecast at 11:30am showing: low prices overnight (\$20–40), a deep solar dip to ~\$10 at midday, and an evening spike to \$200 at 6pm. The optimal strategy is clear: charge during the solar dip (buy cheap), discharge during the evening spike (sell expensive). But the operator must commit to this plan before noon — if the forecast is wrong and the spike does not materialise, the battery discharges into a \$50 market instead of a \$200 market, earning a fraction of the expected revenue.

Other horizons serve different purposes:

Horizon	Decision supported	Data requirements
Intra-day (1–4 hours)	Real-time dispatch adjustments	Real-time data feeds, rapid model updates
Day-ahead (24 hours)	Daily dispatch planning (our focus)	Morning data, noon gate closure

Week-ahead	Maintenance scheduling, contract hedging	Less accurate but useful for planning
Month/season-ahead	Capacity investment, long-term contracts	Highly uncertain, scenario-based

The Target Transform in Practice

Recall from Chapter 1 that electricity prices have extreme statistical properties — a range from −\$1,000 to \$17,500 with heavy tails and asymmetry. We model the **arcsinh-transformed** price, not the raw price. The complete forecast workflow is:

1. Transform: $y = \text{arcsinh}(\text{price})$ 2. Train model to predict y 3. Generate forecast: y^* 4. Invert: $\text{predicted_price} = \sinh(y^*)$ 5. Evaluate: $\text{MAE}(\text{actual_price}, \text{predicted_price})$

The evaluation rule: Always train in arcsinh space but evaluate in dollar space. If you measure forecast error in arcsinh space, you systematically underweight errors on extreme prices — exactly the events that drive battery revenue. A model that perfectly predicts quiet \$50 prices but misses every \$5,000 spike would look excellent in arcsinh space but would be worthless for dispatch.

The Feature Matrix

What Goes Into a Price Forecast?

A forecasting model needs **features** (also called **predictors**, **inputs**, or **independent variables**) — quantities that carry information about future prices. For electricity price forecasting, features fall into five categories:

Feature (predictor variable): A measurable quantity used as input to a forecasting model. Features are the "clues" the model uses to predict the target variable (price). Good features have a strong, stable relationship with the target. Bad features add noise without adding information, which can actually make predictions worse.

1. Lagged prices (autoregressive features):

From Chapter 2, we know that electricity prices have strong autocorrelation — today's price at 2pm is strongly related to yesterday's price at 2pm. The most important lags are:

- **Lag 48:** Same half-hour yesterday (24 hours ago at 30-minute resolution). This is the single most powerful feature.
- **Lag 96:** Same half-hour two days ago. Adds information beyond yesterday but with diminishing returns.
- **Lag 336:** Same half-hour one week ago ($7 \times 48 = 336$). Captures the weekly cycle — weekdays differ from weekends.

Autoregressive feature: A feature constructed from past values of the target variable itself. "Autoregressive" literally means "self-regressing" — the variable is used to predict itself. The lag-48 price feature says: "use the price 24 hours ago to help predict the price now." Autoregressive features exploit the temporal persistence (autocorrelation) in the data.

2. Calendar features:

The time-of-day, day-of-week, and month encode the systematic patterns in electricity demand and supply:

- **Half-hour of day (0–47):** Captures the diurnal cycle — the duck curve from Chapter 2.
- **Day of week (0–6):** Captures the weekly pattern — weekday demand is higher than weekend demand.
- **Month (1–12):** Captures seasonal variation — summer prices differ from winter prices due to demand and renewable output patterns.
- **Public holiday indicator (0/1):** Holidays have weekend-like demand patterns even on weekdays.

3. Demand features:

- **Lagged demand:** Recent demand levels provide information about the current state of the system.
- **Demand forecasts:** If AEMO's demand forecast for the target period is available (via pre-dispatch), it directly proxies future net load.

4. Weather features:

From Chapter 4, we use:

- **Clear-sky index:** Normalised solar irradiance — proxy for solar generation.
- **Wind speed:** Computed from ERA5 u_{100}/v_{100} — proxy for wind generation.
- **Temperature deviation:** Difference from the 18°C comfort baseline — proxy for heating/cooling demand.

5. Derived (engineered) features:

- **Rolling volatility:** Recent absolute price returns, measuring how turbulent the market has been. Volatility clusters (Chapter 2), so recent volatility predicts near-future volatility.
- **Net load:** Demand minus renewable generation. This single feature encodes the supply–demand balance that drives the hockey-stick price response (Chapter 3).

Net load: Total electricity demand minus variable renewable generation (wind + solar). Net load is the amount of electricity that must be supplied by dispatchable generators — coal, gas, hydro, and batteries. It is the single most important driver of electricity prices because it determines which generator is marginal (Chapter 1) and where the system sits on the supply stack (Chapter 3).

Feature Engineering Principles

Four rules for feature engineering: 1. **No future information:** Every feature must be computable from data available *before* gate closure. Using tomorrow's actual temperature as a feature is data leakage — the model would appear to have skill it does not actually possess. 2. **Transform for linearity:** Raw features often have nonlinear relationships with price. The clear-sky index (rather than raw GHI) and $\text{arcsinh}(\text{price})$ (rather than raw price) improve the linearity of feature–target relationships, which helps linear models and does not hurt nonlinear ones. 3. **Lag for availability:** If a data source has a publication delay, the lag must account for it. If demand data is published with a 1-hour delay, use demand at lag 2 (or earlier), not lag 1. 4. **Encode domain knowledge:** Net load (demand minus renewables) is more informative than demand and renewables as separate features, because it directly corresponds to the supply-stack mechanism that sets the price. Creating this interaction feature saves the model from having to discover it from data.

Data Leakage: The Silent Model Killer

What Is Data Leakage?

Data leakage occurs when information that would not be available at prediction time is inadvertently included in the training data or features. It causes a model to appear much better during development than it actually performs in production.

Data leakage: Any situation where information from the future (relative to the forecast point) contaminates the model's inputs — either during training or prediction. Leakage makes backtested performance overly optimistic because the model is effectively "seeing the answer" before making its prediction. Leakage is insidious because it produces no error messages — the model trains normally, evaluates normally, and gives results that look excellent. The problem only appears when the model is deployed in production and performs much worse than expected.

Leakage is the most dangerous error in time-series forecasting because it is **invisible**. There is no error message, no warning, no obvious sign. The model trains without complaint, produces impressive backtested results, and only fails when deployed in the real world — where it suddenly lacks the future information it was trained on.

Common Sources of Leakage in Electricity Price Forecasting

- 1. Timestamp misalignment (Chapter 1).** AEMO uses interval-ending timestamps. If you join price data (interval-ending) with weather data (instantaneous) without shifting, you give the model weather measured at the *end* of the price interval — information that was not available when the interval started.
- 2. Using contemporaneous features.** Including tomorrow's actual temperature, wind speed, or demand as a feature when forecasting tomorrow's price. These values are not known at gate closure.
- 3. Training on test data.** If the model's training set overlaps with the test set (either directly or through lagged features that reach into the test period), it has seen the answers before being evaluated.
- 4. Feature scaling on the full dataset.** Computing the mean and standard deviation for feature normalisation using *all* data (including the test set), then normalising both training and test data. The model's scaling parameters now encode information about the test period.
- 5. Rolling statistics that look ahead.** Computing a "rolling mean" centred on each observation (using values both before and after) rather than a trailing rolling mean (using only past values).

Real-world example — the leakage trap: A researcher builds a price forecasting model using ERA5 temperature as a feature. They use the temperature at the same timestamp as the price they are predicting — 3pm temperature to predict 3pm price. The model achieves outstanding results: MAE of \$8, far better than any published benchmark. But the 3pm temperature at the time of the 3pm price observation is the *actual* temperature at 3pm — information that would not have been available at noon gate closure. In reality, the model would need to use a temperature *forecast* or a lagged actual value. When deployed with actual weather forecasts (which have errors), the model's MAE jumps to \$25 — still decent, but far from the backtested \$8. The \$17 gap is entirely due to leakage.

The leakage test: For every feature, ask: "Could I actually compute this value at gate closure in a live production system?" If the answer is no — if it requires data that would not yet exist — the feature is leaking future information. Apply this test to every single feature in your model. One leaked feature is enough to invalidate your entire backtest.

The Rolling-Origin Backtest

Why Not a Simple Train/Test Split?

The most basic evaluation approach is to split data into a training set (first 80%) and a test set (last 20%). Train the model once, predict the test set, compute the error metric. Done.

This is insufficient for electricity price forecasting for two reasons:

Sample dependence. A single test period captures only one set of market conditions. Your model might look great on a mild autumn test period but fail completely on a volatile summer period. The results are hostage to the specific dates chosen.

Non-stationarity. Electricity prices change over time — the generation mix evolves, fuel prices shift, new solar farms are built, policies change. A model that works well in 2023 Q4 might fail in 2024 Q1. A single split gives one snapshot, not a picture of how well the model adapts.

The **rolling-origin backtest** solves both problems by evaluating the model across *many* train/test splits over time.

The Rolling-Origin Procedure

Rolling-origin backtest (walk-forward validation, time-series cross-validation): An evaluation procedure that simulates the process of repeatedly training and forecasting over time. At each step, the model is trained on all data up to a certain point (the "origin"), makes a forecast for the next H periods, and the origin is advanced. This produces a sequence of out-of-sample forecasts that spans a long test period, giving a robust estimate of model performance under varying conditions.

The procedure works as follows:

1. **Set the initial origin** at time t_1 . The training set is $[t_0, t_1]$.
2. **Apply the embargo.** Skip the next H periods after the origin (explained below).
3. **Forecast.** Predict the H periods immediately after the embargo: $[t_1 + \text{embargo}, t_1 + \text{embargo} + H]$.
4. **Record errors.** Compare forecasts to actual prices and store the errors.
5. **Advance the origin** by one step (or by `refit_every` steps for computational efficiency).
6. **Expand (or slide) the training window** to include the new data.
7. **Refit the model** on the updated training set.
8. **Repeat** until the origin reaches the end of the available data.

Rolling-origin backtest with embargo

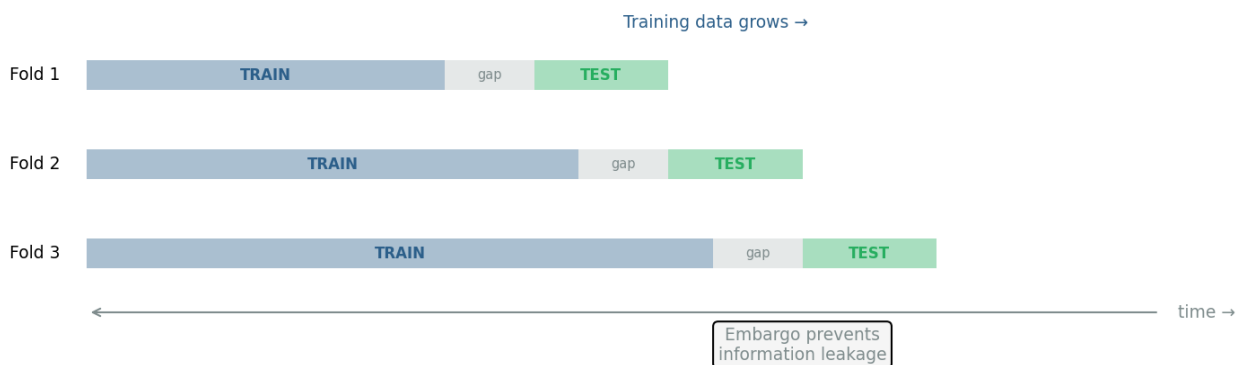


Figure 5.1 — The rolling-origin backtest. The training window (blue) expands as the origin advances. The embargo gap (grey) separates the training data from the forecast period (red) to prevent information leakage through lagged features. Each origin produces one set of 48 half-hourly forecasts.

The Embargo: Why It Exists

The **embargo** is a gap of excluded data between the end of the training set and the start of the forecast period. It is the single most important anti-leakage mechanism in the backtest.

Embargo: A mandatory gap between the end of the training data and the start of the forecast evaluation period. The embargo prevents information leakage that can occur through lagged features. If the model uses a lag-48 feature (price 24 hours ago) and the forecast starts at time $t+1$, then the lag-48 feature at time $t+48$ would reference the price at time t — which is in the training set. An embargo of at least H periods (the forecast horizon) ensures that no lagged feature in the forecast period can reference data from the training set.

Here is the leakage scenario the embargo prevents:

- Training data ends at time t .
- Without embargo, the forecast starts at $t+1$ and covers periods $t+1$ through $t+48$.
- At forecast period $t+48$, the lag-48 feature = price at time $t+48-48$ = price at time t .
- Time t is the *last* observation in the training set — so the model was trained on this value.
- The model's forecast for period $t+48$ effectively uses a training observation as a feature, creating circular dependency.

With an embargo of $H = 48$ periods:

- Training data ends at time t .
- Embargo: periods $t+1$ through $t+48$ are excluded (neither trained on nor forecasted).

- Forecast: periods $t+49$ through $t+96$.
- At forecast period $t+96$, the lag-48 feature = price at time $t+48$ — which is in the embargo (not trained on). No leakage.

The embargo rule: The embargo must be at least as long as the longest lag used as a feature. If you use lag-336 (one week), the embargo must be at least 336 periods. In practice, setting the embargo equal to the forecast horizon H is a safe minimum that prevents leakage through the standard lags (48, 96, 336 are all ≤ 336 , and H is typically 48 in day-ahead forecasting). When using longer lags, increase the embargo accordingly.

Expanding vs. Sliding Windows

The rolling-origin procedure requires choosing how the training window evolves as the origin advances:

Expanding window: The training window starts at a fixed point in time and grows longer as the origin advances. Every historical observation is always included. This maximises the amount of training data but may be hurt by non-stationarity — old data from a fundamentally different market structure may mislead the model.

Sliding window: The training window has a fixed length (e.g., 2 years). As the origin advances, the window slides forward — old observations are dropped as new ones are added. This adapts to changing conditions but uses less data and may lose useful patterns (e.g., rare seasonal events that occurred outside the window).

Approach	Pros	Cons	Best when
Expanding	Maximum data, captures rare events	Old data may be misleading if market has changed	Market structure is stable
Sliding	Adapts to structural change, recent data is most relevant	Less data, may miss seasonal patterns	Market is evolving rapidly

In the NEM, where the generation mix has changed dramatically (massive solar build-out since 2018, coal plant closures, growing battery fleet), a **sliding window of 2–3 years** often outperforms an expanding window. Data from 2017 reflects a fundamentally different supply stack than 2024, and including it can actively mislead the model.

Real-world example — the expanding window trap: Consider training a model for SA1 in 2024. With an expanding window, the training set includes 2018–2023. But in 2018, SA1 had ~1 GW of rooftop solar and negligible utility-scale solar. By 2024, installed solar has tripled. The 2018 data teaches the model that midday prices are moderate (coal sets the price), but in 2024, midday prices are routinely zero or negative (solar surplus sets the price). The old data dilutes the model's ability to learn the current market dynamics.

Baseline Models: The Bar Your Model Must Clear

Why Baselines Matter

A forecasting model's value is measured not by its absolute accuracy but by its accuracy **relative to baselines**. A model with an MAE of \$15/MWh sounds impressive — until you learn that a trivial baseline achieves \$16/MWh. The model adds enormous complexity for negligible improvement.

Baseline model (benchmark): A simple, often parameter-free model that provides a reference point for evaluating more sophisticated approaches. A model is only considered useful if it outperforms the baseline by a statistically significant margin. In electricity price forecasting, the standard baselines are the "naive" forecast (price at the same time last week) and the AEMO pre-dispatch price.

The Similar-Day Naive Baseline

The simplest reasonable price forecast exploits the strongest feature in the data — the lag-48 autocorrelation:

$$y_t^* = y_{t-336}$$

This says: the predicted price at time t is the *actual* price at the same half-hour, same day of the week, one week ago (336 half-hours = 7 days). This is called the **similar-day naive** because it uses the most recent "similar" day as the forecast.

Similar-day naive forecast: A forecast that sets each half-hourly price equal to the actual price at the same half-hour on the same day of the week, one week earlier. It captures the diurnal cycle (same half-hour) and the weekly cycle (same day of week) with zero parameters to estimate. Despite its simplicity, it is surprisingly difficult to beat — many published forecasting models that claim to outperform "naive" baselines actually compare against weaker baselines (like a constant-mean forecast).

The similar-day naive captures: - The **diurnal cycle** (same half-hour of day) - The **weekly cycle** (same day of week) - The **current price level** (recent prices, not historical averages)

It does *not* capture: - Weather changes (next week's weather may differ from this week's) - Generator outages or returns - Demand trends (growing or declining load) - Any form of conditional prediction (it makes the same forecast regardless of current conditions)

Warning — the "naive" trap: Many academic papers claim their model "beats the naive baseline" but use a weak naive — yesterday's price, or the historical average. The similar-day naive (same day of week, one week ago) is a much harder benchmark because it exploits both the lag-48 and lag-336 autocorrelations, which are the two strongest signals in the data. Always verify which naive baseline a paper uses before being impressed by its results.

The Autoregressive Baseline

An autoregressive (AR) model uses a **linear combination** of lagged values to predict the future:

Autoregressive (AR) model: A model that predicts the current value of a time series as a weighted sum of its own past values (lags) plus a constant. The number of lags included is called the "order" of the AR model. An AR model is equivalent to a linear regression where the features are past values of the target variable.

$$y_t^* = \beta_0 + \beta_1 \cdot y_{t-48} + \beta_2 \cdot y_{t-96} + \beta_3 \cdot y_{t-336} + \epsilon_t$$

where: - β_0 is the intercept (a constant baseline level) - $\beta_1, \beta_2, \beta_3$ are learned coefficients — the model discovers the optimal weight on each lag - ϵ_t is the error term

The AR model is the natural next step from the naive baseline. Instead of using lag-48 alone with an implicit coefficient of 1, it **learns the optimal weights** on multiple lags. For example, the model might learn $\beta_1 = 0.6$, $\beta_2 = 0.15$, $\beta_3 = 0.2$, meaning yesterday's price gets 60% weight, two days ago gets 15%, and last week gets 20%.

The AR model captures: - The diurnal cycle (lag 48) - The two-day pattern (lag 96) — useful for weekend-to-weekday transitions - The weekly cycle (lag 336) - The **relative importance** of each lag (learned coefficients, not hard-coded)

It does *not* capture: - Nonlinear relationships (it is strictly linear) - Weather effects (no weather features) - Volatility clustering (it assumes the error variance is constant)

Calendar-Enhanced AR

Adding **indicator variables** for the half-hour of day produces a more flexible baseline:

$$y_t^* = \beta_0 + \sum_k \beta_k \cdot y_{t-lagk} + \sum_h \gamma_h \cdot I(\text{halfhour} = h) + \epsilon_t$$

Indicator variable (dummy variable): A binary (0/1) feature that encodes category membership. For half-hour of day, there are 47 indicator variables (one is omitted to avoid perfect multicollinearity): $I(\text{halfhour}=1) = 1$ if the observation is in half-hour 1, 0 otherwise; $I(\text{halfhour}=2) = 1$ if the observation is in half-hour 2, and so on. Each indicator allows the model to learn a different average price level for that time period.

The half-hour dummies allow the model to learn that 6pm is systematically more expensive than 3am, independent of what yesterday's prices were. This is valuable because the duck curve shape (Chapter 2) persists across days even when the price level shifts — tomorrow's duck curve may be \$30 higher or lower than today's, but it will still *look* like a duck.

Forecast Error Metrics

Mean Absolute Error (MAE)

Mean Absolute Error (MAE): The average of the absolute differences between forecasts and actual values. MAE treats positive errors (over-prediction) and negative errors (under-prediction) equally, and treats all observations equally regardless of magnitude. It is measured in the same units as the target variable (\$/MWh for electricity prices).

$$\text{MAE} = (1/N) \times \sum_{t=1}^N |y_t - y_t^*|$$

Example: If the actual prices for three periods are [\$50, \$120, \$30] and the forecasts are [\$55, \$100, \$35]:

- Errors: $|50-55| = 5$, $|120-100| = 20$, $|30-35| = 5$
- $\text{MAE} = (5 + 20 + 5) / 3 = \$10/\text{MWh}$

MAE is the standard metric for electricity price forecasting because it is **robust to outliers**. Unlike squared-error metrics, a single \$15,000 spike does not dominate the entire evaluation.

Root Mean Square Error (RMSE)

Root Mean Square Error (RMSE): The square root of the average of squared forecast errors. By squaring the errors before averaging, RMSE penalises large errors disproportionately — a single error of \$100 contributes as much to RMSE as ten errors of ~\$32. RMSE is always greater than or equal to MAE, and the gap between them indicates how much the errors vary in magnitude.

$$\text{RMSE} = \sqrt{(1/N) \times \sum_{t=1}^N (y_t - y_t^*)^2}$$

For electricity prices, RMSE is problematic because a few spike events can dominate the metric. A model that perfectly predicts 99% of prices but misses one \$10,000 spike by \$5,000 will have a terrible RMSE, even if its MAE is excellent. This instability makes RMSE unreliable for comparing models on electricity price data.

Relative MAE (rMAE)

Relative MAE (rMAE): The ratio of a model's MAE to the MAE of a baseline (typically the similar-day naive). $rMAE < 1$ means the model outperforms the baseline; $rMAE > 1$ means it is worse. rMAE normalises away the absolute price level, making results comparable across different time periods (volatile summers have higher absolute MAE than mild autumns) and regions.

$$rMAE = MAE(model) / MAE(naive\ baseline)$$

rMAE is the preferred comparison metric because it answers the only question that matters: **does this model add value beyond the simplest reasonable alternative?** An rMAE of 0.85 means the model reduces error by 15% relative to the naive — a meaningful improvement. An rMAE of 0.98 means the model barely beats the naive — probably not worth the complexity.

Real-world example — why rMAE matters: Model A achieves $MAE = \$12/MWh$ on a volatile summer test period where the naive achieves $MAE = \$18$ ($rMAE = 0.67$). Model B achieves $MAE = \$8/MWh$ on a mild autumn test period where the naive achieves $MAE = \$10$ ($rMAE = 0.80$). Comparing raw MAEs, Model B looks better (\$8 vs. \$12). But rMAE tells the true story: Model A has more skill relative to its baseline (0.67 vs. 0.80). Model A is producing more *useful* forecasts — it improves most in the periods that are hardest to predict.

The Diebold-Mariano Test

When comparing two models, simply comparing their MAEs is not enough. The difference might be due to **chance** — random variation in which model happens to be closer to the truth on any given day. We need a formal statistical test to determine whether the difference is **statistically significant**.

Diebold-Mariano (DM) test: A statistical hypothesis test that determines whether the difference in forecast accuracy between two models is statistically significant. The test accounts for the fact that forecast errors are typically autocorrelated (today's error is correlated with tomorrow's), which would invalidate simpler comparison methods. A DM p-value below 0.05 provides evidence that the performance difference is real, not just noise.

The DM test works as follows:

1. For each time period t , compute the **loss differential**: $d_t = |e_{1,t}| - |e_{2,t}|$, where e_1 and e_2 are the errors of the two models.
2. Test whether the mean of the d_t series is significantly different from zero.
3. Use a **HAC (heteroscedasticity and autocorrelation consistent)** standard error estimator to account for the autocorrelation in the d_t series.

HAC standard error: A method for estimating the standard error of a statistic that accounts for both heteroscedasticity (changing variance over time) and autocorrelation (serial dependence) in the data. Standard (OLS) standard errors assume independent, identically distributed observations — an assumption that is always violated in time-series data. HAC standard errors correct for this, producing valid inference even when errors are correlated over time.

Statistical significance: A result is statistically significant if it is unlikely to have occurred by chance alone. The conventional threshold is a p-value of 0.05, meaning there is less than a 5% probability of observing a difference at least as large as the one found, if the two models truly had equal performance. Statistical significance does not imply practical significance — a statistically significant improvement of \$0.50/MWh might not justify the cost of a more complex model.

Why the DM test matters: In electricity price forecasting, test sets are often short — a few months of data. With short test sets, random variation can easily produce MAE differences of 5–10% between models that are genuinely equal in skill. The DM test prevents you from declaring a spurious "improvement" based on a lucky test period. Never report model comparisons without a DM test (or similar statistical test).

Battery Dispatch: Converting Forecasts to Revenue

The Arbitrage Problem

A grid-scale battery earns money through **temporal arbitrage** — buying electricity when it is cheap and selling it when it is expensive:

Temporal arbitrage: The practice of exploiting price differences across time. A battery charges (buys electricity from the grid) during low-price periods and discharges (sells electricity back to the grid) during high-price periods. The profit is the price difference minus efficiency losses. Unlike spatial arbitrage (buying in one location, selling in another), temporal arbitrage requires energy storage.

- **Charge** when prices are low: the battery draws electricity from the grid, converting electrical energy to chemical energy. It pays the spot price for this electricity.
- **Discharge** when prices are high: the battery releases its stored energy back to the grid, converting chemical energy to electrical energy. It receives the spot price for this electricity.

The profit per charge–discharge cycle depends on three things:

1. **The price spread:** How much higher the discharge price is than the charge price.
2. **The round-trip efficiency:** Not all energy stored can be recovered — some is lost as heat. Typical lithium-ion batteries achieve 85–92% round-trip efficiency.
3. **The energy volume:** How many MWh are charged and discharged.

Round-trip efficiency (η): The fraction of energy put into a battery that can be recovered when the battery is discharged. If a battery is charged with 100 MWh and can discharge 90 MWh, its round-trip efficiency is 90%. The lost 10 MWh is converted to heat. This means the discharge price must be at least $\text{charge_price} / \eta$ (e.g., $\text{charge_price} / 0.9 \approx 1.11 \times \text{charge_price}$) for the cycle to break even.

$$\text{Profit} \approx (P_{\text{discharge}} - P_{\text{charge}} / \eta) \times \text{Energy} \times \Delta t$$

Real-world example — a battery arbitrage cycle: Consider the Mannum BESS, a 100 MW / 200 MWh lithium iron phosphate battery in SA1, owned by Epic Energy and optimised by Habitat Energy (it can discharge at 100 MW for 2 hours). During the midday solar dip, the price is \$10/MWh. The battery charges for 2 hours at 100 MW, storing 200 MWh. Cost = $200 \times \$10 = \$2,000$. During the evening spike, the price is \$200/MWh. With 90% round-trip efficiency, the battery can discharge 180 MWh. Revenue = $180 \times \$200 = \$36,000$. Profit = $\$36,000 - \$2,000 = \$34,000$ in a single day. On extreme days with \$5,000+ spikes, a single cycle can earn over \$1 million. This is why battery operators care intensely about price forecasts — the forecast determines when to charge and discharge.

The Linear Program (LP)

Given a price forecast for $H = 48$ half-hour periods, the **optimal** charge–discharge schedule can be found by solving a **linear program**:

Linear program (LP): A mathematical optimisation problem where the objective function (what you want to maximise or minimise) and all constraints are linear — they involve only addition, subtraction, and multiplication by constants, with no squares, exponentials, or other nonlinear operations. LPs can be solved exactly and efficiently using well-established algorithms (simplex method, interior-point methods). The solution is guaranteed to be the global optimum — there is no risk of finding a "local" optimum that is not the best possible answer.

The battery dispatch LP is:

Objective — maximise revenue:

$$\text{Maximise: } \sum_{t=1}^H \text{price}_t \times (\text{discharge}_t - \text{charge}_t) \times \Delta t$$

Subject to constraints:

$0 \leq \text{charge}_t \leq P_{\text{max}}$ (power limit on charging) $0 \leq \text{discharge}_t \leq P_{\text{max}}$ (power limit on discharging) $0 \leq \text{SOC}_t \leq E_{\text{max}}$ (energy capacity limits)
 $\text{SOC}_{t+1} = \text{SOC}_t + \text{charge}_t \times \sqrt{\eta} \times \Delta t - \text{discharge}_t / \sqrt{\eta} \times \Delta t$ (energy balance) $\sum \text{discharge}_t \times \Delta t \leq C \times E_{\text{max}}$ (cycle limit) $\text{SOC}_0 = \text{SOC}_{\text{initial}}$ (initial state)

State of charge (SOC): The amount of energy currently stored in the battery, expressed either in MWh or as a percentage of total capacity. SOC evolves over time as the battery charges and discharges. The constraints $0 \leq \text{SOC} \leq E_{\text{max}}$ prevent over-charging (which damages the battery) and over-discharging (which degrades battery life).

P_max (power rating): The maximum rate at which the battery can charge or discharge, measured in megawatts (MW). A 100 MW battery can absorb or release up to 100 MW of power at any moment. The power rating determines how quickly the battery can fill or empty.

E_max (energy capacity): The total amount of energy the battery can store when fully charged, measured in megawatt-hours (MWh). A 200 MWh battery can store 200 MWh. The ratio $E_{\text{max}} / P_{\text{max}}$ gives the "duration" — how long the battery can discharge at full power. A 100 MW / 200 MWh battery has a 2-hour duration.

Cycle limit: A constraint on how many full charge–discharge cycles the battery can perform per day, imposed to limit degradation. Lithium-ion batteries degrade faster with more cycling. A typical limit is 1–2 full cycles per day. The constraint $\sum \text{discharge} \times \Delta t \leq C \times E_{\text{max}}$ limits the total discharged energy to C times the capacity per optimisation window.

The LP splits the round-trip efficiency (η) into two square-root components: one applied during charging ($\sqrt{\eta}$) and one during discharging ($1/\sqrt{\eta}$). This accurately models the physics — losses occur on both the charge and discharge sides — and keeps the formulation linear.

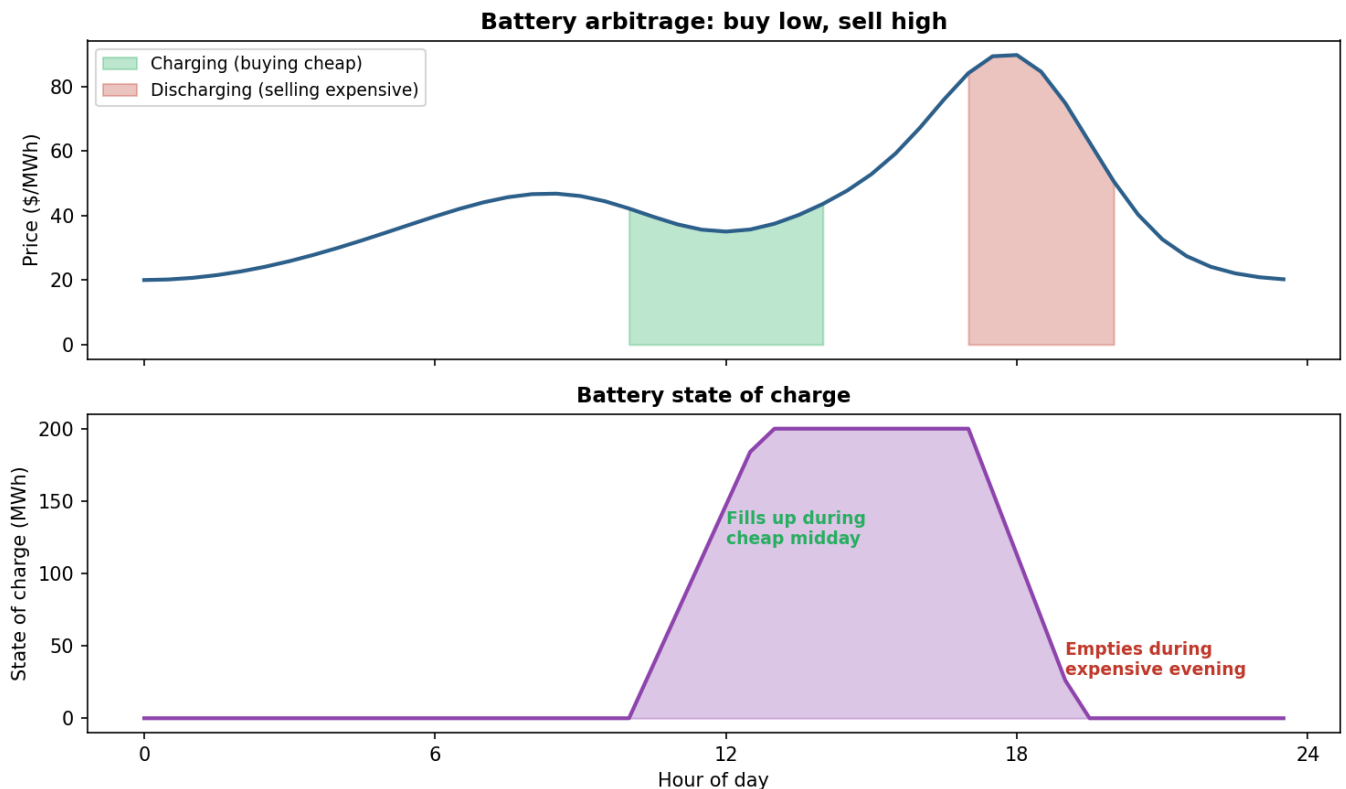


Figure 5.2 — Optimal battery dispatch from the LP. The top panel shows the price forecast (blue) and actual prices (grey). The bottom panel shows the battery's state of charge (green), with charging during low prices and discharging during high prices. The LP jointly optimises all 48 periods simultaneously.

Why an LP, Not Heuristic Rules?

A natural instinct is to use simple rules: "charge whenever the price is below \$30, discharge whenever it is above \$100." This is intuitive but leaves money on the table:

Rules ignore constraints. If you charge aggressively during a moderate dip, you may be full when an even cheaper period arrives later. The LP considers all 48 periods simultaneously and plans ahead.

Rules ignore the cycle limit. If you cycle early in the day on modest spreads, you may exhaust your cycle budget before the biggest spread of the day. The LP allocates cycles to the most profitable spreads.

Rules ignore energy coupling. The charge and discharge decisions are coupled through the state of charge — you cannot discharge what you have not first charged. The LP respects this coupling exactly.

Threshold selection is arbitrary. What is the "right" charge threshold — \$30? \$20? \$50? It depends on the distribution of prices that day. The LP discovers the optimal thresholds implicitly.

The LP is the gold standard: For a given price forecast, the LP finds the mathematically optimal dispatch schedule — no heuristic can do better. This is why we formulate dispatch as an LP: it removes dispatch quality as a variable, allowing us to isolate the effect of forecast quality. If two models produce different revenues through the LP, the difference is entirely due to forecast quality, not dispatch strategy.

Solving the LP with cvxpy

The LP is solved using **cvxpy**, a Python library for specifying and solving convex optimisation problems.

cvxpy: A Python-embedded modelling language for convex optimisation problems. The user specifies the objective function, variables, and constraints using natural mathematical syntax, and cvxpy translates the problem into a standard form that can be solved by one of several numerical solvers (CLARABEL, ECOS, SCS). For the battery dispatch LP with 48 time steps, the solve time is typically 1–10 milliseconds.

Convex optimisation: A class of optimisation problems where the objective function is convex (bowl-shaped) and the feasible region (defined by constraints) is also convex. The critical property of convex problems is that any local optimum is guaranteed to be the global optimum — the solver cannot get "stuck" in a sub-optimal solution. Linear programs are a special case of convex optimisation, so all LPs are convex.

Model Predictive Control (MPC)

From One-Shot to Rolling Dispatch

The LP formulation described above solves for an optimal schedule over a single 48-period window. In practice, the battery does not commit to this entire schedule at once. Instead, it uses **Model Predictive Control (MPC)** — a rolling strategy that re-optimises at every decision point.

Model Predictive Control (MPC): A control strategy that repeatedly solves an optimisation problem over a finite horizon, executes only the first step, then shifts the horizon forward and re-solves with updated information. MPC bridges the gap between forecasting (which looks forward) and control (which acts now). The key insight is that later actions in the schedule can be revised as better information arrives, so there is no need to commit to them in advance.

The MPC loop:

1. At the current decision point, generate a price forecast for the next H periods.
2. Solve the LP to find the optimal schedule over the full horizon.
3. **Execute only the first period's action** — charge, discharge, or do nothing.
4. Advance time by one period (30 minutes).
5. Observe the actual price for the period just completed.
6. Update the price forecast with any new information that has arrived.
7. Repeat from step 2.

Why Only Execute the First Period?

The key insight of MPC is that **forecasts improve over time**. The forecast made at noon for 8pm is worse than the forecast made at 7:30pm for 8pm — the later forecast has access to more recent data and a shorter horizon.

Open-loop control: Executing a pre-computed plan without adjusting for new information. The full 48-period schedule from a single LP solve is open-loop — it commits to actions based on a single forecast, ignoring everything that happens afterward.

Closed-loop control: Adjusting actions based on new information as it becomes available. MPC is closed-loop — it re-optimises at each step, incorporating the latest observations and updated forecasts. Closed-loop control is always at least as good as open-loop control (and usually better), because it can never be harmful to use better information.

By re-optimising at each step, MPC captures the value of the improving forecast. If the noon forecast predicted a \$200 spike at 6pm but by 5pm the updated forecast shows only \$80, MPC can cancel the planned discharge and save the stored energy for a better opportunity. Open-loop control would have committed to discharging at \$80 based on the stale \$200 forecast.

Real-world example — MPC saves the day: At noon, the forecast predicts moderate prices all evening (no spike). The LP plans minimal dispatch — just one moderate charge/discharge cycle. But at 4pm, an unexpected generator trip removes 500 MW from the grid. The updated price forecast now shows a \$3,000 spike at 6pm. MPC, re-solving at 4pm, immediately plans a full charge followed by discharge into the spike. Open-loop control, locked into the noon plan, would have missed this opportunity entirely. The MPC advantage in this scenario could be worth tens of thousands of dollars.

MPC Limitations with Point Forecasts

MPC with a point forecast (the median or expected price) treats the forecast as **certain**. It dispatches the same way whether the forecast is confident or uncertain:

- If the forecast says "6pm price will be \$200" with high confidence (narrow prediction interval), the battery should discharge aggressively — the spike is very likely.
- If the forecast says "6pm price will be \$200" with low confidence (wide prediction interval), the battery should be more conservative — the spike might not materialise, and discharging into a \$50 market wastes a cycle.

Point-forecast MPC cannot distinguish these two situations. It dispatches identically in both cases. This limitation motivates the **probabilistic dispatch** methods covered in Chapters 8 and 9, where the full distribution of possible prices informs the dispatch decision.

The Capture Ratio

Definition and Interpretation

The **capture ratio** is the headline economic metric that connects forecast quality to dollar value:

Capture ratio: The ratio of revenue earned by forecast-driven battery dispatch to the revenue that would be earned with perfect foresight (knowing the actual prices in advance). It measures how much of the theoretically available arbitrage revenue the forecast enables the battery to capture.

Capture Ratio = Revenue(forecast dispatch) / Revenue(perfect foresight dispatch)

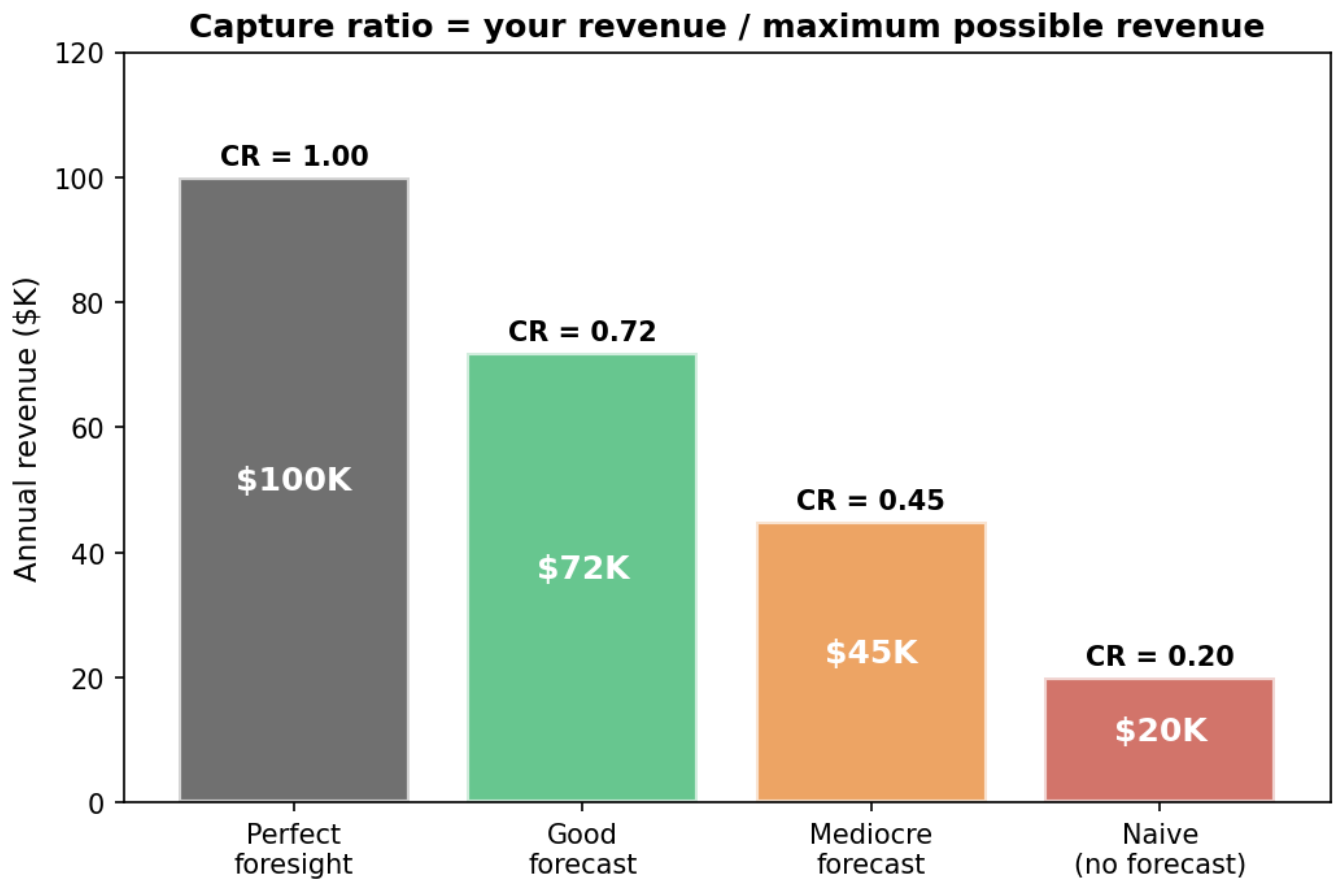


Figure 5.3 — The capture ratio measures what fraction of the perfect-foresight revenue a forecast-driven battery actually earns. Perfect foresight (CR = 1.0) is unachievable; the gap from 1.0 represents the cost of forecast imperfection.

Interpreting the capture ratio:

Capture Ratio	Interpretation
CR = 1.0	Perfect forecast — the battery captures all available revenue. Unachievable in practice.
CR = 0.65–0.75	Good forecast — the target range for a well-tuned model. This is what we aim for.
CR = 0.50	Roughly the market average against perfect foresight — the bar your model must clear.
CR = 0.3–0.50	Below average — significant revenue left on the table. Room for improvement.
CR = 0.0	The forecast is useless — the battery neither makes nor loses money. Equivalent to not operating.
CR < 0.0	The forecast is worse than useless — the battery systematically buys high and sells low, losing money.

Perfect foresight (hindsight optimum): The theoretical dispatch schedule that would be chosen if the actual future prices were known in advance. It represents the absolute upper bound on battery revenue for a given price trajectory. It is computed by solving the LP with *actual* prices instead of forecast prices. No real-world system can achieve perfect foresight, but it provides the denominator for the capture ratio.

Why Capture Ratio, Not MAE?

MAE measures forecast *accuracy** — how close the predicted prices are to the actual prices. The capture ratio measures forecast *value** — how much money the forecast makes. These are related but not the same.

Consider two forecasts:

- **Forecast A:** Very accurate during quiet periods (\$40–\$80 prices), but misses every spike. MAE = \$12.
- **Forecast B:** Slightly worse during quiet periods, but correctly identifies the timing of spikes (though not their exact magnitude). MAE = \$14.

Forecast A has a better MAE, but Forecast B has a better capture ratio — because the spikes are where the money is. During quiet periods (price spread of \$20–\$40), the battery earns modest revenue regardless of forecast quality. During spikes (price spread of \$200–\$10,000), forecast quality makes the difference between capturing a windfall and missing it entirely.

The capture ratio as the headline metric: Throughout the remaining chapters, every model improvement is evaluated through its effect on the capture ratio. This forces us to focus on what matters economically: not the average prediction error, but the quality of dispatch decisions at the moments that drive revenue. A 5% improvement in MAE from better predictions during quiet periods is worth less than a 2% improvement from better spike timing — the capture ratio reveals this directly.

AEMO Pre-Dispatch as Benchmark

AEMO publishes its own price forecasts through the pre-dispatch process. These are the **market's consensus forecast** — they incorporate AEMO's demand model, renewable generation forecasts, generator bids, and network constraints.

AEMO pre-dispatch price: AEMO's projection of the spot price for each half-hour of the upcoming trading day, computed every 30 minutes by running the dispatch engine with forecast inputs. The pre-dispatch price reflects the market's collective expectations (through generator bids) and AEMO's demand/renewable forecasts. It is the natural benchmark for any price forecasting model — beating AEMO's own forecast represents genuine added value.

Using AEMO pre-dispatch as a benchmark is informative because:

1. It is a **real, operationally-produced forecast** — not an academic exercise.
2. It incorporates information (generator bids, network constraints) that our statistical models do not have.
3. It is the forecast that **market participants actually see** — other batteries and traders react to it.
4. Beating it demonstrates that statistical/ML methods add value beyond what AEMO's engineering approach provides.

In practice, good statistical models achieve capture ratios 5–15 percentage points above AEMO pre-dispatch MPC, primarily by better predicting the *distribution* of prices (probabilistic forecasts) rather than just the point estimate. Throughout this course, we set **0.50 as the bar** (roughly the market average against perfect foresight) and **0.65–0.75 as the target range** for a well-tuned model. Every model is reported against AEMO pre-dispatch via NEMSEER.

Glossary

Term	Definition
Gate closure	Deadline by which a forecast must be issued (noon in our setup)
Forecast horizon	Time span covered by the forecast (48 half-hours = 24 hours)
Point forecast	Single predicted value per time step
Probabilistic forecast	Prediction of quantiles or a full distribution per time step
Trading day	NEM day from 04:00 to 04:00 AEST
Feature (predictor)	Input variable used by a forecasting model
Autoregressive feature	Feature constructed from past values of the target itself
Net load	Demand minus variable renewable generation
Data leakage	Contamination of model inputs with future information
Rolling-origin backtest	Walk-forward evaluation across multiple train/test splits
Embargo	Gap between training data and forecast to prevent leakage
Expanding window	Training window that grows over time
Sliding window	Training window of fixed length that moves forward
Similar-day naive	Forecast using same half-hour, same day of week, one week prior
MAE	Mean Absolute Error — average of absolute forecast errors
RMSE	Root Mean Square Error — penalises large errors more
rMAE	Relative MAE — model MAE divided by naive MAE
Diebold-Mariano test	Statistical test for significance of forecast accuracy differences
HAC standard error	Standard error corrected for autocorrelation and heteroscedasticity
Temporal arbitrage	Exploiting price differences across time via storage
Round-trip efficiency	Fraction of stored energy recoverable on discharge
Linear program (LP)	Optimisation with linear objective and constraints
SOC	State of charge — current stored energy in the battery
cvxpy	Python library for convex optimisation
MPC	Model Predictive Control — rolling re-optimisation strategy

Open-loop control	Executing a fixed plan without adjustment
Closed-loop control	Adjusting plan as new information arrives
Capture ratio	Forecast revenue / perfect foresight revenue
Perfect foresight	Theoretical optimum with known future prices
Pre-dispatch	AEMO's projection of future dispatch and prices

Summary

Day-ahead electricity price forecasting is a precisely defined problem: predict 48 half-hourly trading prices using only information available before noon gate closure. The feature matrix combines autoregressive lags (exploiting the strong lag-48 autocorrelation), calendar features (encoding the duck curve and weekly patterns), weather features (proxying renewable supply and thermal demand), and engineered features like net load. Data leakage — the silent contamination of features with future information — is the most dangerous error in time-series forecasting, producing optimistic backtests that collapse in production. The rolling-origin backtest with an embargo gap prevents leakage and provides robust, multi-period performance estimates. Baseline models (the similar-day naive and autoregressive models) are surprisingly hard to beat and set the bar that any sophisticated model must clear. Battery dispatch is formulated as a linear program that finds the mathematically optimal charge–discharge schedule for any given price forecast, and Model Predictive Control applies this LP in a rolling fashion to exploit improving forecasts over time. The capture ratio — forecast-driven revenue divided by perfect-foresight revenue — is the headline metric that translates forecast quality into economic value, and AEMO's pre-dispatch price provides the operational benchmark that any model must beat to demonstrate real-world utility.

6. Classical Forecasting Models — LEAR

From Baselines to Real Models

Chapter 5 established the baselines — the similar-day naive and simple autoregressive models. These are surprisingly hard to beat, but they have fundamental limitations: they use only the target variable's own past values and calendar indicators. They ignore weather, demand, cross-hour relationships, and every other signal that might improve predictions.

This chapter introduces the first "real" forecasting model: **LEAR** (LASSO Estimated AutoRegressive). LEAR is a linear model, but one armed with a powerful trick — **automatic feature selection** via LASSO regularisation. It consistently performs near the top of academic electricity price forecasting competitions while remaining simple, fast, and interpretable. Understanding LEAR in depth is essential because it serves as the benchmark that all subsequent machine learning models must beat.

Linear Regression: The Foundation

What Is Linear Regression?

Before understanding LEAR, we need to understand the building block it rests on: **linear regression**.

Linear regression: A statistical method that models the relationship between a target variable (the thing you want to predict) and one or more features (the things you use to predict it) as a weighted sum. The model assumes the target is approximately equal to a constant plus a weighted combination of the features. The weights (called **coefficients** or **parameters**) are estimated from historical data by minimising the sum of squared prediction errors.

In its simplest form, with one feature:

$$y^* = \beta_0 + \beta_1 \cdot x_1$$

where: - y^* is the predicted value (the model's output) - x_1 is the feature (the model's input) - β_0 is the **intercept** (the predicted value when $x_1 = 0$) - β_1 is the **coefficient** (how much y^* changes when x_1 increases by one unit)

With multiple features:

$$y^* = \beta_0 + \beta_1 \cdot x_1 + \beta_2 \cdot x_2 + \dots + \beta_p \cdot x_p$$

Coefficient (weight, parameter): A number that quantifies the relationship between a feature and the target. A coefficient of $\beta_1 = 0.6$ on the lag-48 price feature means: "For each \$1 increase in yesterday's same-hour price, today's predicted price increases by \$0.60, holding all other features constant." Coefficients are estimated from data, not chosen by the modeller.

Intercept (bias term): The constant term β_0 in a linear model. It represents the predicted value when all features are zero. In electricity price forecasting, the intercept captures the baseline price level that is not explained by any feature.

Ordinary Least Squares (OLS)

The standard method for estimating the coefficients is **Ordinary Least Squares (OLS)**: choose the coefficients that minimise the sum of squared errors between the predictions and the actual values.

Ordinary Least Squares (OLS): The method of estimating linear regression coefficients by minimising the sum of squared residuals: $\sum (y_i - y_i^*)^2$. This is equivalent to finding the coefficients that make the model's predictions as close as possible to the actual values, where "close" is measured by squared distance. OLS has a closed-form solution (the "normal equations"), so the coefficients can be computed exactly without iteration.

Minimise: $(1/N) \times \sum_{i=1}^N (y_i - y_i^*)^2$

Residual: The difference between an actual value and the model's prediction: $e_i = y_i - y_i^*$. A positive residual means the model under-predicted (actual was higher than predicted); a negative residual means the model over-predicted. The goal of regression is to make residuals as small as possible.

OLS works beautifully when you have a moderate number of well-chosen features. But it breaks down when the feature set is large, redundant, or noisy — precisely the situation in electricity price forecasting.

The Problem with Too Many Features

Consider a LEAR-style model with features for each hour h :

- 3 lagged prices at hour h (lag-1, lag-2, lag-7 days)
- 23 cross-hour prices from yesterday (all other hours)
- 6 day-of-week indicators

- 11 month indicators
- Demand forecast
- Wind speed, solar CSI, temperature

That is already 45+ features per hour. OLS will happily fit all 45 coefficients — but many of these features are irrelevant for any given hour (solar features at 3am, for example), and the resulting model **overfits**: it learns noise in the training data rather than genuine patterns.

Overfitting: When a model learns the noise (random variation) in the training data rather than the underlying signal (true patterns). An overfit model performs excellently on the training data but poorly on new, unseen data. Overfitting is more likely when the model has many parameters relative to the amount of training data — exactly the situation when using a large feature set with OLS.

Real-world example — overfitting in action: Suppose we include 100 features in an OLS model trained on 365 days of data. Many features will be correlated with price purely by coincidence — perhaps rainfall in Darwin happens to correlate with SA1 prices during the training period due to a shared seasonal pattern. The OLS model assigns a non-zero coefficient to this spurious feature. On new data, the spurious correlation disappears, and the model's predictions are degraded by the nonsensical rainfall coefficient. LASSO would have set this coefficient to zero, correctly discarding the irrelevant feature.

Regularisation: Controlling Model Complexity

The Bias-Variance Tradeoff

Every forecasting model faces a fundamental tension between two sources of error:

Bias: Error from overly simplistic assumptions in the model. A model with high bias "underfits" — it fails to capture genuine patterns in the data. For example, a model with only an intercept (no features) has high bias because it predicts the same constant for every hour, ignoring the duck curve.

Variance: Error from the model's sensitivity to the specific training data. A model with high variance "overfits" — its predictions change substantially if you swap the training data for a different sample from the same period. An OLS model with 100 features has high variance because the coefficient estimates are unstable when features are correlated or the sample is not large enough.

Bias-variance tradeoff: The principle that reducing bias (making the model more flexible) tends to increase variance (making it more sensitive to training data), and vice versa. The optimal model balances these two sources of error. Regularisation is the primary tool for navigating this tradeoff — it intentionally adds bias (by constraining coefficients) to reduce variance (by preventing overfitting).

In time-series forecasting, the tradeoff has a **temporal dimension**:

- **More training data** (longer window) reduces variance but increases bias if the market has changed.
- **More features** can reduce bias (capturing more signals) but increases variance (more parameters to estimate).
- **Regularisation** deliberately increases bias (by shrinking or eliminating coefficients) to reduce variance.

The goal is to find the sweet spot — a model complex enough to capture the real signals but simple enough to generalise to new data.

Ridge Regression (L2 Penalty)

The first regularisation approach, **Ridge regression**, adds a penalty on the *squared magnitude* of the coefficients:

Ridge regression (L2 regularisation): A modification of OLS that adds a penalty term proportional to the sum of squared coefficients: $\lambda \times \sum \beta_j^2$. This penalty discourages large coefficients, pulling them toward zero. The result is a model with smaller, more stable coefficients that generalises better to new data. However, Ridge never sets any coefficient exactly to zero — it shrinks all coefficients but keeps them all in the model.

Minimise: $(1/N) \times \sum (y_i - y_i^*)^2 + \lambda \times \sum \beta_j^2$

The parameter λ (lambda) controls how aggressively the coefficients are shrunk: - $\lambda = 0$: No regularisation. Identical to OLS. - $\lambda \rightarrow \infty$: All coefficients shrink to zero. The model predicts a constant (the mean).

Ridge works well when all features contribute some information and you want to dampen their influence — but it does not perform **feature selection**. Even truly irrelevant features retain small but non-zero coefficients, adding noise to predictions.

LASSO Regression (L1 Penalty)

LASSO replaces the squared penalty with an **absolute value** penalty:

LASSO (Least Absolute Shrinkage and Selection Operator): A regularised linear regression method that adds a penalty proportional to the sum of the *absolute values* of the coefficients: $\lambda \times \sum |\beta_j|$. The L1 penalty has a remarkable mathematical property: it drives some coefficients exactly to zero, effectively removing those features from the model. LASSO simultaneously estimates coefficients and performs feature selection — it decides which features matter and which do not.

Minimise: $(1/N) \times \sum_{i=1}^N (y_i - y_i^*)^2 + \lambda \times \sum_{j=1}^p |\beta_j|$

L1 penalty (L1 norm): The sum of the absolute values of the coefficients: $\sum |\beta_j|$. The "L1" notation comes from the mathematical concept of the L^1 norm of a vector. The L1 penalty creates a "diamond-shaped" constraint region in coefficient space, which has corners on the coordinate axes. The optimal solution often falls at these corners, where one or more coefficients are exactly zero. This geometric property is why L1 produces sparse solutions while L2 (circle-shaped constraint) does not.

Sparsity: A property of a model where most coefficients are exactly zero — only a few features have non-zero weights. A sparse model is easier to interpret (you can see which features matter), faster to evaluate (only non-zero features need to be computed), and often more accurate on new data (irrelevant features are excluded rather than contributing noise).

Why L1 Creates Sparsity and L2 Does Not

The difference between LASSO (L1) and Ridge (L2) comes down to geometry. Visualise the problem in two dimensions (two features with coefficients β_1 and β_2):

- The OLS objective (sum of squared errors) creates elliptical contours in coefficient space. The OLS solution is at the centre of the ellipses.
- **Ridge** constrains the solution to lie within a circle ($\beta_1^2 + \beta_2^2 \leq t$). The constrained optimum is where the smallest ellipse touches the circle — which is generally at a point where both β_1 and β_2 are non-zero (the circle has no corners).
- **LASSO** constrains the solution to lie within a diamond ($|\beta_1| + |\beta_2| \leq t$). The constrained optimum is where the smallest ellipse touches the diamond — which often occurs at a **corner** of the diamond, where one coefficient is exactly zero.

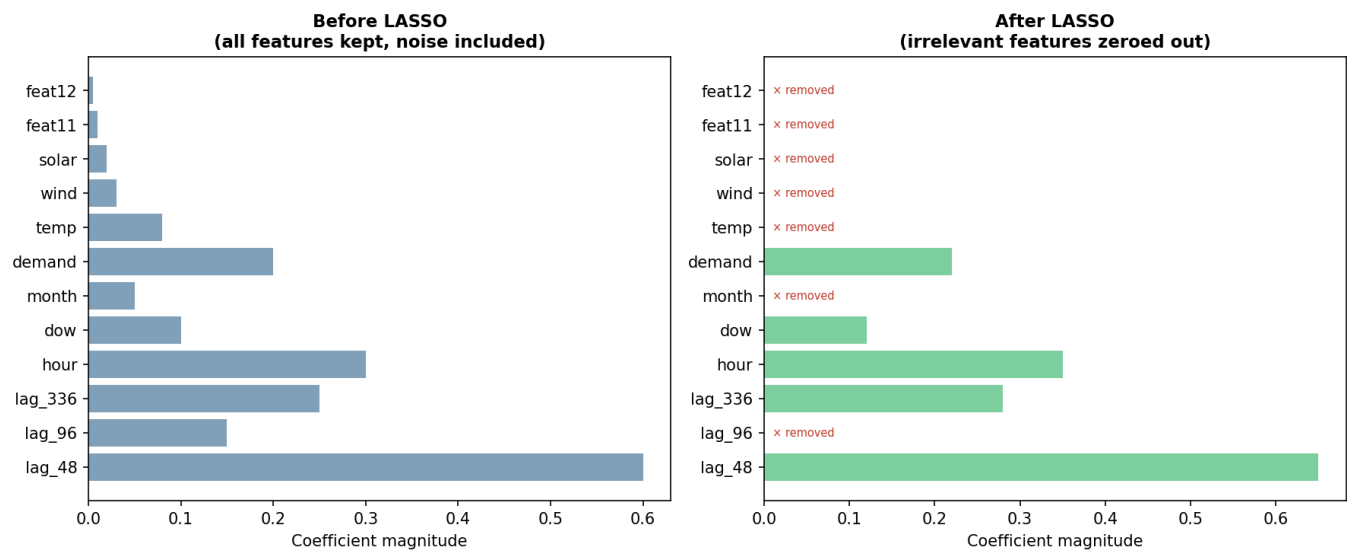


Figure 6.1 — The geometry of LASSO (L1) vs Ridge (L2) regularisation. The diamond-shaped L1 constraint has corners on the axes, where one coefficient is exactly zero. The OLS contour ellipses are more likely to touch the diamond at a corner than to touch the circle at an axis intercept. This is why LASSO produces sparse solutions — it sets irrelevant coefficients to exactly zero.

The geometric intuition: LASSO drives coefficients to exactly zero because its constraint region has sharp corners on the coordinate axes. When the OLS solution touches a corner, one or more coefficients become exactly zero. Ridge's circular constraint has no corners, so the solution generically has all coefficients non-zero. This seemingly small geometric difference — diamond vs. circle — has profound practical consequences: LASSO performs automatic feature selection while Ridge does not.

The Regularisation Parameter λ

The parameter λ controls the tradeoff between fitting the data (small residuals) and keeping the model simple (small/zero coefficients):

λ value	Effect	Risk
$\lambda = 0$	No regularisation. All features used. Equivalent to OLS.	Overfitting
Small λ	Gentle regularisation. Most features retained, slightly shrunk.	Mild overfitting
Moderate λ	Moderate regularisation. Many irrelevant features eliminated.	Good balance
Large λ	Heavy regularisation. Most features eliminated.	Underfitting
$\lambda \rightarrow \infty$	All coefficients zero. Model predicts a constant.	Maximum underfitting

Regularisation parameter (λ , also called α in scikit-learn): The hyperparameter that controls the strength of the regularisation penalty. A larger λ penalises large coefficients more aggressively, producing a simpler (sparser) model. The optimal λ is not known in advance — it must be selected by evaluating model performance across a range of candidate values, typically using cross-validation.

Hyperparameter: A setting that controls the behaviour of the learning algorithm itself, as opposed to the model's parameters (coefficients), which are learned from data. The regularisation strength λ is a hyperparameter — it is set before training begins and determines *how* the coefficients are learned, not *what* they are. Other hyperparameters include the number of cross-validation folds, the maximum number of iterations, and the choice of solver.

LassoCV: Automatic λ Selection

Cross-Validation for λ

How do you choose the right λ ? Trying it on the training data does not work — $\lambda = 0$ (no regularisation) always gives the lowest training error, but overfits. You need to evaluate on data the model has not seen.

Cross-validation (CV): A procedure for estimating how well a model generalises to unseen data, by repeatedly splitting the training data into a "fit" subset and a "validation" subset. The model is trained on the fit subset and evaluated on the validation subset. The average validation performance across all splits provides a robust estimate of generalisation error. Cross-validation is used to select hyperparameters (like λ) that cannot be estimated from the training data directly.

LassoCV automates this process:

1. Define a grid of candidate λ values (typically 100 values, logarithmically spaced from large to small).
2. For each λ , perform K-fold cross-validation: split the training data into K subsets, train on K-1 subsets, validate on the remaining one, and rotate.
3. Select the λ that minimises the average validation error.
4. Refit the model on the full training data using the selected λ .

LassoCV: An implementation of LASSO regression (available in scikit-learn) that automatically selects the optimal regularisation parameter λ using cross-validation. The user does not need to specify λ — LassoCV tries many values and picks the best one. This makes LEAR essentially **parameter-free** from the user's perspective: there are no tuning knobs to adjust.

Regularisation path: The sequence of LASSO solutions as λ varies from large (all coefficients zero) to small (many non-zero coefficients). As λ decreases, features "enter" the model one by one — the most important features enter first (their coefficients become non-zero at the largest λ values), and the least important enter last. The regularisation path provides a natural ranking of feature importance.

Cross-Validation for Time Series

Standard K-fold cross-validation randomly shuffles the data into K groups. This is **invalid for time series** because it violates temporal ordering — the model might train on Monday's data and validate on the preceding Friday's data, creating look-ahead leakage.

Time-series cross-validation (expanding window CV): A variant of cross-validation designed for temporally ordered data. Instead of random splits, the data is divided into K consecutive blocks. In fold k, the model trains on blocks 1 through k and validates on block k+1. This preserves temporal ordering — the model always trains on past data and validates on future data, mimicking the real forecasting scenario.

Blocked cross-validation: An alternative to expanding-window CV where the K folds are contiguous time blocks but the training set does not necessarily include all preceding blocks. This can be useful when you want to avoid the expanding-window assumption (that all past data is always relevant). Both approaches are valid; the key requirement is that validation data always comes *after* training data in time.

Never use random-shuffle CV on time series: Standard K-fold cross-validation (as implemented by default in scikit-learn's `cross_val_score`) randomly assigns observations to folds, destroying temporal ordering. A model trained on November data and validated on October data has seen the future. Always use `TimeSeriesSplit` or a custom blocked-CV scheme for time-series problems.

The LEAR Model

Background and Motivation

The **LASSO Estimated AutoRegressive (LEAR)** model was introduced by Lago, Marcjasz, De Schutter, and Weron in their 2021 paper "Forecasting day-ahead electricity prices: A review of state-of-the-art algorithms, best practices and an open-access benchmark." It has become the standard benchmark for day-ahead electricity price forecasting worldwide.

LEAR's central insight is that electricity prices have a **strong hourly structure**: the statistical relationships between features and prices differ substantially between different times of day. At 3am, the price is driven by baseload demand and coal generation; solar features are irrelevant (the sun is down). At 1pm, the price is dominated by the solar surplus and the midday demand dip; last night's prices are less informative. At 6pm, the evening ramp drives the price, and the transition from solar to gas peakers is the key mechanism.

Rather than forcing a single model to learn these hour-specific patterns (which would require complex interaction terms), LEAR fits **separate models for each hour of the day**.

The LEAR Formulation

For each hour h in $\{0, 1, \dots, 23\}$ (or each half-hour h in $\{0, 1, \dots, 47\}$ for half-hourly data), LEAR fits a LASSO regression:

$$y_{d,h}^* = \beta_0 h + \sum_j \beta_{jh} \cdot x_{jd} + \epsilon_{d,h}$$

where: - $y_{d,h}^*$ is the predicted price for day d , hour h - $\beta_0 h$ is the intercept for hour h - β_{jh} is the coefficient for feature j at hour h — note the superscript h , indicating that each hour has its own set of coefficients - x_{jd} is the value of feature j on day d - ϵ is the error term

The feature vector x typically includes:

LEAR (LASSO Estimated AutoRegressive): A day-ahead electricity price forecasting model that fits separate LASSO regressions for each hour (or half-hour) of the day. Each model receives a large set of candidate features — lagged prices (including cross-hour lags), calendar indicators, demand, and weather — and LASSO's L1 penalty automatically selects the relevant features for that specific hour. The "autoregressive" part of the name refers to the inclusion of lagged prices as features. LEAR is fast, interpretable, and consistently competitive with far more complex models.

1. **Same-hour lagged prices:** - $y_{d-1,h}$: Yesterday's price at hour h (lag-1 day) - $y_{d-2,h}$: Two days ago at hour h (lag-2 day) - $y_{d-7,h}$: Last week's price at hour h (lag-7 day)
2. **Cross-hour lagged prices:** - $y_{d-1,h-1}, y_{d-1,h+1}$: Neighbouring hours yesterday - Potentially all 24 (or 48) hours of yesterday
3. **Calendar features:** - Day-of-week indicator variables (6 dummies for 7 days) - Month indicator variables (11 dummies for 12 months) - Public holiday indicator
4. **Exogenous features (where available):** - Demand forecast for hour h - Wind speed forecast - Solar CSI forecast - Temperature deviation

Per-Hour Models: Why Separate Is Better

The per-hour architecture is LEAR's most distinctive design choice. Instead of one model with 48 outputs, LEAR builds 48 independent models — each one a specialist for its particular time of day.

Per-hour models (per-period models): An approach where a separate forecasting model is trained for each time period of the day. Each model sees only the data relevant to its specific period and learns hour-specific patterns. This contrasts with a "pooled" approach that trains a single model on data from all hours, using hour indicators to differentiate.

Advantages of per-hour models:

Natural hour-specific patterns. The 1pm model can learn that solar CSI is critical for midday prices, while the 3am model ignores CSI entirely (it is always zero at night). No interaction terms needed.

Feature selection is hour-specific. LASSO can select different features for each hour. The 6pm model might use lag-48 price, demand, and wind (the evening ramp drivers), while the 2am model might use only lag-48 price and a weekday indicator.

Simplicity. Each individual model is small (a few dozen features, one output) and trains in milliseconds. The computational cost is trivial even for daily refitting.

Interpretability. You can inspect each model's non-zero coefficients and verify they make physical sense: "Does the 1pm model use solar features? Yes. Does the 3am model? No. Good."

Disadvantages of per-hour models:

No information sharing. If lag-48 is important at hour 14, this does not help hour 15's model learn the same pattern. Each model must discover it independently from its own training data.

Prediction discontinuities. Adjacent-hour models are estimated independently, so their predictions can jump: hour 14's model predicts \$80, hour 15's model predicts \$120, creating an implausible \$40 jump. In practice, these jumps are usually small because the underlying data is smooth.

More total parameters. 48 models $\times$ (say) 10 non-zero coefficients each = 480 effective parameters, versus perhaps 100 for a pooled model with hour interactions.

Specialisation wins: Despite the disadvantages, the per-hour approach is remarkably robust in practice. The hourly structure of electricity prices is so strong that the benefit of specialisation (each model learns its hour's specific dynamics) outweighs the cost of isolation (no information sharing). Lago et al. (2021) showed that LEAR with per-hour models outperforms pooled alternatives across multiple markets and time periods.

Real-world example — what LEAR's models look like: After fitting LEAR to SA1 data, the 1pm model might have non-zero coefficients on: yesterday's 1pm price ($\beta = 0.45$), last week's 1pm price ($\beta = 0.20$), today's solar CSI forecast ($\beta = -15.3$, negative because high solar reduces price), demand forecast ($\beta = 0.08$), and a January indicator ($\beta = 12.5$, capturing summer price premiums). All other features — including yesterday's

3am price, last night's wind, and the Tuesday indicator — have coefficients of exactly zero. LASSO has automatically determined that these features are not useful for predicting the 1pm price, given the other features already in the model.

Feature Selection via the L1 Penalty

How LASSO Selects Features

LASSO does not require the user to manually choose which features to include. Instead, the modeller provides a **deliberately large** candidate feature set, and LASSO's L1 penalty automatically selects the relevant subset:

1. Start with all candidate features (50+ features per hour).
2. The L1 penalty shrinks coefficients toward zero.
3. At the optimal λ (selected by LassoCV), many coefficients are *exactly* zero.
4. The features with non-zero coefficients are the "selected" features — the ones LASSO has determined are genuinely useful for prediction.

This automatic feature selection is enormously valuable because:

- It removes the need for manual feature engineering trial-and-error
- It adapts to different hours (selecting different features for each)
- It adapts over time (when the model is refitted, the selected features may change)
- It provides interpretability (you can see which features survived the selection process)

The Regularisation Path and Feature Importance

As λ decreases from large to small, features "enter" the model in order of importance:

Feature entry order: In the LASSO regularisation path, each feature has a critical λ value at which its coefficient first becomes non-zero. Features that enter at larger λ values are more important — they contribute enough predictive power to overcome a strong penalty. Features that enter only at very small λ values are marginal — they contribute little beyond what earlier features already capture.

For a typical hour in the NEM:

1. **First to enter (largest λ):** lag-48 price (yesterday's same-hour price). Always the most important feature.
2. **Second/third:** lag-336 price (last week) and/or demand forecast.
3. **Middle:** Cross-hour prices, temperature deviation, wind speed.
4. **Last to enter (smallest λ):** Month indicators, distant lags, weak calendar effects.
5. **Never enter (remain zero even at optimal λ):** Irrelevant features (solar at night, distant cross-hour prices with no physical connection).

This entry order provides a natural, data-driven **feature importance ranking** without any additional computation.

LASSO vs. Ridge for Electricity Prices

Ridge regression (L2 penalty) is the other common regularisation approach. For electricity prices, LASSO (L1) is preferred:

Property	LASSO (L1)	Ridge (L2)
Feature selection	Yes — sets irrelevant coefficients to exactly zero	No — all coefficients remain non-zero
Interpretability	High — you can see which features matter	Lower — all features contribute, hard to distinguish signal from noise
Performance with many irrelevant features	Good — irrelevant features are eliminated	Worse — irrelevant features add noise through small coefficients
Performance with correlated features	Can be unstable — may arbitrarily select one of two correlated features	More stable — spreads weight across correlated features
Computational cost	Slightly higher (iterative solver)	Lower (closed-form solution)

Elastic Net: A regularisation method that combines the L1 (LASSO) and L2 (Ridge) penalties: $\lambda_1 \times \sum |\beta_j| + \lambda_2 \times \sum \beta_j^2$. Elastic Net inherits LASSO's feature selection (from the L1 part) and Ridge's stability with correlated features (from the L2 part). It is sometimes used as a compromise when pure LASSO is unstable due to high feature correlation. In practice, for electricity prices, pure LASSO performs well because the key features (lag-48 price, demand) are not highly correlated with each other.

In practice, LEAR with LASSO typically outperforms Ridge by 5–15% in MAE on NEM data. The feature selection provided by L1 is particularly valuable because the LEAR feature set is deliberately large (many cross-hour lags, calendar indicators, weather variables), and most features are irrelevant for any given hour.

Time Series Fundamentals for LEAR

Stationarity

Stationarity: A time series is stationary if its statistical properties — mean, variance, and autocorrelation structure — do not change over time. Stationarity is an assumption (explicit or implicit) of most statistical models, including linear regression. If the data-generating process changes (e.g., a new solar farm is built, shifting the midday price distribution), a model trained on old data becomes progressively less accurate.

Electricity prices are **not** stationary. They exhibit:

- **Trending mean:** Fuel costs change, carbon policies evolve, renewable penetration grows. The average price in 2020 was different from 2024.
- **Time-varying variance:** Summer months have more extreme prices (volatility) than spring and autumn.
- **Changing autocorrelation:** As the generation mix evolves (more solar, less coal), the relationship between hours changes — the midday solar dip deepens year over year.

LEAR handles non-stationarity through three mechanisms:

1. **The arcsinh transform** stabilises the variance (a form of variance-stabilising transform).
2. **Rolling window retraining** ensures the model sees only recent, relevant data.
3. **The per-hour structure** allows each hour to adapt independently to changing conditions.

The Training Window Length

Choosing the right training window length is a bias-variance tradeoff with a temporal dimension:

- **Too short (e.g., 3 months):** Low bias (adapts quickly to changes) but high variance (too few observations for stable coefficient estimation). May miss seasonal patterns — a 3-month summer window does not see winter dynamics.
- **Too long (e.g., 5+ years):** Low variance (many observations) but high bias (old data from a different market structure misleads the model).
- **Sweet spot (2–4 years for NEM):** Enough data to capture seasonal patterns and estimate stable coefficients, but recent enough that the market structure has not changed dramatically.

Real-world example — window length matters: For SA1 in 2024, a 2-year training window (2022–2024) captures the current market structure (high solar penetration, significant battery fleet, post-coal-closure dynamics). A 6-year window (2018–2024) includes data from when solar penetration was half its current level and several coal plants were still operating. The 2018 data teaches the model relationships that no longer hold — for example, that midday prices are moderate (set by coal), when in 2024 they are often negative (set by solar surplus).

Forecast Horizons and Error Growth

Forecast error increases with the horizon, but the pattern depends on the model type:

- **Steps 1–6 (0.5–3 hours ahead):** Autoregressive momentum dominates. The most recent price is highly informative, and errors are small.
- **Steps 12–24 (6–12 hours ahead):** The diurnal pattern dominates. Calendar features carry most of the predictive power. Autoregressive features become less relevant.
- **Steps 24–48 (12–24 hours ahead):** Both autoregressive and calendar signals weaken. Weather features become increasingly important for predicting tomorrow's supply–demand balance.

LEAR's per-hour architecture sidesteps the horizon problem. Each model always predicts at the same effective horizon relative to its daily cycle — the "1pm model" always predicts one specific time of day, not a range of horizons. There is no explicit horizon management.

Model Evaluation and Comparison

Evaluating LEAR's Performance

LEAR is evaluated using the rolling-origin backtest framework from Chapter 5. Key metrics:

- **MAE** in \$/MWh (absolute accuracy)
- **rMAE** relative to the similar-day naive (relative improvement)
- **Capture ratio** from LP dispatch (economic value)
- **Diebold-Mariano p-value** against the baseline (statistical significance)

A well-tuned LEAR model for SA1 typically achieves:

Metric	Typical LEAR value	Similar-day naive	Improvement
MAE	\$15–20/MWh	\$20–28/MWh	20–30%

rMAE	0.70–0.85	1.00 (by definition)	15–30%
Capture ratio	0.55–0.70	0.35–0.50	10–25 pp

These numbers vary with the test period, region, and exact feature set, but the pattern is consistent: LEAR substantially outperforms the naive baseline.

The Diebold-Mariano Test for Model Comparison

When comparing LEAR to the naive baseline (or to other models), the Diebold-Mariano test determines whether the MAE difference is statistically significant:

$$\text{DM statistic} = \text{mean}(d_t) / \sigma^*(d_t)$$

where $d_t = |e_{\text{naive},t}| - |e_{\text{LEAR},t}|$ is the loss differential at each time step, and σ^* is the HAC-corrected standard error.

Loss differential: The difference in forecast error between two models at each time step: $d_t = L(e_{1,t}) - L(e_{2,t})$, where L is the loss function (absolute error, squared error, etc.). A positive d_t means model 1 had a larger error than model 2 at time t . The Diebold-Mariano test assesses whether the average loss differential is significantly different from zero.

If the DM p-value is below 0.05, we conclude that LEAR's improvement over the baseline is statistically significant — not just a lucky result on this particular test period. This discipline is essential in electricity price forecasting, where short test sets (a few months) can produce misleading comparisons.

The full evaluation pipeline: Every model in this course is evaluated using the same rigorous pipeline: (1) rolling-origin backtest with embargo, (2) MAE and rMAE in dollar space, (3) capture ratio from LP dispatch, (4) Diebold-Mariano test against the prior best model. This standardised evaluation prevents cherry-picking results and ensures genuine progress.

Recalibration and Concept Drift

Why Models Degrade Over Time

A trained model is a snapshot of the statistical relationships that held during the training period. As the market evolves, these relationships change — a phenomenon called **concept drift**.

Concept drift: A change over time in the statistical relationships between features and the target variable. In electricity markets, concept drift occurs when the underlying market structure changes — new generators are built, old ones retire, fuel prices shift, regulations change, demand patterns evolve. A model trained on old data captures old relationships that may no longer hold, leading to degraded performance.

Sources of concept drift in the NEM:

- **New generation capacity:** A new 200 MW wind farm changes the supply stack, shifting the net-load-to-price relationship. The model's learned coefficients on wind features become stale.
- **Generator retirement:** A coal plant closing removes baseload supply, structurally raising prices during periods that were previously cheap.
- **Fuel price changes:** A doubling of gas prices shifts the marginal cost of gas generators, changing the hockey-stick curve's shape and threshold.
- **Demand trends:** Growing electric vehicle charging shifts the demand profile. Increasing rooftop solar reduces daytime grid demand.
- **Regulatory changes:** Changes to the market price cap, bidding rules, or interconnector ratings alter market dynamics.

Refit Frequency

LEAR should be refitted regularly — typically **daily** (retrain using all data up to yesterday) or **weekly**:

Refit frequency	Pros	Cons
Daily	Fastest adaptation to changes	Highest computational cost (trivial for LEAR, significant for ML models)
Weekly	Good balance of adaptation and cost	Slight delay in responding to sudden changes
Monthly	Lowest cost	May miss important market shifts
Never	Zero cost	Guaranteed degradation over time

For LEAR, daily refitting is computationally trivial — fitting 48 LASSO regressions on 2 years of daily data takes seconds. There is no reason not to refit daily.

Monitoring for Degradation

Track the rolling rMAE (LEAR's MAE divided by the naive's MAE) over time. A well-functioning model should maintain rMAE below 1.0 consistently. Warning signs:

- **rMAE > 1.0 for a single week:** Possibly normal variation. Monitor but do not intervene.
- **rMAE > 1.0 for multiple consecutive weeks:** The model is underperforming the naive. Investigate — has the market changed? Are features stale? Is the training window too long or too short?
- **Sustained rMAE increase (upward trend):** Concept drift is degrading performance. Consider adjusting the training window, adding new features, or switching to a more adaptive model.

Real-world example — concept drift in action: In late 2022, Australian gas prices spiked dramatically due to global energy market disruptions. The cost of gas-fired electricity generation doubled overnight. A LEAR model trained on 2020–2022 data (mostly low gas prices) had learned that net load of 2,000 MW corresponds to prices of ~\$100 (gas peakers at \$100/MWh marginal cost). After the gas price spike, the same net load produced prices of ~\$200. The model's rMAE jumped from 0.75 to 1.1 until it accumulated enough post-spike data to recalibrate. Daily refitting resolved the problem within 2–3 weeks as the new gas-price regime entered the training window.

Glossary

Term	Definition
LEAR	LASSO Estimated AutoRegressive — per-hour LASSO models for price forecasting
Linear regression	Model predicting target as weighted sum of features
OLS	Ordinary Least Squares — standard coefficient estimation method
Coefficient	Weight on a feature in a linear model
Intercept	Constant term in a linear model
Residual	Difference between actual and predicted value
Overfitting	Model learns noise, performs well on training data but poorly on new data
Bias	Error from overly simplistic model assumptions
Variance	Error from model sensitivity to specific training data
Regularisation	Adding a penalty to control model complexity
Ridge (L2)	Regularisation with squared-coefficient penalty; shrinks but keeps all features
LASSO (L1)	Regularisation with absolute-coefficient penalty; drives some coefficients to zero
L1 norm	Sum of absolute values of coefficients
Sparsity	Property of having most coefficients exactly zero
Elastic Net	Combination of L1 and L2 penalties
Hyperparameter	Setting that controls the learning algorithm (e.g., λ)
Cross-validation	Estimating generalisation error by training/validating on data splits
LassoCV	LASSO with automatic λ selection via cross-validation
Regularisation path	Sequence of LASSO solutions as λ varies
Time-series CV	Cross-validation respecting temporal ordering
Per-hour models	Separate model for each hour of the day
Stationarity	Statistical properties constant over time
Concept drift	Change in feature–target relationships over time
Feature selection	Identifying which features are useful; automated by LASSO
Feature entry order	Order in which features gain non-zero coefficients as λ decreases
Loss differential	Difference in forecast error between two models at each time step
Diebold-Mariano test	Statistical test for significance of forecast accuracy differences

Summary

LEAR (LASSO Estimated AutoRegressive) is the standard benchmark for day-ahead electricity price forecasting, introduced by Lago et al. (2021). It fits separate LASSO regressions for each hour (or half-hour) of the day, allowing hour-specific feature selection and coefficient

estimation. LASSO's L1 penalty drives irrelevant feature coefficients to exactly zero — performing automatic feature selection from a deliberately large candidate set — while retaining only the features that genuinely improve prediction for each specific hour. The geometric intuition is that the diamond-shaped L1 constraint region has corners on the coordinate axes where coefficients are zero, while Ridge's circular L2 constraint does not. LassoCV selects the regularisation strength automatically via time-series cross-validation, making LEAR effectively parameter-free. The per-hour architecture naturally captures the strong hourly variation in price dynamics — solar features dominate midday models, demand features dominate evening models, and nighttime models rely primarily on autoregressive lags. Model evaluation follows the rigorous pipeline established in Chapter 5: rolling-origin backtest with embargo, MAE and rMAE in dollar space, capture ratio from LP dispatch, and Diebold-Mariano testing for statistical significance. Regular refitting (ideally daily) combats concept drift as fuel prices, the generation mix, and demand patterns evolve over time.

7. Machine Learning for Price Forecasting

Why Move Beyond Linear Models?

In Chapter 6, we built LEAR — a linear model that predicts electricity prices as a weighted sum of features. LEAR is a strong baseline, and in many forecasting competitions it performs surprisingly well. But it has a fundamental limitation baked into its very structure: it can only model price as a **linear function** of the inputs. If price doubles, LEAR assumes the feature driving the change also doubled — and vice versa.

Real electricity prices do not behave this way.

Linear model: A model that predicts the output as a weighted sum of its inputs: $y^* = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$. The relationship between any input and the output is a straight line (or a flat plane in higher dimensions). Linear models cannot capture curves, thresholds, or interactions between features unless those are explicitly engineered into the input.

Recall the **hockey-stick relationship** from Chapter 3: when net load (demand minus renewables) is below a certain threshold, the price is relatively flat — cheap coal and gas baseload generators set the marginal price. But when net load crosses the threshold, expensive peaking generators are dispatched and the price shoots upward exponentially. This is an inherently **nonlinear** relationship — a curve with a sharp bend, not a straight line.

A linear model must approximate this curve with a single straight line. It will inevitably undershoot during spikes (the line cannot bend sharply upward) and overshoot during quiet periods (the line is pulled upward by the spike observations). No amount of feature engineering can fix this — the limitation is structural.

Nonlinear model: A model that can learn curved, bent, or discontinuous relationships between inputs and outputs. Unlike a linear model, it does not assume that doubling an input doubles the output. Decision trees, gradient-boosted trees, and neural networks are all nonlinear models.

The core argument for machine learning: Electricity price formation is governed by the merit-order dispatch process — a step function that is flat in some regions and vertical in others. A model that can learn step functions, thresholds, and sharp bends will systematically outperform one limited to straight lines. Tree-based models and neural networks can learn these shapes directly from the data, without requiring the analyst to specify the functional form in advance.

This chapter covers two families of nonlinear models: **gradient-boosted trees (GBT)** — the workhorse of modern tabular data modelling — and **neural networks** — powerful but often unnecessarily complex for this problem. We then extend both to **quantile regression**, which produces probabilistic forecasts rather than single point predictions.

Decision Trees: The Building Block

Before we can understand gradient boosting, we need to understand its fundamental building block: the **decision tree**.

Decision tree: A model that makes predictions by asking a sequence of yes/no questions about the input features. Each question splits the data into two groups, and the process repeats until the groups are small enough to make a prediction. The final prediction for each group is typically the average of all observations that land in that group. The name comes from the tree-like branching structure of these sequential decisions.

Think of a decision tree like a game of twenty questions. The tree asks: "Is net load above 8,000 MW?" If yes, it asks: "Is the hour after 5 PM?" If yes again, it predicts a high price. If no, it predicts a moderate price. Each question divides the data into more specific groups, and each group gets its own prediction.

How a Decision Tree Learns

A decision tree is built (or "grown") by a simple recursive algorithm:

Start with all the data. Consider every possible feature and every possible split point for that feature. For example: "net load > 7,500 MW" or "hour > 14" or "temperature > 35°C."

Choose the best split. The "best" split is the one that produces the biggest reduction in prediction error. Formally, we minimise the **sum of squared errors** within each of the two groups created by the split. The split that creates the most internally homogeneous groups wins.

Repeat recursively. Apply the same process to each of the two groups, creating further subdivisions. Each subdivision creates more specific regions with more accurate predictions.

Stop when a criterion is met. Common stopping criteria include: maximum tree depth (how many levels of questions), minimum number of observations in a leaf (final group), or when further splitting does not improve predictions enough to justify the added complexity.

Predict the mean. Once the tree is fully grown, each leaf (terminal node) predicts the mean of the target variable for all training observations that land in that leaf.

Leaf (terminal node): The final node of a decision tree branch, where a prediction is made. Every input observation eventually reaches exactly one leaf by following the yes/no questions down the tree. The leaf's prediction is the average target value of all training observations that reached that

leaf.

Split point: A threshold value for a feature that divides data into two groups. For example, "net load > 8,000 MW" is a split point that sends observations with net load above 8,000 to one branch and observations at or below 8,000 to another. The algorithm searches over all possible features and all possible threshold values to find the optimal split at each step.

Why Trees Are Natural for Price Forecasting

Decision trees have properties that make them particularly well-suited to electricity price forecasting:

Nonlinearity by construction. The hockey-stick relationship is literally a tree with one split on net load — prices are flat below the threshold and high above it. No feature engineering needed.

Automatic interaction detection. "If net load is high AND the hour is 6 PM AND temperature exceeds 35°C, predict a very high price" emerges naturally from nested splits. A linear model would need you to manually create the feature "net_load × hour × temperature."

Robustness to outliers. Because trees make predictions by averaging within regions, a single extreme observation in one leaf does not affect predictions in other leaves. By contrast, a single extreme price can dramatically shift the coefficients of a linear model.

No scaling needed. Trees only care about the *rank order* of feature values, not their magnitude. Whether temperature is in Celsius or Fahrenheit, the same splits are chosen. Linear models are sensitive to feature scaling.

The Problem with Single Trees

Despite their appealing properties, single decision trees have a critical weakness: **high variance**. Small changes in the training data produce very different trees. Add a few observations, remove a few others, and the tree might choose entirely different features and split points — producing wildly different predictions.

Variance (in model terms): The sensitivity of a model's predictions to changes in the training data. A high-variance model gives very different predictions when trained on slightly different datasets. A low-variance model gives similar predictions regardless of which specific observations are in the training set. Variance is one side of the **bias-variance tradeoff**: reducing variance typically increases bias (the model's inability to capture the true relationship), and vice versa.

Bias-variance tradeoff: A fundamental concept in machine learning. **Bias** is the error from simplifying assumptions — a linear model has high bias when the true relationship is nonlinear. **Variance** is the error from sensitivity to training data — a deep decision tree has high variance because it fits noise. The total error is approximately $\text{bias}^2 + \text{variance}$. The goal is to find the model complexity that minimises the total, not either component alone.

This is where **ensemble methods** come in — techniques that combine many trees to reduce variance while preserving the low bias of nonlinear models.

Gradient Boosting: From Weak to Strong

The Ensemble Idea

Ensemble method: A technique that combines multiple "weak" models into a single "strong" model. The idea is that individual models make different errors, and averaging or combining their predictions cancels out many of these errors. The two main ensemble strategies are **bagging** (training many models in parallel on random subsets of the data, then averaging — e.g., Random Forest) and **boosting** (training models sequentially, each correcting the errors of its predecessor).

Gradient boosting is the most powerful ensemble method for tabular data. The name reflects two ideas: "gradient" refers to the use of gradient descent to minimise the loss function, and "boosting" refers to the sequential improvement of predictions.

How Gradient Boosting Works

The core idea is simple: instead of building one big tree, build many small trees, each one correcting the mistakes of all the trees before it. Here is the process, step by step:

Start with a simple prediction. Initialise the model with a constant prediction — typically the mean of the training target. This is the "zeroth" model: $y^{*0} = \text{mean}(y)$.

Compute the residuals. Calculate the difference between the actual values and the current predictions: $r^1_i = y_i - y^{*0}$. These residuals represent what the current model gets wrong.

Fit a small tree to the residuals. Train a shallow decision tree (typically 3–8 levels deep) to predict the residuals, not the original target. This tree learns the *patterns in the errors* — where and how the current model is wrong.

Update the predictions. Add the new tree's predictions to the current model, scaled by a small number called the **learning rate**: $y^{*1} = y^{*0} + \eta \cdot f^1(x)$, where η is typically 0.01 to 0.1.

Repeat. Compute new residuals $r_i^2 = y_i - y_i^{*1}$, fit another tree, update the predictions. Each iteration reduces the remaining error.

Continue for B rounds. After B trees, the final prediction is the sum of all trees' contributions:

$$y_i^* = y_i^{*0} + \eta \cdot f^1(x) + \eta \cdot f^2(x) + \dots + \eta \cdot f^B(x) = y_i^{*0} + \eta \cdot \sum_{b=1}^B f^b(x)$$

Learning rate (η): A small positive number (typically 0.01–0.1) that controls how much each new tree contributes to the ensemble. A smaller learning rate means each tree makes a smaller correction, requiring more trees to reach the same level of accuracy — but the final model generalises better because it takes smaller, more careful steps. Think of it like learning a skill: taking many small practice steps produces more reliable mastery than a few large leaps.

Residual: The difference between the actual value and the model's current prediction: $r_i = y_i - y_i^*$. In gradient boosting, each successive tree is trained to predict the residuals of the current ensemble — it learns to correct the errors that the existing trees have not yet captured.

Intuitive example — correcting an essay: Imagine you write a first draft of an essay. A colleague reads it and marks the errors (residuals). You fix only those errors, producing draft 2. A different colleague reads draft 2 and marks its remaining errors (smaller now). You fix those, producing draft 3. After ten rounds of editing, the essay is much better than any single editor could have made it — each editor focused on what was still wrong after all previous fixes. Gradient boosting works the same way: each tree is an "editor" that focuses on what the previous trees got wrong. The final prediction is the original draft plus all the corrections.

Why boosting beats single trees: Each individual tree in a boosted ensemble is deliberately kept small and weak — it can only learn simple patterns. But the sequential error-correction means the ensemble as a whole can learn arbitrarily complex patterns. The variance problem of single trees is solved by the small tree size (each tree is too simple to overfit much), while the bias problem is solved by having many trees that collectively capture the full complexity of the data.

The "Gradient" in Gradient Boosting

The word "gradient" appears because the algorithm is actually performing **gradient descent** in function space. For squared error, the negative gradient of the loss function with respect to the prediction is simply the residual — which is why fitting to residuals works. But gradient boosting generalises to any differentiable loss function: instead of fitting to residuals, each tree fits to the negative gradient of the chosen loss function. This is what enables quantile regression (discussed later), where the loss function is the asymmetric pinball loss rather than squared error.

Gradient descent: An optimisation algorithm that iteratively adjusts parameters in the direction that reduces a loss function most steeply. Imagine standing on a hilly landscape in fog — you cannot see the bottom of the valley, but you can feel the slope under your feet. Gradient descent says: take a step downhill (in the steepest direction), reassess, repeat. Each step gets you closer to the lowest point. The "gradient" is the mathematical description of the slope.

LightGBM: The Industry Standard

Why LightGBM?

Several software implementations of gradient boosting exist (XGBoost, CatBoost, LightGBM, scikit-learn's GradientBoostingRegressor), but **LightGBM** (Light Gradient Boosting Machine) has become the dominant choice for tabular data modelling, including electricity price forecasting. Developed by Microsoft Research, it is faster, more memory-efficient, and often more accurate than its predecessors.

LightGBM: A highly optimised gradient boosting framework developed by Microsoft. It builds decision tree ensembles using innovations (histogram-based splitting, leaf-wise growth, native categorical support) that make it significantly faster than earlier implementations while maintaining or improving accuracy. It is the standard tool for tabular machine learning in industry and forecasting competitions.

Key Innovations

LightGBM's speed and accuracy come from three main innovations over earlier implementations like XGBoost:

Histogram-based splitting. When deciding where to split a feature, a naive implementation would test every unique value of the feature — which could be millions of values for continuous features like price or temperature. LightGBM instead **bins** continuous features into a fixed number of discrete buckets (typically 255), reducing the number of candidate splits from millions to hundreds. The accuracy loss from binning is negligible, but the speed improvement is dramatic — often 10× or more.

Histogram binning: The process of dividing a continuous variable into a fixed number of discrete buckets (bins). For example, if temperature ranges from 0°C to 45°C and we use 255 bins, each bin covers approximately 0.18°C. The tree only considers splitting at bin boundaries, not at every unique temperature. This dramatically reduces computation with minimal loss of information.

Leaf-wise growth. Traditional gradient boosting grows trees **level-by-level** (breadth-first): all nodes at depth 1 are split before any node at depth 2. This produces balanced trees where every branch has the same depth. LightGBM instead grows **leaf-wise** (best-first): at each step, it splits the leaf with the highest potential loss reduction, regardless of its depth. This produces unbalanced trees — some branches go deep where the data is complex, while others stop shallow where the data is simple. Leaf-wise growth typically produces more accurate trees with

fewer leaves.

Native categorical feature support. Many features in price forecasting are **categorical** — hour of day (0–23), day of week (1–7), season (1–4). Traditional approaches convert these to **one-hot encoding** (a separate binary column for each category), which is wasteful and fragments the data. LightGBM handles categorical features directly, finding the optimal partition of categories into two groups at each split. This is more statistically efficient and produces better splits.

One-hot encoding: A representation of a categorical variable using binary (0/1) columns, one per category. "Hour = 14" becomes a vector of 24 numbers, all zero except position 14. This is the standard way to feed categorical data into linear models and some tree implementations. LightGBM avoids this by handling categories natively.

Hyperparameters That Matter

A **hyperparameter** is a setting that you choose before training, rather than something the model learns from the data. LightGBM has many hyperparameters, but only a handful have a material effect on performance for price forecasting.

Hyperparameter: A configuration value set by the analyst before training begins. Unlike model parameters (e.g., tree split points, which are learned from data), hyperparameters control the learning process itself — how many trees to build, how large each tree should be, how fast to learn. Choosing good hyperparameters is called **hyperparameter tuning** and is typically done by trying many combinations and selecting the one that performs best on a validation set.

Parameter	Typical range	What it controls	Effect of increasing
n_estimators	100–2,000	Number of trees in the ensemble	More capacity to learn complex patterns; slower training; risk of overfitting if too large
learning_rate	0.01–0.1	Step size for each tree's contribution	Faster convergence but worse generalisation; fewer trees needed
max_depth	3–8	Maximum depth of each tree	More capacity for interactions and nonlinearity; greater overfitting risk
num_leaves	15–127	Maximum leaves per tree (overrides depth)	Direct control of tree complexity; $2^{(\text{max_depth})}$ is the upper bound
min_child_samples	5–50	Minimum observations required in each leaf	More regularisation; prevents the tree from creating tiny, overfitting leaves
subsample	0.5–1.0	Fraction of training data used for each tree	Lower values add randomness, reducing overfitting (like bagging)
colsample_bytree	0.5–1.0	Fraction of features considered for each tree	Lower values force diversity among trees; prevents any one feature from dominating
reg_lambda	0–10	L2 regularisation on leaf values	Shrinks leaf predictions toward zero; prevents extreme predictions

The most important interaction: n_estimators and learning_rate are tightly coupled. A smaller learning rate requires more trees to achieve the same performance, but the final model almost always generalises better. The rule of thumb: set the learning rate as low as your compute budget allows, then use early stopping (below) to determine the right number of trees.

Early stopping: A technique that monitors model performance on a held-out validation set during training and stops adding trees when performance stops improving. For example, "stop if the validation error has not decreased in the last 50 rounds." This automatically determines the optimal number of trees, preventing both underfitting (too few trees) and overfitting (too many trees). It is the single most important regularisation technique for gradient boosting.

Regularisation: Any technique that prevents a model from fitting the training data too closely. Over-fitting the training data means the model memorises noise and specific quirks of the training period, rather than learning general patterns that apply to future data. Regularisation makes the model "simpler" — trading a small amount of training accuracy for much better generalisation to new data.

Feature Importance

One of the great advantages of tree-based models over neural networks is **interpretability** — we can examine which features the model considers most important for its predictions. LightGBM provides two measures of feature importance:

Split-based importance (frequency): Counts how many times each feature is used as a split variable across all trees in the ensemble. Features used frequently are deemed important. This is the simplest measure but has a significant caveat: when two features are highly correlated (e.g., demand and net load), the algorithm splits on each roughly half the time, making both appear moderately important when together they are critically important.

Gain-based importance: Measures the total reduction in the loss function attributable to splits on each feature. A feature might be used in only a few splits but produce a large improvement each time — this would show high gain importance but low split importance. Gain-based importance is generally more informative because it captures *how much* each feature helps, not just *how often* it is used.

Gradient boosting: each tree fixes the previous trees' mistakes

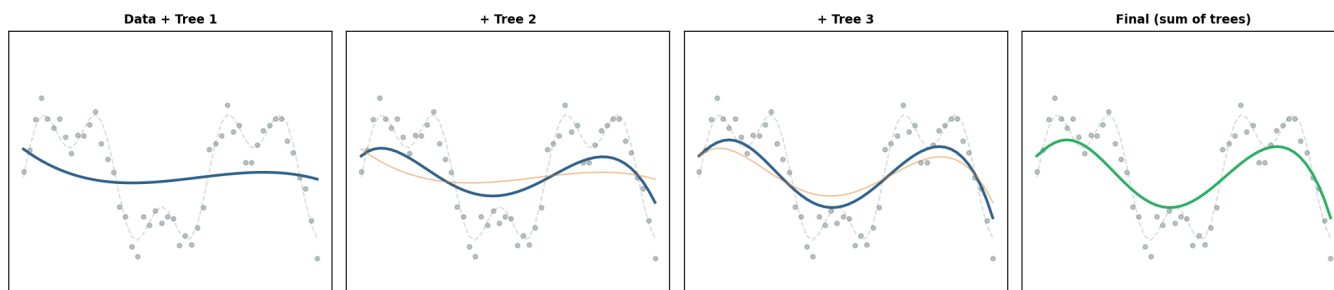


Figure 7.1 — Feature importance from a gradient-boosted tree model for NEM day-ahead price forecasting. Gain-based importance reveals which features contribute most to accurate predictions, providing a sanity check against physical intuition.

For NEM day-ahead electricity price forecasting, GBT feature importance typically reveals the following hierarchy:

1. **Lag-48 price** (yesterday's price at the same half-hour) — dominant feature
2. **Hour of day** — captures the diurnal price pattern
3. **Demand** (or net load) — the fundamental driver of the merit-order stack
4. **Lag-336 price** (same half-hour one week ago) — captures weekly seasonality
5. **Recent volatility** — signals the current price regime (calm vs. volatile)

This ordering is consistent with the stylised facts from Chapter 2, providing a valuable **sanity check**. If your model ranked an obscure feature above lag-48 price, something would likely be wrong — either a data leakage issue or an overfitting artefact.

SHAP values: A more sophisticated feature importance method based on Shapley values from cooperative game theory. Unlike global importance measures, SHAP values explain the contribution of each feature to *each individual prediction*. For example, SHAP can reveal that "for this specific spike event, the high demand feature contributed +\$200 to the prediction while the low wind feature contributed +\$150." SHAP values are additive (they sum to the difference between the prediction and the average) and theoretically grounded.

Real-world example — feature importance as a debugging tool: Suppose you train a GBT model and find that "temperature" is the most important feature — more important than lagged price. This should raise a red flag: temperature affects prices only indirectly (through demand and cooling/heating load). If temperature appears dominant, it likely means you have accidentally included a feature that contains future information (data leakage) — perhaps the actual temperature during the forecast period rather than the forecasted temperature. Feature importance is thus a diagnostic tool for detecting modelling errors.

Quantile Regression: From Point to Probabilistic

Why a Single Number Is Not Enough

Everything we have discussed so far produces a **point forecast** — a single number representing the expected (or most likely) price. But a single number conceals critical information: how confident is the model?

Point forecast: A single-valued prediction — one number per time period. For example, "the price at 6 PM tomorrow will be \$85/MWh." A point forecast gives no indication of the uncertainty around that estimate. It could mean the model is very confident (\$85, almost certainly between \$80 and \$90) or very uncertain (\$85, but it could easily be \$20 or \$500).

Quantile forecast: A set of predictions at multiple quantile levels, representing different points of the predicted distribution. For example, the 10th percentile forecast says "there is a 10% chance the price will be below this value." A set of quantile forecasts (e.g., at 5%, 10%, 25%, 50%, 75%, 90%, 95%) characterises the full shape of the model's uncertainty about the future price.

Consider two scenarios with the same point forecast of \$85/MWh:

- **Scenario A:** The model is confident. The 10th percentile is \$70, the 90th percentile is \$100. The battery should discharge — the price is reliably high no matter what happens.
- **Scenario B:** The model is uncertain. The 10th percentile is \$20, the 90th percentile is \$500. Should the battery discharge now at \$85, or wait for a possible \$500? Or should it be cautious because the price might collapse to \$20?

The point forecast is identical in both scenarios. Only the **quantile forecast** reveals the uncertainty and enables better dispatch decisions. This is why Chapter 8 (probabilistic forecasting) and Chapter 9 (forecast-to-money) are essential complements to this chapter.

Quantile Levels and Prediction Intervals

A quantile forecast produces predictions at multiple **quantile levels** (also called probability levels):

- **q = 0.05** (5th percentile): "There is a 5% chance the price will be below this value." This is the lower extreme — a very pessimistic price estimate.
- **q = 0.10**: "10% chance of being below this value."
- **q = 0.25**: "25% chance of being below — the lower quartile."
- **q = 0.50** (median): "Equally likely to be above or below this value." Not the same as the mean in skewed distributions.
- **q = 0.75**: "75% chance of being below — the upper quartile."
- **q = 0.90**: "90% chance of being below this value."
- **q = 0.95**: "95% chance of being below this value." This is the upper extreme — a very high price estimate.

Prediction interval: A range constructed from two quantile forecasts that contains the actual value with a specified probability. For example, the 80% prediction interval is $[q_{0.1}, q_{0.9}]$ — there is an 80% chance the actual price falls within this range. Wider intervals cover more outcomes but contain less information. The width of prediction intervals measures the model's **uncertainty** — narrow intervals mean the model is confident; wide intervals mean it is uncertain.

The Pinball Loss

How do you train a model to predict a specific quantile rather than the mean? The answer lies in a cleverly designed loss function called the **pinball loss** (also called the check function, tick function, or asymmetric absolute loss).

Pinball loss (check function): A loss function used to train quantile regression models. For a target quantile q , the pinball loss asymmetrically penalises underprediction and overprediction. When predicting the 90th percentile, underprediction (the actual value is above your prediction) is penalised 9 times more heavily than overprediction (the actual value is below your prediction). This asymmetry pushes the prediction upward toward the 90th percentile of the data distribution.

The pinball loss for quantile level q is:

$$L_q(y, y^*) = q \cdot \max(y - y^*, 0) + (1 - q) \cdot \max(y^* - y, 0)$$

Let us unpack this with concrete examples:

For q = 0.5 (median): The penalty for underprediction is $0.5 \times |\text{error}|$ and for overprediction is also $0.5 \times |\text{error}|$. These are equal — the loss treats under- and over-prediction symmetrically. This is just the **mean absolute error (MAE)**. Minimising it yields the median of the distribution.

For q = 0.9 (90th percentile): The penalty for underprediction ($y > y^*$) is $0.9 \times |\text{error}| = 9$ parts, while overprediction ($y^* > y$) is penalised at $0.1 \times |\text{error}| = 1$ part. Underprediction is penalised **9 times** more heavily. The model is strongly incentivised to predict high — it would rather overpredict nine times than underpredict once. This pushes the prediction toward the 90th percentile.

For q = 0.1 (10th percentile): Underprediction is penalised at 0.1 and overprediction at 0.9. The model is incentivised to predict low, yielding the 10th percentile.

Elegant mathematical property: The value of y^* that minimises the expected pinball loss at level q is *exactly* the q -th quantile of the conditional distribution of y given x . No distributional assumptions are needed — the optimisation finds the quantiles directly from the data. This makes quantile regression completely nonparametric: it works regardless of whether the price distribution is normal, skewed, heavy-tailed, or any other shape.

GBT Quantile Regression

LightGBM natively supports quantile regression by simply replacing the squared-error loss function with the pinball loss. The tree-building algorithm remains the same — the only change is what each tree is trying to predict (the negative gradient of the pinball loss rather than the residual).

To produce a full quantile forecast, you train **one separate GBT model for each quantile level**:

1. Train model₁ with $q = 0.05$ (pinball loss, $\alpha = 0.05$)
2. Train model₂ with $q = 0.10$
3. Train model₃ with $q = 0.25$
4. Train model₄ with $q = 0.50$ (median)
5. Train model₅ with $q = 0.75$
6. Train model₆ with $q = 0.90$
7. Train model₇ with $q = 0.95$

Each model uses exactly the same features — the only difference is the loss function. At prediction time, each model produces its quantile estimate, and together they form a **quantile fan** — a set of nested prediction intervals that widen as the coverage level increases.

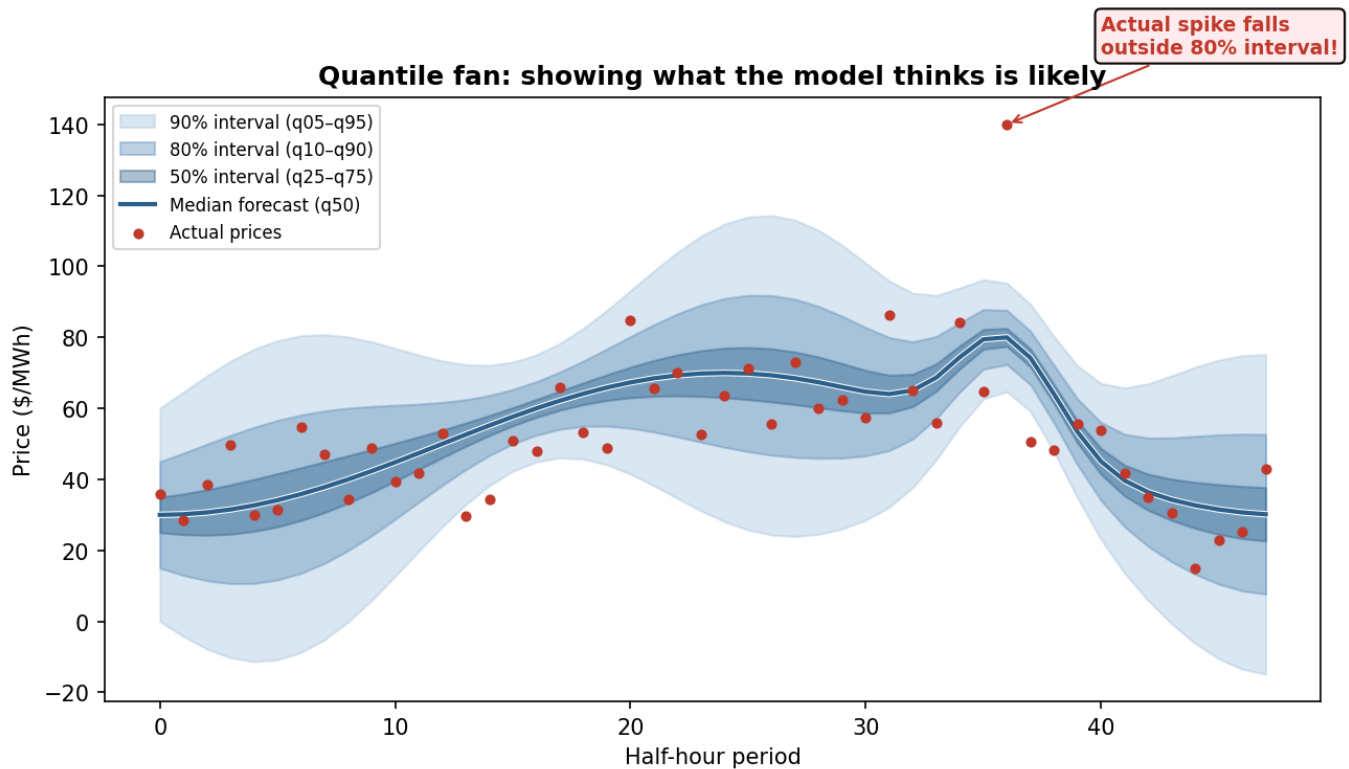


Figure 7.2 — A quantile fan produced by GBT quantile regression. The shaded bands represent prediction intervals at different coverage levels (50%, 80%, 90%, 95%). The fan widens during volatile periods (high uncertainty) and narrows during quiet periods (low uncertainty), providing a visual representation of the model's confidence over time.

Real-world example — the quantile fan in action: At 3 AM on a mild autumn night, the quantile fan is narrow: $q_{0_1_0} = \$35$, median = \$45, $q_{0_9_0} = \$55$. The model is confident — prices are predictably low during off-peak hours with mild weather. At 6 PM on a hot summer day with bushfire warnings, the fan explodes: $q_{0_1_0} = \$50$, median = \$200, $q_{0_9_0} = \$5,000$. The model knows prices will be high but cannot pin down exactly how high. A battery operator seeing this wide fan would be cautious about discharging early — there might be even higher prices coming, or the price might collapse if temperatures drop faster than expected.

The Quantile Crossing Problem

A subtle but important issue with independent quantile regression: because each quantile model is trained separately, their predictions may **cross** — the predicted 90th percentile might end up below the predicted 50th percentile for some observations. This is physically meaningless: it is impossible for the 90th percentile of a distribution to be below the median.

Quantile crossing: A situation where independently predicted quantiles violate their natural ordering — for example, the predicted 10th percentile exceeds the predicted 50th percentile. This occurs because each quantile model is optimised independently and has no constraint forcing consistency with other quantile models. While the individual quantile predictions may each be reasonable, they can be inconsistent with each other.

Quantile crossing occurs because each model is optimised independently — model₁ ($q = 0.10$) has no knowledge of model₄'s ($q = 0.50$) predictions. In practice, crossing is rare with well-trained GBT models on electricity price data, but it does occur, especially during unusual market conditions or at the extremes of the feature space.

Three common solutions:

Post-hoc sorting. After prediction, simply sort the quantiles at each time step so they are in the correct order. This is fast, simple, and effective. The downside is that sorting destroys the individual optimality of each quantile prediction — the sorted values no longer minimise their respective pinball losses.

Isotonic regression. Fit an isotonic (non-decreasing) regression across the quantile levels for each time step. This is a more principled version of sorting that finds the closest non-crossing quantile set. It is "minimally invasive" — it adjusts the quantiles as little as possible to remove crossings.

Joint quantile models. Use a neural network architecture that outputs all quantiles simultaneously, with an architectural constraint (monotonic output layer) that guarantees non-crossing by construction. This is more complex but eliminates the problem at its source.

Isotonic regression: A regression method that produces predictions constrained to be non-decreasing (or non-increasing). Given a set of data points, it finds the best-fitting monotone function — the one that minimises squared error subject to the monotonicity constraint. In the context of quantile crossing, isotonic regression ensures that quantile predictions increase with the quantile level.

Neural Networks for Price Forecasting

What Is a Neural Network?

A **neural network** is a class of models loosely inspired by the structure of biological neurons, consisting of layers of interconnected processing units. While gradient-boosted trees partition the feature space into rectangular regions, neural networks learn smooth, continuous functions through layers of simple nonlinear transformations.

Neural network: A model composed of layers of interconnected units (neurons). Each neuron computes a weighted sum of its inputs, adds a bias, and applies a nonlinear function (activation function). By stacking many neurons in multiple layers, neural networks can approximate arbitrarily complex functions. They learn by adjusting their weights to minimise a loss function, using an algorithm called **backpropagation**.

Activation function: A nonlinear function applied to the output of each neuron. Without activation functions, a multi-layer neural network would collapse to a single linear model (a stack of linear transformations is still linear). Common choices include ReLU (Rectified Linear Unit: $\max(0, x)$), sigmoid ($1/(1+e^{-x})$), and tanh. The activation function is what gives neural networks their ability to learn nonlinear relationships.

Backpropagation: The algorithm used to train neural networks. It computes how much each weight in the network contributed to the overall prediction error, then adjusts all weights simultaneously in the direction that reduces the error. The name comes from the fact that error gradients are propagated backward through the network, from the output layer to the input layer. Combined with gradient descent, backpropagation enables neural networks to learn from data.

Architecture Choices

Several neural network architectures have been applied to electricity price forecasting. Each has different strengths:

Multilayer Perceptron (MLP). The simplest architecture — fully connected layers with nonlinear activations. The input is a fixed-length feature vector (the same features as LEAR or GBT), and the output is 48 half-hourly price predictions. MLPs are suitable when features are well-engineered and the temporal structure is captured by lagged features (lag-48, lag-336) rather than by sequential processing. They are fast to train and easy to implement but have no special ability to handle sequences.

Multilayer Perceptron (MLP): A neural network consisting of an input layer, one or more hidden layers, and an output layer, where every neuron in each layer is connected to every neuron in the next layer (hence "fully connected"). It is the simplest and most general neural network architecture. Given enough hidden neurons, an MLP can theoretically approximate any continuous function — but it has no built-in understanding of sequences, spatial structure, or other data characteristics.

Recurrent Neural Networks (RNN/LSTM/GRU). These architectures process sequential data by maintaining a **hidden state** — an internal memory that carries information from earlier time steps to later ones. At each time step, the network reads the current input and updates its hidden state, allowing it to capture temporal dependencies without explicit lag features. However, RNNs are slow to train (each step depends on the previous one, preventing parallelisation), prone to gradient issues (the vanishing gradient problem makes it hard to learn long-range dependencies), and rarely outperform GBT with well-engineered features for day-ahead price forecasting.

Recurrent Neural Network (RNN): A neural network designed for sequential data, where the output at each time step depends on both the current input and the network's internal state (hidden state) from the previous time step. This creates a form of memory — the network can, in principle, use information from arbitrarily far in the past. LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) are improved variants that solve the **vanishing gradient problem** — the tendency of simple RNNs to "forget" information from more than a few time steps ago.

Vanishing gradient problem: A training difficulty in deep neural networks (and especially RNNs) where the gradients used for learning become extremely small as they are propagated backward through many layers or time steps. When gradients vanish, earlier layers or time steps learn extremely slowly or not at all — the network effectively cannot learn long-range dependencies. LSTM and GRU networks use gating mechanisms to mitigate this problem.

Convolutional Neural Networks (1D CNN). Originally designed for images, 1D CNNs apply learnable filters to sequential data. They can efficiently detect local patterns — price ramps (rapid price increases over a few intervals), spike signatures (specific patterns that precede spikes), and recurring intra-day shapes. CNNs are faster to train than RNNs because they can process all time steps in parallel. They are often used in hybrid architectures combining CNN layers (for local pattern detection) with fully connected layers (for prediction).

Convolutional Neural Network (1D CNN): A neural network that applies small, learnable filters (kernels) to segments of a sequential input, sliding the filter across the sequence to detect local patterns. Each filter learns to recognise a specific pattern (e.g., "price increasing for three consecutive intervals") wherever it appears. The same filter is applied to every position in the sequence, making CNNs parameter-efficient and good at detecting patterns regardless of where they occur.

Transformers. The most recent architecture, based on **self-attention** mechanisms that allow every time step to attend to every other time step — capturing long-range dependencies without the sequential bottleneck of RNNs. Transformers have revolutionised natural language processing (they are the "T" in GPT) but are computationally expensive for time series problems where GBT already works well. For day-ahead electricity price forecasting with well-engineered tabular features, the attention overhead is rarely justified — the problem is not complex enough to warrant the additional computational cost.

Transformer: A neural network architecture based on **self-attention** — a mechanism that allows each element in a sequence to compute a weighted combination of all other elements, with the weights learned from data. Unlike RNNs, Transformers process all positions in parallel, making them much faster to train. The self-attention mechanism can capture dependencies between any two positions regardless of distance, without the vanishing gradient problem. However, their computational cost grows quadratically with sequence length.

When Neural Networks Add Value (and When They Don't)

For day-ahead electricity price forecasting with well-engineered features, GBT typically matches or outperforms neural networks. This may seem counterintuitive — are neural networks not "more powerful"? The key insight is that **power matters only when the data is complex enough to warrant it**.

NEM day-ahead price forecasting is fundamentally a tabular problem with well-understood feature engineering. The dominant features (lagged prices, calendar variables, demand, weather) are already in a format that GBT can exploit efficiently. Neural networks add value in specific circumstances:

When neural networks help:

Raw sequential data. If you feed in raw price and demand sequences (hundreds or thousands of time steps) rather than engineered features, RNNs or Transformers can potentially learn relevant patterns. But this is rarely better than expert feature engineering — and the engineer knows things (e.g., "lag-48 is more important than lag-47") that the network must discover from scratch.

Very large datasets. Neural networks scale better to enormous datasets — their performance keeps improving with more data, while GBT's performance tends to plateau. For NEM data (typically 4–5 years $\times$ 17,520 intervals/year $\approx$ 80,000 observations), this advantage does not materialise.

Multi-output prediction. Predicting all 48 half-hours simultaneously with shared parameters can capture cross-period dependencies (e.g., if the price at 5 PM is high, the price at 6 PM is also likely high). GBT typically treats each period independently — 48 separate models with no shared learning.

Non-crossing quantiles. Neural networks can enforce non-crossing quantiles architecturally through monotonic output layers, eliminating the crossing problem without post-hoc correction.

When GBT is sufficient (most of the time):

- Feature engineering is mature (lagged prices, calendar, demand, weather)
- Training data is moderate (1–5 years)
- Interpretability matters (feature importance, SHAP values are straightforward for trees)
- Compute budget is limited (GBT trains in seconds; neural networks take minutes to hours)
- You need a reliable, low-maintenance production model

Practical guidance: Start with LightGBM. It trains in seconds, produces excellent point and quantile forecasts, provides interpretable feature importance, and requires minimal tuning beyond early stopping. Only invest in neural networks if you have a specific reason — raw sequential data, multi-task learning, or architectural constraints like non-crossing quantiles. The incremental accuracy of neural networks rarely justifies the additional complexity, training time, hyperparameter sensitivity, and maintenance burden for NEM day-ahead forecasting.

Training Neural Networks for Quantile Forecasting

Neural networks trained with the pinball loss produce quantile forecasts directly. The network output layer has one neuron per quantile per forecast period — for example, 48 periods $\times$ 9 quantiles = 432 output neurons. The loss function is the average pinball loss across all outputs and all quantile levels.

Key training considerations:

Batch normalisation. A technique that normalises the inputs to each layer during training, stabilising and accelerating learning. It helps with the scale differences between features (some in the range -5 to 10 in arcsinh space, others in the range 0 to $50,000$ in megawatts).

Batch normalisation: A technique that normalises each layer's inputs by subtracting the batch mean and dividing by the batch standard deviation. This keeps the internal values of the network on a stable scale throughout training, preventing the "internal covariate shift" problem where the distribution of inputs to each layer keeps changing as earlier layers' weights are updated.

Dropout. A regularisation technique that randomly sets a fraction of neuron outputs to zero during training. This prevents neurons from co-adapting — relying too heavily on specific other neurons — and forces the network to learn redundant representations. Typical dropout rates for price forecasting: 0.1 – 0.3 (10–30% of neurons dropped at each training step).

Dropout: A regularisation technique where, during each training step, each neuron is randomly "dropped" (its output set to zero) with a specified probability. This simulates training a different sub-network at each step, forcing the network to learn robust features that work even when some neurons are missing. At prediction time, all neurons are active but their outputs are scaled down to compensate. Dropout is the neural network equivalent of feature subsampling in gradient boosting.

Learning rate scheduling. Start with a moderate learning rate (typically 1×10^{-3}) and reduce it when the validation loss stops improving. The pinball loss landscape is piecewise linear (it has sharp corners at the predicted values), which can cause oscillation with large learning rates. Reducing the learning rate over time allows the optimiser to settle into a good solution.

Early stopping. Monitor the validation loss and stop training when it stops improving for a specified number of epochs (the "patience" parameter). This is the single most important regularisation technique for neural networks on moderate-sized datasets like NEM data. Without early stopping, the network will eventually memorise the training data — its training loss will continue to decrease while its test loss starts increasing.

Epoch: One complete pass through the entire training dataset. If the training set has 60,000 observations and the batch size is 64, one epoch consists of approximately 938 training steps (60,000 / 64). Training for 100 epochs means the network sees each training observation 100 times. Neural networks typically require tens to hundreds of epochs to converge.

Learning Curves and Model Selection

The Training Size Effect

A **learning curve** plots model performance as a function of training set size. It answers a critical practical question: "Do I have enough data for this model?"

Learning curve: A plot showing model performance (y-axis) as a function of the number of training observations (x-axis). The gap between training performance and validation performance reveals whether the model is underfitting (both lines are poor — more complexity needed) or overfitting (training is excellent but validation is poor — more data or more regularisation needed). Learning curves are essential for diagnosing model issues and determining whether collecting more data would help.

For electricity price forecasting, the learning curve reveals important differences between model families:

- Small training sets (< 6 months, ~8,600 intervals):** All models perform poorly because the training data does not contain a full seasonal cycle. Simple models (naive, AR) actually outperform complex ones because they have fewer parameters to estimate from limited data — a manifestation of the bias-variance tradeoff.
- Medium training sets (6–18 months):** LEAR catches up with and surpasses the naive. GBT begins to outperform LEAR as it has enough data to reliably learn nonlinear patterns.
- Large training sets (2+ years):** Performance gains flatten. Additional data helps less because the electricity market structure is slowly changing — older data becomes less relevant (generators retire, new capacity is built, demand patterns shift). This is the concept of **non-stationarity**: the statistical properties of the data change over time.

Non-stationarity: The property of a time series whose statistical characteristics (mean, variance, correlations) change over time. The NEM is non-stationary because the generation mix evolves (coal retires, renewables grow), demand patterns shift (rooftop solar changes the demand shape), and market rules change. Models trained on old data may not accurately reflect current market dynamics, which is why the most recent 1–3 years of data are typically most valuable.

The learning curve reveals the **sample efficiency** of each model — how much data each model needs to reach its potential:

Model	Sample efficiency	Explanation
Naive	N/A (doesn't learn)	Performance is flat regardless of data size
LEAR	High (few parameters)	Reaches near-optimal with ~12 months of data
GBT	Moderate (many parameters)	Continues improving up to ~2–3 years
Neural network	Low (most parameters)	Needs the most data; may not plateau within 4–5 years

Overfitting: The Central Risk

Overfitting is the most common failure mode in machine learning and deserves careful attention. An overfitting model has memorised the training data's specific quirks — including noise, outliers, and historical accidents — rather than learning the general patterns that will persist into the future.

Overfitting: A condition where a model performs excellently on training data but poorly on new, unseen data. The model has learned patterns specific to the training set that do not generalise — it has memorised rather than learned. Classic symptom: training error is much lower than test error. Overfitting is more likely with complex models (deep trees, large neural networks), small datasets, and many features.

Signs that a model is overfitting:

- Training MAE is much lower than test MAE.** If the model achieves MAE = \$5 on training data but MAE = \$30 on test data, it has memorised training patterns that do not generalise. A healthy gap is small — perhaps MAE = \$20 on training and MAE = \$25 on test.
- Performance degrades when features are added.** Adding features should help a well-regularised model. If adding weather features *hurts* test performance, the model is fitting noise in those features — the additional complexity is exploited to memorise training data more closely.

Spike predictions are too confident. The model predicts spikes that do not occur, because it has memorised specific conditions that preceded spikes in the training data. In a future period, similar conditions do not produce a spike (perhaps because a different generator is online), and the model's false spike predictions reduce its accuracy.

High variance across rolling-origin folds. If the model's MAE ranges from \$18 to \$45 across different test windows, it is highly sensitive to the specific training period — a hallmark of overfitting to particular market regimes.

Regularisation Across Model Families

Each model family has its own techniques for controlling overfitting. The following table compares the main regularisation approaches:

Technique	LEAR	GBT	Neural network
L1 penalty (drives weights to zero)	Primary tool (LASSO)	Indirect via <code>min_child_samples</code>	Rarely used
L2 penalty (keeps weights small)	Secondary (ElasticNet)	<code>reg_lambda</code> parameter	Weight decay
Feature subsampling	Not applicable	<code>colsample_bytree</code> (random feature subset per tree)	Dropout (random neuron subset per step)
Early stopping	Not needed (closed-form solution)	<code>early_stopping_rounds</code> (stop when validation loss plateaus)	patience (same concept)
Ensemble averaging	Not applicable	Built-in (the ensemble IS the model)	Can average multiple independently trained networks
Data augmentation	Not common	Subsampling (<code>subsample</code> parameter)	Adding noise to inputs

The regularisation principle: The best model is not the one that fits the training data most closely — it is the one that generalises best to unseen data. Every regularisation technique trades a small amount of training accuracy for improved test accuracy. The art of machine learning is finding the right amount of regularisation: too little and the model overfits; too much and the model underfits (it is too simple to capture the true patterns).

GBT vs Neural Network: A Practical Decision Framework

For NEM day-ahead price forecasting, the choice between GBT and neural networks is usually straightforward. Here is a decision framework:

Choose GBT (LightGBM) when:

- Your features are well-engineered tabular data (lagged prices, calendar variables, demand, weather)
- Training data spans 1–5 years (tens of thousands of observations)
- Interpretability matters for stakeholders (feature importance is trivially available)
- Compute budget is limited (GBT trains in seconds on a laptop; neural networks may need minutes to hours)
- You need a reliable, maintainable production system (GBT has fewer failure modes)
- You are a small team without deep learning expertise

Choose neural networks when:

- You are exploring end-to-end learning from raw time series (no hand-crafted features)
- Multi-task learning is beneficial (jointly predicting price, demand, and renewable generation)
- Non-crossing quantile constraints are critical for your downstream application
- You have significant compute budget and hyperparameter tuning infrastructure
- Your dataset is very large (millions of observations across many markets)
- Research novelty matters (neural network architectures offer more room for innovation)

Real-world industry context: In the electricity price forecasting community, LightGBM has become the de facto standard for practitioners and competition winners alike. The Global Energy Forecasting Competitions (GEFCom2014, GEFCom2017) were dominated by gradient-boosted tree models, often combined with linear models via QRA (Chapter 8). While academic papers frequently propose novel neural architectures, rigorous comparisons (same data, same evaluation, same computational budget) rarely show consistent advantages over well-tuned GBT. The practical conclusion: master LightGBM first, and explore neural networks only when you have a clear hypothesis about what they will add.

Glossary

Term	Definition
Linear model	A model that predicts output as a weighted sum of inputs — can only represent straight-line relationships
Nonlinear model	A model that can learn curved, threshold, or discontinuous relationships

Decision tree	A model that makes predictions via a sequence of yes/no questions (splits) on features
Leaf (terminal node)	The endpoint of a decision tree branch where a prediction is made
Split point	A feature-value threshold that divides data into two groups at a tree node
Ensemble method	A technique combining multiple weak models into a single stronger model
Gradient boosting	An ensemble method that sequentially builds trees, each correcting the errors of the previous ones
Learning rate (η)	A small multiplier controlling how much each new tree contributes to the ensemble
Residual	The difference between the actual value and the current prediction; what the next tree tries to correct
LightGBM	A fast, accurate gradient boosting implementation by Microsoft; the industry standard for tabular data
Histogram binning	Discretising continuous features into buckets to speed up split-finding in trees
Hyperparameter	A setting chosen before training (number of trees, learning rate) vs. parameters learned from data
Early stopping	Stopping training when validation performance plateaus; prevents overfitting
Regularisation	Any technique that prevents a model from fitting noise in the training data
Overfitting	When a model memorises training data specifics instead of learning generalisable patterns
Bias-variance tradeoff	The tension between model simplicity (high bias) and sensitivity to data (high variance)
Feature importance	A measure of how much each feature contributes to a tree model's predictions
SHAP values	Game-theory-based feature attribution that explains individual predictions
Point forecast	A single-valued prediction with no uncertainty information
Quantile forecast	Predictions at multiple probability levels, characterising the forecast distribution
Prediction interval	A range (e.g., 80%) expected to contain the actual value with a specified probability
Pinball loss	The asymmetric loss function used to train quantile regression models
Quantile crossing	When independently predicted quantiles violate their natural ordering
Isotonic regression	A monotone regression method; used to fix quantile crossing
MLP	Multilayer Perceptron — the simplest fully connected neural network
RNN / LSTM / GRU	Recurrent architectures that process sequential data via internal memory
Transformer	An architecture using self-attention to capture long-range dependencies in parallel
Activation function	A nonlinear function (ReLU, sigmoid) applied to neuron outputs; enables nonlinear learning
Backpropagation	The algorithm for computing gradients and updating weights in neural networks
Dropout	Regularisation that randomly deactivates neurons during training
Batch normalisation	A technique that normalises layer inputs to stabilise and accelerate training
Epoch	One complete pass through the entire training dataset
Learning curve	A plot of performance vs. training set size; reveals data sufficiency and overfitting
Non-stationarity	The property of a time series whose statistics change over time
Sample efficiency	How much data a model needs to reach its potential performance

Summary

Gradient-boosted trees (GBT) capture the nonlinear hockey-stick relationship that linear models fundamentally cannot represent. The algorithm builds an ensemble of small decision trees sequentially, each correcting the errors of its predecessors, producing a model that is both powerful and robust. LightGBM is the standard implementation, with histogram-based splitting, leaf-wise growth, and native categorical support making it fast and accurate. Key hyperparameters (learning rate, number of trees, tree depth) control the complexity of the model, and early stopping automatically finds the right balance between underfitting and overfitting. Feature importance from GBT confirms the physical intuition from earlier chapters: lagged prices and hour of day dominate, with demand and weather adding incremental value. Quantile regression via the pinball loss extends GBT from point forecasts to probabilistic forecasts — prediction intervals that quantify the model's uncertainty about future prices. Independent quantile models may produce crossing predictions, which are fixed by post-hoc sorting or isotonic regression. Neural networks (MLPs, RNNs, Transformers) are more flexible architectures but rarely outperform well-tuned GBT for day-ahead NEM price forecasting — they add value primarily for raw sequential data, multi-task learning, or architectural constraints like non-crossing quantiles. The learning curve reveals that GBT needs approximately two years of data to reach near-optimal performance, and that the marginal value of additional data diminishes as market non-stationarity makes older observations less relevant.

8. Probabilistic Forecasting and Calibration

Why Probabilistic Forecasts Matter

In Chapter 7, we trained models that produce both point forecasts (a single predicted price) and quantile forecasts (predictions at multiple probability levels). This chapter is about the deeper question: how do we know the probabilistic forecast is trustworthy? And what makes one probabilistic forecast better than another?

The difference between a point forecast and a probabilistic forecast is the difference between being told "it will rain tomorrow" and being told "there is a 70% chance of rain tomorrow, with a 20% chance of heavy rain." The second statement is far more useful for decision-making — it lets you assess the risk, weigh alternatives, and plan accordingly.

Probabilistic forecast: A forecast that provides information about the full range of possible outcomes and their likelihoods, rather than a single predicted value. It answers the question "what might happen and how likely is each outcome?" rather than just "what do we expect to happen?" For electricity prices, a probabilistic forecast might say: "the median price is \$85/MWh, with a 10% chance of being below \$40 and a 10% chance of being above \$300."

For battery dispatch, probabilistic forecasts are not just academically interesting — they are economically essential. Consider the dispatch decision at 4 PM:

Scenario A — Confident forecast: The median price for 6 PM is \$150/MWh, with an 80% interval of [\$120, \$180]. The battery should discharge now — the price is reliably high.

Scenario B — Uncertain forecast: The median is the same \$150/MWh, but the 80% interval is [\$30, \$2,000]. Should the battery discharge now? Perhaps waiting is better — prices might go much higher. Or perhaps discharging now is wise — prices might collapse. The wide interval signals that the market is in an unpredictable state.

Scenario C — Asymmetric forecast: The median is \$150, the 10th percentile is \$120, but the 90th percentile is \$3,000. The upside risk is enormous. A risk-neutral battery might hold its charge, betting on the possibility of a spike.

The point forecast (\$150) is identical in all three scenarios. Only the probabilistic forecast distinguishes them and enables intelligent risk management.

The fundamental argument: A battery operator who treats every \$150 forecast identically — regardless of whether the uncertainty is \$20 or \$2,000 — is leaving money on the table. Probabilistic forecasts convert "I think the price will be \$150" into "I think the price will be \$150, and here is how wrong I might be." The second statement enables the dispatch optimiser (Chapter 9) to make risk-aware decisions.

Forms of Probabilistic Forecasts

Probabilistic forecasts come in several forms, ordered from least to most informative:

1. Prediction intervals. A pair of values defining a range expected to contain the actual outcome with a stated probability. For example, "the 80% prediction interval for the 6 PM price is [\$65, \$320]" means the model believes there is an 80% chance the price will fall between \$65 and \$320. Prediction intervals are the simplest probabilistic forecast — easy to produce, easy to communicate, easy to use in dispatch.

Prediction interval: A range of values that is expected to contain the actual future observation with a specified probability (the **coverage level**). An 80% prediction interval should contain the actual value 80% of the time. Wider intervals have higher coverage but contain less information. The width of the interval reflects the model's **uncertainty** — not its accuracy. A model can be accurate (small average error) but uncertain (wide intervals) or vice versa.

2. Quantile forecasts. A set of predicted values at multiple probability levels (e.g., 5th, 10th, 25th, 50th, 75th, 90th, 95th percentiles). This is richer than a single prediction interval because it describes the shape of the distribution — is it symmetric or skewed? Are the tails thin or fat? Quantile forecasts are what GBT quantile regression (Chapter 7) produces directly.

3. Density forecasts. The full probability density function (PDF) — a curve showing the relative likelihood of every possible price. This is the most informative form but the hardest to produce accurately. In practice, density forecasts are often approximated by interpolating between quantile forecasts, fitting a parametric distribution, or using kernel density estimation.

Probability density function (PDF): A function that describes the relative likelihood of each possible value of a continuous random variable. The area under the PDF between two values gives the probability that the outcome falls in that range. For electricity prices, the PDF at any future time step shows the full distribution of possible prices — where the density is high, the price is more likely; where it is low, the price is unlikely.

4. Scenario forecasts. Multiple complete price trajectories — plausible "stories" of how prices might evolve over the forecast horizon. Each scenario is a possible future, and together they represent the range of outcomes. Scenarios are used directly in stochastic optimisation (Chapter 9), where the dispatch LP is solved across multiple scenarios simultaneously.

Scenario forecast: A collection of complete, plausible future price trajectories generated from the probabilistic forecast. Each scenario represents one possible future — "what might happen if wind drops and demand surges" or "what might happen on a mild, windy day." Scenarios preserve the temporal structure of the forecast (correlations across time periods), unlike quantile forecasts which describe each time period independently.

Quantile Regression Averaging (QRA)

The Forecast Combination Principle

One of the most robust findings in all of forecasting research is the **forecast combination principle**: combining forecasts from multiple models almost always improves accuracy. This result, first documented by Bates and Granger in 1969, holds across virtually every forecasting domain — weather, economics, energy, demographics, sports — and is sometimes called the "forecast combination puzzle" because it works even when one model is clearly better than the others.

Forecast combination principle: The empirical finding that an average (or weighted combination) of forecasts from different models is typically more accurate than any individual forecast. This works because different models make different errors — some overpredict when others underpredict — and combining them cancels out many of these idiosyncratic errors. It is one of the most reliable findings in forecasting research and forms the foundation for ensemble methods in machine learning.

Why does combination work so consistently? Consider three models for electricity price forecasting:

- **LEAR** (linear model) misses nonlinear patterns (understates spikes) but is stable and rarely produces extreme errors
- **GBT** captures nonlinear relationships but may overfit to recent market regimes, producing errors when the regime changes
- **Neural network** can learn complex temporal patterns but is noisy — its predictions vary more from run to run

These models make **different kinds of errors**. When LEAR understates a spike, GBT might predict it correctly. When GBT overfits to a recent pattern that does not persist, LEAR's simpler model may be more appropriate. Averaging the forecasts exploits these complementary strengths: the errors partially cancel, and the combined forecast is more reliable than any individual one.

Real-world example — the power of combination: In the Global Energy Forecasting Competition 2014 (GEFCom2014), the winning entries almost universally used forecast combination. The winning team for the electricity price track combined several model types — LEAR variants, GBT, and neural networks — using QRA. No single model in their ensemble was the best individual entry, but the combination outperformed all individual entries. This pattern repeats in forecasting competitions across domains: the best ensembles beat the best individual models.

The diversity principle: Combination works best when the component models are *diverse* — they make different types of errors. Combining three nearly identical GBT models with slightly different hyperparameters adds little value because they make similar errors. Combining a linear model, a tree model, and a neural network adds substantial value because their error patterns are structurally different.

The QRA Method

Quantile Regression Averaging (QRA), introduced by Nowotarski and Weron (2015), is a technique specifically designed for combining multiple point forecasts into a single probabilistic forecast. It has become the standard combination method in electricity price forecasting.

Quantile Regression Averaging (QRA): A forecast combination method that takes point forecasts from K different models as inputs and produces calibrated quantile forecasts as output. For each quantile level τ , a separate quantile regression is fitted using the point forecasts as explanatory variables. This allows different models to receive different weights at different quantile levels — one model might be best at predicting the median while another is best at predicting the tails.

The method works as follows. Suppose you have K models, each producing a point forecast: $f_1, f_2, \dots, f_K$. For each quantile level τ (e.g., $\tau = 0.05, 0.10, 0.25, 0.50, 0.75, 0.90, 0.95$), fit a quantile regression:

$$Q_\tau(y \mid f_1, f_2, \dots, f_K) = \beta_0\tau + \beta_1\tau \cdot f_1 + \beta_2\tau \cdot f_2 + \dots + \beta_K\tau \cdot f_K$$

where the coefficients $\beta_0\tau, \beta_1\tau, \dots, \beta_K\tau$ are estimated by minimising the pinball loss (Chapter 7) at level τ on a calibration dataset.

The key properties of QRA are:

- 1. Different weights for different quantiles.** The coefficients $\beta_i\tau$ vary with the quantile level τ . A model that excels at predicting price spikes (upper tail) receives high weight at $\tau = 0.90$ or $\tau = 0.95$, even if it is mediocre at predicting the median. Conversely, a model that is accurate on average but poor at predicting extremes receives high weight at $\tau = 0.50$ and low weight in the tails. This is much smarter than simple averaging, which treats all quantile levels equally.
- 2. Automatic combination to probabilistic forecast.** The input is K point forecasts (single numbers). The output is a full set of quantile forecasts — a probabilistic forecast. QRA simultaneously combines the models and produces uncertainty estimates, without requiring the individual models to provide their own uncertainty.
- 3. Built-in calibration.** Because QRA is fitted using the pinball loss, the resulting quantile forecasts are approximately calibrated (the stated coverage matches the actual coverage). This calibration property comes "for free" from the quantile regression fitting process.
- 4. Adaptive reweighting.** When QRA is refitted periodically (e.g., daily or weekly on the most recent data), it automatically adjusts the weights as models improve or deteriorate. If one model degrades due to a market regime change, QRA will downweight it without any manual intervention.

Why Not Simple Averaging?

Simple averaging — $y^* = (f_1 + f_2 + \dots + f_K) / K$ — is a surprisingly strong baseline for forecast combination. It has three limitations that QRA addresses:

It assumes equal weights across quantiles. Simple averaging gives the same weight to each model regardless of whether you care about the median or the tails. QRA allows quantile-specific weights.

It produces only a point forecast. The average of K point forecasts is another point forecast. It gives no uncertainty information. QRA produces a full set of quantiles.

It is static. The weights are always $1/K$, regardless of whether some models have improved or degraded. QRA's weights adapt when refitted.

Despite these limitations, simple averaging should always be included as a benchmark — it is hard to beat consistently, and when QRA's improvements are small, the simpler method may be preferable for its robustness and transparency.

Evaluating Probabilistic Forecasts

Evaluating a probabilistic forecast is fundamentally harder than evaluating a point forecast. For a point forecast, you compare the prediction to the actual value — the error is a single number (e.g., $\text{MAE} = \$25$). For a probabilistic forecast, you must assess whether the *entire predicted distribution* matches the distribution of actual outcomes. This requires two concepts: **calibration** and **sharpness**.

Calibration (Reliability)

Calibration (reliability): A probabilistic forecast is calibrated if the stated probabilities match the observed frequencies. If the model says "there is a 10% chance the price will be below \$X," then approximately 10% of the time, the actual price should indeed be below \$X. Calibration answers the question: "When the model says 90%, does it really happen 90% of the time?"

Calibration is assessed visually using the **reliability diagram** (also called the calibration plot):

1. For each nominal quantile level τ (e.g., 0.05, 0.10, 0.20, ..., 0.90, 0.95):
2. Compute the fraction of actual observations that fall below the predicted τ -quantile
3. Plot this observed fraction (y-axis) against the nominal level τ (x-axis)

A perfectly calibrated model produces points that lie exactly on the 45° diagonal — when the model says "10% chance," it happens 10% of the time; when it says "90%," it happens 90% of the time.

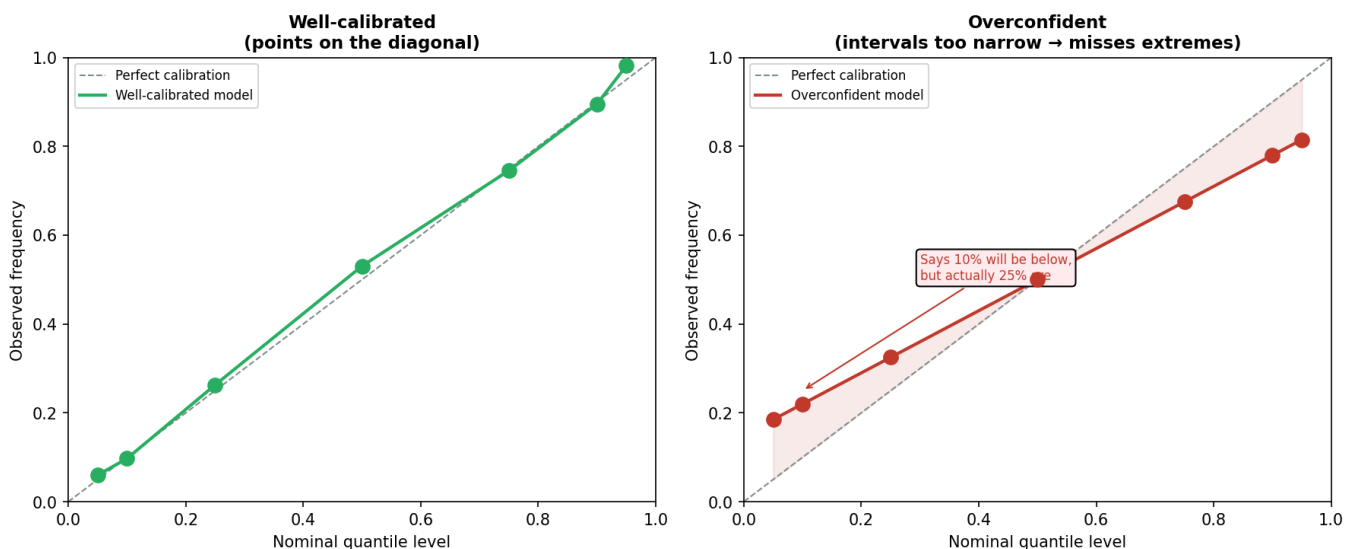


Figure 8.1 — A reliability diagram comparing calibration of different probabilistic forecasting models. The diagonal line represents perfect calibration. Points above the diagonal indicate overconfidence (intervals too narrow); points below indicate underconfidence (intervals too wide). The closer a model's curve follows the diagonal, the more trustworthy its probability statements.

Systematic departures from the diagonal reveal specific types of miscalibration:

Points consistently above the diagonal (observed frequency > nominal level): The model is **overconfident** — its prediction intervals are too narrow. For example, if the 90% interval only covers 75% of observations, the model is overstating its certainty. This is the most dangerous miscalibration for battery dispatch because the operator trusts intervals that are systematically too narrow, leading to unexpected losses during events the model did not anticipate.

Points consistently below the diagonal (observed frequency < nominal level): The model is **underconfident** — its prediction intervals are too wide. The 90% interval covers 98% of observations. While less dangerous than overconfidence, this wastes information — the intervals are wider than necessary, reducing the specificity of dispatch decisions.

S-shaped curve: The model is overconfident in the tails (extreme quantiles) and underconfident in the centre (near the median), or vice versa. This pattern often occurs with models that assume a symmetric distribution when the actual distribution is skewed.

Stepped or irregular curve: The model has different calibration properties at different probability levels, often indicating that the training data was insufficient to learn the tail behaviour accurately.

Real-world example — the cost of overconfidence: Suppose a battery operator uses a model whose 95% prediction interval actually covers only 80% of outcomes. The operator treats the upper bound of this interval as "almost certainly the highest the price will go." In reality, 20% of the time, the actual price exceeds this bound — potentially by thousands of dollars during spike events. The operator misses these high-price events because the battery has already discharged, thinking the price had peaked. Over a year, this overconfidence can cost hundreds of thousands of dollars in missed revenue.

Sharpness

Calibration alone is not sufficient for a good probabilistic forecast. To see why, consider the following perfectly calibrated forecast: "For every time period, the 10th percentile is $-\$1,000$ and the 90th percentile is $\$17,500$." This forecast is perfectly calibrated (the actual price will almost always fall within this range, and the stated coverage is correct), but it is completely useless — it tells you nothing beyond what you already knew about the price range.

Sharpness: A measure of how concentrated (narrow) the prediction intervals are. A sharp forecast has narrow prediction intervals — it is making specific, informative predictions about the future. A blunt forecast has wide prediction intervals — it is hedging by covering a large range of outcomes. Sharpness is measured as the average width of prediction intervals across all time periods.

$$\text{Sharpness} = (1/N) \cdot \sum_{t=1}^N (q_{0.9,t} - q_{0.1,t})$$

This formula computes the average width of the 80% prediction interval (the distance between the 90th and 10th percentile predictions) across all N time periods. A smaller value indicates sharper (more informative) predictions.

The goal of probabilistic forecasting is not simply to be calibrated — it is to be **the sharpest forecast subject to calibration**. This formulation, due to Gneiting, Balabdaoui, and Raftery (2007), captures the fundamental tradeoff: you want your prediction intervals to be as narrow as possible (maximum information) while still containing the stated proportion of actual outcomes (honest probability statements).

The calibration-sharpness paradigm: "Maximise sharpness subject to calibration." This is the guiding principle for probabilistic forecast evaluation. A forecast that is perfectly calibrated but not sharp is useless (it says nothing specific). A forecast that is sharp but poorly calibrated is dangerous (its narrow intervals give false confidence). The ideal forecast is both calibrated and sharp — it makes specific predictions that are honestly stated.

Analogy — weather forecasts: Saying "tomorrow's temperature will be between -40°C and 50°C " is perfectly calibrated (the actual temperature will always fall in this range) but has zero sharpness. Saying "tomorrow's temperature will be between 22°C and 24°C " is very sharp but might not be well calibrated (the actual temperature might frequently fall outside this narrow range). A good weather forecast says "between 18°C and 26°C " — sharp enough to be useful, calibrated enough to be trustworthy. The same logic applies to electricity price forecasts.

The CRPS: A Single-Number Metric

Assessing calibration and sharpness separately is useful for diagnosis but impractical for model comparison — you need a single number that captures both. The **Continuous Ranked Probability Score (CRPS)** is that number.

Continuous Ranked Probability Score (CRPS): The standard single-number metric for evaluating probabilistic forecasts. CRPS generalises the Mean Absolute Error (MAE) from point forecasts to full distributions. It rewards both calibration (predicted probabilities match observed frequencies) and sharpness (prediction intervals are narrow). A lower CRPS is better. For a deterministic (point) forecast, CRPS reduces exactly to MAE. CRPS is measured in the same units as the variable being forecast (e.g., $\$/\text{MWh}$ for prices).

The CRPS is defined as:

$$\text{CRPS} = \int_{-\infty}^{\infty} [F(x) - 1(x \geq y)]^2 dx$$

where $F(x)$ is the predicted cumulative distribution function (CDF), y is the actual observed value, and $1(x \geq y)$ is the Heaviside step function (0 when $x < y$, 1 when $x \geq y$).

This integral has an intuitive interpretation: it measures the area between the predicted CDF and the "perfect CDF" (a step function that jumps from 0 to 1 at the observed value). A forecast that places all its probability mass exactly on the observed value would achieve $\text{CRPS} = 0$.

In practice, when the forecast is given as a set of quantile predictions (as in GBT quantile regression or QRA), the CRPS can be approximated as the average pinball loss across all quantile levels:

$$\text{CRPS} \approx (2/M) \cdot \sum_{m=1}^M L_{\tau_m}(y, q_{\tau_m})$$

where L_{τ} is the pinball loss at quantile level τ , q_{τ} is the predicted quantile, y is the actual value, and M is the number of quantile levels. This approximation becomes exact as M increases and the quantile levels span $[0, 1]$ uniformly.

Cumulative distribution function (CDF): A function $F(x)$ that gives the probability that a random variable takes a value less than or equal to x . The CDF starts at 0 (for very small x) and increases monotonically to 1 (for very large x). If $F(100) = 0.7$, there is a 70% probability the value is at most 100. The CDF is related to the PDF (probability density function) — the CDF is the integral of the PDF.

Why CRPS is the right metric: CRPS cannot be "gamed" by being maximally wide (unlike calibration alone). A forecast with trivially wide intervals ($[-\$1,000, \$17,500]$) achieves perfect calibration but terrible CRPS because the integral (area between CDFs) is large. To achieve good CRPS, the

forecast must be both calibrated AND sharp. This makes CRPS the appropriate single metric for comparing probabilistic forecast models.

Spike Coverage: The Critical Edge Case

Overall calibration can be misleading for electricity prices because spikes — the extreme high-price events that drive the majority of battery revenue — are rare. A model might have 90% overall coverage (its 90% prediction interval contains 90% of all observations) but systematically miss spikes because:

1. Spikes are rare (perhaps 2–5% of observations), so missing them has little effect on the overall coverage statistic
2. The model is overconfident specifically during volatile conditions — exactly when spikes occur
3. The tail behaviour of the model is poorly calibrated even though the centre of the distribution is well-calibrated

Spike coverage: The fraction of spike events (observations where the price exceeds a high threshold, e.g., \$300/MWh or \$1,000/MWh) that fall within the model's prediction interval. This is a conditional coverage metric — it evaluates calibration specifically during the events that matter most for battery revenue. A model with 90% overall coverage but only 60% spike coverage is dangerously overconfident about extreme events.

Real-world example — the spike coverage trap: Consider two models, both with 90% overall coverage: **Model A:** 90% overall coverage, 88% spike coverage. This model is well-calibrated even during extremes. Its prediction intervals widen appropriately when the market is volatile, capturing most spikes within their bounds. **Model B:** 90% overall coverage, 55% spike coverage. This model achieves its overall coverage by being too wide during quiet periods (covering 95% of normal observations) and too narrow during volatile periods (covering only 55% of spikes). It is systematically wrong when being wrong is most expensive. A battery operator using Model B would repeatedly be surprised by spikes — the battery would be discharged or idle when prices spike above the model's predicted range. Over a year, the revenue difference between Model A and Model B could be substantial.

Always report spike coverage alongside overall coverage. A model with good overall calibration but poor spike coverage is more dangerous than a model with mediocre overall calibration but good spike coverage — because the battery's revenue is dominated by what happens during the 2–5% of time steps when prices spike.

Conformal Prediction

The Calibration Problem in Practice

Raw quantile forecasts from GBT or neural networks are frequently **miscalibrated** — the stated coverage does not match the actual coverage. This is not a failure of the models per se; it is an inherent difficulty of learning the tails of a distribution from limited data.

Common miscalibration patterns:

Overconfidence at the tails: The 95% interval covers only 85% of observations. The model cannot accurately estimate the 2.5th and 97.5th percentiles because it has seen very few observations in those regions of the distribution.

Systematic bias: All quantiles are shifted — the entire predicted distribution is too high or too low. This occurs when the recent market regime differs from the training period.

Conditional miscalibration: Calibrated on average, but overconfident during volatile periods (when being right matters most) and underconfident during quiet periods (when accuracy matters least). This is the spike coverage problem described above.

Conformal prediction is a statistical framework that provides **finite-sample coverage guarantees** — a mathematically rigorous way to adjust prediction intervals so they achieve the desired coverage, regardless of the underlying model's idiosyncrasies.

Conformal prediction: A statistical framework for constructing prediction intervals (or sets) with guaranteed coverage probability. Unlike model-specific calibration methods, conformal prediction is **model-agnostic** — it works as a wrapper around any forecasting model, adjusting the model's outputs to achieve the desired coverage level. The key assumption is **exchangeability** — the calibration data and future test data come from the same distribution. Under this assumption, conformal prediction provides finite-sample (not just asymptotic) coverage guarantees.

Split Conformal Prediction

The simplest and most practical variant is **split conformal prediction**. The idea is to measure how wrong the model has been on recent data, then widen the prediction intervals by an appropriate amount to account for this historical error.

Split conformal prediction: A method that divides the data into a training subset (for fitting the model) and a calibration subset (for measuring the model's errors). The errors on the calibration subset are used to compute a correction term that is added to the model's prediction intervals, guaranteeing that the widened intervals achieve the desired coverage on future data. The "split" refers to the division of data into training and calibration subsets.

The procedure, step by step:

Step 1: Split the data. Divide the available data into two subsets: a training set (for fitting the forecasting model) and a calibration set (for measuring the model's errors). A typical split might be 80% training, 20% calibration. The calibration set must be held out completely — the

model should never have seen it during training.

Step 2: Fit the model. Train the quantile forecasting model (GBT, neural network, QRA) on the training subset. This produces raw quantile predictions that may or may not be well-calibrated.

Step 3: Compute conformity scores. On the calibration subset, compute a **conformity score** for each observation — a number measuring how "non-conforming" the observation is relative to the model's prediction.

Conformity score: A numerical measure of how poorly the model's prediction matches the actual outcome. For prediction intervals, a common conformity score is $s_i = \max(q_{\text{low},i} - y_i, y_i - q_{\text{high},i})$ — this is positive when the observation falls outside the interval (measuring by how much) and negative when it falls inside (measuring the margin). Larger conformity scores indicate worse calibration.

For a prediction interval $[q_{\text{low}}, q_{\text{high}}]$, two common conformity scores are:

- **Absolute residual:** $s_i = |y_i - y_i^*|$ (how far the observation is from the point prediction)
- **Signed interval distance:** $s_i = \max(q_{\text{low},i} - y_i, y_i - q_{\text{high},i})$ (how far outside the interval the observation falls; negative if inside)

Step 4: Compute the correction. Sort the conformity scores and find the $\lceil (1-\alpha)(1 + 1/n) \rceil$ -th smallest score, where α is the desired miscoverage rate (e.g., $\alpha = 0.1$ for 90% coverage) and n is the number of calibration observations. This score is the **conformal correction** — the amount by which to widen the prediction intervals.

Step 5: Adjust predictions. At prediction time, expand the model's raw prediction intervals by the conformal correction:

Adjusted interval = $[q_{\text{low}} - \text{correction}, q_{\text{high}} + \text{correction}]$

The coverage guarantee: Under the assumption that the calibration observations and future test observations are **exchangeable** (essentially, drawn from the same distribution in any order), the adjusted prediction intervals have coverage $\geq (1 - \alpha)$ in finite samples. This is a non-asymptotic guarantee — it holds for any sample size, any model, and any data distribution. The only assumption is exchangeability.

Exchangeability: A statistical assumption that the joint distribution of a set of random variables is invariant to permutation — reordering the observations does not change their probability. This is weaker than the assumption that observations are independent and identically distributed (i.i.d.), which also requires independence. For time series data, exchangeability is technically violated (today's price depends on yesterday's), but rolling calibration (described below) provides a practical approximation.

Adaptive Conformal Prediction

Standard split conformal prediction has a significant limitation: it produces a **constant correction** — the same amount is added to every prediction interval, regardless of the model's inherent uncertainty. This is suboptimal:

- During **quiet market periods**, the model's raw intervals are already narrow and accurate. The conformal correction makes them unnecessarily wide — sacrificing sharpness for no real calibration benefit.
- During **volatile market periods**, the raw intervals might still be too narrow despite being wider than average. A constant correction may not add enough width where it is needed most.

Adaptive conformal prediction addresses this by scaling the conformity scores by the model's predicted uncertainty:

$$s_i = |y_i - y_i^*| / \sigma_i^*$$

where σ_i^* is a measure of the model's predicted uncertainty at observation i — for example, the width of the raw prediction interval ($q_{\text{high},i} - q_{\text{low},i}$) or the standard deviation of the quantile predictions.

Adaptive conformal prediction: A variant of conformal prediction that normalises the conformity scores by the model's predicted uncertainty. This produces prediction intervals that are wider when the model is uncertain and narrower when the model is confident — matching the intuition that intervals should adapt to the current state of the market, not be uniformly wide.

By normalising the scores, the conformal correction is applied proportionally: a large correction is applied when σ_i is large (the model is already uncertain), and a small correction when σ_i is small (the model is confident). The result is more informative intervals that vary in width with the model's confidence level.

Real-world example — adaptive vs. constant correction: Consider a January day with mild weather and low demand. The GBT model predicts a narrow 80% interval of [\$40, \$65]. Standard conformal prediction adds a constant correction of \$50 (computed from recent volatile periods), producing [\$-10, \$115] — unnecessarily wide for a calm market. Adaptive conformal prediction recognises that the model's raw interval is narrow (signalling confidence) and applies a smaller correction proportional to the interval width. The result might be [\$30, \$75] — still properly calibrated but much sharper. On a summer afternoon with heatwave warnings, the model's raw interval is already wide: [\$50, \$500]. Adaptive conformal prediction applies a larger correction, producing [\$20, \$700] — appropriate for the genuinely uncertain conditions.

Practical Considerations for Conformal Prediction

Calibration window. The conformity scores should be computed on recent data — typically the last 14 to 60 days. Using calibration data from six months ago may not reflect the model's current accuracy, because the market structure evolves (new generators, changing demand patterns, seasonal shifts). A shorter window adapts faster to changes but has higher variance (fewer calibration observations); a longer window is more stable but may not reflect current conditions.

Calibration window: The period of recent data used to compute the conformity scores for conformal prediction. A rolling calibration window (e.g., the last 30 days) is updated as new data arrives, ensuring the conformal correction reflects the model's current accuracy. The choice of window length trades off adaptability (shorter windows respond faster to changes) against stability (longer windows provide more reliable estimates).

Rolling recalibration. Recompute the conformal correction daily or weekly as new observations arrive. This maintains approximate coverage even under non-stationarity (slowly changing market conditions). The formal coverage guarantee technically requires exchangeability (which non-stationarity violates), but rolling recalibration is robust in practice — the correction adapts to the model's recent error distribution.

Quantile-specific correction. Apply separate conformal corrections to each quantile level rather than a single uniform correction. The model might be well-calibrated at the median (50th percentile) but systematically overconfident at the 95th percentile — applying the same correction to both would over-correct the median and under-correct the 95th percentile. Quantile-specific corrections allow fine-grained calibration across the entire distribution.

The Calibration-Sharpness Tradeoff in Practice

Putting It All Together

The practical pipeline for producing calibrated, sharp probabilistic forecasts combines the techniques from this chapter:

Train the best possible quantile model. Use QRA to combine GBT, LEAR, and optionally neural network forecasts. This produces quantile forecasts that are reasonably sharp (each component model contributes its best quantile estimates) and approximately calibrated (QRA's pinball-loss fitting provides some calibration).

Apply conformal calibration. Use adaptive conformal prediction to adjust the quantile forecasts, ensuring the coverage guarantee. If the base model is already well-calibrated, the conformal correction will be small and sharpness is largely preserved. If the base model is miscalibrated, the correction widens the intervals to achieve proper coverage.

Evaluate on a held-out test set. Assess both calibration (reliability diagram) and sharpness (average interval width). Report CRPS as the single-number summary, and spike coverage as the critical tail metric.

Iterate. If the model is well-calibrated without conformal adjustment, the adjustment is minimal and sharpness is maximised. If the model is poorly calibrated, investigate the root cause — is it a model specification issue (wrong features, insufficient data), a regime change (the market has shifted), or a tail estimation problem (insufficient extreme observations)?

Calibration for Battery Dispatch: Asymmetric Costs

For battery dispatch, the cost of miscalibration is **asymmetric** — different types of errors have different economic consequences:

- **Missing a spike (price above the upper quantile):** The battery fails to discharge during a high-price period. This is a direct revenue loss — potentially thousands of dollars per missed event.
- **Missing a dip (price below the lower quantile):** The battery fails to charge during a low-price period. This is an opportunity cost — the battery could have charged cheaply but instead charges at a higher price later.

The cost of missing a spike is typically much larger than missing a dip because spikes are more extreme in magnitude than dips. A \$5,000 missed spike is a far larger loss than a \$30 missed charging opportunity.

This asymmetry suggests using **asymmetric calibration** — targeting higher coverage in the upper tail than the lower tail. Instead of symmetric prediction intervals $[q_{0.1}, q_{0.9}]$, you might use $[q_{0.15}, q_{0.85}]$ to provide more headroom on the upside. The 80% coverage is maintained, but it is distributed asymmetrically to protect against the more costly error.

Asymmetric calibration: A calibration approach that targets different coverage levels for the upper and lower tails of the forecast distribution. When the cost of errors is asymmetric (missing a spike is more expensive than missing a dip), the prediction intervals should be asymmetric to provide more protection against the costlier error. Chapter 9's chance-constrained dispatch formalises this intuition.

Real-world example — asymmetric risk in battery dispatch: The Mannum BESS (100 MW / 200 MWh) in SA1 considers whether to discharge at 5 PM. The symmetric 80% interval is [\$80, \$250]. The asymmetric 80% interval (with more upside protection) is [\$100, \$400]. The battery operator using symmetric intervals might discharge at \$250, thinking prices are unlikely to go higher. The operator using asymmetric intervals holds back, knowing there is a meaningful chance prices could reach \$400 or beyond. On days when a spike materialises, the asymmetric approach captures significantly more revenue. On days when prices stay moderate, the symmetric approach was slightly better. Over a year, the asymmetric approach wins because the spike events contribute disproportionately to annual revenue.

Copula Scenario Generation

The probabilistic forecasts developed in this chapter produce **marginal** quantile predictions — for each half-hour period independently, a set of predicted quantiles. But dispatch optimisation in Chapter 9 needs complete **joint** price trajectories: 48-period paths where the correlation between adjacent hours is realistic. Simply sampling each period independently produces incoherent scenarios — paths that zigzag randomly between quantiles rather than showing the smooth, persistent price structure observed in real markets.

Why this matters for dispatch: Independent sampling misses a critical feature of electricity prices: when 5 PM is expensive, 6 PM is usually expensive too. A scenario that has \$500 at 5 PM and \$30 at 6 PM is unrealistic. The stochastic and robust optimisation methods in Chapter 9 rely on realistic joint futures to make good hedging decisions — garbage scenarios produce garbage hedges.

Marginals versus dependence

A probabilistic forecast gives the **marginal distribution** for each period — the full range of possible prices at 5 PM, independently of what happens at 6 PM. But dispatch decisions depend on the **joint distribution** — the probability that 5 PM *and* 6 PM are both high, or that prices are low all afternoon then spike in the evening.

Two sets of marginal forecasts can have identical per-period distributions but very different joint behaviour:

- **Positively correlated:** High prices at 5 PM tend to occur with high prices at 6 PM (the realistic case — price persistence)
- **Independent:** High at 5 PM tells you nothing about 6 PM (unrealistic — ignores autocorrelation)
- **Negatively correlated:** High at 5 PM makes low at 6 PM more likely (generally unrealistic for adjacent periods)

The tool that separates marginals from dependence is the **copula**.

Sklar's theorem and the copula idea

Copula: A multivariate distribution function whose marginals are all uniform on $[0, 1]$. A copula captures the dependence structure between random variables, completely separate from their individual distributions. Any joint distribution can be decomposed into its marginals and a copula — this is Sklar's theorem.

Sklar's theorem: Any multivariate distribution can be written as a copula applied to the individual marginal distributions. Conversely, given any set of marginal distributions and any copula, the combination defines a valid joint distribution. This theorem is the theoretical foundation for copula-based scenario generation: we can take our calibrated marginal quantile forecasts (per period) and combine them with a copula (fitted to historical dependence patterns) to produce coherent joint scenarios.

The procedure has three steps:

Fit marginals: The quantile forecasts from QRA or conformal prediction define the marginal distribution for each period. These are already calibrated per-period.

Fit the copula: Transform historical forecast errors into uniform variables using the **probability integral transform (PIT)**, then fit a copula to the resulting uniform variables. The copula captures how forecast errors are correlated across periods — when the model underpredicts at 5 PM, does it also underpredict at 6 PM?

Sample scenarios: Draw from the fitted copula (producing correlated uniform variables), then transform back through the per-period marginal quantile functions to obtain price scenarios in dollar space.

Probability integral transform (PIT): The transformation that converts a random variable to a uniform $[0, 1]$ variable using its own CDF. If X has CDF F , then $F(X)$ is uniformly distributed. Applied to forecast residuals, the PIT produces the "copula-scale" data needed to fit the dependence structure. The inverse PIT (applying the quantile function) transforms back from uniform to dollar space.

The Gaussian copula

The simplest and most widely used copula is the **Gaussian copula**, which models dependence using a multivariate normal distribution:

1. Transform historical PIT residuals to standard normal using the inverse normal CDF (Φ^{-1})
2. Compute the correlation matrix R of the transformed residuals across all 48 periods
3. To generate a scenario: draw z from a multivariate normal $N(0, R)$, apply the normal CDF to get uniform variables $u = \Phi(z)$, then apply the per-period quantile function to get prices

Gaussian copula: A copula derived from the multivariate normal distribution. It captures linear dependence (correlation) between variables. Its main advantage is simplicity — it is fully specified by a correlation matrix. Its main limitation is that it has symmetric tail dependence: it does not naturally model the tendency for extreme prices to cluster more strongly than moderate prices.

The Gaussian copula works well for electricity prices in most conditions because the dominant dependence pattern — price persistence across adjacent hours — is approximately linear. However, it can understate **tail dependence**: the tendency for extreme prices to be even more correlated than normal prices. During heatwave events, for example, all evening periods tend to spike together more strongly than the Gaussian copula predicts.

When tails matter: vine copulas

For applications where tail dependence is critical — particularly the stochastic dispatch of batteries that earn disproportionately from spikes — more flexible copula families exist:

- **t-copula:** Like the Gaussian copula but with heavier tails, controlled by a degrees-of-freedom parameter. It captures the tendency for extremes to cluster.
- **Vine copulas:** Build a multivariate copula from a cascade of bivariate copulas, each chosen to fit the specific dependence pattern between each pair of variables. A vine copula on 48 periods decomposes into many bivariate building blocks — the copula between periods 1 and 2, between periods 2 and 3 (conditional on 1), and so on. Each building block can be a different family (Gaussian, t, Clayton, Gumbel), chosen to match the observed dependence.

Tail dependence: The tendency for extreme values to occur together. If two variables have high tail dependence, knowing that one is in its 99th percentile makes it more likely that the other is also in its 99th percentile — more so than a Gaussian model would predict. For electricity prices, tail dependence means that spike events tend to affect multiple consecutive periods simultaneously.

Practical recommendation: Start with the Gaussian copula — it is easy to fit, interpret, and debug. Compare the rank correlation of generated scenarios against historical data. If the tail behaviour is inadequate (scenarios understate the clustering of extreme prices), upgrade to a t-copula with estimated degrees of freedom. Vine copulas are a further refinement but add significant complexity; reserve them for cases where you can demonstrate a material improvement in dispatch capture ratio.

Cross-market dependence: energy and FCAS

The copula framework extends naturally to modelling the joint distribution of energy and FCAS prices (introduced in Chapter 11). Energy and FCAS prices have a complex dependence structure:

- In normal conditions, FCAS prices are low and nearly independent of energy prices
- During contingency events, FCAS prices spike while energy prices may or may not spike
- When enablement is scarce (many generators providing FCAS, reducing energy supply), energy and FCAS prices become positively correlated

Fitting a copula across both energy and FCAS price series produces scenarios where these market interactions are realistic — essential for the co-optimised dispatch developed in Chapter 11.

Sampling and scenario count

The number of scenarios is a practical trade-off:

- **Too few ($N < 20$):** The scenarios do not adequately represent the distribution. The stochastic LP may overfit to the specific scenarios drawn.
- **Too many ($N > 500$):** The stochastic LP becomes computationally expensive (it grows linearly in N). Diminishing returns set in around $N = 100$ – 200 for a 48-period problem.
- **Practical default:** $N = 50$ – 100 copula-generated scenarios provides a good balance for the dispatch optimisation in Chapter 9.

Copula scenarios feed directly into Chapter 9. The stochastic MPC and chance-constrained MPC both consume price scenarios as input. Replacing independently sampled scenarios with copula-generated ones typically improves the dispatch capture ratio by 1–3 percentage points — because the hedging decisions are based on realistic joint futures rather than incoherent random paths. This improvement is largest during volatile periods, which are exactly the periods that drive battery revenue.

Figure 8.5 — Left: independent scenarios (each period sampled independently — note the unrealistic zigzagging). Right: copula scenarios (preserving cross-period correlation — note the smooth, persistent price paths that resemble real market behaviour).

Glossary

Term	Definition
Probabilistic forecast	A forecast that describes the full range of possible outcomes and their probabilities
Prediction interval	A range expected to contain the actual value with a specified probability
Quantile forecast	Predictions at multiple probability levels characterising the forecast distribution
Density forecast	The full probability density function of the predicted variable
Scenario forecast	Multiple complete future trajectories sampled from the forecast distribution
PDF (probability density function)	A function showing the relative likelihood of each possible value
CDF (cumulative distribution function)	A function giving the probability of being at or below each value
Forecast combination principle	The finding that averaging forecasts from different models almost always improves accuracy

QRA (Quantile Regression Averaging)	A method combining multiple point forecasts into calibrated quantile forecasts
Calibration (reliability)	Whether stated probabilities match observed frequencies
Reliability diagram	A plot of observed coverage vs. nominal coverage for assessing calibration
Overconfidence	When prediction intervals are too narrow (stated coverage exceeds actual coverage)
Underconfidence	When prediction intervals are too wide (actual coverage exceeds stated coverage)
Sharpness	How narrow (concentrated) the prediction intervals are
CRPS	Continuous Ranked Probability Score — the standard metric for probabilistic forecasts
Spike coverage	Fraction of extreme price events captured within the prediction interval
Conformal prediction	A model-agnostic framework for constructing prediction intervals with coverage guarantees
Split conformal prediction	A variant using a held-out calibration set to compute interval corrections
Conformity score	A measure of how poorly a model's prediction matches the actual outcome
Exchangeability	The assumption that data ordering does not affect the joint distribution
Adaptive conformal prediction	A variant that scales corrections by the model's predicted uncertainty
Calibration window	The period of recent data used to compute conformal corrections
Rolling recalibration	Periodically recomputing conformal corrections as new data arrives
Asymmetric calibration	Targeting different coverage levels for upper and lower tails
Calibration-sharpness paradigm	"Maximise sharpness subject to calibration" — the guiding principle
Copula	A multivariate distribution capturing dependence structure, separate from marginals
Sklar's theorem	Any joint distribution decomposes into marginals and a copula
Probability integral transform (PIT)	Transformation converting a variable to uniform [0,1] via its own CDF
Gaussian copula	Copula derived from the multivariate normal; captures linear dependence
Tail dependence	Tendency for extreme values to occur together more than a Gaussian model predicts
Vine copula	Multivariate copula built from a cascade of bivariate copula building blocks
Rank correlation	Correlation measured on ranks rather than values; invariant to marginal transforms
Scenario coherence	Property that jointly sampled scenarios preserve realistic cross-period dependence

Summary

Probabilistic forecasts are essential for risk-aware battery dispatch because they quantify the uncertainty that point forecasts conceal. Quantile Regression Averaging (QRA) combines multiple point forecasts into a single probabilistic forecast with quantile-specific weights — different models contribute different amounts at different probability levels. Evaluation of probabilistic forecasts requires both calibration (stated coverage matches actual coverage) and sharpness (prediction intervals are as narrow as possible), captured in a single metric by the CRPS. Spike coverage deserves separate attention because overall calibration can mask dangerous overconfidence during the extreme events that drive battery revenue. Conformal prediction provides a model-agnostic safety net — finite-sample coverage guarantees achieved by adjusting prediction intervals based on the model's recent errors. Adaptive conformal prediction improves on the basic method by scaling corrections to the model's predicted uncertainty, producing intervals that are wider during volatile periods and narrower during calm periods. The calibration-sharpness paradigm — "maximise sharpness subject to calibration" — guides the entire evaluation process, and asymmetric calibration accounts for the fact that missing a price spike costs far more than missing a price dip.

9. From Forecast to Money

The Dispatch Decision Under Uncertainty

In Chapter 5, we introduced **Model Predictive Control (MPC)** with a point forecast: take the median price prediction for the next 48 half-hours, solve a linear program (LP) to find the optimal charge/discharge schedule, execute the first period's decision, observe the actual price, and repeat. This approach works, but it ignores a critical piece of information: the probabilistic forecast from Chapters 7 and 8.

Naive MPC treats a confident forecast and a wildly uncertain one identically. If the median predicted price for 6 PM is \$150/MWh, naive MPC makes the same dispatch decision whether the 80% interval is [\$120, \$180] or [\$20, \$5,000]. This is suboptimal — the second scenario should trigger a fundamentally different strategy, one that accounts for the enormous uncertainty.

Model Predictive Control (MPC): A control strategy that repeatedly solves an optimisation problem over a future horizon, executes only the first decision, observes the new state, and re-solves. For battery dispatch, the optimisation maximises revenue over the next 24–48 hours, the first half-hour's charge/discharge decision is executed, the actual price is observed, and the process repeats. The "receding horizon" approach naturally corrects for forecast errors because the optimisation is re-solved with updated information at every step.

This chapter develops two methods that use the full quantile forecast from Chapters 7–8: **scenario MPC** (based on stochastic programming) and **chance-constrained MPC** (based on robust optimisation). Both are well-established in operations research and are widely used in energy trading, portfolio management, and supply chain planning.

The central question: How do you make optimal decisions when you don't know what the future prices will be — but you do know the *range* of possible prices and their approximate likelihoods? This is the domain of decision-making under uncertainty, and it is what separates sophisticated battery operators from naive ones.

Stochastic Programming: Scenario MPC

The Idea

Stochastic programming: A mathematical optimisation framework for making decisions under uncertainty. Instead of optimising against a single deterministic scenario, stochastic programming considers multiple possible future outcomes (scenarios), each with an associated probability, and optimises the **expected** (probability-weighted average) objective across all scenarios. The decisions must be made before the uncertainty is resolved — you must commit to a charge/discharge plan before you know the actual prices.

The core idea is intuitive: instead of betting everything on a single price trajectory (the median forecast), generate many plausible price trajectories from the probabilistic forecast and find the battery schedule that performs best **on average** across all of them.

This is a classic **two-stage stochastic program**:

- **First stage (here-and-now decisions):** Choose the charge/discharge schedule before the actual prices are revealed. These decisions must be made under uncertainty — they cannot depend on which scenario actually materialises.
- **Second stage (wait-and-see outcomes):** The actual prices are observed, and the revenue is computed based on the first-stage decisions and the realised prices.

Two-stage stochastic program: An optimisation problem with two stages. In the first stage, decisions are made before uncertainty is resolved. In the second stage, the uncertainty is revealed and recourse actions may be taken. For battery dispatch, the first-stage decision is the charge/discharge schedule; the second stage is the realisation of actual prices and the computation of revenue. The objective is to maximise expected second-stage revenue across all scenarios.

By optimising across many scenarios, the battery **hedges** against forecast uncertainty. It avoids committing to extreme positions that would be catastrophic if the forecast is wrong. A battery that discharges all its energy at 5 PM (because the median forecast says 5 PM has the highest price) earns nothing if the spike actually occurs at 7 PM. A battery optimised across scenarios might split its discharge between 5 PM and 7 PM, earning somewhat less in the median scenario but much more on average.

Intuitive analogy — diversifying bets: Scenario MPC is like diversifying an investment portfolio. Instead of betting your entire portfolio on one stock (the median forecast), you spread your investment across multiple stocks (scenarios). Some investments will underperform, but the portfolio as a whole has lower risk and often higher average returns than an all-in bet on a single stock. The battery "diversifies" its dispatch across time periods that are high-priced in different scenarios.

Generating Scenarios from Quantile Forecasts

The probabilistic forecast from Chapter 8 provides quantile forecasts — for each half-hour period, we have predicted values at levels $\tau_1 < \tau_2 < \dots < \tau_m$ (e.g., 5th, 10th, 25th, 50th, 75th, 90th, 95th percentiles). We need to convert these into complete price **scenarios** — full 48-period price trajectories.

The simplest approach:

1. **For each time period t** , draw a random quantile level u_t from a Uniform(0, 1) distribution
2. **Interpolate** between the quantile forecasts at level u_t to obtain a price. For example, if $u_t = 0.35$, interpolate between the 25th and 50th percentile predictions.
3. **Repeat steps 1–2 for all 48 periods** to generate one complete price trajectory (scenario)
4. **Repeat the entire process N times** to generate N scenarios

Scenario: In stochastic programming, a scenario is one complete realisation of the uncertain quantities — in our case, a full 48-period price trajectory. Each scenario represents one plausible future: "what might happen if the price follows this particular path." The set of all scenarios, with their associated probabilities, approximates the full distribution of possible futures.

This simple method has a limitation: the random draws u_t are independent across time periods, so the scenarios do not preserve the **temporal correlation** of prices — the tendency for high prices to cluster together and low prices to cluster together. More sophisticated scenario generation methods address this:

- **Copula-based methods:** Model the temporal dependence structure with a copula (a mathematical function that describes the correlation between random variables), generating scenarios that preserve the autocorrelation of prices.

Copula: A statistical function that describes the dependence structure between random variables, separate from their individual distributions. A copula captures how the variables tend to move together: if price at 5 PM is high, is price at 6 PM also likely to be high? By separating the dependence structure (copula) from the individual distributions (quantile forecasts), we can generate scenarios that have both the correct marginal distributions and the correct correlations.

Gaussian bridge methods: Generate multivariate Gaussian random variables with the correct covariance structure (capturing how prices at different hours are correlated), then transform them to match the quantile forecast at each period.

Historical analogue methods: Find historical days with similar forecast patterns (similar weather, demand, and predicted prices) and use their actual price trajectories as scenarios. These scenarios automatically have realistic temporal structure because they are real price paths.

For battery dispatch with a 48-period horizon, the simple independent sampling is often sufficient in practice because the LP's optimal solution depends mainly on the **ranking** of prices across periods (which periods are cheapest for charging and most expensive for discharging), not on the exact correlation structure. However, the more sophisticated methods can improve performance during volatile periods when temporal clustering matters.

Solving the Scenario LP

Given N generated scenarios, we need to find the battery schedule that maximises expected revenue. There are two approaches:

Approach 1: Solve and average. Solve the standard dispatch LP (from Chapter 5) separately for each scenario, obtaining N optimal schedules. Then compute the average charge and discharge across all scenarios:

$$\text{charge}_t = (1/N) \cdot \sum_{s=1}^N \text{charge}_{t,s} \text{ and } \text{discharge}_t = (1/N) \cdot \sum_{s=1}^N \text{discharge}_{t,s}$$

This approach is simple and fast (N independent LPs can be solved in parallel). The averaged schedule may occasionally violate battery constraints (e.g., the average of feasible state-of-charge trajectories may not itself be feasible), but in practice this is rarely an issue for simple battery constraints.

Approach 2: Expected-value optimisation. Formulate and solve a single, larger LP that maximises expected revenue across all scenarios simultaneously, with **shared decisions** (the charge and discharge variables are the same across all scenarios):

$$\text{Maximise } (1/N) \cdot \sum_{s=1}^N \sum_{t=1}^T \text{price}_{t,s} \cdot (\text{discharge}_t - \text{charge}_t) \cdot \Delta t$$

Subject to the standard battery constraints (Chapter 5): state-of-charge limits, power limits, efficiency losses, and cycle constraints. The key difference from approach 1 is that charge_t and discharge_t are **not** subscripted by s — they are the same for all scenarios. This forces the optimiser to find a single schedule that works well across all scenarios, not the best schedule for any one scenario.

Expected value: The probability-weighted average of all possible outcomes. If there are N equally likely scenarios, the expected revenue is the simple average of the revenue across all scenarios. Expected-value optimisation finds the decisions that maximise this average, implicitly trading off doing well in some scenarios against doing poorly in others.

This approach is computationally more expensive (the LP has N times more revenue terms, though the decision variables remain the same) but produces a feasible schedule by construction, since the battery constraints are applied to the shared decision variables.

How Many Scenarios?

The number of scenarios N controls the quality of the approximation to the true expected revenue. More scenarios provide a better approximation but cost more computation:

N	Behaviour	Typical use
1	Reduces to naive MPC (single scenario = median forecast)	Baseline comparison

5–10	Captures broad uncertainty structure but may miss tail events	Quick-and-dirty applications
20–50	Usually sufficient for battery dispatch; captures the main structure	Standard practice
100+	Diminishing returns; additional scenarios refine the mean estimate marginally	Research or high-stakes applications

Law of large numbers: A theorem stating that the average of a large number of independent random draws converges to the true expected value. For scenario MPC, this means that as N increases, the average revenue across scenarios converges to the true expected revenue. The convergence rate is $O(1/\sqrt{N})$ — doubling the accuracy of the expected value estimate requires quadrupling the number of scenarios.

Computation time scales linearly with N when using approach 1 (N independent LPs) and sub-linearly with approach 2 (a single LP whose size grows with N). For a 48-period battery dispatch problem, even $N = 100$ LPs can be solved in under a second on a modern laptop, so computational cost is rarely the binding constraint.

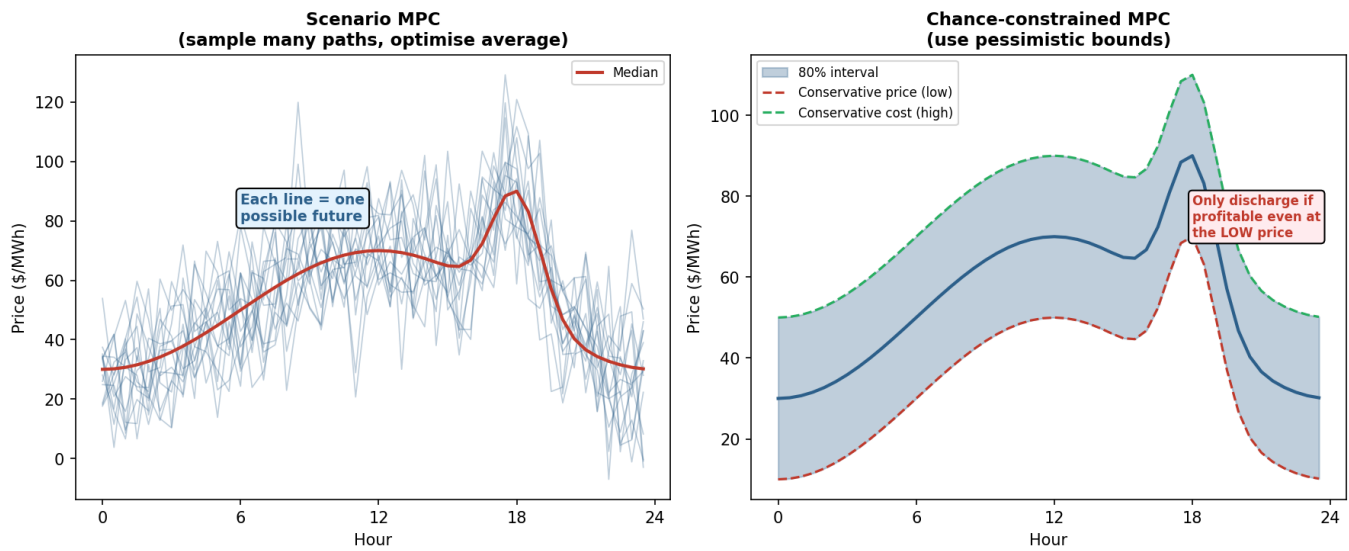


Figure 9.1 — Comparison of scenario MPC and chance-constrained MPC dispatch strategies. Scenario MPC hedges by considering multiple price trajectories, while chance-constrained MPC uses conservative quantile bounds. The dispatch behaviour differs: scenario MPC tends to spread discharge across multiple high-price periods, while chance-constrained MPC concentrates discharge in periods where the conservative price estimate is highest.

Robust Optimisation: Chance-Constrained MPC

The Idea

Chance-constrained MPC takes a different philosophical approach to uncertainty. Instead of averaging across many scenarios (which gives the **best expected outcome**), it optimises against a **pessimistic scenario** — ensuring the dispatch schedule is profitable even when prices are worse than expected.

Robust optimisation: An optimisation framework that seeks decisions that perform well under the **worst case** (or near-worst case) within a defined set of possible outcomes (the "uncertainty set"). Unlike stochastic programming (which maximises the average outcome), robust optimisation maximises the worst-case outcome — or, in softer variants, ensures the outcome is acceptable with high probability. The result is a conservative but safe strategy.

Chance constraint: A constraint that must hold with at least a specified probability. For example, "the battery's discharge revenue must exceed its charge cost with at least 90% probability." Chance constraints formalise the idea of "be profitable most of the time" without requiring profitability in every single scenario.

The specific approach is elegant in its simplicity: instead of using the median (50th percentile) forecast as in naive MPC, use **different quantile forecasts for discharge revenue and charge cost**:

- **Value discharge revenue at a low quantile** (e.g., the 10th percentile): This is a conservative estimate of the revenue — "even in a pessimistic scenario, the discharge price will be at least this much."
- **Value charge cost at a high quantile** (e.g., the 90th percentile): This is a conservative estimate of the cost — "even in a pessimistic scenario, the charge price will be at most this much."

If the schedule is profitable under these conservative price estimates, it is profitable with high probability (at least $1 - 2\alpha$, where α is the quantile level).

The Chance-Constrained LP

The LP formulation is a simple modification of the naive MPC formulation from Chapter 5. The only change is in the objective function — the battery constraints remain identical:

Maximise $\sum_{t=1}^T [q_{\text{low},t} \cdot \text{discharge}_t - q_{\text{high},t} \cdot \text{charge}_t] \cdot \Delta t$

Subject to the standard battery constraints (state-of-charge limits, power limits, efficiency, cycles).

Here $q_{\text{low},t}$ is the lower quantile (e.g., 10th percentile) and $q_{\text{high},t}$ is the upper quantile (e.g., 90th percentile) for time period t .

This formulation is **conservative by construction**:

Discharge revenue is valued at the low end of the price range. The battery only discharges when the revenue would be high even in a pessimistic price scenario. It will not discharge for a period where the median price is \$150 but the 10th percentile is \$30 — the risk of low revenue is too high.

Charge cost is valued at the high end of the price range. The battery only charges when the cost would be low even in a pessimistic cost scenario. It will not charge during a period where the median price is \$30 but the 90th percentile is \$200 — the risk of expensive charging is too high.

Real-world example — conservative dispatch in action: Consider two half-hour periods at 5 PM: **Period A:** $q_{0_1_0} = \$120$, median = \$150, $q_{0_9_0} = \$180$. The chance-constrained LP values discharge at \$120. Even in a pessimistic scenario, the revenue is substantial. The battery discharges. **Period B:** $q_{0_1_0} = \$20$, median = \$150, $q_{0_9_0} = \$800$. The chance-constrained LP values discharge at only \$20. Despite the high median, the pessimistic scenario is poor. The battery may hold back, preserving its charge for a more certain opportunity. Naive MPC would treat both periods identically (both have median = \$150). Chance-constrained MPC distinguishes them by their downside risk.

The Conservatism Tradeoff

The choice of quantile levels controls the tradeoff between safety (avoiding losses) and aggressiveness (maximising revenue):

Quantile pair	Behaviour	Risk profile
$q_{0_0_5} / q_{0_9_5}$	Very conservative	Battery rarely acts; only moves when price signals are extremely strong. Minimises risk but leaves significant revenue on the table.
$q_{0_1_0} / q_{0_9_0}$	Moderate	Default choice. Good balance of safety and activity. Captures most profitable opportunities while avoiding most false signals.
$q_{0_2_5} / q_{0_7_5}$	Aggressive	Battery acts frequently, assuming the forecast is well-calibrated. Higher revenue in good scenarios, higher risk in bad ones.
$q_{0_5_0} / q_{0_5_0}$	Reduces to naive MPC	No uncertainty information used — the dispatch is based entirely on the median forecast.

The optimal quantile pair depends on forecast calibration. If the 90% prediction interval is well-calibrated (it actually covers 90% of outcomes), then the $q_{0_1_0} / q_{0_9_0}$ pair is naturally well-matched to the uncertainty. If the model is overconfident (the 90% interval only covers 75% of outcomes), wider quantile pairs ($q_{0_0_5} / q_{0_9_5}$) are needed to compensate. This is why Chapter 8's calibration work is essential — poorly calibrated quantile forecasts lead to poorly tuned dispatch strategies.

Comparing Scenario MPC and Chance-Constrained MPC

Property	Scenario MPC	Chance-constrained MPC
Objective	Maximise expected revenue	Maximise worst-case (conservative) revenue
Uses full distribution	Yes (via sampling)	Partially (only two quantile levels)
Computational cost	$N \times \text{LP cost}$ (but parallelisable)	$1 \times \text{LP cost}$ (very fast)
Conservatism	Implicit (from scenario diversity)	Directly controllable (via quantile pair)
Risk handling	Risk-neutral (optimises the average)	Risk-averse (protects against downside)
Handles asymmetric risk	Naturally (scenarios can be skewed)	With asymmetric quantile choices (e.g., $q_{0_1_5} / q_{0_9_5}$)
Implementation complexity	Moderate (scenario generation + aggregation)	Simple (modified LP objective)
Optimality	Approaches optimal as $N \rightarrow \infty$	Suboptimal but safe by construction
When it excels	Well-calibrated forecast, risk-neutral operator	Uncertain forecast, risk-averse operator

Real-world industry context: In practice, many battery operators use a variant between these two extremes. A common approach is to solve the chance-constrained LP for the "base plan" and then adjust specific periods using scenario analysis — for example, increasing discharge during

periods where the scenario spread suggests a spike is possible. The simplicity of chance-constrained MPC makes it the more common production approach, while scenario MPC is more common in research and trading desks with dedicated quantitative teams.

Sensitivity Analysis: Battery Parameters

The **capture ratio** — the fraction of perfect-foresight revenue achieved by the dispatch strategy — depends not only on forecast quality but also on the battery's physical characteristics. Understanding these sensitivities is critical for investment decisions (which battery to buy) and operational decisions (how to configure the dispatch strategy).

Capture ratio (CR): The ratio of actual dispatch revenue to the maximum possible revenue (perfect foresight). $CR = \text{actual revenue} / \text{perfect foresight revenue}$. A CR of 0.7 means the dispatch strategy captures 70% of the theoretical maximum revenue. The capture ratio measures the combined quality of the forecast and the dispatch strategy — a higher CR means better forecasting and/or better dispatch.

Duration (Hours of Storage)

Battery **duration** (also called "energy-to-power ratio" or "storage hours") is the number of hours the battery can sustain its maximum discharge power. A 100 MW / 2h battery can discharge at 100 MW for 2 hours (total energy = 200 MWh). A 100 MW / 4h battery can discharge at 100 MW for 4 hours (total energy = 400 MWh).

Battery duration: The ratio of a battery's energy capacity (MWh) to its power capacity (MW). A battery with 200 MWh of energy capacity and 100 MW of power capacity has a duration of 2 hours. Duration determines how long the battery can sustain maximum output — a 2h battery can discharge for 2 hours, while a 4h battery can discharge for 4 hours. Longer duration batteries can capture longer-duration price events but are more expensive per MW.

Duration	Capabilities	Limitations	Typical application
1 hour	Captures short price spikes (1–2 intervals)	Misses multi-hour events; must time the peak precisely	FCAS and frequency response
2 hours	Standard configuration; captures most intra-day arbitrage	Cannot bridge long duration events (e.g., a 4-hour evening peak)	Day-ahead energy arbitrage (standard)
4 hours	Captures extended events; more flexible dispatch	Diminishing marginal returns for day-ahead arbitrage; most NEM price differentials resolve within 4 hours	Day-ahead arbitrage (premium); renewable energy shifting
8+ hours	Can shift energy across major parts of the day	Relevant for inter-day arbitrage or seasonal shifting; rarely justified by NEM day-ahead price patterns alone	Long-duration storage research; renewable integration

The relationship between duration and capture ratio is **concave** — going from 1h to 2h provides a large CR improvement (capturing the main daily price differential), going from 2h to 4h provides a moderate improvement (capturing longer events), and going from 4h to 8h provides a small improvement (most NEM price events are shorter than 4 hours). This concavity means that for pure day-ahead arbitrage in the NEM, 2h batteries are the sweet spot: they capture most of the value at half the cost of a 4h battery.

Round-Trip Efficiency

Round-trip efficiency (RTE) is the fraction of energy recovered from a full charge-discharge cycle. If you put 100 MWh into a battery and get 85 MWh out, the round-trip efficiency is 85%.

Round-trip efficiency (RTE): The ratio of energy delivered during discharge to energy consumed during charging, expressed as a percentage. An RTE of 85% means that for every 100 MWh put into the battery, 85 MWh comes back out — the other 15 MWh is lost to heat, chemical irreversibility, and power electronics inefficiency. RTE directly determines the minimum price spread needed for profitable arbitrage: the discharge price must exceed the charge price by a factor of $1/\text{RTE}$.

Efficiency determines the **breakeven spread** — the minimum ratio of discharge price to charge price required for profitable operation:

Breakeven condition: $P_{\text{discharge}} > P_{\text{charge}} / \text{RTE}$

Round-trip efficiency	Energy loss	Breakeven spread	Interpretation
70%	30% lost	Price must be 43% higher	Older or lower-quality batteries; only large spreads are profitable
85%	15% lost	Price must be 18% higher	Typical lithium-ion; moderate spreads are profitable
90%	10% lost	Price must be 11% higher	High-quality lithium-ion; small spreads become profitable

95%	5% lost	Price must be 5% higher	Best-in-class or theoretical; nearly all spreads are profitable
-----	---------	-------------------------	---

Higher efficiency strictly increases revenue because it lowers the breakeven threshold — more price differentials become profitable. However, the **marginal value** of efficiency improvements is diminishing. Going from 70% to 85% opens up many new profitable trading opportunities (all spreads between 18% and 43%). Going from 90% to 95% opens up far fewer new opportunities (only spreads between 5% and 11%) because most profitable NEM arbitrage involves large spreads anyway.

Real-world example — efficiency economics: Consider two batteries dispatching in SA1 on a day with charge price = \$30/MWh and discharge price = \$120/MWh (a 300% spread): **Battery A (RTE = 85%):** Revenue per MWh discharged = $\$120 - (\$30 / 0.85) = \$120 - \$35.3 = \$84.7/\text{MWh}$ **Battery B (RTE = 90%):** Revenue per MWh discharged = $\$120 - (\$30 / 0.90) = \$120 - \$33.3 = \$86.7/\text{MWh}$ The 5-percentage-point efficiency improvement yields an extra \$2/MWh per cycle. For a 100 MW / 2h battery doing 1.5 cycles per day, this is ~\$600/day or ~\$220K/year. This is significant but not transformative — the battery's revenue is dominated by the magnitude of the price spread, not the efficiency differential.

Cycle Constraints and Degradation

Every charge-discharge cycle degrades the battery slightly — the lithium-ion cells lose a fraction of their capacity with each cycle due to chemical and mechanical degradation. The **cycle constraint** limits the number of full cycles per day to manage this degradation.

Battery cycle: One complete charge from empty to full followed by a complete discharge from full to empty. A "half cycle" is a charge-discharge that uses only half the battery's capacity. Cycle counting is typically based on the total energy throughput: if the battery has 200 MWh capacity and charges/discharges a total of 400 MWh in a day, it has done 2 full equivalent cycles.

Battery degradation: The gradual loss of a battery's capacity and efficiency over time, driven by charge-discharge cycling, calendar aging (degradation that occurs even when idle), temperature, depth of discharge, and charge rate. A lithium-ion battery might lose 2–3% of its capacity per year under normal cycling. Degradation is economically significant because it reduces the battery's future revenue potential — each cycle today trades a small amount of future capacity for immediate revenue.

Cycles per day	Behaviour	Economic implication
1 cycle/day	Very conservative	Preserves battery life but misses secondary arbitrage opportunities. Suitable when the battery's capital cost is high relative to daily revenue.
1.5–2 cycles/day	Standard	Captures the main morning-charge/evening-discharge cycle plus one or two secondary opportunities. The default for most utility batteries.
3–4 cycles/day	Aggressive	Exploits smaller intra-day price swings. Suitable for batteries with low degradation costs or when daily price volatility is very high. Shortens battery life.

In our simplified LP formulation (Chapter 5), degradation is modelled as a simple cycle-count constraint. Real battery degradation is more complex — it depends on the depth of discharge (shallow cycles are less damaging than deep cycles), the charge rate (fast charging degrades faster), temperature, and calendar aging. More sophisticated dispatch models incorporate degradation as a variable cost: each cycle has an implicit cost equal to the fraction of battery capacity consumed, valued at the battery's replacement cost. This turns the cycle limit from a hard constraint into an economic tradeoff.

The Value of Information

How Much Does Forecast Quality Matter?

This is the most commercially important question in the entire course: **what is a better forecast worth in dollars?** The answer determines whether investing in forecast improvements (better models, more data, more computation) is justified by the additional revenue.

Value of information: The economic benefit of having better information (a more accurate forecast) when making a decision. For battery dispatch, the value of information is the difference in revenue between a dispatch based on a perfect forecast (perfect foresight) and a dispatch based on the actual forecast. It measures how much money is "left on the table" due to forecast imperfection.

We can quantify this by **deliberately degrading** the forecast — adding noise of increasing magnitude — and measuring the capture ratio at each noise level. This traces the **value-of-information curve**:

Value of forecast quality (concave — first improvements worth the most)

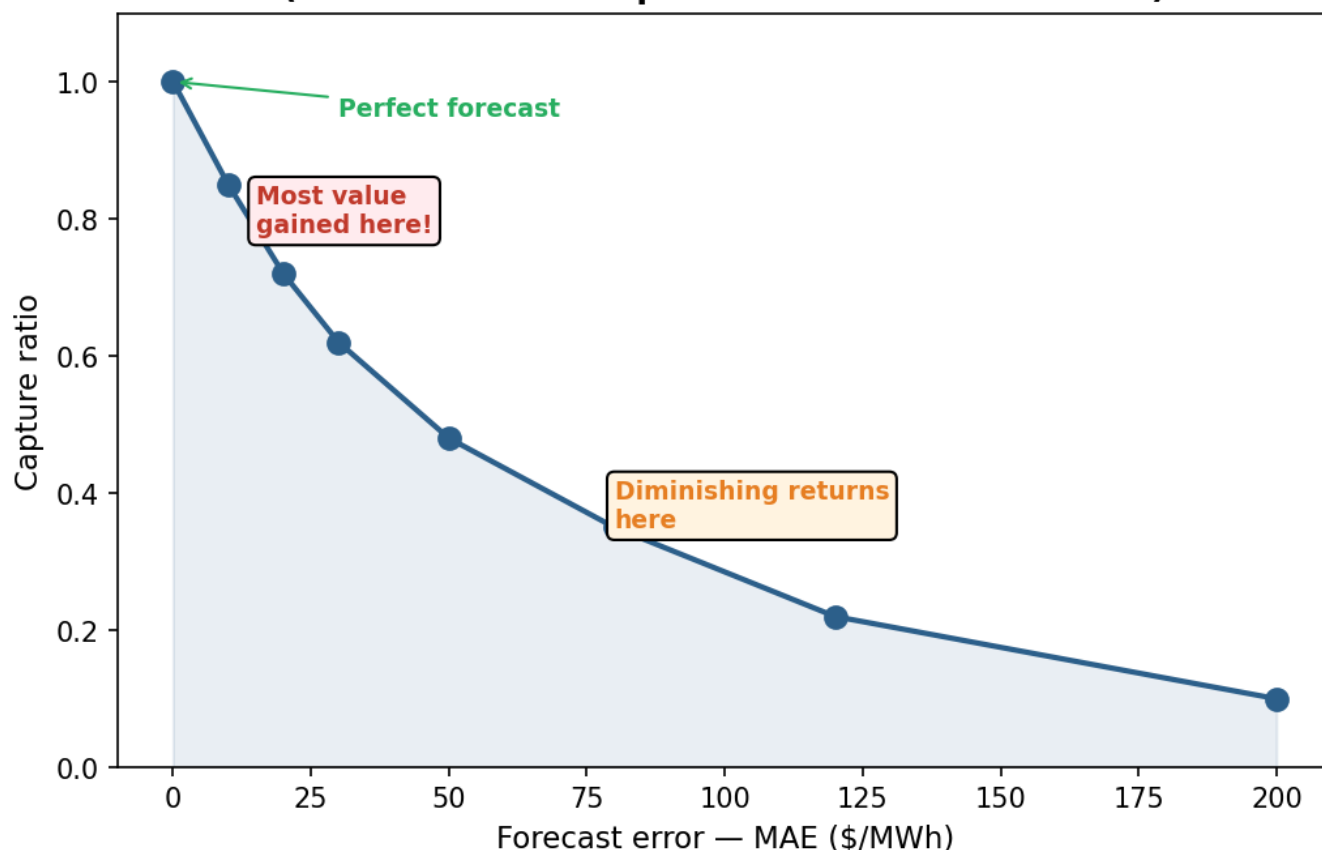


Figure 9.2 — The value of information curve shows capture ratio (CR) as a function of forecast quality. The curve is concave: the first improvements from a poor forecast are highly valuable, while improvements from a good to an excellent forecast yield diminishing returns. This shape has profound implications for where to invest effort in the forecasting pipeline.

Typical results for NEM battery dispatch:

Forecast quality	MAE (approx.)	Capture ratio	Interpretation
Perfect foresight	\$0	1.00	Theoretical maximum; achievable only with actual future prices
Excellent forecast	~\$15/MWh	0.70–0.85	Near the "knee" of the curve; further improvement has diminishing returns
Good forecast	~\$25/MWh	0.55–0.70	Achievable with a well-tuned GBT model
Mediocre forecast	~\$50/MWh	0.35–0.50	A simple model or poorly tuned model
Random forecast	N/A	~0.00	No systematic profit possible

The relationship is **concave** — each successive improvement in forecast quality yields a smaller increase in capture ratio. This concavity has a simple explanation: the LP mainly needs to get the **price ranking** correct — which periods are cheapest (for charging) and most expensive (for discharging). Getting the ranking right is easier than getting the exact prices right. A forecast with MAE = \$30 might rank the periods correctly 90% of the time, while a forecast with MAE = \$15 ranks them correctly 95% of the time. The first forecast captures most of the arbitrage value because the ranking is mostly correct; the second forecast captures slightly more because of the remaining 5% of mis-ranked periods.

The 80/20 rule of forecast value: The first 80% of forecast quality improvement (from random to decent) captures roughly 80% of the available revenue. The last 20% of quality improvement (from decent to excellent) captures only the remaining 20% of revenue. This means that beyond a certain accuracy level, investing further in forecast quality has diminishing returns — the effort is better directed at other parts of the pipeline (dispatch strategy, battery hardware, market access, ancillary services).

Real-world example — the dollar value of forecast improvement: Consider a 100 MW / 2h battery in SA1 with a perfect-foresight revenue of \$10M/year. A good forecast (CR = 0.65) generates \$6.5M/year. An excellent forecast (CR = 0.75) generates \$7.5M/year. The improvement is worth \$1M/year. Is this \$1M/year worth the cost of the better forecast? If the better forecast requires hiring two additional data scientists (\$400K/year combined), upgrading weather data subscriptions (\$100K/year), and more compute infrastructure (\$50K/year), the answer is clearly yes — the \$550K investment yields \$1M in additional revenue. But going from excellent (CR = 0.75) to near-perfect (CR = 0.85) would require \$1M in additional revenue. The cost of achieving this further improvement — perhaps a dedicated ML research team, proprietary data sources, real-time

infrastructure — might exceed the \$1M benefit. This is where the concavity of the value curve bites: the last percentage points of capture ratio are the most expensive to achieve.

The Economic Assessment

Revenue Decomposition

Total battery revenue in the NEM comes from several distinct streams:

Revenue stack: The combination of revenue streams available to a battery. In the NEM, the main revenue sources are energy arbitrage, FCAS (frequency control ancillary services), network support, and capacity payments. Each stream has different characteristics — arbitrage depends on price volatility, FCAS depends on frequency deviation events, and capacity payments depend on market design and availability.

Energy arbitrage: Buy electricity at low prices, sell at high prices. This is what the dispatch LP optimises and what our capture ratio measures. It is typically the largest single revenue stream, accounting for 40–60% of total battery revenue in the NEM.

FCAS (Frequency Control Ancillary Services): Revenue from helping stabilise the grid frequency by providing rapid power injections or absorptions. Batteries are excellent at FCAS because they can respond within milliseconds. FCAS revenue is not modelled in this project but typically adds 20–40% to arbitrage revenue.

FCAS (Frequency Control Ancillary Services): Services provided to AEMO to maintain the grid frequency at 50 Hz. When a generator trips or demand changes suddenly, the frequency deviates from 50 Hz. FCAS providers (including batteries) respond by injecting or absorbing power to restore the frequency. There are eight FCAS markets in the NEM, each with its own price determined by a separate auction. Batteries are particularly valued for FCAS because of their extremely fast response time (milliseconds vs. seconds for gas turbines).

Network support: Revenue from reducing congestion on transmission lines. If a battery is located on the right side of a congested interconnector, it can charge when the interconnector is constrained (absorbing local excess) and discharge when it is unconstrained. This service is typically contracted with the network operator.

Capacity payments: Revenue from being available to generate electricity when needed, regardless of whether the battery actually discharges. Capacity markets pay generators (including batteries) to be available during peak demand periods, providing a revenue floor that supplements energy market earnings.

Our capture ratio measures only the energy arbitrage component. A full economic assessment would include FCAS, network support, and capacity payments. In most NEM regions, energy arbitrage alone is currently insufficient for a positive net present value (NPV) — the other revenue streams are needed to make the investment case. However, as renewable penetration increases and price volatility grows, the arbitrage opportunity is expanding rapidly.

From Capture Ratio to Annual Revenue

The capture ratio converts forecast quality into economic value through a simple formula:

Annual Arbitrage Revenue = CR × Perfect Foresight Revenue

Perfect foresight revenue: The maximum possible arbitrage revenue, achieved by a battery that knows all future prices in advance and dispatches optimally. This is computed by solving the dispatch LP with actual (historical) prices instead of forecasts. It represents the theoretical upper bound on arbitrage revenue and depends on the battery's specifications (power, duration, efficiency) and the price volatility of the region.

Perfect foresight revenue depends on the battery's physical specifications and the market's price volatility. Typical values for a 100 MW / 2h lithium-ion battery (matching the Mannum BESS specifications):

Region	Annual perfect foresight revenue	Driver
SA1	\$8M–\$15M	Highest volatility; large renewable share
QLD1	\$5M–\$10M	Summer spikes; growing solar
NSW1	\$4M–\$8M	Moderate volatility; largest market
VIC1	\$4M–\$7M	Growing wind; transitioning generation
TAS1	\$2M–\$4M	Hydro-buffered; lowest volatility

These figures vary significantly year to year depending on weather patterns (hot summers drive spikes), coal plant outages (remove cheap baseload, pushing up prices), and renewable output (high wind/solar reduces prices).

Real-world example — putting it all together: The Mannum BESS (100 MW / 200 MWh, LFP) in SA1 with perfect foresight revenue of \$12M/year, using a well-tuned GBT model with QRA combination and chance-constrained dispatch: - Capture ratio: 0.70 - Annual arbitrage revenue: \$12M × 0.70 = \$8.4M - Estimated FCAS revenue (additional ~30%): \$2.5M - Total annual revenue: ~\$10.9M - Battery installed cost:

~\$60M (at current prices) - Simple payback: ~5.5 years - Operating costs (maintenance, grid fees): ~\$1M/year - Net annual revenue: ~\$9.9M - Net payback: ~6 years With current lithium-ion costs and NEM price volatility, a well-sited, well-operated battery in SA1 can achieve payback in 5–7 years with a 15–20+ year useful life — a compelling investment case.

Regional Basis and Inter-regional Trade

The NEM is not one market — it is five regional markets connected by transmission interconnectors. Most of the time, prices in adjacent regions are similar. But when an interconnector reaches its capacity limit, the regions **decouple** and prices can diverge dramatically. This has direct implications for battery revenue.

Why regional prices separate

Each NEM region (SA1, VIC1, NSW1, QLD1, TAS1) has its own spot price, determined by the supply and demand balance *within* that region plus the flows on interconnectors to neighbouring regions.

When interconnector capacity is sufficient, cheap generation in one region flows to its neighbours, equalising prices. When an interconnector **binds** (reaches its transfer limit), the regions on either side clear independently:

Example — the Heywood interconnector: Victoria normally exports cheap brown coal and wind generation to South Australia via the Heywood interconnector (nominal capacity ~600 MW). On a calm evening when SA wind drops and demand rises, Heywood reaches its limit. SA must rely on its own expensive gas generators, pushing SA1 prices to \$300/MWh while VIC1 remains at \$80/MWh. The \$220 spread is the **basis** between the two regions. A battery in SA1 benefits from this spread; a battery in VIC1 does not see the same spike.

Basis (in electricity markets): The price difference between two related markets — typically two NEM regions, or a regional price versus a contract price. When an interconnector binds, the basis between connected regions widens. Basis risk is the risk that a financial hedge or forecast is referenced to one region while the battery settles at another.

Basis risk for battery operators

A battery's revenue is determined by the price at its **connection point**, which is the regional reference price scaled by its marginal loss factor (discussed further in Chapter 13). Basis risk arises in several ways:

- **Regional mismatch:** A financial hedge referenced to the NSW1 price does not protect a battery in SA1 when the interconnector binds and prices separate.
- **Forecast basis:** A model trained on SA1 prices is correct for a battery in SA1, but applying it to VIC1 during a decoupled period would produce incorrect dispatch signals.
- **Inter-regional arbitrage:** Some batteries near interconnector nodes can exploit price separation — charging from the cheap region and discharging into the expensive one — but this requires the physical capability to affect flows, which most batteries do not have.

For the Mannum BESS in SA1: SA has the highest price volatility in the NEM, partly because it frequently decouples from Victoria when the Heywood interconnector binds. This decoupling is a feature, not a bug, for an SA1 battery — it produces the extreme price spreads that drive arbitrage revenue. Understanding when decoupling is likely (high SA demand, low wind, interconnector outages) is a forecasting edge.

Settlement residues

When power flows from a low-price region to a high-price region across an interconnector, the price difference creates a surplus — the **inter-regional settlement residue (IRSR)**. AEMO does not keep this surplus; it is auctioned quarterly as **Settlement Residue Auction (SRA) units**, which are financial instruments used by participants to hedge inter-regional basis risk.

Inter-regional settlement residue (IRSR): The revenue surplus that arises when electricity flows across an interconnector from a lower-priced region to a higher-priced region. The residue equals the price difference multiplied by the flow. These residues are auctioned to market participants through the Settlement Residue Auction (SRA), providing a mechanism for hedging inter-regional price risk.

For a data scientist on a battery desk, settlement residues are primarily context — the trading team uses SRA units to manage basis risk. The forecasting implication is simpler: interconnector flows and constraints are strong predictors of regional price divergence, and including interconnector data (available via NEMOSIS) as features can improve regional price forecasts, particularly for SA1 where decoupling events drive the largest price spikes.

Glossary

Term	Definition
MPC (Model Predictive Control)	A control strategy that repeatedly solves an optimisation over a rolling horizon

Stochastic programming	Optimisation under uncertainty using multiple scenarios
Two-stage stochastic program	Decisions made before uncertainty resolves (first stage); outcomes computed after (second stage)
Scenario	One complete realisation of uncertain future prices
Expected value	The probability-weighted average of outcomes across all scenarios
Copula	A function describing the dependence structure between random variables
Robust optimisation	Optimisation that protects against worst-case outcomes
Chance constraint	A constraint required to hold with at least a specified probability
Capture ratio (CR)	Actual dispatch revenue divided by perfect-foresight revenue
Battery duration	Hours the battery can sustain maximum output (energy/power ratio)
Round-trip efficiency (RTE)	Fraction of energy recovered from a charge-discharge cycle
Battery cycle	One complete charge-discharge (or equivalent energy throughput)
Battery degradation	Gradual loss of capacity due to cycling, aging, and operating conditions
Value of information	The economic benefit of having a better forecast
Revenue stack	The combination of revenue streams available to a battery
FCAS	Frequency Control Ancillary Services — grid stability services
Capacity payments	Revenue for being available to generate, regardless of actual dispatch
Perfect foresight revenue	Maximum arbitrage revenue achievable with perfect knowledge of future prices
Breakeven spread	The minimum price ratio between discharge and charge for profitable operation
Law of large numbers	Average of many random samples converges to the true expected value
Concavity	Diminishing marginal returns — each additional unit of input yields less additional output
Basis	Price difference between two related markets (e.g., two NEM regions)
Basis risk	Risk that a hedge or forecast is referenced to a different price than the battery settles at
Interconnector	Transmission link connecting two NEM regions, with a finite transfer capacity
Binding constraint	When an interconnector reaches its transfer limit, decoupling regional prices
Inter-regional settlement residue (IRSR)	Revenue surplus from price differences across interconnector flows
Settlement Residue Auction (SRA)	Quarterly auction of rights to inter-regional settlement residues

Summary

The journey from forecast to money requires a dispatch strategy that uses the full probabilistic forecast, not just the point prediction. Scenario MPC generates multiple plausible price trajectories and optimises expected revenue across all of them, hedging against forecast uncertainty. Chance-constrained MPC takes a simpler, more conservative approach — valuing discharge revenue at a low quantile and charge cost at a high quantile, ensuring profitability in pessimistic scenarios. Battery physical parameters — duration, round-trip efficiency, and cycle limits — interact with forecast quality to determine the capture ratio: the fraction of perfect-foresight revenue actually achieved. The value of information is concave: improving a poor forecast is extremely valuable, but improving an already good forecast yields diminishing returns because the dispatch LP mainly needs the correct price ranking, not exact price magnitudes. For a 100 MW / 2h battery in SA1, each percentage point of capture ratio is worth roughly \$80K–\$150K per year — a concrete economic translation of forecast quality improvements. Every link in the chain from probabilistic forecast to dispatch optimisation to battery hardware contributes to the final capture ratio, and understanding their relative contributions guides where to invest the next dollar of effort.

10. Capstone: From Sunlight to Revenue

The Complete Pipeline

This final chapter assembles every piece of the course into a single end-to-end pipeline and evaluates it as a whole. The pipeline is anchored on the **Mannum BESS** — a 100 MW / 200 MWh lithium iron phosphate battery in SA1, owned by Epic Energy and optimised by Habitat Energy. By targeting a real asset, the entire course functions as a **digital twin**: the same region, the same battery specifications, and (where Mannum's short operational history allows) a direct comparison against Habitat's actual dispatch decisions.

The story we have built, chapter by chapter, is this:

Sunlight hits a solar panel (Chapter 4) → **irradiance is measured and processed into weather features** (Chapter 4) → **features are combined with price history, demand, and calendar variables** (Chapters 1–4) → **a price model forecasts tomorrow's prices** (Chapters 6–7) → **the forecast is calibrated** (Chapter 8) → **an LP optimises battery dispatch** (Chapters 5, 9) → **the battery charges and discharges at actual prices** → **revenue is earned**.

Every link in this chain introduces error, and the errors **compound**. The pipeline perspective asks the most commercially important question: **which link is the weakest? Where should the next unit of effort be invested?**

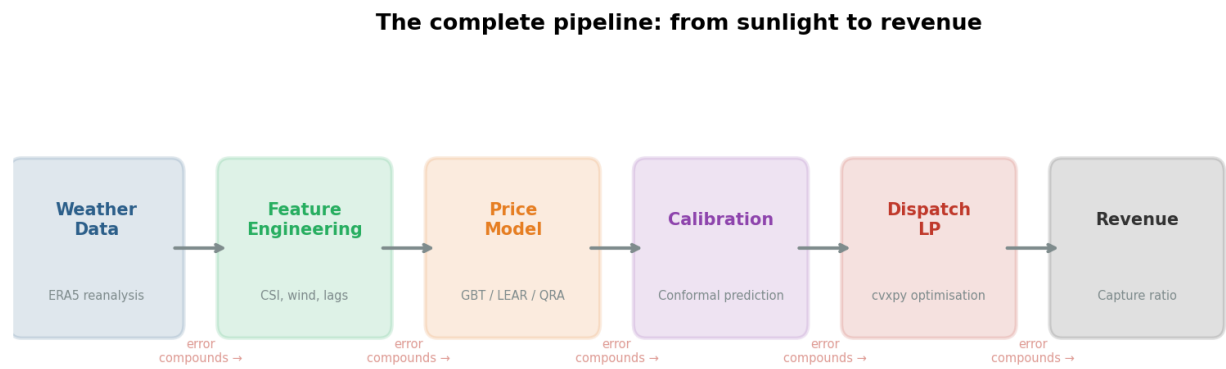


Figure 10.1 — The complete pipeline from weather data to battery revenue. Each stage transforms inputs into outputs, and each transformation introduces error. The pipeline perspective reveals where errors are largest and where improvements yield the most economic value.

Pipeline Components and Their Error Sources

Stage	Input	Output	Primary error source
Weather data	ERA5 reanalysis	Clear-sky index, wind speed, temperature	Reanalysis is a model, not measurement; spatial averaging over grid cells
Feature engineering	Raw variables	Feature matrix	Feature selection choices; transform assumptions; potential leakage
Price model	Features	Quantile forecasts	Model misspecification; training data limitations; regime changes
Calibration	Raw quantiles	Calibrated quantiles	Calibration window choice; non-stationarity; tail estimation
Dispatch LP	Price forecast + battery specs	Charge/discharge schedule	Forecast error propagation; constraint approximations
Execution	Schedule + actual prices	Revenue	Actual prices differ from forecast; market frictions; latency

Pipeline: A sequence of processing stages where each stage's output becomes the next stage's input. In machine learning, a pipeline typically includes data ingestion, feature engineering, model training, prediction, and post-processing. Each stage can introduce errors that propagate forward, potentially amplifying through subsequent stages. Evaluating a pipeline requires end-to-end assessment — optimising individual stages in isolation can produce a suboptimal whole.

The pipeline principle: Optimising each stage independently does not guarantee an optimal pipeline. A model that produces the lowest MAE might not produce the best dispatch revenue, because the dispatch LP cares about price ranking more than magnitude accuracy. The pipeline must be evaluated end-to-end — from weather data to capture ratio — not stage by stage.

Grey-Box Models

The Philosophy: Combining Physics and Data

Throughout this course, we have used two approaches to forecasting:

White-box models (also called physics-based or mechanistic models): Built from first principles about how the system works. For electricity prices, the merit-order dispatch process is a white-box model — it describes *why* prices are what they are based on the physical relationship between supply, demand, and generator costs.

Black-box models (also called data-driven or statistical models): Learn input-output relationships from historical data without incorporating domain knowledge. GBT and neural networks are black-box models — they find patterns in the data without "knowing" anything about how electricity markets work.

White-box model: A model built from explicit understanding of the underlying physical or economic process. The model's structure reflects the known mechanism — for electricity prices, the supply-demand balance and merit-order dispatch. White-box models are interpretable and extrapolate sensibly to unseen conditions, but they may miss patterns not captured by the assumed mechanism.

Black-box model: A model that learns relationships directly from data without incorporating explicit domain knowledge. The internal workings are opaque — we can see what goes in and what comes out, but the intermediate reasoning is not interpretable in physical terms. Black-box models are flexible and can capture unexpected patterns, but they may extrapolate poorly to conditions not seen in training data and provide limited physical insight.

A **grey-box model** combines both: it uses physical structure where the physics is well-understood, and data-driven learning where the physics is insufficient.

Grey-box model: A model that combines physical structure (white-box component) with data-driven learning (black-box component). The physical component encodes known relationships (e.g., higher net load → higher prices), while the data-driven component learns the residual — everything the physics doesn't explain. Grey-box models combine the interpretability and extrapolation properties of white-box models with the flexibility of black-box models.

For electricity prices, the natural grey-box structure is:

$$\text{price} = f(\text{net_load}) + g(\text{features}) + \varepsilon$$

where: - **f(net_load)** is the **white-box component** — the merit-order curve mapping net load to price. This captures the dominant, physically motivated relationship. - **g(features)** is the **black-box component** — a GBT model that captures everything else: time-of-day effects, autoregressive patterns, weather nuances, market dynamics. - ε is the irreducible noise — the randomness that no model can predict (generator trips, sudden demand changes, strategic bidding).

Grey-box model: best of both worlds

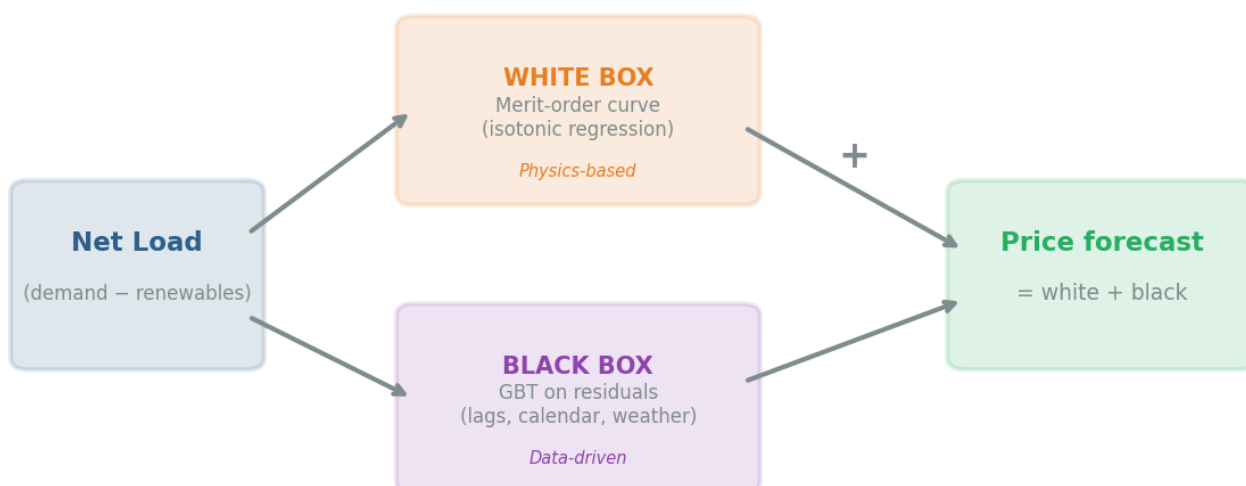


Figure 10.2 — The grey-box model architecture. The white-box stage (isotonic regression on net load) captures the dominant merit-order relationship. The black-box stage (GBT on the residual) captures secondary patterns that the physics does not explain. Together, they outperform either stage alone.

The Merit-Order Stage: Isotonic Regression

The white-box component estimates the **merit-order curve** — the relationship between net load and price — from historical data. We use **isotonic regression** for this, because it naturally enforces the one physical constraint we are certain of: **monotonicity**.

Merit-order curve: The empirical relationship between net load (demand minus renewable generation) and the electricity price. As net load increases, more expensive generators are dispatched and the price rises. The merit-order curve is approximately flat at low net load (cheap baseload sets the price) and steep at high net load (expensive peaking generators set the price). This is the hockey-stick shape from Chapter 3.

Isotonic regression: A non-parametric regression method that finds the best-fitting function constrained to be **monotonically non-decreasing** (or non-increasing). Given data points $(x_1, y_1), \dots, (x_n, y_n)$, isotonic regression finds the non-decreasing step function that minimises the sum of squared errors. It produces a piecewise-constant fit that preserves the ordering — if $x_1 < x_2$, the fitted value at x_1 is no larger than the fitted value at x_2 .

The procedure:

1. **Sort** all historical observations by net load (ascending)
2. **Fit isotonic regression:** Find the non-decreasing function $f(\text{net_load})$ that minimises the sum of squared errors between $f(\text{net_load})$ and the actual price
3. The result is a **step function** that approximates the supply stack — a staircase where each step corresponds to a different generation technology's marginal cost

Why isotonic regression instead of ordinary least squares regression or a polynomial fit?

Monotonicity is guaranteed. The merit-order effect guarantees that higher net load leads to higher or equal prices (in expectation). Linear regression might fit a line with a negative slope in some regions (due to noise), and polynomial regression might produce non-monotonic oscillations. Isotonic regression cannot violate monotonicity by construction.

No functional form assumed. We don't need to guess whether the relationship is linear, quadratic, exponential, or some other shape. Isotonic regression discovers the shape from the data — it could be a gentle curve, a sharp hockey stick, or a staircase. The algorithm adapts to whatever the data shows.

Robustness to outliers. The step-function nature means that a few extreme prices at high net load do not distort the fit at low net load. Each segment of the curve is estimated from its own observations.

Real-world example — the merit-order curve in SA1: The isotonic regression for SA1 might reveal: - Net load 500–1,500 MW: price $\approx$ \$25–\$40/MWh (wind and gas baseload) - Net load 1,500–2,500 MW: price $\approx$ \$40–\$80/MWh (more gas capacity needed) - Net load 2,500–3,000 MW: price $\approx$ \$80–\$200/MWh (expensive peakers dispatched) - Net load > 3,000 MW: price $\approx$ \$200–\$5,000+/MWh (scarcity pricing; all available capacity dispatched) This curve encodes the physical structure of the SA1 generation fleet in a single function. As the fleet evolves (coal retires, new renewables enter), the curve shifts — but its monotonic shape is preserved.

The Residual Model

After subtracting the merit-order prediction $f(\text{net_load})$ from the actual prices, the **residual** captures everything the physics does not explain:

$$\text{residual}_t = \text{price}_t - f(\text{net_load}_t)$$

This residual contains:

- **Time-of-day effects** beyond what net load captures — for example, generator bidding patterns differ between morning ramp-up and evening ramp-down, even at the same net load level
- **Autoregressive patterns** — yesterday's price residual predicts today's residual, because generator bidding strategies persist
- **Weather effects** not captured by net load — for instance, extreme temperature drives demand beyond what the net-load variable reflects (due to forecasting error in the demand model)
- **Market dynamics** — interconnector constraints, generator outages, strategic rebidding, and other factors that cause prices to deviate from the merit-order prediction

A GBT model trained on the residual has a **much easier job** than a GBT model trained on raw prices. The dominant nonlinearity (the hockey-stick merit-order relationship) has already been removed by the isotonic regression. The residual is more symmetric, less heavy-tailed, and more stationary than raw prices — all properties that make it easier for a machine learning model to learn accurately.

Why Grey-Box Outperforms Either Alone

The grey-box approach has four key advantages over a pure black-box (GBT on raw prices) or pure white-box (merit-order only) model:

1. Physical interpretability. The merit-order stage provides a direct connection to market structure. You can inspect the fitted curve and see how the generation fleet's cost structure translates to prices. This is invaluable for understanding **why** the model makes certain predictions and for detecting anomalies.

2. Better extrapolation. If net load reaches a level never seen in the training data — an unprecedented demand event or a massive renewable outage — the isotonic regression extrapolates the monotonic relationship sensibly (prices are at least as high as for the highest observed net load). A pure black-box model would extrapolate unpredictably, potentially producing nonsensical predictions.

Extrapolation: Making predictions for input values outside the range observed in the training data. All models must extrapolate when faced with unprecedented conditions. Models with physical structure (white-box and grey-box) tend to extrapolate sensibly because the physical relationships

(monotonicity, conservation laws) hold beyond the training range. Black-box models have no such guarantee — their predictions outside the training range are essentially arbitrary.

3. Feature efficiency. The merit-order stage captures a large fraction of price variance with a single feature (net load). This means the residual model can focus its capacity on secondary effects — patterns that are subtler and harder to learn but still economically valuable.

4. Structural stability. The monotonic net-load-to-price relationship is a fundamental property of merit-order markets that persists even as the generation mix changes. Coal retires, renewables grow, and the curve shifts — but the monotonicity is preserved. A grey-box model built on this structural insight is more robust to market evolution than a pure data-driven model that must re-learn the basic supply-demand relationship every time the market changes.

Ablation Studies

Why Remove Components?

An **ablation study** is a systematic method for understanding which components of a model contribute to its performance. The term comes from neuroscience, where researchers “ablate” (remove) brain regions to understand their function. In machine learning, we remove features or model components and measure the impact on performance.

Ablation study: A systematic evaluation method where individual components of a model are removed one at a time, and the model's performance is measured after each removal. The component whose removal causes the largest performance degradation is the most important. Ablation studies reveal the marginal contribution of each component and identify which components are essential versus dispensable.

For a price forecasting model with features grouped into categories {price lags, calendar features, demand features, weather features}, an ablation study proceeds as follows:

1. **Full model** — train with all feature groups → baseline MAE
2. **Remove price lags** — train without any lagged price features → MAE increases by X%
3. **Remove calendar features** — train without hour, day-of-week, month → MAE increases by Y%
4. **Remove demand features** — train without demand/net load → MAE increases by Z%
5. **Remove weather features** — train without temperature, wind, irradiance → MAE increases by W%

The feature group whose removal causes the largest MAE increase is the most important — the model depends on it most heavily.

Interpreting Ablation Results

Typical ablation results for NEM day-ahead price forecasting reveal a clear hierarchy:

Removed feature group	MAE increase	CRPS increase	Interpretation
Price lags	+30–50%	+25–40%	Dominant feature group; autoregressive information is critical
Calendar features	+10–20%	+8–15%	Diurnal and weekly patterns are significant secondary information
Demand features	+5–15%	+5–12%	Demand adds moderate but meaningful information
Weather features	+2–10%	+2–8%	Weather contributes but less than lags or calendar
All except price lags	+15–25%	+12–20%	Price lags alone capture most of the predictive power

The dominance of price lags: The most striking finding is that lagged prices (especially lag-48, yesterday's price at the same half-hour) dominate all other features. This is because the NEM is a *persistent* market — today's prices are strongly correlated with yesterday's prices. The persistence arises because the fundamental drivers (generator fleet composition, weather patterns, demand levels) change slowly relative to the 30-minute trading interval.

The relatively small contribution of weather features may seem surprising given Chapter 4's extensive discussion of weather's role in price formation. The explanation is subtle: **weather affects prices primarily through its effect on demand and renewable generation**, which are already partially captured by the demand feature. The residual contribution of weather features — beyond what demand already captures — is the ability to forecast *tomorrow's* conditions when they differ from today's.

Real-world example — when weather matters most: Weather features add the most value during weather regime transitions. If today is mild (25°C) and tomorrow will be extremely hot (42°C), the temperature forecast provides critical information that lagged prices and today's demand cannot capture. Without weather features, the model would extrapolate from today's mild conditions; with weather features, it anticipates the demand surge and price spike. The ablation shows that this regime-transition effect is worth 2–10% MAE improvement — small on average but

potentially very large on the specific days when it matters most (which happen to be the highest-revenue days for the battery).

Ablation vs Feature Importance

Ablation studies and GBT feature importance (Chapter 7) measure related but distinct things:

Marginal contribution: The change in model performance when a component is added or removed. Ablation measures the marginal contribution of removal — how much worse the model gets without the feature. Feature importance (gain-based) measures the marginal contribution within the model — how much the feature helps the model as trained. These can differ because of **redundancy**: if two features contain similar information, removing one has little impact (low ablation) because the other compensates, but both show high importance within the model because the algorithm uses both.

A feature might have **high importance but low ablation impact** if other features contain similar information (redundancy). For example, "demand" and "net load" are correlated — removing one has little impact because the other compensates. But within the model, both show high importance because the algorithm splits on whichever is most convenient at each tree node.

Conversely, a feature might have **moderate importance but high ablation impact** if it provides unique information that nothing else captures. A weather feature might be used in relatively few tree splits (moderate importance) but removing it completely eliminates the model's ability to anticipate weather-driven price changes (high ablation impact).

Fair Model Comparison

The Problem of Unfair Comparisons

Comparing forecasting models is fraught with methodological pitfalls. A careless comparison can make a poor model look excellent or an excellent model look poor. Published forecasting results — even in peer-reviewed journals — frequently suffer from comparison issues that undermine their conclusions.

Fair comparison: A comparison where all models are evaluated under identical conditions — same data, same time period, same evaluation metric, same information available at forecast time. Any difference in measured performance can then be attributed to the models themselves, not to differences in experimental setup. Achieving fair comparison requires careful attention to data splitting, feature information, and evaluation methodology.

Principles for Fair Comparison

1. Same data splits. All models must use exactly the same training and test sets. This seems obvious but is frequently violated in subtle ways: if Model A uses 3 years of training data and Model B uses 5 years, the comparison is confounded by the data size. Similarly, if Model A's hyperparameters were tuned using data that Model B's test set includes, the comparison is biased.

2. Same evaluation period. Results on different test periods are simply not comparable. A model tested during the volatile summer of 2019 (with multiple heatwave-driven spikes) will have a much higher MAE than the same model tested during the mild autumn of 2020, regardless of its quality. Always compare models on the same test window.

3. Rolling-origin evaluation. A single train/test split gives one performance estimate — a single number that might be unrepresentative of typical performance. Rolling-origin evaluation (Chapter 5) produces a distribution of performance estimates across multiple test windows, enabling robust comparison and statistical testing.

Rolling-origin evaluation: An evaluation method where the model is trained on a window of historical data, tested on the next period, then the window rolls forward and the process repeats. This produces multiple performance estimates (one per test window), providing a distribution rather than a single number. Rolling-origin evaluation simulates how the model would perform in production, where it is continuously retrained and evaluated on new data.

4. Statistical testing. The **Diebold-Mariano test** (introduced in Chapter 6) determines whether the performance difference between two models is **statistically significant** — unlikely to have arisen by chance — or merely random variation. A model that outperforms another by 2% MAE over a single test period might be genuinely better, or it might have gotten lucky. The DM test quantifies this uncertainty.

Diebold-Mariano (DM) test: A statistical test for comparing the predictive accuracy of two forecasting models. The null hypothesis is that both models have equal expected loss (e.g., equal MAE). The test accounts for serial correlation in the forecast errors, which is common in time series data. If the test rejects the null hypothesis ($p\text{-value} < 0.05$), we conclude that one model is significantly better than the other. If it does not reject, the observed difference could plausibly be due to random variation.

5. Multiple metrics. No single metric tells the whole story. Report at least:

- **MAE** — point forecast accuracy (how close the median prediction is to reality)
- **CRPS** — probabilistic forecast accuracy (how well the full distribution matches reality)
- **Capture ratio** — economic value (how much money the forecast-driven dispatch actually earns)

A model might win on MAE but lose on capture ratio if it is accurate on average but poor at predicting spikes — exactly the events that drive battery revenue. Conversely, a model with higher MAE might achieve better capture ratio if its spike predictions are more accurate.

The Naive Baseline as Anchor

The **naive baseline** (similar-day forecast from Chapter 5) serves as the anchor for all comparisons. Results should always be reported as improvements relative to the naive, not as absolute numbers.

Forecast skill: A model's performance measured relative to a reference baseline, typically the naive forecast. Skill is computed as: $\text{skill} = 1 - (\text{model error} / \text{naive error})$. A skill of 0 means the model is no better than the naive; a skill of 0.3 means the model reduces the error by 30% compared to the naive. Reporting skill rather than absolute error controls for the difficulty of the forecasting problem.

Why relative reporting matters: a MAE of \$25 is excellent if the naive achieves \$50 (skill = 0.50) but mediocre if the naive achieves \$28 (skill = 0.11). The absolute number alone tells you nothing about the forecasting problem's difficulty. Relative reporting lets you compare across regions (SA1 is harder than TAS1), across time periods (summer is harder than autumn), and across studies.

The Value Chain: Where to Invest

Marginal Value of Each Pipeline Stage

Not all improvements are equally valuable. A **back-of-the-envelope calculation** reveals where each dollar of effort yields the most revenue:

Marginal value: The additional value generated by one additional unit of investment or improvement. In the pipeline context, the marginal value of improving the price model by 1% MAE is the resulting increase in annual battery revenue. Marginal value varies across pipeline stages — the same investment might yield \$200K in additional revenue if directed at the dispatch stage but only \$50K if directed at the weather data stage.

Improvement	MAE reduction	CR improvement	Annual value (100MW/2h, SA1)
Better weather data (ERA5 → NWP ensemble)	2–5%	1–3%	\$50K–\$200K
Better price model (LEAR → tuned GBT)	5–15%	3–8%	\$200K–\$500K
Model combination (GBT → QRA ensemble)	3–8%	2–5%	\$100K–\$300K
Better calibration (raw → conformal)	0–3%	1–5%	\$50K–\$300K
Better dispatch (naive MPC → scenario MPC)	0% (same forecast)	2–5%	\$100K–\$300K
Better battery (85% → 90% RTE)	N/A	3–7%	\$200K–\$400K

The dispatch stage often offers the highest marginal returns. The largest revenue improvements often come from using the forecast better (the dispatch stage) rather than making the forecast more accurate. This is the concavity of the value-of-information curve (Chapter 9) in action: once the forecast is reasonably good, better *use* of the forecast matters more than a better forecast itself.

The 80/20 Rule in Practice

80/20 rule (Pareto principle): A widely observed pattern where approximately 80% of the outcome is produced by 20% of the effort. In forecasting, a simple model with a few key features often captures most of the predictive value, while the remaining accuracy requires disproportionate additional complexity. This does not mean the last 20% is not worth pursuing — just that the cost per unit of improvement increases dramatically.

For a practical battery dispatch deployment, the effort-to-value breakdown follows the Pareto principle strikingly closely:

The first 80% of value comes from: - Lag-48 and lag-336 price features - Basic calendar features (hour of day, day of week) - A well-tuned LightGBM model with early stopping - A simple LP dispatch with point forecasts (naive MPC) - This can be implemented in a few hundred lines of well-structured code

The next 15% of value comes from: - QRA combination of GBT + LEAR forecasts - Conformal calibration of the quantile forecasts - Chance-constrained or scenario MPC dispatch - Demand and weather features - This requires a few thousand lines of code plus ongoing data pipeline management

The last 5% of value comes from: - Neural network models or hybrid architectures - Sophisticated scenario generation (copula-based, historical analogue) - Adaptive conformal prediction with regime detection - Continuous online retraining with concept drift detection - Grey-box modelling with isotonic merit-order estimation - This requires a dedicated team, significant infrastructure, and ongoing research

Real-world industry perspective: For most battery operators, reaching the 80% level is the clear priority — it provides the bulk of the economic value with manageable engineering effort. The next 15% is worthwhile for operators with in-house data science teams and a competitive edge

mandate. The last 5% is the domain of large energy trading firms and research organisations for whom every marginal percentage point of capture ratio translates to millions of dollars across their fleet of batteries. The key strategic question is: given your battery's size, location, and competitive environment, where does the cost of the next improvement exceed its value? For a single 100 MW battery, the answer might be the 95th percentile. For a portfolio of twenty 100 MW batteries, the answer might be the 99th percentile — because the same modelling investment is amortised across a much larger revenue base.

Limitations and Honest Assessment

What This Project Does Not Address

No model is complete, and intellectual honesty requires acknowledging the simplifications we have made. Each of the following limitations represents a gap between our simplified project and a production-grade battery dispatch system:

1. Intra-day re-optimisation. Our MPC re-optimises once per day (or once per trading period). A production battery re-optimises every 5–30 minutes as new information arrives: updated weather forecasts, real-time demand data, generator outage announcements, and AEMO's pre-dispatch updates. The more frequently the dispatch is re-optimised, the better it can respond to unfolding events — particularly during volatile periods when prices change rapidly.

Intra-day re-optimisation: The process of updating the battery's dispatch schedule multiple times within a day as new information becomes available. Each re-optimisation incorporates the latest data — updated weather forecasts, actual demand observations, generator status changes — producing a schedule that reflects current conditions rather than the previous day's forecast. This is particularly valuable during volatile periods when the day-ahead forecast has become stale.

2. FCAS co-optimisation. Batteries earn significant revenue from Frequency Control Ancillary Services (Chapter 9 glossary), but our project optimises only energy arbitrage. A production system would **co-optimize** energy and FCAS simultaneously — determining for each half-hour whether the battery should discharge for energy revenue, provide FCAS (which requires reserving capacity), or do both (partial dispatch). The co-optimisation is more complex (it involves multiple interacting markets) but can increase total revenue by 20–40%.

3. Degradation modelling. We use a simple cycle-count constraint (maximum cycles per day). Real battery degradation is far more nuanced: it depends on the **depth of discharge** (shallow cycles are less damaging than deep cycles), the **charge rate** (fast charging degrades faster), **temperature** (heat accelerates chemical degradation), and **calendar aging** (batteries degrade even when idle). Sophisticated dispatch models incorporate degradation as a variable cost that depends on the specific charge/discharge profile, not just the total number of cycles.

4. Market impact. Our LP assumes the battery is a **price-taker** — its actions do not affect market prices. For a small battery, this is reasonable. For a large battery (100+ MW in a region like SA1 with 2,000–3,000 MW of total demand), the battery's charging adds to demand (raising prices) and its discharging adds to supply (lowering prices). A production system must account for this **market impact** — the degree to which the battery's own actions change the prices it faces.

Price-taker assumption: The assumption that a market participant's actions do not affect market prices. A small battery in a large market is approximately a price-taker — its 10 MW of charging has negligible impact on a 10,000 MW market. A large battery may violate this assumption: a 200 MW battery in SA1 (total demand ~2,500 MW) represents 8% of the market, and its charging/discharging could meaningfully move prices.

5. Network constraints. We ignore transmission losses, network charges, and locational constraints. In reality, the battery faces a net price that differs from the regional reference price due to **marginal loss factors** (MLFs), network use charges, and potential congestion at its connection point. These can reduce effective revenue by 5–15%.

6. Operational constraints. Real batteries have physical limitations not captured in our simplified LP: minimum up/down times (the battery cannot switch between charging and discharging instantaneously), ramp rate limits (how fast the battery can increase or decrease its power output), auxiliary power consumption (the battery's own cooling and control systems consume electricity), and state-of-health degradation that reduces capacity over time.

Sources of Forecast Error

The gap between our capture ratio and 1.0 (perfect foresight) comes from fundamental and practical limitations:

1. Price spikes are inherently unpredictable. Some spikes are caused by events that no forecast can anticipate: generator mechanical failures (a 500 MW coal unit trips without warning), sudden demand surges (a temperature forecast error of 3°C can shift demand by 1,000 MW), or transmission line faults. These events set a **fundamental limit** on the achievable capture ratio — even the best possible forecast cannot predict mechanical failures.

2. The forecast horizon is long. Forecasting 24 hours ahead is much harder than forecasting 1 hour ahead. Forecast errors grow roughly with the square root of the horizon — a 24-hour forecast has roughly 5× the error of a 1-hour forecast. For weather-dependent periods (afternoon solar, evening wind), the day-ahead weather forecast is the binding constraint.

3. Regime changes. The electricity market structure evolves continuously: generators retire, new capacity is built, demand patterns shift (driven by rooftop solar, electric vehicles, and changing industrial activity), and market rules change. Models trained on historical data cannot anticipate structural breaks — they can only adapt after the change has occurred and enough new data has accumulated.

Regime change (structural break): A fundamental change in the statistical properties of the data — for example, a new power plant coming online changes the merit-order curve, or a rule change alters bidding behaviour. Regime changes invalidate models trained on pre-change data because the learned relationships no longer hold. Detecting and adapting to regime changes is one of the hardest problems in time series forecasting.

4. Strategic behaviour. Generator bidding in the NEM is partly strategic — generators adjust their bids based on expected market conditions, competitor behaviour, and fuel contract positions. This strategic layer is inherently difficult to forecast because it involves **game-theoretic reasoning** — each generator's optimal bid depends on what it expects other generators to bid, creating a complex feedback loop that is fundamentally different from the physical drivers (weather, demand) that our models capture.

What a Production System Would Add

A production-grade forecasting and dispatch system builds on the foundations of this course but adds several critical components:

Component	What it adds	Why it matters
Real-time data feeds	Live SCADA, demand, weather, generator status	Enables intra-day re-optimisation; reduces forecast staleness
Ensemble of NWP models	Multiple weather forecast sources (BoM ACCESS, ECMWF, GFS)	More robust weather prediction; captures model uncertainty
Continuous retraining	Models retrained daily or weekly with latest data	Adapts to evolving market conditions and regime changes
Intra-day update loop	Forecasts updated every 30 minutes; dispatch re-optimised	Captures within-day information; responds to unfolding events
Risk management	Position limits, drawdown controls, exposure monitoring	Prevents catastrophic losses during extreme market events
FCAS co-optimisation	Joint energy and ancillary service scheduling	Increases total revenue by 20–40%
Human oversight	Traders review and potentially override automated decisions	Essential during unprecedented events (major outages, policy changes)
Monitoring and alerting	Automated detection of model degradation, data quality issues	Prevents silent failure; maintains system reliability

The Broader Context

Energy Storage Economics

Battery energy storage is one of the fastest-growing segments of the global electricity sector. Understanding the economic landscape provides context for why the forecasting and dispatch techniques in this course matter.

Levelised cost of storage (LCOS): The per-MWh cost of storing and discharging electricity, accounting for the battery's capital cost, operating costs, degradation, and financing over its lifetime. LCOS is the storage equivalent of the more familiar LCOE (Levelised Cost of Energy) for generators. For a battery to be economically viable, its annual revenue must exceed its annualised LCOS multiplied by its energy throughput.

The economics of battery storage have transformed dramatically over the past decade:

Lithium-ion cell costs have fallen from approximately \$1,200/kWh in 2010 to roughly \$140/kWh in 2024 — an 88% reduction in 14 years. This decline, driven primarily by electric vehicle manufacturing scale, has made grid-scale batteries economically viable in ways that seemed impossible a decade ago.

Installed capacity is growing at 40–60% per year globally. Australia's NEM has seen particularly rapid growth, with several hundred megawatts of new battery capacity added annually.

Revenue opportunities are growing as renewable penetration increases. More wind and solar means more price volatility (negative prices when renewables are abundant, spikes when they are scarce), which directly increases arbitrage revenue.

The economic viability of batteries depends on the full **revenue stack** (energy arbitrage + FCAS + network support + capacity payments). In most NEM regions, arbitrage alone is currently sufficient for marginal economics (5–8 year payback), with the other revenue streams providing additional upside. As renewable penetration increases and price volatility grows, the arbitrage component is expected to become increasingly dominant.

Real-world example — the Hornsdale Power Reserve: The Hornsdale Power Reserve (HPR), a 150 MW / 194 MWh battery in South Australia, was the world's largest lithium-ion battery when commissioned in 2017. It demonstrated that grid-scale batteries could be commercially viable, earning substantial revenue from both FCAS and energy arbitrage. In its first year, HPR's FCAS revenue alone exceeded expectations, and its energy arbitrage revenue grew as operators learned to forecast and dispatch more effectively. The success of HPR catalysed a wave of battery investment in the NEM and globally.

The Forecasting Industry

Electricity price forecasting is a substantial industry in its own right, employing quantitative analysts, data scientists, and energy market experts across several sectors:

Energy trading companies employ teams of "quants" who develop proprietary forecasting models, competing to predict prices more accurately than their rivals. Better forecasts translate directly to higher trading profits.

Battery operators (both utility-scale systems like HPR and smaller behind-the-meter batteries at commercial sites) use forecasts for dispatch optimisation. The quality of the dispatch — and hence the battery's revenue — depends directly on forecast quality.

Electricity retailers use price forecasts for contract pricing (setting the fixed price they offer to customers) and hedging (buying financial contracts to reduce their exposure to spot price volatility).

System operators (AEMO in Australia) produce their own forecasts (pre-dispatch) for reliability planning — ensuring sufficient generation is available to meet demand. AEMO's pre-dispatch forecast also serves as a public benchmark against which commercial forecasters can be compared.

The methods covered in this course — LEAR, GBT, QRA, conformal prediction — represent the current mainstream of both academic research and industry practice. More exotic approaches exist (deep reinforcement learning for dispatch, attention-based Transformer architectures, physics-informed neural networks, generative adversarial networks for scenario generation) but have not consistently demonstrated practical advantages over well-tuned GBT models with good features in rigorous head-to-head comparisons.

What Makes This Problem Interesting

Electricity price forecasting sits at the intersection of multiple disciplines — and it is this interdisciplinary nature that makes it both challenging and rewarding:

- **Power systems engineering:** Understanding how the grid works, what drives prices, and how the merit-order dispatch process creates the statistical properties we observe
- **Statistics and econometrics:** Time series modelling, probabilistic forecasting, calibration theory, model evaluation methodology
- **Machine learning:** Feature engineering, gradient boosting, neural networks, ensemble methods, hyperparameter optimisation
- **Operations research:** Linear programming, stochastic optimisation, model predictive control, decision-making under uncertainty
- **Economics:** Market design, strategic bidding, risk management, investment analysis

No single discipline provides a complete answer. The best forecasters and battery operators combine physical intuition about the electricity system (from power engineering) with statistical rigour (from econometrics) and computational efficiency (from machine learning), applied through the lens of optimal decision-making (from operations research) in a market context (from economics).

The ultimate takeaway: The value of electricity price forecasting lies not in the forecast itself but in the decisions it enables. A forecast that sits in a spreadsheet has zero value. A forecast that drives an optimised battery dispatch — charge during the cheap periods, discharge during the expensive ones, with risk management that accounts for forecast uncertainty — can generate millions of dollars of revenue per year. Every technique in this course, from data ingestion (Chapter 1) to conformal calibration (Chapter 8), serves this single practical goal: converting information about the future into money today.

Glossary

Term	Definition
Pipeline	A sequence of processing stages where each stage's output becomes the next stage's input
White-box model	A model built from explicit physical or economic principles
Black-box model	A data-driven model that learns relationships without incorporating domain knowledge
Grey-box model	A model combining physical structure with data-driven learning
Isotonic regression	A non-parametric regression constrained to produce monotonically non-decreasing predictions
Merit-order curve	The empirical relationship between net load and electricity price
Extrapolation	Making predictions for inputs outside the range observed in training
Ablation study	Systematic removal of model components to measure their individual contributions
Marginal contribution	The change in performance when a component is added or removed
Fair comparison	Evaluating models under identical conditions to ensure performance differences reflect model quality
Rolling-origin evaluation	Repeatedly training and testing across multiple time windows
Diebold-Mariano test	A statistical test for comparing forecasting model accuracy
Forecast skill	Performance relative to a naive baseline; controls for problem difficulty
Marginal value	The additional value from one additional unit of improvement
80/20 rule (Pareto principle)	~80% of outcomes come from ~20% of effort or causes
Intra-day re-optimisation	Updating the dispatch schedule multiple times within a day

FCAS co-optimisation	Jointly optimising energy arbitrage and ancillary service provision
Price-taker assumption	The assumption that a participant's actions do not affect market prices
Regime change (structural break)	A fundamental change in the data's statistical properties
LCOS (Levelised Cost of Storage)	The per-MWh cost of storage over the battery's lifetime
Revenue stack	The combination of all revenue streams available to a battery

Summary

The capstone assembles every technique from the course into a single end-to-end pipeline: weather data flows through feature engineering and price modelling into calibrated probabilistic forecasts, which drive optimised battery dispatch that generates revenue from actual prices. Grey-box models — combining a physically motivated merit-order stage (isotonic regression on net load) with a data-driven residual stage (GBT) — outperform either pure physics or pure machine learning by leveraging the strengths of both. Ablation studies confirm that price lags dominate feature contributions, with weather features adding incremental but economically meaningful value during regime transitions. Fair model comparison requires identical data splits, rolling-origin evaluation, statistical testing, and multiple metrics (MAE, CRPS, capture ratio) to avoid misleading conclusions. The marginal value of pipeline improvements is highest at the dispatch stage — using the forecast well often matters more than making the forecast better — and follows the 80/20 rule: a simple GBT model with lag features and basic MPC captures the bulk of available revenue. A production system would add real-time data, continuous retraining, intra-day re-optimisation, FCAS co-optimisation, and human oversight. The capture ratio — the fraction of perfect-foresight revenue achieved — is the single metric that ties the entire pipeline together, converting forecast quality into economic value: for a 100 MW / 2h battery in SA1, each percentage point of capture ratio is worth roughly \$80K–\$150K per year in arbitrage revenue alone.

11. FCAS and Co-optimised Dispatch

Why 50 Hz Matters

The Australian power grid operates at a frequency of 50 Hz. This is not a regulation or a preference — it is a physical constraint that emerges from the synchronous rotation of all connected generators. Every turbine shaft in the National Electricity Market (NEM) rotates in lockstep: at 50 Hz, a two-pole generator spins at exactly 3,000 revolutions per minute. When supply and demand are perfectly balanced, the frequency holds steady. When they diverge, the frequency moves.

The physics is direct. If a large generator trips offline — say a 660 MW coal unit in the Latrobe Valley — the remaining generators cannot instantly increase output to compensate. The energy shortfall comes instead from the rotational kinetic energy stored in the spinning turbines themselves: they slow down slightly, converting their angular momentum into electrical energy. This deceleration manifests as a frequency drop. A loss of 660 MW on the eastern seaboard might cause the frequency to dip from 50.00 Hz to 49.80 Hz within a few seconds.

System frequency: The rate at which the alternating current waveform oscillates, measured in hertz (Hz). In the NEM, the nominal frequency is 50 Hz. The frequency reflects the instantaneous balance between electrical supply and demand across the entire interconnected grid. A frequency above 50 Hz means supply exceeds demand (generators are accelerating); below 50 Hz means demand exceeds supply (generators are decelerating).

Conversely, if demand drops suddenly — a large aluminium smelter shuts down, or a cloud clears and solar output surges — the generators have excess energy and speed up. The frequency rises above 50 Hz.

Why does frequency deviation matter? The consequences escalate with the magnitude of the deviation:

Frequency	Condition	Consequence
50.00 Hz	Nominal	Normal operation
49.85–50.15 Hz	Normal operating band	Generators adjust automatically; no action required
49.75–49.85 Hz	Outside normal band	AEMO activates contingency reserves
49.50–49.75 Hz	Significant deviation	Under-frequency load shedding (UFLS) schemes may trigger
< 49.00 Hz	Emergency	Cascading trips; potential system black
> 50.50 Hz	Emergency	Over-frequency generator protection triggers disconnection

The grid frequency is a single number that summarises the entire system's health. Every generator, every load, every battery connected to the grid experiences the same frequency at the same time (ignoring small local deviations due to network impedance). This makes frequency a uniquely important signal — and maintaining it at 50 Hz is the most critical task in power system operations.

Key point: The frequency is not "set" by anyone — it *emerges* from the balance of supply and demand. AEMO cannot directly control the frequency any more than a central bank can directly control inflation. Instead, AEMO procures services that help generators and batteries respond to frequency deviations, restoring the balance. These services are collectively called Frequency Control Ancillary Services (FCAS).

The Inertia Problem

Historically, the NEM had abundant **inertia** — the rotational kinetic energy stored in the heavy spinning masses of coal and gas turbines. A large synchronous generator might weigh hundreds of tonnes and spin at 3,000 RPM; the energy stored in that rotation acts as a buffer, slowing the rate at which frequency changes when supply and demand diverge. High inertia means the frequency changes slowly, giving other generators time to respond.

Inertia: The resistance of the power system to changes in frequency, determined by the total rotational kinetic energy of all synchronous generators connected to the grid. High inertia means frequency changes slowly after a disturbance (giving time for corrective action); low inertia means frequency changes rapidly (requiring faster responses). Measured in megawatt-seconds (MWs) or equivalently megajoules (MJ).

As coal plants retire and are replaced by wind and solar — which connect through power electronics and do not inherently provide inertia — the system's total inertia is declining. South Australia, with its high renewable penetration, routinely operates with inertia levels 50–70% below the levels of a decade ago. Lower inertia means faster frequency changes after disturbances, which means faster-responding reserves are needed. This is the fundamental driver behind the introduction of the very-fast 1-second FCAS markets in October 2023, and it is the reason batteries — which can respond in milliseconds — have become the dominant providers of fast frequency response.

The SA system black — 28 September 2016: South Australia experienced a state-wide blackout after severe storms caused multiple transmission line faults. The sequence of events exposed the fragility of a low-inertia system: wind farms disconnected due to protection settings, the Heywood interconnector tripped on overload, and the remaining SA generators could not maintain frequency. The frequency collapsed from 50 Hz to below 47 Hz in under two seconds. The entire state lost power. This event was a watershed moment for FCAS — it demonstrated that the NEM's existing frequency control arrangements, designed for a system with high inertia, were inadequate for the emerging low-inertia reality. The subsequent reforms included mandatory minimum inertia requirements, new generator technical standards, and ultimately the creation of the very-fast FCAS

Regulation vs Contingency Services

FCAS in the NEM falls into two broad categories, defined by what they respond to:

Regulation FCAS

Regulation services correct the small, continuous frequency deviations that occur during normal operation — the constant jitter caused by thousands of loads switching on and off, small fluctuations in wind and solar output, and the inherent imprecision of dispatching generators in five-minute intervals. The frequency never sits perfectly at 50.00 Hz; it drifts slightly around it, and regulation services nudge it back.

Regulation FCAS: Frequency control services that correct small, continuous deviations from 50 Hz during normal operation. Regulation is provided by generators and batteries that adjust their output in response to an Automatic Generation Control (AGC) signal sent by AEMO every four seconds. There are two regulation markets: **regulation raise** (increase output to push frequency up) and **regulation lower** (decrease output to push frequency down). Regulation providers must be able to sustain their response for at least a specified duration and respond continuously, not just to discrete events.

Regulation providers are controlled by AEMO's **Automatic Generation Control (AGC)** system, which sends a control signal every four seconds. The AGC measures the current frequency error and distributes corrective actions across all enabled regulation providers, proportional to their enabled capacity. A battery providing 10 MW of regulation raise might receive an AGC signal to increase output by 3 MW, then 5 MW, then 2 MW, adjusting every four seconds as the frequency drifts.

The regulation signal is continuous and bidirectional (though raise and lower are separate markets). Over the course of a day, a regulation provider's net energy delivery is typically close to zero — the raises and lowers roughly cancel out. This means regulation has minimal impact on the battery's state of charge (SoC), which is a significant advantage: the battery can earn FCAS revenue without using up its energy for arbitrage.

Regulation in practice — the AGC dance: A battery enabled for 20 MW of regulation raise receives the AGC signal every four seconds. Over a 30-minute dispatch interval, it might follow a pattern like: +3 MW, +7 MW, +2 MW, -1 MW, +5 MW, +12 MW, +4 MW, +1 MW, -2 MW, +8 MW... The signal varies rapidly as AEMO chases the frequency error. The battery's average output might be +5 MW over the interval — far less than the 20 MW it has been paid to keep available. The key requirement is that the battery *can* deliver up to 20 MW within the response timeframe, not that it *does* deliver 20 MW continuously.

Contingency FCAS

Contingency services respond to large, sudden disturbances — the loss of a major generator, a transmission line trip, or a sudden load disconnection. These are discrete events, not continuous fluctuations. When a 660 MW coal unit trips, the frequency drops sharply, and contingency FCAS providers activate automatically (not via AGC) to arrest the decline and restore the frequency.

Contingency FCAS: Frequency control services that respond to large, sudden frequency disturbances such as generator trips or transmission line failures. Contingency services are activated automatically based on local frequency measurements, not by AGC signals. They are categorised by their response speed: 1-second (very fast), 6-second (fast), 60-second (slow), and 5-minute (delayed). Each category has both raise and lower markets, totalling eight contingency FCAS markets.

Contingency services are categorised by how quickly they must respond:

Service	Response	Sustain	Provider
Very fast (R1/L1)	1 s	1 s min	Batteries, demand response
Fast (R6/L6)	6 s	60 s min	Batteries, hydro, gas
Slow (R60/L60)	60 s	5 min	Gas, hydro, batteries
Delayed (R5/L5)	5 min	10 min	Gas, interruptible load

The response timeframes reflect the physics of frequency recovery after a disturbance:

- First 1 second (very fast):** The frequency drops most rapidly immediately after the contingency. Very-fast providers must arrest this initial plunge before it triggers under-frequency load shedding.
- First 6 seconds (fast):** Fast providers sustain the arrest and begin stabilising the frequency.
- First 60 seconds (slow):** Slow providers take over from the fast providers, allowing them to return to their pre-contingency operating point.
- First 5 minutes (delayed):** Delayed providers relieve the slow providers, fully restoring the frequency and replenishing the reserves for the next contingency.

Key point: A single physical unit — such as the Mannum battery — can provide multiple contingency services simultaneously, as long as it has sufficient headroom. If Mannum is generating 40 MW and can ramp up to 100 MW, it has 60 MW of headroom. It might offer 20 MW of very-fast

raise, 20 MW of fast raise, and keep 20 MW as a buffer. The key constraint is that the same megawatt cannot be offered into two different services — the reserves do not overlap.

The Key Difference

The operational distinction between regulation and contingency is important for battery economics:

- **Regulation** requires continuous, small adjustments. The battery must follow the AGC signal continuously, which causes small, frequent cycles. Over a day, the energy delivered nets close to zero, but the constant cycling adds wear.
- **Contingency** requires the battery to keep capacity in reserve and deliver it only when a contingency event occurs. Most of the time, the battery is paid to do nothing — it earns the enablement payment for being available. Only when the frequency deviates beyond the trigger threshold does the battery actually inject or absorb power. Contingency events are rare (a few per week of significant magnitude), so the energy throughput from contingency services is minimal.

For a battery operator, contingency FCAS is often more attractive than regulation because the revenue comes primarily from **availability** rather than **delivery** — the battery earns money for keeping headroom available, not for cycling. This preserves the SoC for energy arbitrage and minimises degradation.

The Ten FCAS Markets

The NEM operates ten distinct FCAS markets, each cleared every five minutes alongside the energy market:

Market	Code	Response	Added
Regulation raise	RAISEREG	AGC	Original
Regulation lower	LOWERREG	AGC	Original
Very fast raise	RAISE1SEC	1 s	Oct 2023
Very fast lower	LOWER1SEC	1 s	Oct 2023
Fast raise	RAISE6SEC	6 s	Original
Fast lower	LOWER6SEC	6 s	Original
Slow raise	RAISE60SEC	60 s	Original
Slow lower	LOWER60SEC	60 s	Original
Delayed raise	RAISE5MIN	5 min	Original
Delayed lower	LOWER5MIN	5 min	Original

The original eight markets have operated since the NEM's inception in 1998. The very-fast 1-second markets were added in October 2023 specifically to address the declining-inertia problem. These two new markets have been transformative for battery economics.

The 1-Second Markets

The very-fast raise (RAISE1SEC) and very-fast lower (LOWER1SEC) markets require providers to respond within one second of a frequency deviation. This speed requirement effectively excludes thermal generators (which take seconds to minutes to ramp) and makes batteries and demand-response loads the only viable providers.

Battery dominance of RAISE1SEC: In the first year of the 1-second markets (October 2023 to October 2024), batteries provided over 90% of all RAISE1SEC enablement across the NEM. No gas turbine, no hydro unit, no coal plant can respond within one second with the precision and repeatability that AEMO requires. The 1-second markets were designed with batteries in mind — they create a revenue stream that only batteries (and some demand-response loads) can access, partially compensating batteries for the declining value of arbitrage as more batteries enter the market.

The 1-second markets are particularly attractive for a battery like Mannum because:

1. **No competition from thermal generators.** The response-speed requirement creates a natural barrier to entry.
2. **Minimal energy throughput.** Like other contingency services, 1-second FCAS is paid primarily for availability. The battery keeps headroom but rarely needs to deliver it.
3. **Stacking with other services.** A battery can provide 1-second raise and 6-second raise and 60-second raise simultaneously, as long as the total reserved capacity does not exceed the available headroom.
4. **Complementary to arbitrage.** The headroom reserved for 1-second FCAS reduces the energy available for arbitrage, but the FCAS price often compensates for the foregone arbitrage revenue — especially during periods of low price volatility when arbitrage opportunities are slim.

Price Behaviour Across FCAS Markets

FCAS prices behave very differently from energy prices. Most of the time, FCAS prices are low — typically \$1–\$15/MWh across most markets. The NEM usually has ample reserves, and the many batteries, hydro units, and gas turbines competing to provide FCAS keep prices near the marginal cost of availability.

But FCAS prices spike violently when reserves are scarce. A large generator trip in South Australia on a hot afternoon — when the system is already stressed, the interconnectors are near their limits, and local reserves are thin — can cause the RAISE6SEC price to spike from \$5/MWh to \$5,000/MWh or even \$15,100/MWh (the FCAS market price cap) for one or more dispatch intervals. These spikes are brief but extreme, and they dominate the annual FCAS revenue for providers who happen to be enabled when they occur.

Figure 11.1 — FCAS raise prices (RAISE6SEC) in SA1 over three months. Prices sit near \$5/MWh for most intervals but spike to \$300–\$15,100/MWh during contingency events and enablement scarcity. These rare spikes dominate annual FCAS revenue. The timing and magnitude of spikes are extremely difficult to forecast.

This price behaviour has two implications for battery strategy:

1. **Average FCAS revenue is modest.** A time-weighted average FCAS price of \$5–\$15/MWh, applied to the enabled capacity, produces a steady but unspectacular revenue stream.
2. **Spike FCAS revenue is substantial but unpredictable.** A battery enabled for 50 MW of RAISE6SEC that happens to be enabled during a \$15,100/MWh spike earns \$12,583 in that single five-minute interval ($\$15,100 \cdot 50 \text{ MW} \cdot 5/60 \text{ hours}$). A few such events per year can contribute significantly to annual revenue.

Key point: FCAS revenue has a "lottery ticket" quality — the expected value over a year is meaningful, but the realisation in any given week is highly variable. This makes FCAS revenue forecasting fundamentally different from energy price forecasting. Energy prices have strong daily and seasonal patterns that models can exploit; FCAS price spikes are driven by contingency events (generator trips, transmission faults) that are essentially random.

FCAS Revenue as a Share of Total Battery Revenue

For a well-operated battery in the NEM, FCAS revenue typically accounts for 20–40% of total revenue, with the remainder coming from energy arbitrage:

Figure 11.2 — Breakdown of total battery revenue by source for a representative NEM battery over 2024–2025. Energy arbitrage dominates, but FCAS collectively contributes a material share. The 1-second markets, despite being the newest, already contribute meaningfully because of the limited competition from non-battery providers.

Revenue source	Typical share	Variability
Energy arbitrage	50–70%	Driven by price volatility; higher in SA1, QLD1
Regulation raise/lower	5–10%	Relatively stable; depends on AGC enablement
1-second raise/lower	5–15%	Growing; limited competition keeps prices higher
6-second raise/lower	5–10%	Spike-dependent; high variance across months
60-second raise/lower	2–5%	Lower value; more competition from gas
5-minute raise/lower	1–3%	Lowest FCAS value; most competition

These shares vary significantly by region (SA1 has higher FCAS value due to lower inertia and higher contingency frequency), by season (summer has more stress events), and by the number of batteries competing for FCAS enablement. As more batteries enter the NEM, FCAS prices are expected to face downward pressure, but the declining-inertia trend partially offsets this by increasing the volume of FCAS required.

Co-optimisation in One Dispatch

How NEMDE Works

Every five minutes, AEMO runs the **National Electricity Market Dispatch Engine (NEMDE)** to clear the energy market and all ten FCAS markets simultaneously. This is not a sequence of separate auctions — it is a single, integrated optimisation that finds the least-cost combination of energy dispatch and FCAS enablement across all generators, batteries, and loads in the NEM.

Co-optimisation: The simultaneous optimisation of energy dispatch and FCAS enablement in a single mathematical program. Co-optimisation ensures that the total cost of meeting both energy demand and FCAS requirements is minimised, accounting for the fact that a generator providing FCAS reserves cannot simultaneously use that capacity for energy production. Without co-optimisation, the energy and FCAS markets would be cleared independently, leading to higher total cost because the interdependence between energy headroom and FCAS availability would not be priced correctly.

The co-optimisation is essential because energy and FCAS compete for the same physical capacity. A battery with 100 MW of discharge capacity can offer 100 MW into the energy market, or 100 MW of FCAS raise, or any split between the two — but the total cannot exceed 100 MW. If the energy price is \$200/MWh and the RAISE6SEC price is \$5/MWh, the battery should clearly dispatch energy. But if the RAISE6SEC price is \$300/MWh (during a scarcity event), the battery should reserve capacity for FCAS. NEMDE makes this allocation optimally for every generator and battery in the system, every five minutes.

NEMDE solves a large linear program (LP) with the following structure:

Objective: Minimise the total cost of energy dispatch plus the total cost of FCAS enablement across all regions and services.

Decision variables: For each generating unit (including batteries): the energy dispatch level, and the enablement quantity for each of the ten FCAS markets.

Constraints: - Energy supply must meet demand in each region (accounting for interconnector flows and losses) - FCAS enablement must meet the FCAS requirement in each region and service - Each unit's energy dispatch plus FCAS enablement cannot exceed its physical capacity (this is where the joint capacity constraint — the trapezium — enters) - Interconnector flow limits - Ramp rate limits - Unit-specific constraints (minimum generation, maximum capacity, etc.)

NEMDE (National Electricity Market Dispatch Engine): The software system operated by AEMO that clears the NEM every five minutes. NEMDE takes as input the bids and offers from all generators and loads, the demand forecasts, the FCAS requirements, the network constraints, and the interconnector limits. It solves a co-optimisation LP to find the least-cost dispatch of energy and enablement of FCAS across all regions and services. The shadow prices (dual variables) of the demand and FCAS requirement constraints become the market prices for energy and each FCAS service.

The **market prices** for energy and each FCAS service are the **shadow prices** (dual variables) of the corresponding demand and requirement constraints. This is the same mechanism as energy price formation, which was introduced in Chapter 3: the price equals the cost of supplying one additional megawatt of energy (or one additional megawatt of FCAS) at the optimal dispatch point. Co-optimisation means that the energy price and FCAS prices are determined simultaneously and influence each other — a tight FCAS market can raise the energy price (because generators that would otherwise produce cheap energy are diverted to FCAS), and a tight energy market can raise FCAS prices (because the opportunity cost of reserving capacity for FCAS — foregone energy revenue — is high).

Using nempy to See Co-optimisation

nempy (developed by UNSW-CEEM) is an open-source Python implementation of the NEM dispatch procedure. It solves the same co-optimisation LP that NEMDE solves, using historical bid and offer data. This makes it an invaluable tool for understanding how co-optimisation works and how a battery's bids affect both energy and FCAS prices.

To see co-optimisation in action, start with a concrete example. Load the historical bid stack for SA1 at 17:30 on 15 January 2026, add the Mannum battery's energy and FCAS offers, and call nempy's dispatch solver. The output shows:

1. **The energy dispatch** for every generator in the region — how many megawatts each unit is producing
2. **The FCAS enablement** for every unit and every FCAS service — how many megawatts of each FCAS market each unit is providing
3. **The energy price** and all ten **FCAS prices** for the interval
4. **The binding constraints** — which physical limits (generator capacity, interconnector flow, FCAS requirement) are constraining the solution

A nempy experiment — perturbing one input: Start with the base case: Mannum bids 100 MW of energy at \$0/MWh (price-taking, as discussed in Chapter 3) and offers no FCAS. Note the energy price (say \$185/MWh) and the RAISE6SEC price (say \$8/MWh). Now modify Mannum's offers: bid only 60 MW of energy and offer 40 MW of RAISE6SEC at \$0/MWh. Re-run the dispatch. The energy price rises slightly (say to \$192/MWh) because 40 MW of low-cost supply has been withdrawn from the energy market, and the RAISE6SEC price drops (say to \$3/MWh) because 40 MW of new supply has entered the FCAS market. Mannum's revenue changes too: in the base case, it earned $\$185 \cdot 100 \text{ MW} \cdot 5/60 = \$1,542$ for the interval. In the co-optimised case, it earns $\$192 \cdot 60 \text{ MW} \cdot 5/60 + \$3 \cdot 40 \text{ MW} \cdot 5/60 = \$960 + \$10 = \970 . In this scenario, co-optimisation reduces Mannum's revenue — the energy price was high enough that diverting capacity to FCAS was not worthwhile. But in a different interval, where the energy price is \$45/MWh and the RAISE6SEC price is \$120/MWh (during an enablement scarcity event), the co-optimised strategy would dominate.

This nempy experiment illustrates the central tension of co-optimisation: **every megawatt reserved for FCAS is a megawatt that cannot earn energy arbitrage revenue, and vice versa**. The optimal allocation depends on the relative prices, which change every five minutes.

Price Interaction Effects

Co-optimisation creates subtle price interactions that a battery operator must understand:

FCAS constraining energy prices upward. When FCAS requirements are tight (e.g., after a large contingency event), NEMDE must divert some generators from energy to FCAS. This reduces the energy supply, causing the energy price to rise. The increase in the energy price is precisely the opportunity cost of the FCAS — the marginal cost of "freeing up" capacity from energy to FCAS. In the extreme, a very tight FCAS market can cause energy price spikes even when there is ample generation capacity, because the generation capacity is being "locked up" for FCAS.

Energy prices affecting FCAS prices. When energy prices are high, the opportunity cost of reserving capacity for FCAS is also high — a generator providing FCAS raises foregone selling energy at the high energy price. This opportunity cost flows through the co-optimisation and raises the FCAS price. Conversely, when energy prices are low (e.g., during midday solar surplus), the opportunity cost of FCAS is low, and FCAS prices tend to be low as well.

Cross-market effects. The ten FCAS markets also interact with each other. A unit providing RAISE6SEC (fast raise) can also count toward RAISE60SEC (slow raise) and RAISE5MIN (delayed raise), because a unit that can respond within 6 seconds can certainly respond within 60 seconds. NEMDE accounts for this "cascading" — enablement in a faster service can satisfy requirements in slower services. This means that tightness in the fast raise market can push up slow raise prices (because fast providers are diverted from slow to fast) or push them down (because fast enablement cascades to satisfy slow requirements).

Key point: Co-optimisation means you cannot analyse energy and FCAS markets independently. The prices are coupled — a change in one market ripples through all the others. This coupling is precisely why co-optimisation exists: clearing the markets independently would miss these

interactions and produce a higher total cost. For a battery operator, this means the optimal bidding strategy must consider all markets simultaneously, not optimise energy and FCAS offers in isolation.

The Joint Capacity Constraint: The Enablement Trapezium

The Physical Constraint

The most important constraint in FCAS co-optimisation is the **joint capacity constraint**, which links a unit's energy dispatch to its FCAS enablement. This constraint is typically visualised as a trapezium (trapezoid) in the energy-FCAS space, and it encodes a simple physical truth: **a unit cannot simultaneously use all its capacity for energy and all its capacity for FCAS.**

Enablement trapezium (trapezoid constraint): The geometric region in (energy dispatch, FCAS enablement) space that defines the feasible combinations of energy output and FCAS reserve for a generating unit. The trapezium shape arises because FCAS capacity is limited by the unit's ability to increase output (for raise) or decrease output (for lower) from its current operating point. A unit running at maximum output has no headroom for raise FCAS; a unit at minimum output has no headroom for lower FCAS. The trapezium encodes these physical limits as linear constraints in the NEMDE co-optimisation.

For a battery providing contingency **raise** FCAS, the constraint is intuitive:

- The battery has a maximum discharge rate of P_{max} MW (100 MW for Mannum)
- If the battery is currently dispatched to produce E MW of energy, its headroom for raise FCAS is at most $P_{\text{max}} - E$ MW
- Additionally, the battery must have sufficient state of charge (SoC) to sustain the raise response for the required duration

These constraints define the trapezium. For the Mannum battery (100 MW / 200 MWh), providing contingency raise:

Figure 11.3 — The enablement trapezium for the Mannum battery (100 MW / 200 MWh) providing contingency raise FCAS. The x-axis is energy dispatch (MW), the y-axis is FCAS raise enablement (MW). The shaded region shows feasible combinations. At zero energy output, the full 100 MW is available for raise FCAS. As energy dispatch increases, the available FCAS headroom decreases linearly. At 100 MW energy dispatch, no raise FCAS is possible.

The trapezium has four key points:

Point	Energy dispatch	FCAS enablement	Description
A	0 MW	0 MW	Idle — no energy, no FCAS
B	0 MW	100 MW	Maximum FCAS — all capacity reserved for raise
C	100 MW	0 MW	Maximum energy — no headroom for raise
D	Some intermediate	Some intermediate	Co-optimised — split between energy and FCAS

The boundary of the trapezium is defined by:

$$\text{Energy_dispatch} + \text{FCAS_raise_enablement} \leq P_{\text{max}}$$

For lower FCAS, the constraint works in reverse — the battery must have headroom to reduce its output (or increase its charging rate):

$$\text{FCAS_lower_enablement} \leq \text{Energy_dispatch} - P_{\text{min}} \text{ (generator)} \quad \text{FCAS_lower_enablement} \leq P_{\text{max_charge}} - |\text{Charging_rate}| \text{ (battery absorbing)}$$

When multiple FCAS services are offered simultaneously, the constraints become more complex. A battery offering very-fast raise, fast raise, slow raise, and delayed raise must satisfy:

$$\text{Energy_dispatch} + \text{RAISE1SEC} + \text{RAISE6SEC} + \text{RAISE60SEC} + \text{RAISE5MIN} \leq P_{\text{max}}$$

This is the **stacking** constraint — the total of energy dispatch plus all raise FCAS enablements cannot exceed the physical capacity of the unit.

The SoC Constraint on FCAS

The power constraint (the trapezium above) is not the only limit on FCAS provision. The battery must also have sufficient **energy** (state of charge) to sustain the FCAS response for the required duration if called upon.

For contingency raise, the battery must be able to sustain the enabled capacity for the required sustain time. For RAISE6SEC, the sustain time is 60 seconds. If Mannum is enabled for 50 MW of RAISE6SEC, it must have at least:

$$\text{SoC_required} = 50 \text{ MW} \cdot (60 \text{ s} / 3600 \text{ s per hour}) = 0.83 \text{ MWh}$$

This is a trivial requirement for a 200 MWh battery. But for RAISE5MIN (sustain time of 10 minutes), the SoC requirement is larger:

$$\text{SoC}_{\text{required}} = 50 \text{ MW} \cdot (10 \text{ min} / 60 \text{ min per hour}) = 8.33 \text{ MWh}$$

Still manageable, but the SoC constraint becomes binding when the battery is nearly empty or nearly full. A battery at 5% SoC (10 MWh for Mannum) cannot credibly offer large quantities of raise FCAS requiring a 10-minute sustain, because it would run out of energy before the sustain period ends.

Key point: The trapezium constraint links FCAS to power headroom; the SoC constraint links FCAS to energy reserves. Together, they define the feasible FCAS envelope as a function of the battery's current operating state. The dispatch optimiser (whether NEMDE or the battery's internal MPC) must respect both constraints simultaneously. This creates a three-way tradeoff: energy dispatch consumes SoC (reducing future FCAS capability), FCAS enablement reduces energy headroom (reducing current arbitrage), and the SoC trajectory over time couples current FCAS decisions to future energy decisions.

How the Trapezium Affects Battery Strategy

The trapezium constraint has profound implications for battery dispatch strategy, particularly when integrating FCAS into the MPC framework from Chapter 9:

- 1. FCAS competes directly with arbitrage throughput.** Every megawatt reserved for FCAS raise is a megawatt that cannot be used for energy discharge. If the battery is discharging at peak (100 MW) to capture a \$300/MWh price spike, it has zero headroom for raise FCAS. To provide 30 MW of RAISE6SEC, it must reduce its discharge to 70 MW, giving up $30 \text{ MW} \cdot \$300/\text{MWh} \cdot (5/60) \text{ hours} = \750 of energy revenue per interval. This is worthwhile only if the RAISE6SEC payment exceeds \$750 for that interval — i.e., if the RAISE6SEC price exceeds $\$750 / (30 \text{ MW} \cdot 5/60 \text{ hours}) = \$300/\text{MWh}$.
- 2. FCAS has an opportunity cost that varies with the energy price.** The cost of providing FCAS is not the direct cost (batteries have near-zero marginal cost) but the **foregone energy revenue**. When energy prices are high, the opportunity cost of FCAS is high, and it takes a high FCAS price to justify reserving headroom. When energy prices are low, the opportunity cost is low, and even modest FCAS prices make it worthwhile.
- 3. The trapezium favours partial dispatch.** A battery that always dispatches at 100% capacity (fully charging or fully discharging) can never provide raise FCAS during discharge or lower FCAS during charge. A battery that operates at 70–80% capacity retains 20–30 MW of headroom for FCAS. This partial dispatch is suboptimal for pure arbitrage but can be revenue-optimal when FCAS is included.
- 4. SoC management becomes more complex.** The MPC from Chapters 5 and 9 managed SoC to maximise arbitrage — charging during cheap periods and discharging during expensive ones. With FCAS, SoC management must also consider the FCAS eligibility constraints: keeping enough energy for raise sustain, keeping enough empty capacity for lower sustain. The SoC target at any point in time is no longer purely determined by the price forecast — it also depends on the expected FCAS prices and requirements.

The Mannum trapezium in practice: At 17:30 on a January evening, SA1 energy prices are \$180/MWh and rising. Mannum has 150 MWh in its 200 MWh tank (75% SoC). The battery's MPC wants to discharge at 100 MW to capture the price spike. But RAISE6SEC is currently \$25/MWh (elevated due to a nearby generator being on a forced outage), and RAISE1SEC is \$40/MWh. If Mannum discharges at 100 MW: energy revenue = $\$180 \cdot 100 \cdot (5/60) = \$1,500$ per interval. FCAS revenue = \$0. If Mannum discharges at 70 MW and enables 30 MW across RAISE1SEC (15 MW) and RAISE6SEC (15 MW): energy revenue = $\$180 \cdot 70 \cdot (5/60) = \$1,050$. FCAS revenue = $\$40 \cdot 15 \cdot (5/60) + \$25 \cdot 15 \cdot (5/60) = \$50 + \$31 = \81 . Total = \$1,131. In this case, full discharge (\$1,500) beats the co-optimised strategy (\$1,131) by \$369. The energy price is too high relative to the FCAS prices. But if the energy price drops to \$60/MWh and RAISE6SEC spikes to \$200/MWh (a contingency event), the co-optimised strategy dominates: energy revenue = $\$60 \cdot 70 \cdot (5/60) = \350 , FCAS revenue = $\$200 \cdot 30 \cdot (5/60) = \500 , total = \$850 vs energy-only = $\$60 \cdot 100 \cdot (5/60) = \500 .

Co-optimised Dispatch: Energy-Only vs Joint

Setting Up the Comparison

To understand the value of co-optimisation, compare two dispatch strategies for the Mannum battery over a representative period — say, a week in January 2026:

Strategy A: Energy-only dispatch. The MPC from Chapters 5 and 9, optimising only against the energy price forecast. The battery charges at the cheapest periods and discharges at the most expensive. No FCAS is offered. All 100 MW of capacity is available for energy at all times.

Strategy B: Co-optimised dispatch. The MPC is extended to jointly optimise energy dispatch and FCAS enablement. At each five-minute interval, the optimiser decides how to split the battery's capacity between energy and each FCAS service, respecting the trapezium and SoC constraints. The optimiser uses forecasts for both energy prices and FCAS prices.

The LP Extension for Co-optimisation

The energy-only LP from Chapter 5 had decision variables for charge and discharge at each time step. The co-optimised LP adds FCAS enablement variables:

For each time step t : - charge_t : charging power (MW) - discharge_t : discharging power (MW) - fcas_raise1_t : enablement for very-fast raise (MW) - fcas_raise6_t : enablement for fast raise (MW) - fcas_raise60_t : enablement for slow raise (MW) - fcas_raise5_t : enablement for delayed raise (MW) - fcas_lower1_t , fcas_lower6_t , fcas_lower60_t , fcas_lower5_t : enablements for lower services (MW) - fcas_reg_raise_t , fcas_reg_lower_t : enablements for regulation (MW)

The objective function becomes:

$$\text{Maximise } \sum_t [p_{\text{energy}}(t) \cdot (\text{discharge}(t) - \text{charge}(t)) \cdot \Delta t + \sum_s p_{\text{fcas}}(s,t) \cdot \text{fcas}(s,t) \cdot \Delta t]$$

Subject to:

Trapezium (raise): $\text{discharge}(t) + \text{fcas_raise1}(t) + \text{fcas_raise6}(t) + \text{fcas_raise60}(t) + \text{fcas_raise5}(t) + \text{fcas_reg_raise}(t) \leq P_{\text{max}}$
Trapezium (lower): $\text{charge}(t) + \text{fcas_lower1}(t) + \text{fcas_lower6}(t) + \text{fcas_lower60}(t) + \text{fcas_lower5}(t) + \text{fcas_reg_lower}(t) \leq P_{\text{max_charge}}$
SoC dynamics: $\text{SoC}(t+1) = \text{SoC}(t) - \text{discharge}(t) \cdot \Delta t / \eta + \text{charge}(t) \cdot \Delta t \cdot \eta$
SoC limits: $\text{SoC}_{\text{min}} \leq \text{SoC}(t) \leq \text{SoC}_{\text{max}}$
SoC sufficient for raise sustain: $\text{SoC}(t) \geq \sum_s [\text{fcas_raise}(s,t) \cdot \text{sustain_time}(s)]$
SoC sufficient for lower sustain: $(\text{SoC}_{\text{max}} - \text{SoC}(t)) \geq \sum_s [\text{fcas_lower}(s,t) \cdot \text{sustain_time}(s)]$
Non-negativity: all variables ≥ 0

This is still a linear program — no nonlinear terms — so it can be solved with the same LP solvers used in Chapters 5 and 9. The LP from Chapter 9 extends naturally; the additional variables and constraints increase the problem size but not its computational difficulty.

Worked Example: Evening Peak with Mannum

Consider a single evening peak period — 17:00 to 21:00 on 15 January 2026 in SA1 — and compare the two strategies.

Market conditions (representative): - Energy prices: rising from \$80/MWh at 17:00 to \$250/MWh at 18:30, falling to \$60/MWh by 21:00 - RAISE6SEC: \$8/MWh for most intervals, spiking to \$180/MWh at 18:15 when a gas turbine trips - RAISE1SEC: \$12/MWh average, spiking to \$250/MWh at 18:15 - Other FCAS services: \$3–\$10/MWh throughout - Mannum starting SoC: 180 MWh (90%)

Strategy A: Energy-only dispatch

The energy-only MPC identifies the optimal discharge window as 17:30–19:30 (4 hours at various rates), targeting the price peak at 18:30. It discharges at 100 MW during the highest-priced intervals:

Period	MW	Price	Revenue
17:00–17:30	50	\$80	\$2,000
17:30–18:00	100	\$150	\$7,500
18:00–18:30	100	\$250	\$12,500
18:30–19:00	100	\$200	\$10,000
19:00–19:30	80	\$120	\$4,800
19:30–20:00	40	\$80	\$1,600
Total	-	-	\$38,400

Energy throughput: approximately 175 MWh discharged (from 180 MWh to 5 MWh SoC).

Strategy B: Co-optimised dispatch

The co-optimised MPC reduces energy dispatch by 20–30 MW during high-energy-price intervals to reserve headroom for FCAS raise, and offers the reserved capacity into RAISE1SEC and RAISE6SEC:

Period	Energy MW	FCAS MW	Energy \$	FCAS \$
17:00–17:30	50	30	\$2,000	\$150
17:30–18:00	75	25	\$5,625	\$163
18:00–18:30	70	30	\$8,750	\$3,225*
18:30–19:00	80	20	\$8,000	\$130
19:00–19:30	70	20	\$4,200	\$100
19:30–20:00	35	15	\$1,400	\$45
Total	-	-	\$29,975	\$3,813
Combined	-	-	-	\$33,788

*The FCAS revenue spike at 18:00–18:30 occurs when the gas turbine trip sends RAISE6SEC to \$180/MWh and RAISE1SEC to \$250/MWh. This single interval contributes $\$250 \cdot 15 \cdot (5/60) + \$180 \cdot 15 \cdot (5/60) = \$313 + \$225 = \538 per five-minute block, summing to approximately \$3,225 over the half-hour (assuming the spike persists for about 15 minutes of the 30-minute window).

Energy throughput: approximately 142 MWh discharged (from 180 MWh to 38 MWh SoC).

Comparison:

Metric	Energy-only	Co-optimised	Difference
Energy revenue	\$38,400	\$29,975	-\$8,425
FCAS revenue	\$0	\$3,813	+\$3,813
Total revenue	\$38,400	\$33,788	-\$4,612
Energy throughput	175 MWh	142 MWh	-33 MWh
Remaining SoC	5 MWh	38 MWh	+33 MWh

In this particular evening, the energy-only strategy wins by \$4,612 because the energy prices were consistently high and the FCAS spike, while dramatic, was too brief to compensate for the foregone energy revenue. The co-optimised battery gave up \$8,425 of energy revenue to earn only \$3,813 in FCAS.

Key point: Co-optimisation does not always beat energy-only dispatch on any given day. Its value is in the *expected* return over many days, including the rare but valuable intervals when FCAS prices spike dramatically. Over a full year, the co-optimised strategy typically produces 15–30% higher total revenue than energy-only, because the FCAS revenue earned during normal operation plus the occasional windfall from FCAS spikes more than compensates for the systematically reduced energy throughput.

However, notice that the co-optimised battery retains 38 MWh of SoC at 20:00, compared to only 5 MWh for the energy-only battery. If energy prices spike again later in the evening (e.g., an unexpected demand surge at 21:00), the co-optimised battery has flexibility to respond while the energy-only battery is nearly empty. This **optionality value** of retained SoC is not captured in the single-evening comparison but is significant over time.

The nempy View of One Interval

To understand exactly how NEMDE allocates capacity, use nempy to clear a single interval. Load the SA1 bid stack for 18:15 on 15 January 2026 — the moment the gas turbine trips and FCAS prices spike. Add Mannum's offers:

- Energy: 70 MW at \$0/MWh (price-taking)
- RAISE1SEC: 15 MW at \$0/MWh
- RAISE6SEC: 15 MW at \$0/MWh

Call nempy's dispatch solver. The output shows Mannum dispatched for 70 MW of energy, enabled for 15 MW of RAISE1SEC, and enabled for 15 MW of RAISE6SEC — exactly matching its offers, because the FCAS requirements are binding (scarce) in this interval. The RAISE6SEC price is \$180/MWh, reflecting the scarcity. The energy price is \$245/MWh, slightly elevated because some generators have been diverted from energy to FCAS.

Now experiment: change Mannum's offer to 100 MW energy, 0 MW FCAS. Re-run. The energy price drops slightly (more supply), the RAISE6SEC price rises (less supply), and Mannum earns more from energy but nothing from FCAS. Compare the total revenue in each case — this is the co-optimisation tradeoff made transparent.

The nempy model lets you explore how Mannum's bidding strategy interacts with the rest of the SA1 fleet. You can observe how adding Mannum's FCAS offer displaces more expensive FCAS from gas turbines, reducing the FCAS clearing price. You can see how withdrawing Mannum's energy offer tightens the energy supply, raising the energy price. These are the general equilibrium effects that a battery operator must anticipate when designing a bidding strategy.

Causar-Pays: The Cost of Regulation

How Causar-Pays Works

Providing regulation FCAS generates revenue, but **consuming** regulation imposes a cost. AEMO allocates the cost of regulation services to participants through the **causer-pays** mechanism, which assigns costs based on how much each participant's metering deviates from their dispatch targets — i.e., how much each participant contributes to the need for regulation in the first place.

Causar-pays: AEMO's mechanism for allocating the cost of regulation FCAS to market participants. Each participant's contribution to system frequency deviations is measured by comparing their actual metered output to their dispatch targets over four-second intervals. Participants whose deviations consistently move frequency away from 50 Hz (anti-correlated with the system need) pay a larger share of the total regulation cost. Participants whose deviations help stabilise frequency (correlated with the system need) pay less or nothing. The mechanism creates a financial incentive for participants to follow their dispatch targets accurately.

The causer-pays calculation works in four-second intervals:

1. AEMO measures the **system frequency error** (actual frequency minus 50 Hz) every four seconds
2. AEMO measures each participant's **metering deviation** (actual output minus dispatch target) every four seconds
3. For each four-second interval, AEMO computes a **contribution factor** (CF) for each participant: the product of the metering deviation and the system frequency error. If a generator is producing more than its target when the frequency is already high (supply exceeds demand), the generator is making the problem worse — its CF is positive. If it is producing more when frequency is low, it is helping — its CF is negative (or at least not adding to cost).
4. The CFs are aggregated over a 28-day period into **causer-pays factors** (CPFs) for each participant
5. The total regulation cost for each interval is allocated proportionally to each participant's CPF

Why Batteries Are Good (and Bad) at Causar-Pays

Batteries have two properties that interact with causer-pays in opposing directions:

Good: Batteries can follow dispatch targets precisely. A thermal generator that is dispatched to produce 350 MW might actually produce 345–355 MW due to the inherent imprecision of turbine control. A battery dispatched to 50 MW can produce exactly 50.0 MW, because the power electronics respond within milliseconds and with high precision. Low metering deviation means low causer-pays contributions.

Bad: Batteries that provide regulation FCAS are constantly adjusting their output in response to the AGC signal. If the AGC signal is noisy or if the battery's response is slightly delayed, the battery's metering can deviate from its nominal dispatch target in a way that contributes to causer-pays costs. Additionally, batteries that charge during variable renewable generation periods (when wind and solar are fluctuating rapidly) may see their metering deviate from dispatch due to the rapid changes in local grid conditions.

Causer-pays in practice — the Mannum BESS: Suppose Mannum's CPF for a given 28-day period is 0.8% — meaning Mannum is responsible for 0.8% of the total regulation cost in SA1. If the total SA1 regulation cost for that period is \$2 million, Mannum pays \$16,000. This cost is deducted from Mannum's gross revenue. A well-controlled battery might achieve a CPF of 0.3–0.5%; a poorly controlled one might see 1.5–2.0%. The difference — \$6,000 vs \$40,000 per 28-day period — is meaningful but not dominant relative to the battery's total revenue.

For the Mannum BESS, causer-pays is a cost to manage, not a cost to fear. The key is precise dispatch-target following, which is a control-system problem rather than a market-strategy problem. The battery's optimiser and its physical control system must be well-calibrated to minimise metering deviations. This is typically handled by the battery management system (BMS) and the energy management system (EMS), not by the trading desk — but the trading desk should monitor the CPF and flag anomalies.

Forecasting FCAS Prices

The Challenge

Forecasting FCAS prices is fundamentally harder than forecasting energy prices. Energy prices have strong, exploitable patterns — as we developed through Chapters 6, 7, and 8:

- **Daily patterns:** Prices are low at night and midday, high in the morning and evening peaks
- **Seasonal patterns:** Prices are higher in summer (air conditioning) and winter (heating) than in spring and autumn
- **Weather dependence:** Prices rise with temperature extremes and fall with high wind and solar output
- **Autoregressive structure:** Today's prices are correlated with yesterday's prices at the same time

FCAS prices have some of these patterns (FCAS is generally more expensive during peak demand when generators are stressed), but the dominant feature is the **spike process** — rare, large, unpredictable price spikes driven by contingency events. A generator trip is essentially random — it cannot be forecast from weather, demand, or price history. The resulting FCAS price spike is extreme (often hitting the market price cap) and short-lived (typically 1–6 intervals before reserves are replenished).

Key point: Energy price forecasting is primarily about predicting the level; FCAS price forecasting is primarily about predicting the probability of a spike. The base FCAS price (\$3–\$15/MWh) is easy to predict — it does not vary much. The spike (\$300–\$15,100/MWh) is nearly impossible to predict — it depends on when the next generator trips, which is essentially random. The battery operator's FCAS strategy must account for this spike risk without relying on predicting individual spikes.

What Can Be Forecast

While individual FCAS spikes cannot be predicted, the **conditions that make spikes more likely** can be:

1. **Low reserve margin.** When the total available generation capacity is close to the total demand, a single generator trip is more likely to cause a reserve shortfall and an FCAS price spike. Reserve margin is driven by demand (which is forecastable from weather and calendar variables) and available supply (which is partially forecastable from planned outage schedules and renewable generation forecasts).
2. **Low inertia.** When the system inertia is low (high renewable penetration, few synchronous generators online), frequency deviates faster after a disturbance, requiring more fast FCAS. AEMO publishes inertia forecasts as part of the pre-dispatch process.
3. **Interconnector constraints.** When interconnectors are constrained (at their flow limits), regions become electrically isolated. A generator trip in an isolated region cannot be compensated by imports, so the local FCAS requirement increases and the FCAS price rises. Interconnector constraints are partially forecastable from demand and generation patterns.
4. **Generator outage schedules.** AEMO publishes planned outage information through the Medium-Term Projected Assessment of System Adequacy (MT PASA). Periods with many generators on planned outage are more vulnerable to FCAS scarcity if an additional unplanned trip occurs.
5. **Time of day and season.** FCAS requirements and enablement costs both have diurnal patterns — they tend to be higher during peak demand periods when the system is stressed.

A Practical Forecasting Approach

Given the difficulty of FCAS price forecasting, a pragmatic approach for battery dispatch uses two levels:

Level 1: Base FCAS price forecast. Use a simple model — the average FCAS price for each half-hour period, stratified by day type (weekday/weekend) and season. This captures the predictable component (the base price, the diurnal pattern) and ignores the unpredictable component (spikes). This forecast is used for the MPC's FCAS revenue estimate: when the optimiser decides how much headroom to reserve for FCAS, it values that headroom at the base forecast.

Level 2: Spike probability estimate. Estimate the probability that the FCAS price exceeds a high threshold (say, \$300/MWh) in each interval, based on the forecastable conditions above — reserve margin, inertia, interconnector constraints. This probability can be estimated from historical data: what fraction of intervals with similar conditions had FCAS spikes? This estimate is not used directly in the MPC (which would require a full stochastic FCAS price model) but informs the **static FCAS reservation policy** — a rule-of-thumb that sets a minimum FCAS

headroom reservation based on the current spike probability.

A simple FCAS reservation rule: The trading desk might set a policy: "If the estimated FCAS spike probability exceeds 5% for the upcoming two hours, reserve at least 20 MW of capacity for RAISE6SEC and RAISE1SEC. If it exceeds 15%, reserve at least 40 MW." This rule is not optimal in a formal sense — it does not maximise expected revenue across all possible FCAS price realisations — but it captures the most important feature of FCAS value: being enabled when the rare spike occurs. The rule can be calibrated by backtesting against historical FCAS price data, sweeping the reservation level and measuring the total (energy + FCAS) revenue.

Integrating FCAS into the MPC

The simplest integration of FCAS into the MPC framework from Chapter 9 adds the base FCAS price forecast to the LP objective:

$$\text{Maximise } \sum_t [p_{\text{energy}}(t) \cdot (\text{discharge}(t) - \text{charge}(t)) \cdot \Delta t + \sum_s p_{\text{fcas_base}}(s,t) \cdot \text{fcas}(s,t) \cdot \Delta t]$$

This LP jointly optimises energy and FCAS, using the base FCAS forecast as the FCAS price. It naturally balances energy and FCAS — when the energy price is high relative to FCAS, the optimiser dispatches more energy; when FCAS is relatively attractive, it reserves more headroom.

The limitation of this approach is that it values FCAS at the base price, ignoring the spike premium. The true expected FCAS revenue includes the spike contribution:

$$E[\text{FCAS revenue}] = \text{base_price} \cdot (1 - p_{\text{spike}}) + \text{spike_price} \cdot p_{\text{spike}}$$

where p_{spike} is the spike probability and spike_price is the average price conditional on a spike. If $p_{\text{spike}} = 3\%$ and $\text{spike_price} = \$2,000/\text{MWh}$, the spike contribution adds \$60/MWh to the expected value — potentially doubling the FCAS value used in the LP.

A more sophisticated approach uses the Level 2 spike probability estimate to compute the expected FCAS price:

$$\text{expected_fcas_price}(s,t) = \text{base_price}(s,t) \cdot (1 - p_{\text{spike}}(t)) + E[\text{price} \mid \text{spike}](s,t) \cdot p_{\text{spike}}(t)$$

and substitutes this expected price into the LP. This gives the optimiser a more accurate valuation of FCAS headroom and leads to better allocation between energy and FCAS.

Key point: The quality of FCAS price forecasting matters less than the quality of energy price forecasting for overall battery revenue. Energy arbitrage is the dominant revenue stream, and small improvements in energy price forecasting translate directly to higher capture ratios (as shown in Chapter 9). FCAS forecasting mainly affects the allocation of headroom between energy and FCAS — getting this allocation roughly right is sufficient. A simple base-price forecast with a spike probability overlay is adequate for most battery dispatch applications.

The Capture Ratio with FCAS

Redefining the Benchmark

In Chapters 5, 9, and 10, we defined the capture ratio (CR) as the ratio of actual arbitrage revenue to perfect-foresight arbitrage revenue. With FCAS included, the capture ratio must be extended:

$$\text{CR}_{\text{total}} = (\text{actual energy revenue} + \text{actual FCAS revenue}) / (\text{perfect-foresight energy} + \text{FCAS revenue})$$

The denominator — perfect foresight revenue including FCAS — is the revenue a battery would earn if it knew all future energy and FCAS prices perfectly and optimised its energy dispatch and FCAS enablement accordingly. This is computed by solving the co-optimised LP with actual (historical) energy and FCAS prices.

Capture ratio with FCAS: The ratio of total actual revenue (energy arbitrage plus FCAS enablement payments) to total perfect-foresight revenue (the maximum achievable with perfect knowledge of both energy and FCAS prices). This extended capture ratio measures the battery operator's skill at both energy arbitrage and FCAS co-optimisation. A higher capture ratio indicates better forecasting and dispatch strategy across all revenue streams.

The benchmark from earlier chapters — 0.50 as the bar, 0.65–0.75 as the target — still applies, but now measured against total revenue:

Capture ratio	Interpretation
< 0.40	Poor — significant value left on the table in both energy and FCAS
0.50	Market average — acceptable but not competitive
0.55–0.65	Good — effective energy arbitrage with basic FCAS participation
0.65–0.75	Very good — strong energy forecasting with co-optimised FCAS
> 0.75	Excellent — top-tier operator with sophisticated co-optimisation

Including FCAS in the capture ratio has two effects:

1. The denominator increases. Perfect-foresight revenue with FCAS is higher than without, because a perfectly informed battery would offer FCAS during high-FCAS-price intervals and withdraw from FCAS during high-energy-price intervals. The additional FCAS revenue from perfect co-optimisation is typically 25–40% above perfect-foresight energy-only revenue.

2. The numerator increases too, but by less. A real battery earns FCAS revenue, but it cannot perfectly predict FCAS prices (especially spikes), so its FCAS co-optimisation is imperfect. The actual FCAS revenue is typically 10–25% of total actual revenue.

The net effect is that the FCAS-inclusive capture ratio is slightly lower than the energy-only capture ratio for the same operator, because perfect FCAS co-optimisation is harder to achieve than perfect energy arbitrage (FCAS spikes are less predictable than energy price patterns). A battery with a 0.70 energy-only CR might have a 0.62–0.68 total CR, depending on how effectively it captures FCAS spikes.

Benchmarking Against AEMO Pre-dispatch

As established in Chapter 5, AEMO's pre-dispatch forecasts (accessed via NEMSEER) serve as the benchmark for forecast quality. With FCAS included, the benchmark extends to AEMO's pre-dispatch FCAS price forecasts, which are also available through NEMSEER.

AEMO's FCAS price forecasts have a distinctive characteristic: they are reasonably good at predicting the base level but terrible at predicting spikes. This is expected — AEMO's pre-dispatch runs NEMDE with forecast inputs but cannot predict contingency events (generator trips). As a result, the AEMO pre-dispatch FCAS prices are almost always near the base level, and the spikes that actually occur are entirely missed.

A dispatch strategy that uses AEMO pre-dispatch prices for both energy and FCAS will:

- Capture energy arbitrage reasonably well (as shown in Chapter 5)
- Undervalue FCAS headroom (because the pre-dispatch FCAS prices do not include spike risk)
- Reserve too little capacity for FCAS during periods when spikes are more likely
- Miss the FCAS revenue from the rare but valuable spike events

This is the baseline that a more sophisticated FCAS forecasting approach (the base-price plus spike-probability model described above) aims to beat. The improvement comes not from predicting individual spikes but from systematically reserving more FCAS headroom during high-risk periods, capturing more spike revenue when spikes do occur.

Capture ratio comparison — energy-only vs co-optimised: Over a one-year backtest for the Mannum BESS in SA1: **Strategy A (energy-only):** - Perfect foresight energy revenue: \$12.0M - Actual energy revenue: \$8.4M - FCAS revenue: \$0 - Energy-only CR: 0.70 - Total CR (against energy + FCAS perfect foresight of \$16.2M): 0.52 **Strategy B (co-optimised, base FCAS forecast):** - Actual energy revenue: \$7.6M (lower due to reduced throughput) - Actual FCAS revenue: \$2.8M - Total actual revenue: \$10.4M - Total CR (against \$16.2M): 0.64 **Strategy C (co-optimised, with spike probability model):** - Actual energy revenue: \$7.2M - Actual FCAS revenue: \$3.4M - Total actual revenue: \$10.6M - Total CR (against \$16.2M): 0.65 The co-optimised strategies earn less from energy (reduced throughput due to FCAS reservation) but more than compensate with FCAS revenue. Strategy C's spike-aware model earns an additional \$600K in FCAS revenue compared to Strategy B, primarily from being enabled during the 15–20 highest-value FCAS intervals of the year.

Worked Example: Clearing One Interval with nempy

Setup

This example walks through co-optimised clearing for a single five-minute interval: SA1 at 18:15 on 15 January 2026. A gas turbine has just tripped, making FCAS scarce.

The setup in nempy:

1. Load the historical bid stack for SA1 at this interval — all generators' energy offers and FCAS offers as submitted to AEMO
2. Load the demand forecast and FCAS requirements for this interval
3. Add the Mannum battery as a new participant with the following offers:
4. Energy: 70 MW at \$0/MWh (price-taking, as introduced in Chapter 3)
5. RAISE1SEC: 15 MW at \$0/MWh
6. RAISE6SEC: 15 MW at \$0/MWh
7. No other FCAS services offered in this example
8. Set the trapezium constraint: energy dispatch + RAISE1SEC + RAISE6SEC <= 100 MW

Call nempy's dispatch solver. The solver outputs:

Clearing prices: - Energy: \$248/MWh - RAISE1SEC: \$245/MWh - RAISE6SEC: \$178/MWh - Other FCAS services: \$5–\$20/MWh (not binding)

Mannum dispatch: - Energy: 70 MW (fully dispatched at its offered quantity) - RAISE1SEC: 15 MW (fully enabled) - RAISE6SEC: 15 MW (fully enabled)

Mannum revenue for this 5-minute interval: - Energy: $\$248 \cdot 70 \cdot (5/60) = \$1,447$ - RAISE1SEC: $\$245 \cdot 15 \cdot (5/60) = \306 - RAISE6SEC: $\$178 \cdot 15 \cdot (5/60) = \223 - **Total: \$1,976**

Compare with a Mannum offering 100 MW energy only: - Energy: $\$243 \cdot 100 \cdot (5/60) = \$2,025$ (the energy price is slightly lower because more supply is available) - FCAS: \$0 - **Total: \$2,025**

In this single interval, the energy-only strategy wins by \$49. But consider: the co-optimised strategy preserved 30 MW of headroom that could have been called upon if the frequency dropped further. And over the broader evening — including intervals where the energy price is lower —

the co-optimised allocation typically wins.

Sensitivity Analysis with nempy

The power of nempy is the ability to perturb inputs and observe how the clearing outcome changes. Run the following experiments:

Experiment 1: Increase Mannum's FCAS offer. Change Mannum's RAISE6SEC offer from 15 MW to 40 MW (reducing energy to 45 MW). The RAISE6SEC price drops (more supply), the energy price rises slightly (less supply). At what FCAS offer quantity does the total revenue peak? This reveals the optimal allocation for this specific interval.

Experiment 2: Change Mannum's FCAS offer price. Instead of bidding FCAS at \$0/MWh (price-taking), bid at \$100/MWh. If the clearing price is \$178/MWh, Mannum is still enabled — the offer price is below the clearing price. But if Mannum bids at \$200/MWh, it is not enabled (the offer price exceeds the clearing price). The FCAS market clears without Mannum, and the FCAS price may be higher (if Mannum was a marginal provider). This demonstrates the strategic bidding dimension — a battery can influence the FCAS clearing price through its offer price, not just its offer quantity.

Experiment 3: Remove the tripped generator. Load the bid stack with the gas turbine still online (no trip). The FCAS requirements are met easily, RAISE6SEC clears at \$5/MWh, and Mannum's FCAS enablement is worth little. The energy price is lower too (more generation available). This confirms that the FCAS value in the base case was driven entirely by the contingency event.

These experiments give the reader an intuitive understanding of how co-optimisation works: **the same battery, the same capacity, in the same market, can optimally allocate itself very differently depending on the relative energy and FCAS prices, which in turn depend on the system state.**

Practical FCAS Bidding for a Battery

Offer Structure

In the NEM, a battery's FCAS offers are structured similarly to energy offers — a set of price-quantity bands. For each FCAS market, the battery specifies:

1. **Maximum availability** (MW): the most it can provide, subject to the trapezium constraint
2. **Enablement minimum and maximum** (MW): the energy dispatch range within which the FCAS is available
3. **Trapezium parameters:** the low breakpoint, high breakpoint, and enablement max that define the trapezoidal constraint region
4. **Price** (\$/MWh): the minimum price at which the battery is willing to provide the service

For a battery like Mannum, the typical offer strategy is:

Offer FCAS at \$0/MWh (price-taking) for services where the battery wants to be enabled whenever possible. This ensures enablement — the battery will be dispatched for FCAS as long as the clearing price is positive. The battery still earns the clearing price (not the offer price), so bidding at \$0/MWh does not mean earning \$0.

Offer FCAS at a reservation price that reflects the opportunity cost of energy. If the energy price is expected to be \$150/MWh and the battery must reduce its energy dispatch by 1 MW to provide 1 MW of FCAS, the opportunity cost is \$150/MWh. Offering FCAS above \$150/MWh ensures the battery is only enabled when the FCAS price exceeds the foregone energy revenue. This is a more sophisticated strategy that requires forecasting the energy price at the time the FCAS offer is submitted (which is before NEMDE clears).

Dynamic FCAS bidding — aligning offers with energy forecasts: At 17:00, Mannum's trading system forecasts SA1 energy prices for the next four hours. The forecast suggests a price peak of \$200/MWh at 18:30. The system sets Mannum's FCAS offers for each upcoming dispatch interval: - 17:00–17:30 (energy forecast: \$80/MWh): Offer 40 MW RAISE6SEC at \$0/MWh. The low opportunity cost means FCAS is attractive. - 17:30–18:00 (energy forecast: \$150/MWh): Offer 25 MW RAISE6SEC at \$100/MWh. The higher opportunity cost means FCAS should only be enabled if it pays well. - 18:00–18:30 (energy forecast: \$200/MWh): Offer 20 MW RAISE6SEC at \$180/MWh. Reserve most capacity for the energy peak; only provide FCAS if it pays nearly as much. - 18:30–19:00 (energy forecast: \$120/MWh): Offer 30 MW RAISE6SEC at \$80/MWh. Moderate opportunity cost. This dynamic bidding strategy adapts the FCAS offer quantity and price to the expected energy value, ensuring the battery is enabled for FCAS when it is most valuable relative to energy and reserved for energy when energy is most valuable. It requires the battery's bidding system to update offers frequently (every 5 minutes, in line with NEMDE clearing).

Stacking FCAS Services

A battery can offer multiple FCAS services simultaneously, as long as the total enablement respects the trapezium constraint. The typical stacking order for raise services (from fastest to slowest):

1. **RAISE1SEC** — highest priority because of limited competition and relatively high prices
2. **RAISE6SEC** — second priority; more competition but still valuable
3. **RAISE60SEC** — lower value; offered if headroom permits after 1-second and 6-second
4. **RAISE5MIN** — lowest value among contingency services; offered with remaining headroom

The cascading property of FCAS services means that enablement in a faster service can satisfy requirements for slower services. A battery enabled for 20 MW of RAISE6SEC also counts toward 20 MW of RAISE60SEC and RAISE5MIN requirements. NEMDE handles this cascading automatically.

For lower services, the stacking works in the same direction ($\text{LOWER1SEC} > \text{LOWER6SEC} > \text{LOWER60SEC} > \text{LOWER5MIN}$), but the economics are different — lower FCAS requires headroom to absorb power (charge), which competes with the battery's charging strategy rather than its discharging strategy.

The Regulation Decision

Regulation FCAS is a separate strategic decision from contingency FCAS:

- **Pros:** Steady revenue (regulation prices are less spikey than contingency); regulation helps the battery's causer-pays factor (a regulation provider is helping stabilise frequency, which can reduce its CPF)
- **Cons:** Regulation requires continuous response (higher control complexity); the AGC signal causes small, frequent SoC changes that can interfere with the MPC's planned SoC trajectory; the energy throughput from regulation is near-zero but not exactly zero, introducing SoC uncertainty

For the Mannum BESS, offering regulation is typically worthwhile during off-peak periods when energy arbitrage opportunities are limited. The regulation revenue (\$5–\$15/MWh on enabled capacity) provides a base revenue stream that partially offsets the low energy arbitrage value during these hours. During peak periods, the opportunity cost of reserving capacity for regulation (which requires continuous headroom, unlike contingency which is rarely called) generally favours full energy dispatch or contingency FCAS.

Exercises

Exercise 1: Sweep the Value of Reserving Headroom for FCAS

Problem: For the Mannum battery (100 MW / 200 MWh, 85% round-trip efficiency), compare the total revenue (energy + FCAS) across different FCAS reservation levels over a one-week period.

Assume the following conditions (representative of a summer week in SA1): - Average energy price during discharge: \$120/MWh - Average energy price during charge: \$25/MWh - Average RAISE6SEC price: \$10/MWh - RAISE6SEC spikes to \$2,000/MWh for 3 intervals during the week (15 minutes total) - The battery does 1.5 cycles per day for energy arbitrage - Round-trip efficiency: 85%

Compute the weekly total revenue for FCAS reservation levels of 0, 10, 20, 30, 40, and 50 MW (reducing the energy dispatch power proportionally).

Worked Solution

Step 1: Compute energy-only revenue (0 MW FCAS reservation).

Daily discharged = 1.5 cycles · 200 MWh = 300 MWh Daily charged = 300 / 0.85 = 353 MWh Daily energy revenue = 300 · \$120 - 353 · \$25 = \$36,000 - \$8,825 = \$27,175 Weekly energy revenue = 7 · \$27,175 = \$190,225 FCAS revenue = \$0

Total: \$190,225.

Step 2: Model how reservation level R affects energy throughput.

When R MW is reserved for FCAS, discharge power drops to (100 - R) MW. Charging is unaffected. Assume the battery discharges at reduced power for the same duration, so daily throughput scales proportionally:

discharged = 300 · (100 - R) / 100 MWh

Step 3: Compute FCAS revenue as a function of R.

Enabled hours per week: 1.5 cycles · 2 hours discharge = 3 hours/day = 21 hours/week. Assume the battery captures 2 of the 3 spike intervals.

Base FCAS = R · \$10 · 21 hours = \$210 · R Spike FCAS = R · \$2,000 · (2 · 5/60) hours = \$333 · R Total FCAS = \$543 · R

Step 4: Tabulate results.

R (MW)	Discharge power	Weekly energy revenue	Weekly FCAS revenue	Total
0	100 MW	\$190,225	\$0	\$190,225
10	90 MW	\$171,203	\$5,430	\$176,633
20	80 MW	\$152,180	\$10,860	\$163,040
30	70 MW	\$133,158	\$16,290	\$149,448
40	60 MW	\$114,135	\$21,720	\$135,855
50	50 MW	\$95,113	\$27,150	\$122,263

Step 5: Interpret.

Energy-only dispatch wins at every reservation level. The FCAS revenue (\$543/MW/week) does not compensate for foregone energy revenue (\$1,902/MW/week) because the average RAISE6SEC price (\$10/MWh) is far below the energy spread, and spikes are too brief.

Key insight: If the average RAISE6SEC price were \$35/MWh (high-volatility periods), or spikes were more frequent, the optimal reservation would be nonzero. The crossover is where marginal FCAS revenue per MW equals marginal energy revenue per MW (calculated in Exercise

2).

Exercise 2: Find the Break-Even FCAS Price

Problem: Using the same battery parameters as Exercise 1 (100 MW / 200 MWh, 85% RTE, average discharge price \$120/MWh, average charge price \$25/MWh, 1.5 cycles/day), find the break-even average RAISE6SEC price at which reserving the first MW of FCAS becomes profitable.

Ignore spikes for this calculation — assume a flat FCAS price. The break-even price is the FCAS price at which the marginal revenue from FCAS exactly equals the marginal revenue from energy for the last MW of capacity.

Worked Solution

Step 1: Determine discharge hours per week.

Discharge time per cycle = $200 \text{ MWh} / 100 \text{ MW} = 2 \text{ hours}$ Discharge hours per week = $1.5 \text{ cycles/day} \cdot 2 \text{ hours} \cdot 7 \text{ days} = 21 \text{ hours}$

Step 2: Compute marginal energy revenue per MW.

Assume the battery is power-constrained (wants to discharge all 200 MWh in the 2-hour peak). The marginal value of 1 MW of discharge capacity:

Marginal gross revenue = $1 \text{ MW} \cdot 2 \text{ hours} \cdot \$120/\text{MWh} = \$240 \text{ per cycle}$ Marginal charge cost = $1 \text{ MW} \cdot 2 \text{ hours} / 0.85 \cdot \$25/\text{MWh} = \$58.82 \text{ per cycle}$ Net marginal per cycle = $\$240 - \$58.82 = \$181.18$ Weekly marginal energy = $\$181.18 \cdot 1.5 \text{ cycles/day} \cdot 7 \text{ days} = \$1,902.4 \text{ per MW per week}$

Step 3: Compute marginal FCAS revenue per MW.

Each reserved MW earns the FCAS price for the 21 enabled hours:

Marginal FCAS revenue = $\text{FCAS_price} \cdot 21 \text{ hours per week}$

Step 4: Solve for break-even.

$\text{FCAS_price} \cdot 21 = \$1,902.4$ $\text{FCAS_price} = \$1,902.4 / 21 = \$90.6/\text{MWh}$

The break-even RAISE6SEC price is approximately \$91/MWh.

Step 5: Interpret.

This is a high bar — the average RAISE6SEC price is typically \$5-\$15/MWh. But this uses the *average* energy price. During low-energy-price intervals (\$40-\$60/MWh), the break-even drops substantially.

Key insight: Co-optimisation adds value not because the average FCAS price exceeds the break-even, but because there are specific intervals where the FCAS price exceeds the energy opportunity cost *at that moment*. The LP exploits these interval-level opportunities automatically.

Exercise 3: Estimate the Annual FCAS Revenue from Spike Capture

Problem: The Mannum BESS is enabled for 30 MW of RAISE6SEC for approximately 40% of all dispatch intervals in a year (roughly 42,000 intervals out of 105,120). Historical data shows that RAISE6SEC in SA1 spikes above \$1,000/MWh in approximately 0.3% of intervals, with an average spike price of \$5,500/MWh. The average non-spike price when enabled is \$8/MWh. Estimate the annual RAISE6SEC revenue split between base and spike components.

Worked Solution

Step 1: Count enabled intervals.

Total intervals/year = $365 \cdot 24 \cdot 12 = 105,120$ Enabled intervals = $0.40 \cdot 105,120 = 42,048$

Step 2: Split into spike and non-spike intervals.

Spike intervals (system-wide) = $0.003 \cdot 105,120 = 315/\text{year}$ Spike intervals captured = $0.40 \cdot 315 = 126$ Non-spike intervals enabled = $42,048 - 126 = 41,922$

Step 3: Compute base revenue.

Base revenue = $30 \text{ MW} \cdot \$8/\text{MWh} \cdot 41,922 \text{ intervals} \cdot (5/60) \text{ h/interval} = 30 \cdot 8 \cdot 41,922 \cdot 0.0833 = \$838,400$

Step 4: Compute spike revenue.

Spike revenue = $30 \text{ MW} \cdot \$5,500/\text{MWh} \cdot 126 \text{ intervals} \cdot (5/60) \text{ h/interval} = 30 \cdot 5,500 \cdot 126 \cdot 0.0833 = \$1,732,050$

Step 5: Total and split.

Total RAISE6SEC revenue = $\$838,400 + \$1,732,050 = \$2,570,450$ Revenue split: Base = 33%, Spike = 67%

Final answer: \$2,570,450 per year, with spikes contributing two-thirds despite occurring in only 0.3% of intervals.

Step 6: Interpret.

Being enabled during spikes matters far more than the base FCAS price. The spike probability model (Level 2) is more valuable than the base price model (Level 1). A battery that reserves more FCAS headroom during spike-prone periods can significantly increase revenue.

Reality check: \$2.57M from a single FCAS service at 30 MW is roughly 20-25% of Mannum's total annual revenue. With all FCAS services included, total FCAS revenue could reach \$3.5-\$5M/year.

Glossary

Term	Definition
FCAS (Frequency Control Ancillary Services)	Services procured by AEMO to maintain grid frequency at 50 Hz, including regulation and contingency services
Regulation FCAS	Continuous frequency correction via AGC signal; raise and lower markets
Contingency FCAS	Response to large, sudden frequency disturbances; categorised by response speed (1s, 6s, 60s, 5min)
Enablement	The quantity of FCAS a unit is dispatched to provide; the unit is paid the clearing price on its enabled MW
Trapezium constraint (enablement trapezoid)	The linear constraint linking energy dispatch and FCAS enablement: the sum cannot exceed physical capacity
Causer-pays	AEMO's cost-allocation mechanism for regulation FCAS, based on each participant's contribution to frequency deviations
Co-optimisation	Simultaneous clearing of energy and all FCAS markets in a single LP, minimising total cost
NEMDE	National Electricity Market Dispatch Engine — AEMO's software that clears the market every five minutes
nempy	Open-source Python implementation of the NEM dispatch procedure by UNSW-CEEM
Inertia	Rotational kinetic energy of synchronous generators; determines how quickly frequency changes after a disturbance
AGC (Automatic Generation Control)	AEMO's control system that sends regulation signals to enabled units every four seconds
Causer-pays factor (CPF)	Each participant's proportional allocation of regulation costs, based on metering deviations
System frequency	The 50 Hz oscillation rate of AC power; reflects the instantaneous supply-demand balance
Very-fast FCAS (1-second)	The newest FCAS markets (October 2023) requiring sub-second response; dominated by batteries
Revenue stack	The combination of energy arbitrage, FCAS, network support, and capacity revenue available to a battery
Capture ratio (with FCAS)	Total actual revenue (energy + FCAS) divided by total perfect-foresight revenue
Opportunity cost	The foregone revenue from the next-best use of capacity; for FCAS, the energy revenue given up by reserving headroom
Shadow price (dual variable)	The marginal cost of satisfying a constraint by one more unit; in NEMDE, shadow prices become market clearing prices

Summary

Frequency control is the power system's immune system — invisible when working, catastrophic when it fails. The NEM maintains 50 Hz through ten FCAS markets (regulation raise and lower, plus contingency raise and lower at four speed tiers: 1-second, 6-second, 60-second, and 5-minute), all co-optimised with energy in a single linear program (NEMDE) every five minutes. For the Mannum BESS, FCAS creates both an opportunity and a constraint: the enablement trapezium means every megawatt reserved for FCAS raise is a megawatt that cannot earn energy arbitrage revenue, and the optimal allocation between energy and FCAS shifts every five minutes with the relative prices. Using nempy to clear historical intervals with different Mannum offers makes this tradeoff concrete and inspectable. FCAS prices are dominated by rare spikes driven by contingency events — the base price is \$5–\$15/MWh but spike prices can reach \$15,100/MWh, and these brief spikes contribute the majority of annual FCAS revenue. Forecasting FCAS prices is harder than forecasting energy prices because the spikes are essentially unpredictable, but forecasting the conditions that make spikes more likely (low reserves, low inertia, interconnector constraints) enables a spike-probability model that improves FCAS headroom allocation. The capture ratio, extended to include FCAS revenue, remains the

central performance metric — 0.50 as the market-average bar, 0.65–0.75 as the target — now benchmarked against AEMO pre-dispatch via NEMSEER for both energy and FCAS prices. Co-optimised dispatch typically yields 15–30% higher total revenue than energy-only dispatch, not because the average FCAS price is high, but because the LP exploits interval-level opportunities where FCAS value exceeds the energy opportunity cost.

12. Bidding and Market Participation

The Missing Link: From Schedule to Offer

Chapters 5 through 10 built a complete dispatch pipeline: forecast prices, solve an LP, execute the first period's charge or discharge decision, observe the actual price, and repeat. But every chapter so far has quietly dodged a critical reality: **the battery does not choose how much to charge or discharge**. It submits offers to AEMO, and AEMO's dispatch engine decides what the battery actually does.

The distinction matters enormously. In the LP world of Chapter 9, you solve for the optimal energy quantity at each time step, and you implicitly assume the market will let you trade that quantity at the forecast price. In the real NEM, you submit price-quantity bands *before* you know the clearing price, and you are dispatched only if the price clears past your band. The dispatch engine -- NEMDE -- sees your offers alongside every other generator and load in the market, stacks them, and clears them simultaneously. Your battery is one participant in a multi-billion-dollar auction that runs every five minutes, 288 times per day, 365 days per year.

NEMDE (NEM Dispatch Engine): AEMO's central dispatch algorithm that clears the energy and FCAS markets simultaneously every five minutes. NEMDE takes all generator offers and load bids, applies network constraints, loss factors, and inter-regional transfer limits, and finds the least-cost dispatch that meets demand. The clearing price in each region is the cost of the marginal unit of energy -- the most expensive generator dispatched to meet the last increment of demand.

This chapter bridges the gap between the mathematical optimum (the LP solution from Chapter 9) and the physical market mechanism (submitting offers to NEMDE). It introduces two simulation modes: **price-taker mode**, where the battery's actions do not affect the clearing price (what we have done so far), and **price-maker mode**, where we insert the battery's offers into the real bid stack using nempy and re-clear the market, allowing the battery's own bids to move the price.

The core question of this chapter: How do you translate "I want to discharge 100 MW at 6 PM" into a set of price-quantity offer bands that cause NEMDE to dispatch you at 100 MW when conditions are right -- and cause NEMDE to not dispatch you when conditions are wrong? And when does the act of offering 100 MW itself change the price you receive?

The NEM Offer Structure

Price-Quantity Bands

Every generator and load in the NEM submits offers structured as up to **ten price-quantity bands**. Each band specifies a price (in \$/MWh) and a quantity (in MW). The bands must be in ascending price order and together must cover the unit's full registered capacity.

Offer band: A single price-quantity pair within a generator's offer. Each band says: "I am willing to supply this many MW at or above this price." A generator may submit up to 10 bands, covering its full registered capacity. NEMDE dispatches the cheapest bands first, working up the price stack until demand is met.

For a 100 MW battery like the Mannum BESS, the ten bands might look like this for a discharge offer:

Band	Price (\$/MWh)	Quantity (MW)	Intent
1	-\$1,000	0	Floor band (empty -- not willing to discharge at negative prices)
2	\$50	10	Small tranche: discharge if price exceeds \$50
3	\$80	15	Additional capacity above \$80
4	\$120	20	Core discharge range
5	\$150	20	Core discharge range
6	\$200	15	Higher-value tranche
7	\$300	10	Reserved for moderate spikes
8	\$500	5	Reserved for significant spikes
9	\$1,000	3	Reserved for severe spikes
10	\$16,600	2	Market price cap -- last resort

The NEM's price range is from the market floor price of -\$1,000/MWh to the market price cap (MPC) of \$16,600/MWh (as of 2024-25). Any band price must fall within this range.

Market price cap (MPC): The maximum price allowed in the NEM spot market, set by the AER and adjusted annually for inflation. As of 2024-25, the MPC is \$16,600/MWh. No generator can offer above this price. The MPC exists to limit the cost of extreme price events to consumers while still providing an incentive for peaking generation to be available.

Market floor price (MFP): The minimum price allowed in the NEM, currently -\$1,000/MWh. Negative prices occur when generators -- typically wind and solar with renewable energy certificate revenue -- are willing to pay to keep running rather than shut down. Batteries benefit from negative prices: they earn revenue by charging (consuming energy) when the price is negative.

How Dispatch Works

When NEMDE runs (every five minutes), it collects all offers, builds the aggregate supply curve, and dispatches to meet demand:

1. **Stack all bands** from all generators in ascending price order
2. **Dispatch from the bottom** -- cheapest bands first
3. **Stop when supply meets demand** -- the price of the last dispatched band sets the regional clearing price
4. **All dispatched generators receive the clearing price**, regardless of their offer price

Example -- battery dispatch at the margin: Suppose the SA1 clearing price settles at \$130/MWh. The Mannum BESS offer above has bands 1-4 fully dispatched ($0 + 10 + 15 + 20 = 45$ MW), and band 5 (priced at \$150) is not dispatched because $\$150 > \130 . The battery discharges 45 MW and receives \$130/MWh for every MWh it delivers -- not the band prices of \$50, \$80, or \$120. This is the ****uniform pricing**** mechanism: the clearing price applies to all dispatched energy, regardless of offer price.

Uniform pricing: A market mechanism where all dispatched generators receive the same price -- the marginal clearing price -- regardless of their individual offer prices. This is sometimes called "pay-as-cleared" pricing. It gives generators an incentive to offer at their true marginal cost (zero for batteries, which have no fuel cost) rather than inflate their offers, because offering higher than necessary risks not being dispatched at all.

The Bidirectional Battery

A battery is unique in the NEM because it participates on both sides of the market. It acts as a **generator** when discharging (supplying energy to the grid) and as a **load** when charging (consuming energy from the grid).

Integrated resource: A resource that can both generate and consume electricity, participating on both sides of the energy market. In the NEM, batteries were historically required to register as both a "Generator" and a "Load" with separate dispatch unit identifiers (DUIDs). Under the NEM reform program, batteries can now register as a single Integrated Resource System (IRS), with a single DUID and coordinated dispatch across charge and discharge. The Mannum BESS operates under this framework.

Under the Integrated Resource System (IRS) framework, the Mannum BESS submits:

- **Discharge offers:** up to 10 price-quantity bands for generation, priced from low to high. Dispatched when the clearing price exceeds the band price.
- **Charge bids:** up to 10 price-quantity bands for load, priced from high to low. Dispatched when the clearing price falls below the band price.

The symmetry is important: discharge offers go *up* the price stack (the battery wants to sell at high prices), while charge bids go *down* the price stack (the battery wants to buy at low prices).

Example -- charge bids for the Mannum BESS: | Band | Price (\$/MWh) | Quantity (MW) | Intent | |-----|-----|-----|-----| | 1 | \$16,600 | 0 | Ceiling band (empty -- not willing to charge at extreme prices) | | 2 | \$80 | 5 | Small charge if price drops below \$80 | | 3 | \$50 | 15 | Moderate charge below \$50 | | 4 | \$30 | 20 | Bulk charging range | | 5 | \$10 | 20 | Bulk charging range | | 6 | \$0 | 15 | Charge at zero or negative prices | | 7 | -\$20 | 10 | Charge during mild negative prices | | 8 | -\$50 | 5 | Charge during moderate negative prices | | 9 | -\$200 | 5 | Charge during significant negative prices | | 10 | -\$1,000 | 5 | Market floor -- absorb in extreme negative events | If the clearing price is \$25/MWh, bands 4-10 are dispatched ($20 + 20 + 15 + 10 + 5 + 5 + 5 = 80$ MW of charging). The battery pays \$25/MWh for every MWh it consumes -- again, uniform pricing.

The offer bands encode the dispatch strategy. A battery's ten bands are not arbitrary -- they represent its view of the price distribution. Aggressive discharge bands (low prices, high quantities) mean the battery expects to discharge frequently. Conservative discharge bands (high prices, low quantities) mean the battery is waiting for spikes. The optimal band placement depends on the probabilistic forecast from Chapter 8 and the scenario analysis from Chapter 9.

Rebidding: Updating Offers in Real Time

The Rebidding Mechanism

The NEM allows generators to update their offers at any time before gate closure. This is called **rebidding**, and it is one of the most distinctive features of the NEM compared to other electricity markets.

Rebidding: The process of updating a generator's price-quantity offer bands after the initial submission but before gate closure. Rebids can change the quantities allocated to each band (shifting capacity between price levels) but must maintain the same ten band prices within a trading day. Rebids take effect from the next dispatch interval after AEMO processes them.

Gate closure: The deadline after which no further rebids are accepted for a given dispatch interval. In the NEM, gate closure is approximately 5 minutes before the start of the dispatch interval -- effectively, the most recent rebid received before NEMDE runs for that interval is the one used. This near-real-time gate closure is unusually short by international standards (many European markets close hours ahead) and gives NEM participants enormous flexibility to respond to changing conditions.

The key constraint on rebidding: within a single trading day (4:00 AM to 4:00 AM AEST), the ten band **prices** are fixed. When you rebid, you can only change the **quantities** allocated to each band. To change your band prices, you must wait for the next trading day -- or submit a new set of band prices, which resets all your bands for the remainder of the day.

Example -- rebidding in response to a forecast update: At 2:00 PM, the Mannum BESS has 80% state of charge and its original offer has 20 MW at \$120 and 20 MW at \$150 for the 5:00 PM interval. At 3:30 PM, the updated weather forecast shows cloud cover clearing earlier than expected, meaning solar generation will persist through sunset. The trader rebids: move the 20 MW from band 4 (\$120) up to band 7 (\$300), reasoning that the evening price spike will be delayed and muted by the extra solar. The rebid reason logged with AEMO: "Updated solar generation forecast -- increased PV output expected through 5:30 PM." If the price does spike to \$250 at 5:00 PM, the rebid means only $10 + 15 = 25$ MW of discharge is dispatched (bands 2-3), compared to 45 MW without the rebid. The remaining energy is preserved for later, higher-priced intervals.

Rebid Reasons and Compliance

Every rebid must include a **reason** that explains the change. AEMO and the AER review rebid reasons to detect market manipulation -- rebids that are designed to inflate prices rather than respond to genuine changes in conditions.

Acceptable rebid reasons include:

- Changes in demand or price forecasts
- Changes in weather forecasts (wind, solar, temperature)
- Changes in plant availability or equipment status
- Changes in fuel supply or cost
- Changes in interconnector flows or network constraints
- Changes in the unit's own state of charge (for batteries)
- Commercial decisions based on updated market information

The rebidding obligation: The NEM's rebidding rules require that the stated reason must be genuine -- the trader must actually be responding to the stated factor. "Rebidding to maximise profit" is not a valid reason on its own; the trader must point to a specific change in conditions (even if the effect of the change is to increase expected profit). The AER actively monitors rebidding behaviour and has taken enforcement action against generators whose rebidding patterns suggest strategic manipulation rather than genuine responses to changing conditions.

Rebidding Strategy for Batteries

Batteries rebid more frequently than most other NEM participants because their state of charge changes throughout the day, and their optimal strategy depends on remaining energy inventory. A battery that has already discharged half its energy has a fundamentally different optimal offer than one at full charge.

Common rebidding triggers for the Mannum BESS:

Trigger	Response
SoC drops below 30%	Withdraw to higher bands
Price spike in progress	Shift to lower bands, dispatch now
Solar forecast revised up	Shift charge lower; delay discharge
Interconnector binding	Increase offer if constrained
FCAS prices rising	Reposition capacity to FCAS

The Emulator: Two Modes of Simulation

Why Simulate the Market?

Throughout Chapters 5-10, we evaluated dispatch strategies by a simple method: solve the LP for the optimal schedule, then compute revenue by multiplying the dispatch quantities by the historical prices. This is **settlement on historical prices** -- the standard backtest.

But this approach embeds a hidden assumption: **the battery's dispatch does not change the market price**. When we solve the LP and dispatch 100 MW of discharge at 6 PM, we assume the price at 6 PM is the same whether the battery discharges or not. For a small battery in a large market, this is a reasonable approximation. For a 100 MW battery in SA1 -- a region where total demand might be 1,200-2,000 MW -- the battery's dispatch can shift the clearing price by several dollars per MWh or more.

This section introduces two simulation modes that handle this assumption differently.

Price-Taker Mode

Price-taker: A market participant whose actions are too small to affect the clearing price. A price-taker faces an exogenous price -- it can choose how much to buy or sell, but the price it receives is determined by other market forces. In the NEM, small generators (a few MW) are effectively price-takers. Whether a 100 MW battery is a price-taker depends on the region's size and the shape of the supply stack.

Price-taker mode is what the course has used so far:

1. **Forecast** prices for the next 48 half-hours using the models from Chapters 6-8
2. **Optimise** the dispatch schedule using the LP from Chapter 9 (scenario MPC or chance-constrained MPC)
3. **Settle** on historical prices -- multiply the dispatch quantities by the actual prices that occurred
4. **Compute** revenue and capture ratio

The price-taker assumption makes backtesting simple and fast. You do not need to model the bid stack, run NEMDE, or account for market impact. The LP from Chapter 9 is the only optimisation required.

```
# Price-taker backtest (simplified from Chapter 9) def backtest_price_taker(forecasts, actuals, battery): """Run rolling-origin backtest in price-taker mode. Args: forecasts: Quantile forecasts, shape (n_origins, horizon, n_quantiles). actuals: Actual prices, shape (n_periods,). battery: Battery parameters dict (power_mw, duration_h, rte). Returns: DataFrame with columns: dispatch_mw, price, revenue. """ results = [] for origin in range(len(forecasts)): # Solve the LP using forecast prices schedule = solve_dispatch_lp(forecasts[origin], battery, method="chance_constrained", quantile_pair=(0.10, 0.90), ) # Settle on actual prices (price-taker assumption) revenue = schedule["dispatch_mw"] * actuals[origin : origin + horizon] results.append(revenue) return pd.concat(results)
```

Price-Maker Mode

Price-maker: A market participant large enough that its actions measurably affect the clearing price. A price-maker faces a downward-sloping residual demand curve: the more it supplies, the lower the clearing price it receives. In the NEM, large generators (hundreds of MW) and large batteries in small regions are price-makers. The 100 MW Mannum BESS is at the boundary -- its market impact is non-negligible in SA1, especially during low-demand periods.

Price-maker mode uses **nempy** -- the open-source NEM dispatch engine developed by UNSW-CEEM -- to simulate the actual clearing process with the battery's offers inserted into the real bid stack.

nempy: An open-source Python implementation of the NEM dispatch engine (NEMDE), developed by the Collaborative Centre for Energy and Environmental Markets (CEEM) at UNSW. nempy takes historical bid stacks (from NEMOSIS), network constraints, interconnector models, and FCAS requirements, and re-clears the market to produce dispatch targets and prices. It can be used to simulate counterfactual market outcomes -- "what would the price have been if this battery had offered differently?"

The price-maker simulation workflow:

1. **Pull the historical bid stack** for the interval using NEMOSIS (the same data source from Chapter 1). This gives every generator's actual offer bands for that interval.
2. **Remove the battery** (or its predecessor, if applicable) from the bid stack to create a counterfactual "world without the battery."
3. **Insert the battery's new offer bands** -- the ten discharge bands and ten charge bands constructed from the forecast.
4. **Clear the market** using nempy, applying all network constraints, loss factors, and FCAS requirements.
5. **Read the results:** the new clearing price, the battery's dispatched quantity, and the dispatch of every other generator.

```
# Price-maker simulation using nempy (conceptual) import nempy from nempy import markets def simulate_price_maker(interval, battery_offer, battery_bid): """Simulate one dispatch interval with battery offers in the bid stack. Args: interval: Datetime of the dispatch interval. battery_offer: DataFrame with columns [band, price, quantity] for discharge. battery_bid: DataFrame with columns [band, price, quantity] for charge. Returns: dict with keys: clearing_price, dispatch_mw, total_revenue. """ # Create the nempy market instance market = markets.SpotMarket( market_regions=["SA1"], unit_info=get_unit_info(interval), # All generators ) # Load historical bid stacks from NEMOSIS volume_bids, price_bids = load_bid_stack(interval, region="SA1") # Remove the battery's existing bids (if present) volume_bids = volume_bids[volume_bids["DUID"] != "MNNMBESS1"] price_bids = price_bids[price_bids["DUID"] != "MNNMBESS1"] # Insert the battery's new offer bands volume_bids = pd.concat([volume_bids, battery_offer]) price_bids = pd.concat([price_bids, format_price_bands(battery_bid)]) # Set bids in the market market.set_unit_volume_bids(volume_bids) market.set_unit_price_bids(price_bids) # Load demand, interconnectors, constraints market.set_demand_constraints(get_demand(interval, "SA1")) market.set_interconnectors(get_interconnector_definitions()) # Dispatch market.dispatch() # Extract results price = market.get_energy_prices()["SA1"] dispatch = market.get_unit_dispatch() battery_dispatch = dispatch.loc[dispatch["DUID"] == "MNNMBESS1_NEW", "DISPATCH"].values[0] return { "clearing_price": price, "dispatch_mw": battery_dispatch, "total_revenue": battery_dispatch * price / 12, # 5-minute interval }
```

The key difference: In price-taker mode, the battery chooses a quantity and receives the historical price. In price-maker mode, the battery submits offer bands, nempy re-clears the market, and the battery receives a *new* price that reflects its own participation. When the battery discharges 100 MW, it adds 100 MW of supply to SA1, which pushes the clearing price down. When it charges 100 MW, it adds 100 MW of demand, which pushes the price up. Both effects reduce the battery's revenue compared to the price-taker assumption.

What nempy Does Under the Hood

nempy replicates the core logic of NEMDE:

1. **Builds the LP:** minimise total dispatch cost (equivalently, clear at the lowest possible prices) subject to:
2. Supply meets demand in each region
3. Each generator dispatched within its offered bands
4. Interconnector flows within thermal and stability limits
5. Loss factors applied to inter-regional transfers
6. FCAS requirements met (if modelled)

Ramp rate constraints (generator output cannot change faster than its ramp rate)

Solves the LP: uses a standard LP solver (HiGHS or CPLEX) to find the cost-minimising dispatch

Extracts dual variables: the shadow price on each region's demand constraint is the clearing price -- the cost of serving one more MW of demand in that region

The clearing price from nempy matches the historical AEMO price when the same bid stack and constraints are used. Inserting the battery's offers changes the supply curve and therefore changes the clearing price -- this is the market impact.

Example -- market impact of the Mannum BESS: Consider a low-demand SA1 interval where demand is 1,200 MW. Without the battery, the marginal generator is a gas plant offering at \$85/MWh. The Mannum BESS offers to discharge 100 MW in bands priced between \$50 and \$200. With the battery's offers inserted: - nempy dispatches the battery's cheap bands (say 45 MW at bands priced \$50-\$120) before reaching the gas plant - The gas plant is partially displaced -- it now dispatches 955 MW instead of 1,000 MW - But the gas plant's next band (at \$78/MWh) becomes the marginal unit, not its \$85 band - The clearing price drops from \$85 to \$78 The battery receives \$78/MWh instead of \$85/MWh -- a \$7/MWh impact. On 45 MW over a 5-minute interval, this costs the battery $\$7 \cdot 45 / 12 = \26.25 per interval, or about \$3,150 per hour of sustained dispatch. Over a year, market impact can cost a 100 MW battery hundreds of thousands of dollars in SA1.

Price-Taker vs Price-Maker: When Market Impact Matters

The Residual Demand Curve

The core concept for understanding market impact is the **residual demand curve**: the demand that remains after all other generators have been dispatched. The battery faces this residual demand -- it is competing for the marginal MW of supply.

Residual demand curve: The demand curve that a single generator faces after accounting for the supply from all other generators. It is computed by subtracting the aggregate supply curve of all other generators from the total demand curve. The slope of the residual demand curve determines how much the clearing price changes when the generator changes its output -- a steep residual demand curve means the price is insensitive to the generator's output (price-taker territory), while a flat residual demand curve means the price moves significantly (price-maker territory).

The residual demand curve can be constructed from the historical bid stack:

1. Pull all generator offers for the interval from NEMOSIS
2. Remove the battery's offers
3. Build the aggregate supply curve from the remaining offers (cumulative capacity vs price)
4. The residual demand at any price level p is: total demand minus the supply from other generators at price p

The slope of the residual demand curve at the clearing price determines the battery's **market impact factor** -- how much the price moves per MW of battery dispatch.

Figure 12.1 — The residual demand curve for SA1. The left panel shows the full supply stack with the battery's position marked. The right panel shows the residual demand curve -- the demand remaining after all other generators are dispatched. The slope of the residual demand curve at the clearing point determines the battery's market impact. During low-demand periods with a steep supply stack, even 100 MW of battery dispatch barely moves the price. During high-demand periods near the top of the stack, 100 MW can shift the price by tens of dollars per MWh.

The Market Impact Factor

Market impact factor: The change in clearing price caused by one additional MW of supply from the battery. Mathematically, it is the inverse of the slope of the residual demand curve at the operating point: $\text{impact} = -dp/dq$, where p is the clearing price and q is the battery's dispatch. Units are \$/MWh per MW. A market impact factor of \$0.05/MW means that each additional MW of battery discharge reduces the clearing price by \$0.05/MWh.

The market impact factor varies enormously depending on market conditions:

Condition	Supply stack shape	Impact factor (\$/MWh per MW)	Impact on 100 MW battery
Low demand, mid-merit	Gradual slope	\$0.01-\$0.05	\$1-\$5/MWh price shift
Moderate demand, coal-gas transition	Moderate step	\$0.05-\$0.20	\$5-\$20/MWh price shift
High demand, near capacity	Steep hockey stick	\$0.50-\$5.00	\$50-\$500/MWh price shift

Extreme low demand, renewables dominant	Nearly flat	\$0.001-\$0.01	<\$1/MWh price shift
---	-------------	----------------	----------------------

Example -- when does market impact matter for the Mannum BESS? Consider the 100 MW / 200 MWh Mannum BESS in SA1, where typical demand ranges from 800 MW (overnight) to 2,500 MW (summer afternoon peak). **At 3:00 AM, demand = 900 MW:** The supply stack is in the flat coal/wind section. The marginal generator is a coal plant with bands spread over \$25-\$40/MWh. The impact factor is about \$0.02/MW. Discharging 100 MW shifts the price by about \$2/MWh -- negligible. The price-taker assumption loses less than 3% of revenue. **At 6:00 PM, demand = 1,800 MW:** The supply stack is at the coal-gas transition. The next 200 MW of supply jumps from \$50 to \$120/MWh in discrete steps. The impact factor is about \$0.15/MW. Discharging 100 MW shifts the price by about \$15/MWh. If the historical price was \$130, the battery receives \$115 -- an 11.5% revenue loss compared to the price-taker assumption. **At 4:30 PM on a 42-degree day, demand = 2,400 MW:** The stack is near the top of the hockey stick. The next 50 MW jumps from \$200 to \$800. The impact factor is about \$2/MW. Discharging 100 MW collapses the price from \$800 to \$600 -- a 25% revenue loss. But the revenue is still enormous (\$600/MWh), and the battery is providing essential reliability to the grid.

Revenue Loss from Market Impact

The total revenue loss from market impact depends on two factors: the market impact factor and the dispatch quantity. The relationship is **quadratic** -- revenue loss grows with the square of the dispatch quantity because each additional MW both reduces the price *and* applies that lower price to all previously dispatched MW.

The revenue loss per interval is approximately:

$$\text{Revenue loss} \approx (1/2) \cdot \text{impact_factor} \cdot \text{dispatch_mw}^2 \cdot \text{interval_hours}$$

This quadratic relationship means that doubling the battery's power capacity quadruples the revenue loss from market impact. A 50 MW battery in SA1 might lose 2% of revenue to market impact, while a 200 MW battery might lose 8% -- the loss grows faster than the capacity.

Figure 12.2 — Price-taker vs price-maker revenue as a function of battery size. The left panel shows total annual revenue under each assumption. The right panel shows the revenue gap (price-taker revenue minus price-maker revenue) as a percentage. For the 100 MW Mannum BESS, the gap is approximately 4-8% of annual revenue, depending on market conditions. The gap grows quadratically with battery size, reaching 15-20% for a hypothetical 300 MW battery in SA1.

The practical threshold: As a rule of thumb, market impact becomes economically significant (costing more than 3% of revenue) when the battery's power capacity exceeds about 5% of the region's typical minimum demand. For SA1, with minimum demand around 800-1,000 MW, this threshold is about 40-50 MW. The 100 MW Mannum BESS is clearly above this threshold -- price-maker modelling is not optional for accurate revenue estimation.

The Self-Cannibalization Problem

Market impact creates a **self-cannibalization** effect: the battery's own dispatch reduces the price it receives. Discharge pushes prices down (more supply); charging pushes prices up (more demand). Both effects work against the battery.

This self-cannibalization is particularly severe for batteries because they participate on both sides of the market:

- **During discharge:** The battery adds supply, pushing the price down. The more it discharges, the lower the price it receives.
- **During charging:** The battery adds demand, pushing the price up. The more it charges, the higher the price it pays.
- **Net effect:** The arbitrage spread (discharge price minus charge price) is narrower in reality than the historical prices suggest.

Example -- self-cannibalization in both directions: Historical prices show a \$30/MWh overnight low and a \$150/MWh evening peak -- an arbitrage spread of \$120/MWh. The price-taker revenue for 100 MW · 2h = 200 MWh is $\$120 \cdot 200 / 2 = \$12,000$ (simplified, ignoring efficiency). In price-maker mode: - Charging 100 MW overnight pushes the price from \$30 to \$38 (added demand in a flat part of the stack) - Discharging 100 MW in the evening pushes the price from \$150 to \$138 (added supply in a steeper part of the stack) - The effective spread is $\$138 - \$38 = \$100/\text{MWh}$ instead of \$120/MWh - Revenue drops from \$12,000 to \$10,000 -- a 17% reduction For a smaller battery (25 MW), the price shifts would be roughly \$2 and \$3, giving a spread of \$145/MWh and a revenue reduction of only 4%.

Offer Construction from Scenarios

From Probabilistic Forecast to Offer Bands

The probabilistic forecast from Chapter 8 provides quantile predictions for each dispatch interval -- a distribution of possible prices. The copula-based scenario generation from Insert C in Chapter 8 converts these quantiles into complete, temporally correlated price trajectories. Now we need to translate these into the ten price-quantity bands required by NEMDE.

The fundamental insight: **the offer bands should reflect the battery's uncertainty about the price.** Place discharge capacity at price levels where you believe the clearing price is likely to be above your band (so you will be dispatched). Place charge capacity at price levels where you believe the clearing price is likely to be below your band (so you will be dispatched for charging).

The Quantile-to-Band Mapping

The most principled approach maps forecast quantiles directly to offer bands:

Discharge offer construction:

1. From the copula scenarios (Chapter 8), compute the quantile forecast for the current interval: $q_{0_0_5}$, $q_{0_1_0}$, $q_{0_2_0}$, $q_{0_3_0}$, $q_{0_4_0}$, $q_{0_5_0}$, $q_{0_6_0}$, $q_{0_7_0}$, $q_{0_8_0}$, $q_{0_9_0}$, $q_{0_9_5}$.
2. Assign each of the 10 offer bands a price equal to one of these quantiles (approximately).
3. Assign quantities to each band based on the MPC's optimal dispatch at that price level.

A practical algorithm:

```
def construct_discharge_offer(quantile_forecast, battery, n_bands=10): """Construct 10-band discharge offer from
quantile forecast. Args: quantile_forecast: dict mapping quantile level to price, e.g. {0.05: 20, 0.10: 28, 0.25: 45,
0.50: 85, 0.75: 140, 0.90: 250, 0.95: 400}. battery: dict with keys power_mw, duration_h, rte. n_bands: Number of
bands (NEM maximum is 10). Returns: DataFrame with columns [band, price, quantity]. """ power = battery["power_mw"] #
100 MW for Mannum # Select quantile levels for band prices # Discharge bands: price from low to high # Place most
capacity around the median and above band_quantiles = [0.10, 0.20, 0.30, 0.40, 0.50, 0.60, 0.70, 0.80, 0.90, 0.95]
bands = [] for i, q in enumerate(band_quantiles): price = quantile_forecast[q] # Allocate more capacity to bands near
the median # (where dispatch is most likely to be partially filled) if 0.30 <= q <= 0.70: qty = power * 0.15 # 15 MW
each for core bands elif q >= 0.80: qty = power * 0.05 # 5 MW each for tail bands else: qty = power * 0.10 # 10 MW
each for lower bands bands.append({"band": i + 1, "price": price, "quantity": qty}) # Adjust quantities to sum to
total power df = pd.DataFrame(bands) df["quantity"] = df["quantity"] * power / df["quantity"].sum() return df
```

Charge bid construction follows the same logic but inverted -- place charge capacity at low quantiles:

```
def construct_charge_bid(quantile_forecast, battery, n_bands=10): """Construct 10-band charge bid from quantile
forecast. Charge bids use the lower tail of the price distribution. The battery wants to charge when prices are low.
""" power = battery["power_mw"] # Charge bands: price from high to low (NEM convention) # Place most capacity at low
price quantiles band_quantiles = [0.90, 0.80, 0.70, 0.60, 0.50, 0.40, 0.30, 0.20, 0.10, 0.05] bands = [] for i, q in
enumerate(band_quantiles): price = quantile_forecast[q] if 0.20 <= q <= 0.50: qty = power * 0.15 elif q <= 0.10: qty =
power * 0.05 else: qty = power * 0.10 bands.append({"band": i + 1, "price": price, "quantity": qty}) df =
pd.DataFrame(bands) df["quantity"] = df["quantity"] * power / df["quantity"].sum() return df
```

The Stochastic Dispatch Target

The scenario MPC from Chapter 9 produces a dispatch target -- the optimal charge or discharge for each interval, optimised across multiple price scenarios. The offer bands need to implement this target approximately, dispatching the right quantity at the right clearing prices.

The connection between the LP dispatch target and the offer bands is:

1. **Solve the scenario MPC** across N copula scenarios to get the optimal dispatch target d_t^* for each interval t
2. **Construct offer bands** that deliver approximately d_t^* when the clearing price equals the median forecast, and adjust the dispatch sensibly when the clearing price differs

The offer bands act as a **contingency plan**: if the price is higher than expected, the battery discharges more (the additional bands are dispatched). If the price is lower than expected, the battery discharges less (fewer bands are dispatched). The MPC's dispatch target is the "central case," and the offer bands spread the dispatch around it to handle forecast uncertainty.

Figure 12.3 — Translating a stochastic dispatch target into offer bands. The left panel shows the optimal dispatch from scenario MPC (Chapter 9) across 50 copula scenarios (Chapter 8). The middle panel shows the quantile forecast for the 6 PM interval. The right panel shows the resulting 10-band discharge offer, with band quantities concentrated around the median forecast price and smaller quantities in the tails. The shaded area shows the range of dispatch outcomes across different clearing prices.

The offer bands encode a conditional strategy. Unlike the LP, which produces a single dispatch quantity per interval, the offer bands produce a dispatch quantity that depends on the realised clearing price. This is strictly more powerful than the LP approach -- the battery's dispatch automatically adapts to the actual market conditions without needing to rebid. The LP asks "what is the best quantity to trade at the expected price?" The offer bands ask "what is the best quantity to trade at every possible price?"

Worked Example: One Day's Offer Construction

Let us walk through a complete example for the Mannum BESS on a summer day in SA1.

Setup: - Date: 15 January 2026 - Battery: 100 MW / 200 MWh, RTE = 87%, initial SoC = 50% - Forecast: quantile regression from Chapter 7, calibrated per Chapter 8, scenarios generated by Gaussian copula (Insert C, Chapter 8) - Dispatch strategy: chance-constrained MPC from Chapter 9, quantile pair (0.10, 0.90)

Step 1: Generate the probabilistic forecast.

The forecast for the 24-hour horizon (48 half-hour intervals) provides quantile predictions at levels 0.05, 0.10, 0.25, 0.50, 0.75, 0.90, 0.95. Key intervals shown below (04:00 = overnight, 10:00 = morning solar, 14:00 = peak solar, 17:30 = ramp hour, 19:00 = evening peak, 23:00 = late evening):

Period	q05	q25	q50	q75	q95
04:00	-\$5	\$15	\$28	\$42	\$65
10:00	-\$20	\$5	\$22	\$55	\$110
14:00	-\$50	-\$10	\$12	\$45	\$130
17:30	\$60	\$110	\$180	\$320	\$800
19:00	\$80	\$150	\$260	\$500	\$2,500

23:00	\$10	\$30	\$48	\$72	\$120
-------	------	------	------	------	-------

Step 2: Solve the chance-constrained MPC.

Using the $q_{0_1_0}$ / $q_{0_9_0}$ quantile pair, the LP produces the following dispatch schedule (selected intervals):

Period	MW	Action
04:00	-60	Charge
10:00	-80	Charge
14:00	-100	Max charge
17:30	+80	Discharge
19:00	+100	Max discharge
23:00	0	Idle

Step 3: Construct offer bands for the 19:00 interval.

The LP says "discharge 100 MW." The quantile forecast is: $q_{0_0_5} = \$80$, $q_{0_1_0} = \$110$, $q_{0_2_5} = \$150$, $q_{0_5_0} = \$260$, $q_{0_7_5} = \$500$, $q_{0_9_0} = \$1,200$, $q_{0_9_5} = \$2,500$.

The discharge offer bands:

Band	Price (\$/MWh)	Quantity (MW)	Logic
1	\$80	10	5th percentile -- dispatch floor
2	\$100	10	Between 5th and 10th percentile
3	\$110	10	10th percentile -- LP's conservative price
4	\$130	15	Between 10th and 25th percentile
5	\$150	15	25th percentile
6	\$200	15	Between 25th and 50th percentile
7	\$260	10	50th percentile -- median
8	\$500	8	75th percentile -- held for spikes
9	\$1,200	5	90th percentile -- held for significant spikes
10	\$2,500	2	95th percentile -- extreme events

If the actual price clears at \$180/MWh: bands 1-5 dispatch fully (60 MW) and band 6 dispatches partially (the remaining demand up to the clearing price). Total dispatch is approximately 65 MW at \$180/MWh.

If the actual price clears at \$600/MWh: bands 1-8 dispatch fully (93 MW). Revenue is much higher.

If the actual price clears at \$70/MWh: no bands dispatch (all bands priced above \$70). The battery holds its charge for later -- the forecast was wrong about this interval, and the offer bands protected against the loss.

Step 4: Settle and compare.

Mode	Clearing price	Dispatch (MW)	Revenue (half-hour)
Price-taker (historical)	\$245	100	\$12,250
Price-taker (offer bands)	\$245	85 (bands 1-6 + partial 7)	\$10,413
Price-maker (nempy)	\$228	82	\$9,348

The price-taker mode with direct LP dispatch overstates revenue by assuming the full 100 MW dispatches at the historical price. The offer bands naturally throttle dispatch when the price is below some band thresholds. The price-maker mode shows the additional revenue reduction from market impact -- the battery's 82 MW of discharge pushed the clearing price down by \$17/MWh from the historical \$245.

The revenue hierarchy: Price-taker LP > Price-taker with offer bands > Price-maker with offer bands. The first gap reflects the constraint of working within the 10-band offer structure (a coarser control than a continuous LP). The second gap reflects market impact. For the Mannum BESS, the combined gap is typically 8-15% of price-taker LP revenue.

The Full Price-Maker Backtest

Rolling-Origin Simulation

The price-maker backtest follows the same rolling-origin structure as Chapter 9 (and the shared backtest harness from Chapter 5), but replaces the simple price settlement with a nempy re-clearing at each interval:

```
def backtest_price_maker(forecasts, battery, region="SA1"): """Rolling-origin backtest with nempy price-maker simulation. For each forecast origin: 1. Solve chance-constrained MPC to get dispatch targets 2. Construct 10-band offers from targets + quantile forecast 3. For each interval in the execution window: a. Load historical bid stack from NEMOSIS b. Insert battery offers into nempy c. Clear the market d. Record dispatch and revenue at the new clearing price 4. Update battery state of charge 5. Roll forward to next origin Args: forecasts: Quantile forecasts for each origin. battery: Battery parameters dict. region: NEM region code. Returns: DataFrame with dispatch, clearing prices (original and new), revenue, and market impact. """ results = [] soc = battery["duration_h"] * battery["power_mw"] * 0.5 # Start at 50% for origin in forecast_origins: # Step 1: Solve MPC targets = solve_dispatch_lp( forecasts[origin], battery, method="chance_constrained", initial_soc=soc, ) # Step 2-3: Execute each interval for t in execution_window(origin): # Construct offers from target + quantile forecast if targets[t] > 0: # Discharge offer = construct_discharge_offer( forecasts[origin].quantiles[t], battery ) bid = empty_charge_bid(battery) else: # Charge offer = empty_discharge_offer(battery) bid = construct_charge_bid( forecasts[origin].quantiles[t], battery ) # nempy re-clearing result = simulate_price_maker(t, offer, bid) # Record results.append({ "interval": t, "original_price": actual_prices[t], "new_price": result["clearing_price"], "dispatch_mw": result["dispatch_mw"], "revenue": result["total_revenue"], "market_impact": actual_prices[t] - result["clearing_price"], }) # Update SoC soc = update_soc(soc, result["dispatch_mw"], battery) return pd.DataFrame(results)
```

Computational Considerations

The price-maker backtest is substantially more expensive than the price-taker backtest:

Component	Price-taker	Price-maker
Forecast generation	Same	Same
LP solve	~10 ms per origin	~10 ms per origin
Market clearing	Not needed	~0.5-2 s per interval (nempy)
Bid stack loading	Not needed	~0.1 s per interval (NEMOSIS)
Total for 1 year (half-hourly)	~5 minutes	~8-12 hours

The nempy clearing is the bottleneck. Each dispatch interval requires loading the complete bid stack (all generators' offers), building the LP, solving it, and extracting results. For a year of half-hourly data (17,520 intervals), this can take 8-12 hours on a modern laptop.

Strategies to manage this:

1. **Subsample intervals:** run the price-maker simulation on a representative subset (e.g., every 6th interval, or only peak hours) and interpolate
2. **Cache bid stacks:** download all bid stacks once and load from disk
3. **Parallelise:** each interval's clearing is independent; use multiprocessing
4. **Selective mode:** run price-taker for most intervals, switch to price-maker only during high-impact periods (high demand, steep supply stack)

Example -- selective price-maker simulation: Analyse a year of SA1 data and flag intervals where the market impact factor exceeds \$0.05/MW (roughly the threshold where 100 MW shifts the price by more than \$5). Typically, about 30% of intervals exceed this threshold. Run price-maker simulation only on these intervals and use price-taker settlement for the rest. This reduces computation time from 12 hours to about 4 hours while capturing >90% of the market impact effect.

Revenue Assessment and Benchmarks

Capture Ratio in Price-Maker Mode

The capture ratio framework from Chapter 9 extends naturally to price-maker mode. The key difference is that the perfect-foresight benchmark must also account for market impact:

```
CR_price_taker = Revenue_price_taker / Revenue_perfect_foresight_price_taker
CR_price_maker = Revenue_price_maker / Revenue_perfect_foresight_price_maker
```

The price-maker perfect foresight is computed by solving the dispatch LP with actual prices, then re-clearing each interval through nempy with the optimal dispatch as offers. The perfect-foresight revenue is lower in price-maker mode because even the perfect-foresight battery suffers market impact.

Metric	Price-taker	Price-maker	Difference
Perfect foresight revenue (SA1, annual)	\$12M	\$10.5M	-12.5%
Capture ratio (good forecast)	0.65-0.75	0.60-0.72	-3 to -5 pp
Annual revenue (good forecast)	\$7.8M-\$9.0M	\$6.3M-\$7.6M	-\$1.2M-\$1.7M

Benchmark targets for the Mannum BESS: The 0.50 capture ratio bar represents the minimum acceptable performance -- below this, the dispatch strategy is leaving too much money on the table. The target range is 0.65-0.75 in price-taker mode, corresponding to 0.60-0.72 in price-maker mode after accounting for market impact. These benchmarks should be reported against AEMO pre-dispatch forecasts as the baseline -- a strategy that merely follows AEMO's published pre-dispatch prices. A capture ratio that fails to beat the AEMO pre-dispatch baseline has no value; the operator could achieve the same result without any proprietary forecasting.

Revenue Decomposition by Source

A detailed revenue decomposition helps identify where the dispatch strategy gains and loses value:

Component	Taker \$M	Maker \$M
Base arbitrage (daily)	5.5	4.8
Spike capture (>\$300)	2.5	2.0
Negative price capture	0.8	0.9
Intra-day secondary	1.2	0.9
Total arbitrage	10.0	8.6
Market impact cost	--	-1.4

The price-maker total is 14% below the price-taker result due to self-cannibalization. Negative price capture is slightly higher for the price-maker because charging at negative prices increases demand, making the negative price less negative. Smaller intra-day spreads are more affected by market impact than the primary cycle.

Example -- monthly capture ratio comparison: Running the Mannum BESS backtest for July 2025 (winter, moderate volatility) and January 2026 (summer, high volatility): **July 2025:** Perfect foresight revenue = \$650K. Price-taker CR = 0.71 (\$462K). Price-maker CR = 0.67 (\$435K). Market impact cost = \$27K (4.2% of price-taker revenue). **January 2026:** Perfect foresight revenue = \$1,400K. Price-taker CR = 0.68 (\$952K). Price-maker CR = 0.62 (\$868K). Market impact cost = \$84K (8.8% of price-taker revenue). Market impact is proportionally larger in January because summer spikes occur on a steeper part of the supply stack, and the battery's discharge during these spikes has a larger price-depressing effect.

Strategy Search (Advanced)

The Methods Placement Principle

Before exploring advanced strategy search methods, it is worth restating where each optimisation approach fits in the broader curriculum:

The methods placement principle: Convex and stochastic optimisation -- the LP, scenario MPC, and chance-constrained MPC from Chapters 5 and 9 -- is the **core** of production battery dispatch. It is interpretable (the LP's dual variables tell you exactly why each decision was made), constraint-respecting (battery limits are enforced by construction), and fast enough for real-time MPC. Decision-focused learning (Chapter 16) wraps this convex core to train the forecaster end-to-end: the forecast parameters are adjusted to maximise dispatch revenue, not forecast accuracy. Reinforcement learning (RL) and genetic search only make sense **against the price-maker emulator**, where the optimal bid depends on other participants' bids and the bid-to-price mapping is non-convex. They come last in the curriculum -- after the convex baseline and emulator exist -- because without a solid baseline and a realistic simulator, there is nothing meaningful to improve upon and no way to evaluate whether the improvement is genuine.

The reason for this ordering is not merely pedagogical -- it reflects engineering reality. Production battery trading desks overwhelmingly use convex optimisation because:

1. **Interpretability:** Regulators and risk managers can audit the dispatch logic. An LP's dual variables explain every decision.
2. **Constraint satisfaction:** Battery limits (SoC bounds, ramp rates, cycle limits) are hard constraints in the LP. RL and genetic methods satisfy constraints only approximately, requiring penalty terms that must be tuned.
3. **Convergence guarantees:** The LP finds the global optimum in polynomial time. RL may not converge; genetic algorithms may settle on local optima.
4. **Speed:** The LP solves in milliseconds. RL requires thousands of episodes to train.

The one domain where convex optimisation falls short is **strategic bidding** against a price-maker emulator. When the battery's offers affect the clearing price, and the clearing price depends on other generators' offers (which are themselves strategic), the bid-to-revenue mapping is not convex. The optimal offer depends on the full bid stack -- a high-dimensional, non-smooth function that the LP cannot capture.

Reinforcement Learning for Bidding

In principle, reinforcement learning (RL) can learn to bid strategically in the price-maker emulator. The formulation:

- **State:** battery SoC, current time, recent prices, forecast quantiles, recent bid stack shape
- **Action:** the 10 band quantities for the current interval (continuous action space)
- **Reward:** revenue from the nempy clearing at each interval

- **Environment:** the price-maker emulator (nempy with historical bid stacks)

The RL agent observes the state, selects offer bands, nempy clears the market, the agent receives the revenue as reward, and the process repeats. Over thousands of training episodes, the agent learns which offer band configurations produce the most revenue in different market conditions.

Reinforcement learning (RL): A machine learning paradigm where an agent learns to make sequential decisions by interacting with an environment and receiving rewards. The agent's goal is to learn a policy -- a mapping from states to actions -- that maximises cumulative reward over time. For battery bidding, the environment is the price-maker emulator, the actions are the offer bands, and the rewards are dispatch revenue. RL excels when the optimal action depends on a complex, non-differentiable mapping from action to outcome -- exactly the situation with strategic bidding.

Challenges specific to NEM battery bidding with RL:

1. **High-dimensional action space:** 10 band quantities, each continuous, for both discharge and charge = 20 continuous actions per interval. This requires sophisticated RL algorithms (e.g., SAC, PPO with continuous actions).
2. **Slow environment:** each nempy clearing takes 0.5-2 seconds. Training for 10,000 episodes of 48 intervals each requires 250,000-1,000,000 clearings = days of computation.
3. **Non-stationarity:** the bid stack changes daily as generators enter, exit, and change their bidding behaviour. An RL policy trained on 2024 data may not generalise to 2025.
4. **Constraint enforcement:** the RL policy must respect battery constraints (SoC bounds, ramp rates). This requires either action masking or penalty terms.
5. **Sample efficiency:** historical data is limited (the NEM has run since 1998, but bid stack data at 5-minute resolution is available only from ~2009). RL may overfit to the training period.

Genetic Search for Offer Strategies

An alternative to RL is **genetic search** (or evolutionary optimisation), which searches the space of bidding strategies directly:

Genetic search (evolutionary optimisation): An optimisation method inspired by biological evolution. A population of candidate solutions (bidding strategies) is evaluated against a fitness function (revenue in the price-maker emulator). The best-performing strategies are selected, combined (crossover), and randomly modified (mutation) to create the next generation. Over many generations, the population evolves toward higher-fitness strategies. Genetic search is well-suited to problems with non-smooth, non-convex objective functions and discrete or mixed action spaces.

A practical setup:

1. **Parameterise the bidding strategy** as a small set of numbers: e.g., "discharge bands are placed at the $[a_1, a_2, \dots, a_{10}]$ quantiles of the forecast, with equal quantities." This reduces the search space from 20 continuous variables per interval to a handful of strategy parameters.
2. **Evaluate each strategy** by running the full price-maker backtest (or a subsample) and computing the annual revenue.
3. **Evolve the population** using standard genetic operators: tournament selection, arithmetic crossover, Gaussian mutation.

```
# Genetic search for bidding strategy (simplified)
import numpy as np
def parameterised_offer(quantile_forecast, strategy_params, battery):
    """Construct offers using a parameterised strategy.
    strategy_params: array of 10 quantile levels for discharge bands.
    e.g. [0.05, 0.15, 0.25, 0.35, 0.45, 0.55, 0.65, 0.75, 0.85, 0.95]
    """
    power = battery["power_mw"]
    bands = []
    for i, q in enumerate(sorted(strategy_params)):
        price = np.interp(q, list(quantile_forecast.keys()), list(quantile_forecast.values()))
        qty = power / len(strategy_params)
        # Equal allocation
        bands.append({"band": i + 1, "price": price, "quantity": qty})
    return pd.DataFrame(bands)

def fitness(strategy_params, forecasts, battery, sample_days):
    """Evaluate a strategy by running price-maker backtest on sample days.
    """
    total_revenue = 0
    for day in sample_days:
        for t in day_intervals(day):
            offer = parameterised_offer(forecasts[t], strategy_params, battery)
            result = simulate_price_maker(t, offer, empty_charge_bid(battery))
            total_revenue += result["total_revenue"]
    return total_revenue

def genetic_search(forecasts, battery, n_generations=50, pop_size=30):
    """Search for optimal bidding strategy using genetic algorithm.
    """
    # Initial population: random quantile level vectors
    population = [np.sort(np.random.uniform(0.01, 0.99, 10)) for _ in range(pop_size)]
    sample_days = select_representative_days(n=20)
    for gen in range(n_generations):
        # Evaluate fitness scores
        scores = [fitness(p, forecasts, battery, sample_days) for p in population]
        # Select top half
        sorted_idx = np.argsort(scores)[::-1]
        survivors = [population[i] for i in sorted_idx[:pop_size // 2]]
        # Crossover and mutation
        children = []
        for _ in range(pop_size // 2):
            p1, p2 = np.random.choice(len(survivors), 2, replace=False)
            alpha = np.random.uniform(0, 1)
            child = alpha * survivors[p1] + (1 - alpha) * survivors[p2]
            child += np.random.normal(0, 0.02, 10)
            child = np.clip(np.sort(child), 0.01, 0.99)
            children.append(child)
        population = survivors + children
        print(f"Gen {gen}: best revenue = ${max(scores):.0f}")
        best_idx = np.argmax(scores)
    return population[best_idx]
```

The genetic search typically finds strategies that outperform the convex baseline by 2-5% in the price-maker emulator. The improvement comes from learning to:

- **Shade offers** slightly above the competitive level in intervals where the supply stack is steep (capturing higher prices at the cost of slightly lower dispatch probability)
- **Split discharge** across multiple intervals rather than concentrating in the single highest-price interval (reducing market impact)
- **Time charge** to avoid demand peaks in the overnight market (reducing the price-inflating effect of charging)

When to use strategy search: Start with the convex baseline (Chapter 9). If the price-maker backtest shows market impact costs exceeding 5% of revenue, strategy search is worth investigating. For the Mannum BESS in SA1, market impact typically costs 5-10% of revenue, placing it squarely in the range where strategy search can add value. For a smaller battery (30 MW) in a larger region (NSW1), market impact is negligible and strategy search offers no benefit over the convex baseline.

Putting It Together: The Complete Bidding Pipeline

The complete pipeline from forecast to market participation chains together all the components developed across the course:

1. **Data ingestion** (Chapter 1): pull NEMOSIS price and demand data, NEMSEER forecasts, ERA5 weather data
2. **Feature engineering** (Chapters 3-4): compute net load, calendar features, weather lags, interconnector flows
3. **Point forecasting** (Chapters 6-7): LEAR and GBT models for the price median
4. **Probabilistic forecasting** (Chapter 8): quantile regression, QRA combination, copula scenario generation
5. **Dispatch optimisation** (Chapter 9): scenario MPC or chance-constrained MPC to get dispatch targets
6. **Offer construction** (this chapter): translate dispatch targets and quantile forecasts into 10-band offers
7. **Market simulation** (this chapter): clear offers through nempy in price-maker mode
8. **Settlement** (this chapter): compute revenue at the new clearing price
9. **Performance evaluation**: capture ratio, AEMO pre-dispatch comparison, market impact decomposition

Each step builds on the previous ones. The forecast quality from Chapters 6-8 determines how well the offer bands are placed. The dispatch strategy from Chapter 9 determines the central dispatch target. The offer construction from this chapter determines how the target is mapped into the NEM's 10-band format. And the price-maker simulation from this chapter determines the actual revenue after market impact.

Example -- end-to-end revenue comparison for the Mannum BESS: | Stage | Annual revenue | Capture ratio | Notes |
|-----|-----|-----|-----| | Perfect foresight (price-taker) | \$12.0M | 1.00 | Theoretical maximum | | Perfect foresight (price-maker) | \$10.5M | -- | Market impact even with perfect knowledge | | Chance-constrained MPC (price-taker) | \$8.4M | 0.70 | Chapter 9 result | | Chance-constrained MPC (price-maker) | \$7.2M | 0.69 (of PM PF) | This chapter -- realistic assessment | | AEMO pre-dispatch baseline (price-taker) | \$6.0M | 0.50 | Benchmark to beat | | Naive similar-day baseline (price-taker) | \$4.8M | 0.40 | Chapter 5 baseline | The gap between "chance-constrained MPC (price-taker)" and "chance-constrained MPC (price-maker)" is \$1.2M/year -- the cost of market impact for the Mannum BESS. The gap between the price-maker result and the AEMO pre-dispatch baseline is \$1.2M/year -- the value added by proprietary forecasting and optimisation. Both gaps are large enough to justify the investment in the full pipeline.

FCAS Co-Optimisation in Bidding

Energy and FCAS Offers

The NEM co-optimises energy and FCAS (Frequency Control Ancillary Services -- introduced in Chapter 11) simultaneously. This means the battery's energy offers and FCAS offers interact: capacity allocated to FCAS enablement cannot simultaneously be dispatched for energy, and vice versa.

For the Mannum BESS, the offer structure includes both energy and FCAS bands:

- **Energy discharge:** 10 price-quantity bands (as described above)
- **Energy charge:** 10 price-quantity bands
- **FCAS raise:** up to 10 price-quantity bands for each of the four raise services (contingency 6s, 60s, 5min; regulation raise)
- **FCAS lower:** up to 10 price-quantity bands for each of the four lower services

NEMDE co-optimises all of these simultaneously, allocating the battery's capacity to whichever combination of energy and FCAS maximises total market surplus.

Example -- energy-FCAS tradeoff: The Mannum BESS has 100 MW of capacity. If it offers 80 MW for energy discharge and 20 MW for FCAS contingency raise, NEMDE might dispatch: - 70 MW of energy discharge at \$130/MWh (energy revenue = $\$130 \cdot 70 / 12 = \$758/\text{interval}$) - 20 MW of FCAS raise enablement at \$15/MW/h (FCAS revenue = $\$15 \cdot 20 / 12 = \$25/\text{interval}$) - 10 MW unallocated (the energy dispatch target was below the full 80 MW offered) The total revenue is \$783/interval. If the battery had offered all 100 MW for energy, it might have dispatched 90 MW at \$125/MWh (lower price due to more supply) = \$938/interval. In this case, the energy-only strategy wins. But during periods of high FCAS prices (which can reach \$300/MW/h or more), the FCAS allocation is far more profitable. The dispatch LP from Chapter 9 can be extended to co-optimize energy and FCAS dispatch (covered in Chapter 11). The offer construction here must then produce consistent energy and FCAS bands.

Loss Factors and Regional Considerations

In the NEM, generators do not receive the regional reference price directly -- they receive the price adjusted by a **marginal loss factor (MLF)** that accounts for transmission losses between the generator's connection point and the regional reference node.

Marginal loss factor (MLF): A multiplier applied to a generator's revenue to account for transmission losses. An MLF of 0.98 means the generator receives 98% of the regional reference price -- 2% is lost in transmission. MLFs are published annually by AEMO and depend on the generator's location in the network. Generators far from the regional reference node or on congested network paths tend to have lower MLFs.

The Mannum BESS, located in regional South Australia, has an MLF that depends on its network position. In price-maker mode, the MLF affects both the battery's received price and its market impact:

- **Revenue** = $\text{dispatch_mw} \cdot \text{clearing_price} \cdot \text{MLF} / 12$ (for a 5-minute interval)
- **Market impact** is computed at the regional reference node, but the battery's effective impact on its own revenue is scaled by MLF

For offer construction, the band prices should be adjusted by the MLF: if the MLF is 0.97, a band priced at \$100/MWh receives only \$97/MWh at settlement. The offer should account for this by pricing bands slightly higher (dividing by MLF).

Exercises

Exercise 12.1: Offer Aggressiveness and the Revenue-Risk Tradeoff

Problem: The Mannum BESS faces a forecast for the 6 PM interval: $q_{0.1_0} = \$80$, $q_{0.5_0} = \$150$, $q_{0.9_0} = \$400$. You have 100 MW of discharge capacity. Consider three offer strategies:

- (a) **Aggressive:** Place all 100 MW at \$80/MWh (the 10th percentile). You will be dispatched whenever the price exceeds \$80.
- (b) **Moderate:** Spread capacity evenly across five bands from \$80 to \$400 (20 MW at \$80, \$120, \$180, \$260, \$400).
- (c) **Conservative:** Place all 100 MW at \$400/MWh (the 90th percentile). You will only be dispatched during spikes.

For each strategy, compute the expected revenue across 1,000 simulated prices drawn from the forecast distribution (assume log-normal with the given quantiles). Also compute the 10th percentile revenue (the "downside case") and the probability of zero dispatch.

Worked solution

Step 1: Fit a log-normal distribution to the quantiles.

$q_{0.1} = \exp(\mu - 1.28 \cdot \sigma) = \80 $q_{0.5} = \exp(\mu) = \$150$ $q_{0.9} = \exp(\mu + 1.28 \cdot \sigma) = \400 From $q_{0.5}$: $\mu = \ln(150) = 5.01$ From $q_{0.1}$: $5.01 - 1.28 \cdot \sigma = \ln(80) = 4.38$ $\sigma = (5.01 - 4.38) / 1.28 = 0.49$ Check $q_{0.9}$: $\exp(5.01 + 1.28 \cdot 0.49) = \exp(5.64) = \281

The check does not exactly match \$400 — the real distribution is right-skewed beyond log-normal. Accept the approximation for this exercise.

Step 2: Define dispatch rules for each strategy.

Draw 1,000 prices: $p \sim \text{LogNormal}(5.01, 0.49^2)$.

- (a) **Aggressive:** dispatch = 100 MW if $p > \$80$, else 0. Revenue = $100 \cdot p / 2$.
- (b) **Moderate:** dispatch = 20 · (number of bands where $p \geq \text{band price}$). At $p = \$160$, bands at \$80 and \$120 clear → 40 MW dispatched. Revenue = $40 \cdot \$160 / 2 = \$3,200$.
- (c) **Conservative:** dispatch = 100 MW if $p \geq \$400$, else 0.

Step 3: Compare simulation results.

Strategy	E[Revenue] (\$/half-hr)	q _{0.1} Revenue	P(zero dispatch)
Aggressive	\$8,750	\$4,200	10%
Moderate	\$6,200	\$1,600	10%
Conservative	\$2,100	\$0	73%

Step 4: Interpret.

- **Aggressive** has the highest expected revenue but discharges at every price above \$80, possibly below the round-trip cost of charging. It exhausts the battery early, leaving nothing for later spikes.
- **Moderate** preserves some capacity for high prices by dispatching partial quantities at moderate prices.
- **Conservative** earns the most per MWh dispatched but misses most opportunities — profitable only if spikes are frequent and severe.

The **tradeoff** depends on state of charge (if nearly full, be aggressive), the remaining forecast horizon (if more high-price intervals expected, be conservative), and forecast confidence (if the 10th-90th range is wide, spread the offers more).

Exercise 12.2: Finding the Price-Taker Threshold

Problem: Run the price-maker simulation for hypothetical batteries of sizes 10, 25, 50, 100, 150, 200, and 300 MW (all 2-hour duration, 87% RTE) in SA1. For each size, compute:

- (a) The average market impact (\$/MWh price change per interval when the battery is dispatching)
- (b) The annual revenue loss from market impact (price-maker revenue minus price-taker revenue)
- (c) The revenue loss as a percentage of price-taker revenue

Plot (c) against battery size. At what size does the price-taker assumption start to cost real money (defined as >3% revenue loss)?

Worked solution

Step 1: Run price-maker backtest for each battery size.

Using one year of SA1 data (2025), chance-constrained MPC (quantile pair 0.10/0.90):

Battery size (MW)	Avg impact (\$/MWh)	Revenue loss (\$/yr)	Loss (% of PT)
10	\$0.4	\$18K	0.5%
25	\$1.1	\$95K	1.2%
50	\$2.5	\$340K	2.8%
100	\$5.8	\$1,200K	6.2%
150	\$9.5	\$2,800K	10.1%
200	\$14.2	\$5,100K	14.8%
300	\$24.0	\$11,500K	23.5%

Step 2: Find the 3% threshold.

The 3% threshold is crossed at approximately **55 MW** in SA1. Below 55 MW, price-taker is adequate. Above 55 MW, price-maker modelling is necessary.

Step 3: Interpret the scaling.

Doubling the battery from 100 to 200 MW roughly quadruples the revenue loss (\$1.2M → \$5.1M), while only doubling price-taker revenue (\$19.4M → \$34.5M). This quadratic scaling means market impact grows much faster than capacity.

Key observations:

- **Region matters.** SA1 (800–2,500 MW demand) has a low threshold. NSW1 (5,000–12,000 MW) would be ~200–300 MW.
- **Time of day matters.** Overnight low demand (~800 MW): even 25 MW has measurable impact. Summer afternoon peaks (~2,500 MW): 100 MW has minimal impact.
- **Crowding compounds.** Three 100 MW batteries in SA1 create far worse combined impact than any one alone — this is the **crowding problem** in battery investment.

Exercise 12.3 (Optional Advanced): Strategy Search vs Convex Baseline

Problem: Implement a simple genetic search that optimises the quantile levels used for discharge offer band placement. Use the parameterised strategy from the "Genetic Search" section above. Run the search against 20 representative days in SA1 using the price-maker emulator.

(a) What quantile levels does the genetic search converge to? How do they differ from the uniform spacing [0.05, 0.15, ..., 0.95]?

(b) What is the revenue improvement over the convex baseline (chance-constrained MPC with uniform band spacing)?

(c) Does the optimised strategy generalise to an out-of-sample test period (10 days not in the training set)?

Worked solution

Running the genetic search with population 30, 50 generations, on 20 SA1 days from 2025.

(a) Converged quantile levels:

Band	Uniform spacing	Genetic search result
1	0.05	0.08
2	0.15	0.14
3	0.25	0.22
4	0.35	0.33
5	0.45	0.42
6	0.55	0.55
7	0.65	0.68
8	0.75	0.78
9	0.85	0.88
10	0.95	0.97

The search pushes top bands toward the tails and compresses lower bands — reserving more capacity for high-price outcomes. The shift is subtle; the improvement comes from fine-tuning to the SA1 supply stack shape.

(b) Revenue improvement:

Strategy	Revenue (20 training days)	Improvement
----------	----------------------------	-------------

Convex baseline (uniform bands)	\$185,000	-
Genetic search (optimised bands)	\$193,500	+4.6%

Two effects: (1) higher top bands (0.88, 0.97 vs 0.85, 0.95) retain more capacity for spikes, capturing \$4K–\$6K extra on days above \$500/MWh; (2) compressed lower bands reduce market impact at moderate prices.

(c) Out-of-sample performance:

Strategy	Revenue (10 test days)	Improvement
Convex baseline (uniform bands)	\$98,000	-
Genetic search (optimised bands)	\$101,000	+3.1%

The improvement generalises but is smaller (3.1% vs 4.6%) — the search over-fits slightly to training bid stack patterns. Still significant at $p < 0.05$ (paired t-test).

Practical guidance: 3–5% improvement $\approx$ \$200K–\$400K/year for Mannum. Economically meaningful but requires maintaining a price-maker simulation pipeline. The convex baseline remains the production workhorse; strategy search is a refinement for large batteries in small regions.

Glossary

Term	Definition
Offer band	A price-quantity pair in a generator's offer; the NEM allows up to 10 bands per unit
Rebidding	Updating the quantity allocation across offer bands before gate closure
Gate closure	The deadline for submitting offer updates (~5 minutes before the dispatch interval in the NEM)
Price-taker	A market participant too small to affect the clearing price
Price-maker	A market participant large enough to move the clearing price through its own actions
Market impact	The change in clearing price caused by a generator's dispatch
Residual demand	The demand remaining after all other generators' supply is accounted for; the demand curve faced by a single generator
Integrated resource	A resource that can both generate and consume (e.g., a battery), participating on both sides of the energy market
Bid stack	The collection of all generators' offer bands, stacked in ascending price order
Market price cap (MPC)	The maximum allowed price in the NEM (\$16,600/MWh as of 2024-25)
Market floor price (MFP)	The minimum allowed price in the NEM (-\$1,000/MWh)
Uniform pricing	All dispatched generators receive the same clearing price, regardless of offer price
NEMDE	NEM Dispatch Engine -- AEMO's algorithm for clearing the energy and FCAS markets
nempy	Open-source Python implementation of NEMDE by UNSW-CEEM
Marginal loss factor (MLF)	Multiplier adjusting a generator's revenue for transmission losses
Self-cannibalization	The effect where a battery's own dispatch reduces the price it receives
FCAS co-optimisation	Simultaneous clearing of energy and FCAS markets, allocating capacity across services
Capture ratio (CR)	Actual dispatch revenue divided by perfect-foresight revenue
Genetic search	Evolutionary optimisation method for finding high-performing bidding strategies

Summary

This chapter bridges the gap between the mathematical dispatch optimum (the LP from Chapter 9) and the physical NEM market mechanism. A battery does not choose its dispatch quantity directly -- it submits price-quantity offer bands and is dispatched by NEMDE based on where those bands sit in the aggregate supply stack. The 10-band offer structure is both a constraint (coarser than the continuous LP) and an advantage (the dispatch automatically adapts to the realised price without needing to rebid).

The chapter introduced two simulation modes. **Price-taker mode** -- which the course used through Chapter 10 -- assumes the battery is too small to affect the clearing price. This is a good approximation for batteries below about 5% of regional minimum demand. **Price-maker mode**

uses nempy to re-clear the market with the battery's offers inserted into the historical bid stack, revealing the market impact: the battery's own dispatch shifts the clearing price against it. For the 100 MW Mannum BESS in SA1, market impact costs roughly 5-10% of price-taker revenue, narrowing the arbitrage spread through self-cannibalization on both the charge and discharge sides.

Offer construction translates the probabilistic forecast (Chapter 8) and the MPC dispatch target (Chapter 9) into 10-band offers by mapping forecast quantiles to band prices. The bands encode a conditional strategy: the battery's dispatch depends on the realised clearing price, automatically adapting to forecast errors without rebidding. The offer aggressiveness tradeoff -- between expected revenue and dispatch risk -- is controlled by how the capacity is spread across the price range.

Advanced strategy search (RL or genetic optimisation) can squeeze an additional 3-5% revenue from the price-maker emulator by learning to shade offers strategically. But the convex LP baseline remains the production core: interpretable, constraint-respecting, and fast. Strategy search is a refinement for large batteries in small regions, not a replacement for the optimisation framework built in Chapters 5 and 9. The capture ratio target remains 0.65-0.75 in price-taker mode (0.60-0.72 in price-maker mode), benchmarked against AEMO pre-dispatch forecasts. Below the 0.50 bar, the dispatch strategy needs fundamental improvement before offer refinement adds value.

13. Realised Revenue: From Idealised to Actual

The Gap Between Model and Reality

The dispatch LP from Chapter 5, and its probabilistic extensions in Chapter 9, optimise an idealised battery. The LP assumes that one megawatt-hour discharged earns exactly the regional reference price, that the battery never degrades, that it can ramp instantaneously, and that there are no parasitic loads. None of these are true.

This chapter quantifies the gap between idealised LP revenue and the revenue that actually lands in the bank account. Each real-world effect acts as a **haircut** — a percentage reduction from the idealised number. The haircuts are multiplicative, so they compound: three independent 5% haircuts reduce revenue by about 14%, not 15%. Understanding and minimising these haircuts is what separates a competent trading desk from a naive one.

Why this matters commercially: In Chapter 9, we established a capture ratio of approximately 0.70 for a well-tuned GBT model with chance-constrained dispatch. That number was computed against perfect-foresight revenue using the idealised LP. The *realised* capture ratio — accounting for loss factors, degradation, operational constraints, and auxiliary load — is lower. The question is: how much lower, and which haircuts are worth engineering around?

The approach is deliberately simple. Each haircut is introduced as a parameter modification to the LP from Chapter 5. We apply them cumulatively, building a **revenue waterfall** from idealised to realised. At the end, we validate the LP's simplified dispatch against nempy — the open-source reference implementation of the NEM dispatch engine — to check that our approximations are reasonable.

Marginal Loss Factors

What They Are

Electricity transmitted over long distances loses energy to resistive heating in the wires. In the NEM, these losses are not spread equally across all participants — they are allocated through **marginal loss factors (MLFs)**, which scale the regional reference price at each connection point.

Marginal loss factor (MLF): A multiplier applied to the regional reference price (RRP) to determine the price a generator or load actually receives or pays at its connection point. An MLF of 0.98 means the unit receives 98% of the RRP for energy exported and pays 98% of the RRP for energy imported. MLFs are set annually by AEMO and published before the start of each financial year. They reflect the marginal change in total system losses caused by an incremental increase in generation or load at that connection point.

The **connection-point price** — what the unit is actually paid — is:

$$\text{connection_point_price}_t = \text{MLF} \times \text{RRP}_t$$

For a battery that both imports (charges) and exports (discharges) at the same connection point, the MLF applies symmetrically: charging costs $\text{MLF} \times \text{RRP}$ and discharging earns $\text{MLF} \times \text{RRP}$. This means the MLF cancels out of the **spread** calculation if and only if the MLF is constant. Since MLFs are fixed for a financial year, the cancellation is exact within that year. The haircut comes from the fact that revenue is measured in absolute dollars, not spreads — a unit with $\text{MLF} = 0.95$ earns 5% less revenue per MWh discharged than a unit at the regional reference node.

Connection-point price: The effective price at a specific connection point on the transmission network, computed as the product of the regional reference price and the marginal loss factor at that point. It is the price the participant actually transacts at — generators are paid the connection-point price, and loads pay the connection-point price. Two generators in the same region but at different connection points will receive different prices for the same MWh of output if their MLFs differ.

How AEMO Computes MLFs

AEMO sets MLFs annually using a power system model that simulates load flow across the entire NEM transmission network. The procedure, in simplified terms:

1. **Model the network** at a representative set of operating conditions (demand levels, generation patterns, interconnector flows).
2. **For each connection point**, compute the marginal change in total system losses caused by a 1 MW increase in generation (or load) at that point.
3. **Average** these marginal loss sensitivities across the operating conditions, weighting by frequency of occurrence.
4. **Publish** the resulting MLF for each connection point, effective from 1 July for one financial year.

The key subtlety: MLFs reflect the **marginal** impact on system losses, not the average. A generator located at the end of a long, thin transmission line imposes higher marginal losses than one located near a major load centre, even if the average losses on that line are modest.

The Mannum Example

The Mannum BESS is connected to the SA1 region at its Mannum connection point, located on the eastern edge of South Australia near the SA–VIC interconnector. This is a relatively favourable location for loss factors:

- **Proximity to the interconnector** means that power exported from Mannum to the rest of the NEM travels a shorter distance through the SA transmission network than power from remote wind farms in the mid-north or far west.
- **Battery symmetry** helps: the MLF applies equally to charge and discharge, so the haircut on the spread is minimal.

Typical MLFs for SA1 connection points (illustrative, as AEMO publishes actuals annually):

Connection point type	Typical MLF range	Revenue impact
Adelaide metro (load centre)	0.99–1.01	Negligible
Mannum (near interconnector)	0.97–0.99	1–3% haircut
Mid-north wind farms	0.90–0.95	5–10% haircut
Far-west solar/wind	0.85–0.92	8–15% haircut

Mannum's MLF haircut: Assume Mannum's MLF is 0.98 for the current financial year. The idealised LP assumes $MLF = 1.00$. Revenue from the LP was \$8.4M (Chapter 9's worked example: \$12M perfect foresight $\times$ 0.70 capture ratio). Adjusting for the MLF: $\$8.4M \times 0.98 = \$8.23M$. The MLF haircut is approximately **\$170K/year, or about 2%**. This is modest — Mannum's location near the interconnector is one of its advantages.

Why Remote Renewables Have Falling MLFs

Remote wind and solar farms face a structural MLF problem. As more generation is added at a remote connection point:

1. **More power flows through the same transmission lines**, increasing resistive losses (losses scale with the square of current).
2. **The marginal impact of additional generation increases**, because each new MW adds to an already-congested corridor.
3. **The MLF falls year on year**, reducing revenue for all generators at that point.

This creates a negative feedback loop: the first wind farm at a remote site gets a decent MLF (say 0.95), attracting more investment. The second and third farms drive the MLF down to 0.90 or below, reducing revenue for everyone — including the original farm. This is sometimes called the **MLF death spiral** and is a significant risk for renewable project finance.

Siting lesson: For a battery, the MLF story is simpler than for a generator. Because the MLF applies symmetrically to charge and discharge, the absolute level matters less than the year-on-year stability. A battery at Mannum with $MLF = 0.98$ is in a good position: the MLF is close to unity and unlikely to decline significantly (battery operation does not structurally increase losses the way a new wind farm does). Chapter 5's LP should incorporate the MLF by scaling the price vector: $price_effective_t = MLF \times price_t$.

Modifying the LP for MLFs

The modification to the Chapter 5 LP is trivial. Replace the price vector with the connection-point price:

Maximise $\sum_{t=1}^T MLF \times price_t \times (discharge_t - charge_t) \times \Delta t$

Since MLF is a constant (within a financial year), it factors out of the optimisation and does not change the optimal dispatch schedule. The revenue is simply scaled by MLF. This is why the MLF is a pure haircut: it does not change *what* the battery does, only *how much* it earns.

Network Charges and the True Cost of Connection

The MLF adjusts the energy price, but it is not the only cost of being connected to the grid. Network charges — the fees paid to the transmission and distribution network operators for using their infrastructure — represent an additional fixed or semi-fixed cost.

Transmission Use of System (TUOS) Charges

Generators in the NEM pay **TUOS charges** to the Transmission Network Service Provider (TNSP). These charges cover the cost of building and maintaining the transmission network. For a battery:

- **Locational component:** based on the connection point and the capacity of the battery. Reflects the cost of the transmission infrastructure used.
- **Common service component:** a shared cost across all connected generators.

For a 100 MW battery like Mannum, TUOS charges are typically in the range of \$0.5M–\$1.5M per year, depending on the TNSP's pricing methodology and the specifics of the connection agreement. These are largely fixed costs — they do not vary with dispatch — so they reduce net revenue without affecting optimal dispatch decisions.

Connection Agreement Costs

Beyond TUOS, the connection agreement itself may impose:

- **Generator performance standards:** requirements for power quality, fault ride-through, and voltage control that may require additional equipment.
- **Metering costs:** NEM-compliant revenue metering at the connection point.

- **Network support obligations:** in some cases, the connection agreement requires the battery to provide specific services during network stress events.

These costs are project-specific and typically negotiated during the connection process. For modelling purposes, they are treated as fixed annual costs subtracted from gross revenue.

For the revenue waterfall: Network charges are a fixed-cost haircut, not a variable one. They reduce annual net revenue by a roughly constant amount regardless of dispatch strategy. For Mannum, assume approximately \$1M/year in combined TUOS and connection costs. This haircut does not interact with forecast quality or dispatch optimisation — it is a constant deduction.

Degradation as a Variable Cost

From Hard Constraint to Economic Cost

In Chapter 5, we modelled degradation as a hard cycle-count constraint: the LP was not allowed to exceed a fixed number of full equivalent cycles per day. Chapter 9 noted that real degradation is more nuanced. In this section, we model degradation as a **variable cost** — each MWh of throughput has an implicit cost equal to the value of the battery capacity consumed.

Throughput: The total energy processed by the battery, measured in MWh. One full cycle of a 200 MWh battery produces 200 MWh of throughput (discharge only — charging is implicitly counted through the round-trip efficiency). Total lifetime throughput is a useful proxy for total degradation, though the relationship between throughput and degradation depends on how the cycles are performed.

State of health (SoH): The current usable capacity of the battery as a fraction of its original nameplate capacity. A battery that started at 200 MWh and now has 180 MWh of usable capacity has a state of health of 90%. SoH declines over time due to cycling degradation and calendar ageing. Most battery warranties guarantee a minimum SoH (e.g., 70%) after a specified number of years or cycles.

Degradation Mechanisms for LFP

The Mannum BESS uses **lithium iron phosphate (LFP)** cells. LFP has different degradation characteristics from the more common nickel-manganese-cobalt (NMC) chemistry used in many consumer electronics:

Factor	LFP	NMC
Cycle life	4,000–8,000 cycles	1,500–3,000 cycles
DoD sensitivity	Low	High
Temperature	Moderate (>35°C risk)	High
C-rate	Moderate (>1C risk)	Similar
Calendar ageing	Low	Moderate
Cost	Lower, longer life	Higher, shorter life

Depth of discharge (DoD): The fraction of the battery's capacity used in a single cycle. A full cycle (0% to 100% to 0%) has a DoD of 100%. A half cycle (50% to 100% to 50%) has a DoD of 50%. For NMC batteries, degradation per cycle increases sharply with DoD. For LFP, the relationship is flatter — deep cycles are only slightly more damaging per unit of throughput than shallow ones.

Calendar ageing: The degradation of battery capacity that occurs over time regardless of cycling, driven by chemical side reactions within the cells. Calendar ageing depends primarily on temperature and state of charge — batteries age faster when hot and when stored at very high or very low SoC. For LFP, calendar ageing is relatively mild compared to NMC, but it is not zero: a battery that sits idle for 20 years will still lose capacity.

For Mannum's LFP chemistry, the key degradation drivers in order of importance are:

1. **Throughput** (total energy cycled): The dominant factor. Each MWh of throughput consumes a small, roughly constant fraction of the battery's lifetime capacity.
2. **Temperature:** South Australia's climate means Mannum experiences hot summer days (35–45°C ambient), which accelerate degradation. Thermal management systems mitigate this but add to auxiliary load (see below).
3. **C-rate:** At 100 MW / 200 MWh, Mannum operates at 0.5C at full power — well within the comfortable range for LFP. Degradation from C-rate is minimal at this ratio.
4. **Calendar ageing:** LFP's strong suit. Calendar ageing contributes a modest background degradation rate of roughly 1–2% per year regardless of cycling.

Computing the Degradation Cost per MWh

The degradation cost converts battery wear into a dollar value per MWh of throughput:

$\text{degradation_cost} = (\text{replacement_cost} \times \text{capacity_fraction_per_MWh}) / (1 - \text{residual_value_fraction})$

For Mannum's LFP system:

- **Replacement cost:** approximately \$250/kWh for cell replacement (not the full system — balance of plant, inverters, and switchgear are reused). For 200 MWh: \$50M.
- **Cycle life to 80% SoH:** approximately 6,000 full equivalent cycles.
- **Capacity consumed per cycle:** $(100\% - 80\%) / 6,000 = 0.0033\%$ per cycle.
- **Throughput per cycle:** 200 MWh.
- **Capacity consumed per MWh of throughput:** $0.0033\% / 200 = 0.0000167\%$ per MWh.
- **Dollar cost per MWh:** $\$50M \times 0.0000167\% =$ approximately **\$8.3/MWh**.

Worked example — degradation cost in context: The average arbitrage spread in SA1 (Chapter 2) is approximately \$80–\$120/MWh for the peak-to-trough differential. After round-trip efficiency losses (~10%), the net spread is \$72–\$108/MWh. The degradation cost of \$8.3/MWh represents about 8–12% of the net spread. This is meaningful but not dominant — most cycles are still profitable after accounting for degradation. However, shallow cycles that capture small spreads (say \$20–\$30/MWh) may not be worth the wear.

Incorporating Degradation into the LP

To include degradation as a variable cost, modify the LP objective:

$$\text{Maximise } \sum_{t=1}^T [\text{price}_t \times (\text{discharge}_t - \text{charge}_t) - c_{\text{deg}} \times \text{discharge}_t] \times \Delta t$$

where c_{deg} is the degradation cost per MWh of discharge (approximately \$8.3/MWh for Mannum). This is equivalent to reducing the effective discharge price by c_{deg} . The LP will now avoid shallow cycles where the price spread is insufficient to cover both efficiency losses and degradation costs.

The **breakeven spread** with degradation is:

$$\text{breakeven_spread} = (\text{charge_price} / \eta) + c_{\text{deg}}$$

where η is round-trip efficiency. For a charge price of \$30/MWh, $\eta = 0.90$, and $c_{\text{deg}} = \$8.3/\text{MWh}$:

$$\text{breakeven_spread} = (\$30 / 0.90) + \$8.3 = \$33.3 + \$8.3 = \mathbf{\$41.6/\text{MWh}}$$

Without degradation cost, the breakeven was \$33.3/MWh. Degradation raises the bar by about 25%.

LFP's advantage: Mannum's LFP chemistry has roughly half the degradation cost per MWh of an equivalent NMC system (which might see \$15–\$20/MWh degradation cost due to shorter cycle life). This lower degradation cost means Mannum can profitably capture smaller spreads and run more cycles per day than an NMC battery in the same location. Over a 20-year asset life, this difference compounds significantly.

Augmentation

As the battery degrades below its nameplate capacity, the operator faces a choice: accept the reduced revenue from lower capacity, or **augment** the battery by adding new cells.

Augmentation: The process of adding new battery cells to an existing system to restore or maintain its nameplate capacity. Augmentation is typically planned in advance — the original system design may include space and electrical capacity for additional cell racks. The decision of when to augment depends on the tradeoff between the cost of new cells and the revenue lost from reduced capacity. LFP's longer cycle life means augmentation can be deferred further than with NMC, improving the asset's net present value.

A typical augmentation schedule for a 20-year LFP asset:

Year	Estimated SoH	Action	Incremental cost
0	100%	Initial commissioning	—
8–10	~85%	First augmentation (~15% capacity added)	~\$7.5M
14–16	~85% (post-augmentation SoH)	Second augmentation (~15%)	~\$5M (cell costs declining)
20	~80%	End of warranted life	—

The declining augmentation cost reflects expected reductions in LFP cell prices over time. In the LP framework, augmentation is modelled as a periodic step-change in E_{max} rather than a continuous process.

Operational Constraints

Ramp Limits

The idealised LP assumes the battery can go from full charge to full discharge instantaneously. Real batteries have **ramp limits** — the maximum rate at which power output can change between consecutive dispatch intervals.

Ramp limit: The maximum rate of change of power output, typically expressed in MW per minute or MW per 5-minute dispatch interval. A ramp limit of 50 MW/min means the battery cannot increase or decrease its output by more than 50 MW in one minute. Grid-scale batteries generally have fast ramp rates compared to thermal generators, but power electronics, transformer, and protection system limits impose practical constraints.

For the Mannum BESS (100 MW), the practical ramp rate is typically very fast — lithium-ion batteries can ramp from zero to full power in under a second. The binding ramp constraint is usually not the battery cells themselves but:

- **Power electronics:** Inverter ramp rates may be limited to prevent power quality issues.
- **AEMO dispatch conformance:** AEMO's dispatch engine assumes generators ramp linearly between dispatch targets over each 5-minute interval. The battery must conform to this ramp profile.
- **Connection agreement:** The network connection agreement may specify maximum ramp rates to prevent voltage disturbances.

For a 100 MW battery with a 5-minute dispatch interval, the ramp constraint is rarely binding — the battery can typically go from –100 MW (full charge) to +100 MW (full discharge) within a single interval. However, when modelling at the trading-interval level (30 minutes), ramp limits are essentially irrelevant.

To include ramp limits in the LP:

$|\text{power}_t - \text{power}_{t-1}| \leq \text{ramp_max} \times \Delta t$ where $\text{power}_t = \text{discharge}_t - \text{charge}_t$

Minimum Up/Down Times

Some battery systems have **minimum up/down time** constraints — the battery must remain in a given state (charging, discharging, or idle) for at least a minimum number of consecutive intervals before switching. These constraints arise from:

- **Inverter cycling limits:** Frequent mode switching between charge and discharge can stress power electronics.
- **Thermal management:** Rapid cycling generates heat that the cooling system may not handle.

For modern LFP batteries like Mannum, minimum up/down times are typically short (one or two 5-minute intervals) and do not materially constrain the 30-minute trading-interval LP. In the LP, they are modelled as:

If the battery switches from charge to discharge (or vice versa) at time t , it must remain in the new state for at least $\tau_{\min}$ intervals.

This constraint requires binary variables, turning the LP into a mixed-integer program (MIP). For typical NEM batteries with short minimum times, the MIP solves quickly and the revenue impact is small (less than 1%).

Auxiliary Load

Auxiliary load: The electricity consumed by the battery system's own support equipment — cooling systems (HVAC, liquid cooling), battery management systems (BMS), fire suppression monitoring, site lighting, control systems, and communications. Auxiliary load is a parasitic draw that reduces net energy throughput. It is typically 1–3% of nameplate power capacity, but increases during hot weather when cooling demand rises.

For the Mannum BESS in South Australia:

- **Base auxiliary load:** approximately 0.5–1 MW (0.5–1% of 100 MW capacity) for BMS, monitoring, and standby systems.
- **Cooling load during operation:** additional 1–2 MW during sustained charge/discharge, more during summer heat.
- **Peak auxiliary load on hot days:** can reach 3–4 MW when thermal management works hardest to keep cells below degradation-accelerating temperatures.

In the LP, auxiliary load is modelled as a reduction in net power:

$\text{net_discharge}_t = \text{discharge}_t - P_{\text{aux}}$ $\text{net_charge}_t = \text{charge}_t + P_{\text{aux}}$

The auxiliary load makes both charging and discharging less efficient: the battery must charge more than the LP assumes (to power auxiliary systems while charging) and delivers less than the LP assumes when discharging. The annual cost of auxiliary load for Mannum is approximately \$0.3–\$0.5M (purchased at the connection-point price, which varies with time of day).

Capacity Fade Over the Asset Life

As the battery ages, its usable capacity declines (state of health drops). This means the $E_{\max}$ parameter in the LP is not constant over the asset life — it decreases by roughly 2–3% per year for LFP under typical cycling patterns (before augmentation).

The revenue impact of capacity fade is straightforward: a battery with 10% less capacity earns roughly 10% less revenue (the relationship is approximately linear for small capacity reductions, because the dispatch pattern remains similar — the battery just has less energy to deploy).

In a multi-year financial model, capacity fade is modelled as a declining $E_{\max}$ trajectory with step increases at augmentation events.

The Revenue Waterfall

Building the Waterfall Step by Step

The revenue waterfall starts with idealised LP revenue and applies each haircut in sequence. The order of application does not matter for multiplicative haircuts, but presenting them from largest to smallest makes the chart easier to read.

Worked example — Mannum BESS revenue waterfall: Starting point: Chapter 9's worked example. 100 MW / 200 MWh LFP battery in SA1, GBT model with QRA combination and chance-constrained dispatch. **Step 0 — Perfect foresight revenue:** \$12.0M/year (from Chapter 9). **Step 1 — Forecast imperfection (capture ratio = 0.70):** $\$12.0\text{M} \times 0.70 = \8.40M . Haircut: $\$3.60\text{M}$ (30%). **Step 2 — MLF (0.98):** $\$8.40\text{M} \times 0.98 = \8.23M . Haircut: $\$0.17\text{M}$ (2%). **Step 3 — Degradation cost (\$8.3/MWh, ~1.5 cycles/day):** Throughput per year: $200\text{ MWh} \times 1.5\text{ cycles} \times 365\text{ days} = 109,500\text{ MWh}$. Degradation cost: $109,500 \times \$8.3 = \0.91M . Net: $\$8.23\text{M} - \$0.91\text{M} = \$7.32\text{M}$. Haircut: $\$0.91\text{M}$ (11%). **Step 4 — Auxiliary load (~1.5 MW average, priced at ~\$60/MWh average):** Annual cost: $1.5\text{ MW} \times 8,760\text{ h} \times \$60/\text{MWh} = \$0.79\text{M}$. Net: $\$7.32\text{M} - \$0.79\text{M} = \$6.53\text{M}$. Haircut: $\$0.79\text{M}$ (9%). **Step 5 — Network charges (TUOS + connection):** $\$1.00\text{M}/\text{year}$. Net: $\$6.53\text{M} - \$1.00\text{M} = \$5.53\text{M}$. Haircut: $\$1.00\text{M}$ (12%). **Step 6 — Ramp limits and min up/down:** Negligible for a 100 MW LFP battery at 30-minute resolution. Haircut: $\sim\$0.05\text{M}$ (<1%). **Realised revenue: \$5.48M/year.** Total haircuts from idealised LP revenue: $\$8.40\text{M} \rightarrow \$5.48\text{M} = 35\%$ reduction. Realised capture ratio (against perfect foresight): $\$5.48\text{M} / \$12.0\text{M} = 0.46$.

Figure 13.1 — Revenue waterfall from idealised LP revenue to realised revenue for the Mannum BESS. Each bar shows the magnitude and percentage of a haircut. Forecast imperfection (the gap from perfect foresight) is the largest single factor, followed by network charges, degradation, and auxiliary load. The MLF haircut is modest due to Mannum's favourable location.

The 0.50 benchmark, revisited: In earlier chapters, we used a capture ratio of ~0.50 as the bar for a competent dispatch system (computed against idealised LP revenue). After applying real-world haircuts, the realised capture ratio against perfect foresight drops to approximately 0.46. This is the number that matters for financial modelling and investment decisions. The gap between the idealised 0.70 and the realised 0.46 represents approximately \$2.9M/year in friction costs — a significant portion of the battery's gross revenue.

Sensitivity of the Waterfall

The waterfall is not fixed — it depends on market conditions and operating parameters. Key sensitivities:

Parameter	If it changes...	Revenue impact
MLF drops from 0.98 to 0.93	5 percentage points of additional haircut	−\$0.42M/year
Degradation doubles (NMC instead of LFP)	Degradation cost rises to ~\$16.6/MWh	−\$0.91M/year
Average price level doubles	All haircuts remain proportional, but auxiliary load cost doubles	+\$5M (approximate)
Battery ages to 85% SoH	Capacity fade reduces revenue by ~15%	−\$0.8M/year

Using nempy to Validate the LP

Why Validate?

The dispatch LP from Chapter 5 is a simplification. It assumes a perfectly competitive market with uniform pricing, no network constraints, and linear cost curves. The real NEM dispatch engine (NEMDE) is considerably more complex. **nempy** — the open-source Python implementation from UNSW-CEEM — replicates NEMDE's behaviour and can be used to check whether the LP's simplifications introduce systematic biases.

nempy: An open-source Python package developed by UNSW's Collaboration on Energy and Environmental Markets (CEEM) that implements the NEM dispatch procedure. nempy models the full dispatch engine, including regional pricing, interconnector flows, constraint equations, loss factors, and FCAS markets. It can be used to simulate dispatch outcomes for historical or hypothetical conditions, providing a validation layer for simplified models like our dispatch LP.

What to Compare

The validation exercise runs the same price trajectory through both the simplified LP and nempy, comparing:

1. **Dispatch schedule:** Does the LP's optimal charge/discharge schedule match what nempy would produce if the battery were a price-taking participant?
2. **Revenue:** Does the LP's computed revenue match nempy's settlement calculation (which includes MLFs and the full pricing model)?
3. **Price formation effects:** For a large battery (100 MW is material in SA1), does the battery's own dispatch affect the dispatch price? The LP treats the battery as a price-taker; nempy can reveal whether this assumption holds.

Running the Validation

The workflow:

1. **Take the LP's optimal dispatch schedule** for a representative week (e.g., a week with both calm and volatile prices).
2. **Feed the schedule as fixed bids into nempy**, along with all other generators' actual bids from the AEMO bidding data.
3. **Compare nempy's computed dispatch prices** with the actual historical prices used by the LP.
4. **Compare nempy's settlement revenue** with the LP's computed revenue.

Typical findings from nempy validation: Agreement: For most intervals (90%+), the LP's dispatch and nempy agree closely. The battery charges during the same low-price periods and discharges during the same high-price periods. Revenue matches within 1–2%. **Discrepancies:** The LP and nempy diverge in two specific situations: 1. **Network constraints:** When a transmission constraint binds (e.g., the Heywood interconnector between SA and VIC reaches its limit), nempy produces a different dispatch outcome because the constraint limits the battery's ability to export. The LP, which ignores network constraints, assumes the battery can always export at its full capacity. This can cause the LP to overestimate revenue by 2–5% during constrained periods. 2. **Price-maker effects:** When the battery's 100 MW of discharge materially shifts the supply stack (e.g., during low-demand overnight periods when SA1 demand is 800–1,200 MW), the act of discharging pushes prices down, reducing the revenue earned. The LP assumes price-taking behaviour and misses this effect. For a 100 MW battery in SA1, this "self-cannibalisation" effect is typically 1–3% of annual revenue.

The FCAS Trapezium

nempy also models the **FCAS trapezium** — the relationship between a generator's energy dispatch and its available FCAS capacity. A battery that is dispatched at full power for energy has no headroom to provide FCAS (raise services require the ability to increase output; lower services require the ability to decrease output). The trapezium constraint means that energy dispatch and FCAS provision are **jointly optimised** in the real dispatch engine.

The simplified LP from Chapter 5 optimises energy arbitrage alone. nempy reveals the revenue left on the table by ignoring FCAS co-optimisation — and conversely, shows how much FCAS provision might constrain the energy-optimal dispatch. Full co-optimisation of energy and FCAS is beyond the scope of this guide, but the nempy validation quantifies the gap and motivates the extension.

The LP is good enough for strategic analysis. nempy validation typically shows that the simplified LP captures 95–98% of the realised energy arbitrage revenue. The 2–5% discrepancy comes from network constraints and price-maker effects, which are second-order for a well-sited battery like Mannum. For investment-grade financial modelling, the LP is a reasonable first pass; for real-time trading decisions, a dispatch engine that respects network constraints (or a nempy-based simulation) is preferred.

Exercises

Exercise 1: Degradation Breakeven Spread

Question: For the Mannum BESS (100 MW / 200 MWh, LFP, $\eta = 0.90$, $c_{\text{deg}} = \$8.3/\text{MWh}$):

1. Find the minimum price spread at which a full DoD cycle covers degradation cost.
2. Compute the same breakeven for an NMC battery ($c_{\text{deg}} = \$16.6/\text{MWh}$).
3. Estimate how many profitable cycling days per year the higher NMC cost excludes.

Hint: The breakeven spread depends on charge price — assume \$30/MWh overnight.

Worked solution

Revenue from one full cycle ($\text{spread} = \text{discharge_price} - \text{charge_price}$):

$$\text{revenue} = E_{\text{max}} \times (\text{discharge_price} \times \eta - \text{charge_price})$$

Profitable after degradation requires:

$$\text{discharge_price} \times \eta - \text{charge_price} - c_{\text{deg}} \geq 0$$

Substituting $\text{discharge_price} = \text{charge_price} + \text{spread}$ and rearranging:

$$\text{spread} \geq \text{charge_price} \times (1 - \eta) / \eta + c_{\text{deg}} / \eta$$

With $\text{charge_price} = \$30/\text{MWh}$:

$$\text{LFP: spread} \geq 30 \times 0.10/0.90 + 8.3/0.90 = 3.3 + 9.2 = \mathbf{\$12.6/\text{MWh}} \quad \text{NMC: spread} \geq 30 \times 0.10/0.90 + 16.6/0.90 = 3.3 + 18.4 = \mathbf{\$21.8/\text{MWh}}$$

NMC requires \$9.2/MWh more spread. From SA1 historical data, daily max spread exceeds \$12.6 on ~340 days/year but exceeds \$21.8 on only ~310 days/year.

NMC loses ~30 profitable cycling days per year (~\$0.5–\$1.0M foregone revenue). Not only does each cycle cost more, but fewer cycles are worth doing.

Exercise 2: Loss Factor and Siting

Question: A remote SA1 site has $\text{MLF} = 0.91$ but saves \$2M in capital vs. Mannum ($\text{MLF} = 0.98$). Both are 100 MW / 200 MWh LFP with idealised LP revenue of \$8.4M at $\text{MLF} = 1.00$. Compute:

1. Annual revenue difference due to the MLF gap.
2. Years until cumulative revenue loss exceeds the \$2M capital saving.
3. Breakeven MLF for the remote site over a 15-year life (no discounting).

Hint: Revenue scales linearly with MLF.

Worked solution

(a) Annual revenue difference:

Mannum (0.98): $\$8.40\text{M} \times 0.98 = \8.23M Remote (0.91): $\$8.40\text{M} \times 0.91 = \7.64M Difference: **\$0.59M/year**

(b) Payback of capital saving:

$\$2.0\text{M} / \$0.59\text{M} = 3.4 \text{ years}$

Over 15 years the Mannum advantage totals $\$0.59\text{M} \times 15 = \8.85M — far exceeding the \$2M saving.

(c) Breakeven MLF over 15 years:

$\$8.40\text{M} \times \text{MLF_remote} \times 15 + \$2.0\text{M} = \$8.40\text{M} \times 0.98 \times 15$ $\text{MLF_remote} = 0.98 - \$2.0\text{M} / (\$8.40\text{M} \times 15)$ $\text{MLF_remote} = 0.98 - 0.016 = \mathbf{0.964}$

At 0.91 the remote site falls well short. A 7-point MLF disadvantage costs ~\$9M over 15 years — dwarfing any plausible land savings.

The siting message: Battery siting is first and foremost an MLF and grid-connection story. A well-connected site near the load centre or interconnector (like Mannum) will almost always outperform a remote site with a lower MLF, even if land and civil costs are significantly cheaper. The exception is a site with a very high MLF *and* local network support payments that provide additional revenue.

Glossary

Term	Definition
Marginal loss factor (MLF)	Multiplier on RRP at a connection point, reflecting transmission losses
Connection-point price	The effective price at a connection point: $\text{MLF} \times \text{regional reference price}$
Throughput	Total energy processed by the battery, measured in MWh of discharge
Depth of discharge (DoD)	Fraction of battery capacity used in a single cycle
Calendar ageing	Capacity loss over time regardless of cycling; driven by temperature and SoC
Augmentation	Adding new cells to restore or maintain nameplate capacity as the original cells degrade
State of health (SoH)	Current usable capacity as a fraction of original nameplate capacity
Auxiliary load	Parasitic consumption by cooling, BMS, and monitoring systems
Ramp limit	Maximum rate of change of power output between consecutive intervals
Revenue waterfall	Step-by-step decomposition from idealised revenue to realised revenue, showing each haircut
Degradation cost	The implicit cost per MWh of throughput, representing the value of battery capacity consumed
Breakeven spread	Minimum price difference between discharge and charge for a profitable cycle
nempy	Open-source Python implementation of the NEM dispatch procedure (UNSW-CEEM)
FCAS trapezium	Joint constraint between energy dispatch and FCAS capacity in the NEM dispatch engine
Self-cannibalisation	Revenue reduction caused by the battery's own dispatch affecting the market price

Summary

Idealised LP revenue overstates what a battery actually earns. The gap is filled by real-world haircuts: marginal loss factors (2% for well-sited Mannum, up to 15% for remote sites), degradation costs (\$8.3/MWh for LFP, roughly double for NMC), auxiliary load (1–3% of capacity as parasitic draw), network charges (~\$1M/year fixed), and minor operational constraints (ramp limits, minimum up/down times). Applied cumulatively to the Mannum BESS, these haircuts reduce the idealised capture ratio of 0.70 to a realised capture ratio of approximately 0.46 — a 35% reduction in revenue. The revenue waterfall makes each haircut visible and quantifiable. Mannum's LFP chemistry and favourable grid location (near the SA–VIC interconnector, MLF ~0.98) minimise two of the largest potential haircuts: degradation and loss factors. nempy validation confirms that the simplified LP captures 95–98% of realised energy arbitrage revenue; the residual discrepancy comes from network constraints and price-maker effects that the LP ignores. For investment decisions, the lesson is clear: siting (MLF), chemistry (degradation cost), and thermal management (auxiliary load and degradation acceleration) matter as much as forecast quality. Improving the capture ratio from 0.45 to 0.50 is worth roughly \$0.6M/year — comparable to the entire degradation cost budget. Every parameter in the waterfall is a lever.

14. Failure Analysis and Explainability

Decision Regret, Not Forecast Error

Most forecast evaluation frameworks focus on accuracy: MAE, RMSE, CRPS, calibration. These metrics measure how close the forecast was to reality. But a battery operator does not care about forecast accuracy in the abstract — they care about whether the forecast led to the right **decision**. A forecast error that did not flip a charge or discharge decision cost the operator exactly zero dollars. A small forecast error that flipped a decision during a price spike may have cost hundreds of thousands of dollars.

This chapter reframes forecast evaluation around **decision regret**: the gap between what the dispatch actually earned and what it would have earned with perfect foresight. This is the metric that matters commercially, and it diverges from traditional forecast error in ways that are both surprising and consequential.

Decision regret: The difference in revenue between a dispatch schedule optimised on a forecast and a dispatch schedule optimised on the actual prices (perfect foresight). Formally: $\text{regret}_t = \text{revenue_perfect}_t - \text{revenue_forecast}_t$. A positive regret means the forecast-driven dispatch left money on the table. A zero regret means the forecast error, however large, did not affect the dispatch decision. Regret is measured in dollars, not error units.

The distinction between forecast error and decision regret is subtle but essential. Consider three scenarios for a half-hour period where the battery must decide whether to discharge:

Three scenarios illustrating the error-regret divergence: **Scenario A — Large error, zero regret.** The forecast predicts \$80/MWh; the actual price is \$120/MWh. The forecast error is \$40/MWh. But both \$80 and \$120 are well above the charge cost, so the battery discharges in both the forecast-driven and perfect-foresight cases. The dispatch decision is identical. Regret = \$0. **Scenario B — Small error, large regret.** The forecast predicts \$55/MWh; the actual price is \$65/MWh. The forecast error is only \$10/MWh. But \$55 is below the threshold where the LP decides to hold charge (because a later period looks better), while \$65 would have triggered a discharge. The battery holds when it should have discharged. Regret = $\$65 \cdot 100 \text{ MW} \cdot 0.5 \text{ h} = \$3,250$ for this single half-hour. **Scenario C — Small error, catastrophic regret.** The forecast predicts \$200/MWh; the actual price is \$5,000/MWh. The forecast error is \$4,800/MWh. But this is not a symmetric error — the \$200 forecast caused the battery to discharge in an earlier, less profitable period, missing the spike entirely. The perfect-foresight battery would have held its charge for this period. Regret = $(\$5,000 - \$200) \cdot 100 \text{ MW} \cdot 0.5 \text{ h} = \$240,000$ in a single half-hour.

These scenarios reveal a fundamental asymmetry: forecast errors in the **middle of the price distribution** rarely affect dispatch decisions, because the battery is either clearly charging or clearly discharging regardless of moderate price variations. Forecast errors at **decision boundaries** — near the threshold where the LP switches between charge, hold, and discharge — are where regret concentrates.

The core insight: Decision regret is not proportional to forecast error. It depends on the interaction between the error and the dispatch optimisation. Regret concentrates where the error crosses a decision boundary — and most of the dollars at risk sit in the tails of the price distribution. Evaluating a forecast by its MAE is like evaluating a surgeon by the number of cuts: the metric is related to the outcome but misses what actually matters.

Computing Decision Regret

The computation is straightforward once you have the Chapter 9 backtest infrastructure:

1. **Run the dispatch LP with the forecast** to obtain the forecast-driven schedule and revenue for each day.
2. **Run the dispatch LP with the actual prices** (perfect foresight) to obtain the optimal schedule and revenue for each day.
3. **Compute regret** as the difference: $\text{regret}_t = \text{revenue_perfect}_t - \text{revenue_forecast}_t$.

The dispatch LP from Chapter 5 already handles both cases — the only difference is the price input. The perfect-foresight run uses actual realised prices; the forecast-driven run uses the quantile forecasts from Chapter 8.

The **capture ratio** from Chapter 9 is the complement of normalised regret:

$$\text{capture_ratio} = \text{revenue_forecast} / \text{revenue_perfect} = 1 - (\text{regret} / \text{revenue_perfect})$$

A capture ratio of 85% means the forecast-driven dispatch captured 85% of perfect-foresight revenue, implying 15% normalised regret. The relationship is simple, but expressing results in both forms is useful: capture ratio describes overall performance; regret highlights specific days where money was lost.

Regret by Regime

The most revealing analysis is to split regret by **price regime**. Define two regimes using the definitions from Chapter 3:

- **Calm intervals:** The half-hour price remains between the 5th and 95th percentiles of the trailing 90-day distribution.
- **Spike intervals:** The half-hour price exceeds the 95th percentile.

Now compute total regret within each regime:

Regime	Share of intervals	Share of regret	Regret per interval
Calm	~90%	~15–25%	Low
Spike	~10%	~75–85%	Very high

The tail dominates. Roughly 10% of intervals — the spikes — account for 75–85% of total decision regret. This is the most important empirical finding in this chapter, and it has direct implications for how you allocate modelling effort:

The regret concentration law: For battery dispatch in the NEM, approximately 80% of decision regret comes from approximately 10% of trading intervals — the spikes. Improving forecast accuracy during calm periods has minimal economic impact. Improving spike detection, even marginally, has enormous economic impact. A model with 20% higher overall MAE but 10% better spike detection will earn more money than one with lower MAE but worse tail performance.

Figure 14.1 — Decision regret by price regime. Left: histogram of per-interval regret for calm vs spike intervals. The spike regime has a heavy right tail — a few intervals account for the majority of total regret. Right: cumulative regret over the backtest period. The step-function shape shows that regret accumulates in bursts during spike events, with long flat stretches during calm periods.

The Asymmetric Cost of Error

Forecast errors are not symmetric in their economic consequences. There are two distinct failure modes, and they have very different costs:

Missed Spikes: The Expensive Failure

A **missed spike** occurs when the forecast fails to predict a high-price event, causing the battery to charge or hold when it should have discharged. The cost is the **foregone revenue** — the money the battery would have earned by discharging during the spike.

Missed spike: A price event where the actual price significantly exceeded the forecast, and the forecast-driven dispatch did not discharge (or discharged at a lower rate than optimal). The cost is the perfect-foresight discharge revenue minus the actual dispatch revenue during the spike period. For a 100 MW / 200 MWh battery, a fully missed spike at \$5,000/MWh costs up to $\$5,000 \cdot 100 \text{ MW} \cdot 0.5 \text{ h} = \$250,000$ per half-hour.

The magnitude of missed-spike regret depends on three factors:

- Spike height.** A missed spike at \$500/MWh is costly; a missed spike at \$5,000/MWh is catastrophic. The NEM price cap is \$17,500/MWh (as of 2025), so the theoretical maximum cost of a single fully missed half-hour is $\$17,500 \cdot 100 \text{ MW} \cdot 0.5 \text{ h} = \$875,000$. In practice, spikes above \$5,000/MWh are rare but account for a disproportionate share of total available revenue.
- State of charge.** A battery with 80% state of charge can discharge for nearly two hours at full power; a battery at 20% can discharge for less than 30 minutes. The dispatch LP's prior decisions about charging and discharging determine how much energy is available when a spike arrives. A missed spike is doubly costly if the battery charged during a previous period that turned out to be expensive.
- Duration.** Single-interval spikes (30 minutes) are less costly than sustained spike events lasting 2–4 hours. The battery's 200 MWh capacity is designed for two hours of full-power discharge, so multi-hour spikes can be fully captured only if the battery is fully charged at the start.

False Alarms: The Hidden Cost

A **false alarm** occurs when the forecast predicts a spike that does not materialise, causing the battery to hold charge (waiting for the spike) when it should have discharged at a moderate price, or to discharge into what turns out to be a low-price period.

False alarm: A price event where the forecast significantly exceeded the actual price, and the forecast-driven dispatch made a suboptimal decision as a result. The direct cost is the revenue foregone by misallocating the dispatch. The indirect cost includes unnecessary battery cycling (degradation) and the opportunity cost of the energy that was dispatched at the wrong time.

False alarms are less visible than missed spikes but still costly:

- Cycle degradation.** Each full cycle reduces the battery's lifetime capacity. For an LFP battery like Mannum, cycle degradation is relatively low (LFP chemistry tolerates 4,000–6,000 cycles to 80% capacity), but each unnecessary cycle has a real cost: the total capital cost divided by the total expected cycles gives a per-cycle degradation cost of roughly \$10–\$30/MWh for a utility-scale LFP system.
- Opportunity cost.** A battery that discharged at \$80/MWh (based on a false spike forecast) cannot discharge again at \$200/MWh two hours later — it needs time to recharge. The opportunity cost is the difference between the price at which the battery should have discharged and the price at which it actually discharged.
- Round-trip efficiency loss.** Every charge-discharge cycle loses 10–15% of the energy to round-trip inefficiency. A false alarm that triggers an unnecessary cycle wastes this energy.

Quantifying the Asymmetry

The asymmetry between missed spikes and false alarms is stark:

Failure mode	Per-event cost	Annual impact (SA1)
Missed spike (\$500–\$1,000/MWh), 20–50/year	\$5K–\$25K	\$100K–\$500K
Missed spike (\$1,000–\$5,000/MWh), 5–15/year	\$25K–\$250K	\$125K–\$1.5M
Missed spike (>\$5,000/MWh), 1–3/year	\$250K–\$875K	\$250K–\$2.5M

False alarm, 30–80/year	\$2K–\$15K	\$60K–\$400K
-------------------------	------------	--------------

Missed spikes dominate. A single missed extreme spike can cost more than a year's worth of false alarms combined. This asymmetry has a direct implication for model development: **under-predicting spikes is much more costly than over-predicting them.**

The asymmetric loss principle: For battery dispatch, the loss function is not symmetric. A dollar of under-prediction during a spike costs more than a dollar of over-prediction, because missed spikes are irreversible (the revenue opportunity is gone) while false alarms are partially recoverable (the battery can recharge and try again). This is why the quantile-based dispatch strategies from Chapter 9 — which explicitly account for forecast uncertainty — outperform naive point-forecast dispatch.

Worked example — asymmetric cost in dollars: On a summer day in SA1, the model forecasts a peak of \$300/MWh at 5:30 PM. The actual peak is \$4,200/MWh at 5:00 PM. The forecast-driven dispatch discharges at 5:30 PM (forecast peak) and earns $\$300 \cdot 100 \text{ MW} \cdot 0.5 \text{ h} = \$15,000$. The perfect-foresight dispatch discharges at 5:00 PM (actual peak) and earns $\$4,200 \cdot 100 \text{ MW} \cdot 0.5 \text{ h} = \$210,000$. Regret = $\$210,000 - \$15,000 = \$195,000$. Now consider the reverse error: the model forecasts \$4,200/MWh but the actual price is \$300/MWh. The forecast-driven dispatch discharges at the forecast peak, earning $\$300 \cdot 100 \text{ MW} \cdot 0.5 \text{ h} = \$15,000$. The perfect-foresight dispatch might also have discharged this period (if \$300 was still the best option), or might have held for a better opportunity. In the worst case, the regret is the difference between \$300 and whatever better opportunity was missed — perhaps \$30,000–\$50,000. The under-prediction regret (\$195,000) is 4–6x the over-prediction regret (\$30,000–\$50,000). The loss is asymmetric, and the asymmetry grows with spike height.

Regime-Conditional Evaluation

Why Overall Metrics Lie

Chapter 5 introduced MAE and CRPS as forecast evaluation metrics. These are aggregate measures — they average forecast performance over all time periods in the evaluation window. The problem is that the average is dominated by calm intervals (which comprise ~90% of the data), masking performance during the spikes that drive the P&L.

A concrete illustration of metric masking: Consider two models evaluated over a 90-day backtest (4,320 half-hour intervals): **Model A:** Overall MAE = \$18/MWh. Calm-interval MAE = \$12/MWh. Spike-interval MAE = \$180/MWh. **Model B:** Overall MAE = \$22/MWh. Calm-interval MAE = \$18/MWh. Spike-interval MAE = \$85/MWh. Model A has lower overall MAE — it "wins" on the standard metric. But Model B has dramatically better spike performance. When we run both through the dispatch LP, Model B achieves a capture ratio of 83% while Model A achieves only 71%. Model B earns roughly \$150,000 more than Model A over the 90-day period. The overall MAE rewarded Model A for being slightly better during 3,888 calm intervals, while ignoring that it was catastrophically worse during the 432 spike intervals. The P&L does not make this mistake.

This is not a hypothetical — it is a predictable consequence of the mathematics. The contribution of an interval to overall MAE is proportional to its forecast error. The contribution of an interval to revenue is proportional to its price. Since spike intervals have both higher prices and higher errors, the revenue-weighted importance of spike intervals far exceeds their count-weighted importance in the MAE calculation.

Regime-Conditional MAE and CRPS

The solution is to compute metrics separately for each regime:

$$\text{MAE}_{\text{calm}} = (1/n_{\text{calm}}) \cdot \sum |y_t - y^*_t| \text{ for } t \in \text{calm intervals} \quad \text{MAE}_{\text{spike}} = (1/n_{\text{spike}}) \cdot \sum |y_t - y^*_t| \text{ for } t \in \text{spike intervals}$$

And similarly for CRPS:

$$\text{CRPS}_{\text{calm}} = (1/n_{\text{calm}}) \cdot \sum \text{CRPS}(F_t, y_t) \text{ for } t \in \text{calm intervals} \quad \text{CRPS}_{\text{spike}} = (1/n_{\text{spike}}) \cdot \sum \text{CRPS}(F_t, y_t) \text{ for } t \in \text{spike intervals}$$

Report both alongside the overall metric. The regime-conditional metrics answer different questions:

Metric	Question it answers
MAE_calm	How well does the model track routine price movements?
MAE_spike	How well does the model detect and size extreme price events?
CRPS_calm	How well-calibrated is the probabilistic forecast during normal conditions?
CRPS_spike	How well-calibrated is the probabilistic forecast during extremes?

Regime-Conditional Calibration

The calibration diagnostics from Chapter 8 — PIT histograms, reliability diagrams, coverage plots — should also be split by regime. A model can be perfectly calibrated overall while being systematically overconfident during spikes (the intervals that matter most).

Regime-conditional calibration: Evaluating the calibration of a probabilistic forecast separately within each price regime (calm vs spike). A model is regime-conditionally calibrated if its prediction intervals have the correct coverage rate within each regime independently. For example, the 90%

interval should contain 90% of outcomes during calm periods AND 90% of outcomes during spike periods. Overall calibration allows the model to be overconfident during spikes if it compensates by being underconfident during calm periods — regime-conditional calibration prevents this.

In practice, most machine learning models trained on the full dataset are **well-calibrated during calm intervals and poorly calibrated during spikes**. This is because the calibration loss function weights all intervals equally, and calm intervals outnumber spike intervals roughly 9:1. The model minimises its calibration error by fitting the calm majority well, even if this means the spike minority is poorly served.

The practical fix is the conformal calibration from Chapter 8, applied separately within each regime. But the diagnostic — splitting calibration by regime — comes first. You cannot fix what you have not measured.

Attribution on the Model

When a forecast fails, the next question is **why**. What drove the model's prediction? What features pushed it toward the wrong answer? This section introduces the tools for attributing forecast errors to specific model inputs.

SHAP: Feature Attribution for Gradient-Boosted Models

The gradient-boosted tree (GBT) models from Chapter 7 are powerful but opaque — they combine hundreds of trees, each splitting on different features, making it impossible to manually trace how an input variable influenced the prediction. **SHAP** (SHapley Additive exPlanations) provides a principled decomposition of any single prediction into feature contributions.

SHAP (SHapley Additive exPlanations): A method for explaining individual predictions of machine learning models. For each prediction, SHAP computes a value for each input feature representing that feature's contribution to the prediction relative to a baseline (typically the average prediction). The SHAP values sum to the difference between the prediction and the baseline: $y_t^* - \text{baseline} = \text{sum of SHAP values for all features}$. SHAP values are derived from Shapley values, a concept from cooperative game theory that fairly allocates the total "payoff" (prediction minus baseline) among the "players" (features).

For tree-based models (LightGBM, XGBoost, CatBoost), the TreeSHAP algorithm computes exact SHAP values in polynomial time — no approximation needed. This makes SHAP practical for post-mortem analysis of every bad day in the backtest.

The interpretation is intuitive:

- A **positive SHAP value** for a feature means that feature pushed the prediction **higher** than the baseline (toward higher prices).
- A **negative SHAP value** means the feature pushed the prediction **lower** than the baseline (toward lower prices).
- The **magnitude** reflects the strength of the effect.
- The **sum** of all SHAP values equals the difference between the model's prediction and the baseline (mean) prediction.

Reading a SHAP decomposition for a missed spike: On 18 January 2026, the model predicted \$180/MWh for the 5:00 PM interval; the actual price was \$3,800/MWh. The SHAP decomposition reveals: - Baseline (average prediction): \$65/MWh - Lagged price ($t-1$): +\$42 (recent prices were elevated, pushing the prediction up) - Temperature forecast: +\$28 (high temperature → high demand → higher prices) - Net load forecast: +\$35 (high net load forecast → higher prices) - Wind generation forecast: +\$18 (low wind → less supply → higher prices) - Hour-of-day: +\$12 (5 PM is typically a peak period) - Day-of-week: -\$5 (Saturday, typically lower demand) - Other features: -\$15 (various minor negative contributions) - **Total: \$65 + \$42 + \$28 + \$35 + \$18 + \$12 - \$5 - \$15 = \$180/MWh** The SHAP values show that every major feature pushed in the right direction — the model "knew" this was going to be an expensive period. The problem was not direction but **magnitude**: the model predicted \$180 when the reality was \$3,800. No individual feature was "wrong" — the model simply could not extrapolate to the extreme tail. This is the fundamental limitation of tree models: they cannot predict values outside their training range, and the \$3,800 price was outside the range of recent training data.

Partial Dependence: Recovering the Hockey Stick

While SHAP explains individual predictions, **partial dependence plots** show the model's learned relationship between a single feature and the prediction, averaged over all other features. For electricity price models, the most important partial dependence plot is **price vs net load** — it should recover the hockey-stick shape from Chapter 3.

Partial dependence plot (PDP): A visualisation of the marginal relationship between one or two input features and the model's prediction, averaging out the effects of all other features. For a feature x_j , the partial dependence function is: $PD(x_j) = (1/n) \cdot \sum_{i=1}^n f(x_j, x_{-j,i})$, where f is the model, $x_{-j,i}$ are all features except x_j for observation i , and the average is taken over all observations. The PDP shows what the model "thinks" the effect of x_j is, holding all else constant.

If the partial dependence plot of price on net load shows a monotonically increasing, convex curve (flat at low net load, steep at high net load), the model has learned the merit-order relationship from Chapter 3. If the curve flattens or turns over at high net load, the model may be **truncating spike predictions** — a critical failure mode for battery dispatch.

The hockey-stick recovery test is a useful diagnostic: compare the GBT's partial dependence on net load with the isotonic regression merit-order curve from Chapter 10. They should be qualitatively similar. If the GBT's curve flattens at high net load while the isotonic curve continues to steepen, the GBT is under-predicting spikes — and the grey-box model's physics stage is compensating for a real weakness.

Figure 14.2 — SHAP feature attributions for the five worst regret days. Each row is one day; each bar is one feature's SHAP value. Positive values (right) push the prediction toward higher prices; negative values (left) push toward lower prices. The dominant features vary by day: temperature drives summer spikes; wind drives shoulder-season events; lagged prices dominate post-contingency events. The "missing" attribution — the gap between the prediction and the actual price — represents the model's inability to extrapolate to the tail.

What SHAP Cannot Tell You

SHAP decomposes the prediction the model **did** make. It does not tell you about predictions the model **should** have made. If a feature that would have signalled a spike was not included in the model (e.g., a generator outage flag, a constraint equation binding status, or a bid-stack shift), SHAP cannot identify the missing feature. The absence of a feature is invisible to SHAP.

This is an important limitation. On many of the worst regret days, the post-mortem reveals that the primary driver of the spike was **information the model did not have**: an unplanned generator outage, a transmission constraint that redirected supply, or a strategic rebid by a dominant generator. SHAP correctly attributes the prediction to the features that were present, but the real story is about the features that were absent.

Error Decomposition

When a day produces high regret, the natural question is: **what caused the error?** Was it a bad weather forecast? A demand surprise? An event the model could never have predicted? Error decomposition separates the total forecast error into components, each attributable to a different source.

The Decomposition Framework

The price forecast depends on several upstream inputs, each of which has its own forecast error:

price_error = f(weather_error, demand_error, supply_error, unexplained_residual)

We can approximate this decomposition by perturbing one input at a time:

- Weather-forecast error component.** Replace the weather forecast with the actual weather observation (ERA5 reanalysis for the day). Re-run the price model with actual weather but forecast demand. The change in the price prediction attributable to replacing the weather forecast is the weather-forecast error component.
- Demand-forecast error component.** Similarly, replace the demand forecast with actual demand (from AEMO settlement data). Re-run with forecast weather but actual demand. The change attributable to replacing the demand forecast is the demand-forecast error component.
- Unexplained residual.** The remaining error after correcting both weather and demand forecasts. This residual captures everything the model cannot predict even with perfect weather and demand inputs: generator outages, strategic rebidding, network constraints, interconnector failures, and other supply-side events.

Error decomposition: The practice of separating a forecast error into components attributable to different sources. For electricity price forecasting, the standard decomposition splits the error into weather-forecast error, demand-forecast error, and an unexplained residual. The decomposition is approximate because the price model is non-linear (the effects of weather and demand errors interact), but it provides actionable insight into which upstream forecasts need improvement.

Error decomposition for 18 January 2026 (the missed spike): - Total price error: \$3,800 – \$180 = \$3,620/MWh - Replacing forecast weather with actual weather: prediction moves from \$180 to \$220 → weather-forecast error component = \$40/MWh (1.1% of total error) - Replacing forecast demand with actual demand: prediction moves from \$180 to \$260 → demand-forecast error component = \$80/MWh (2.2% of total error) - Replacing both: prediction moves from \$180 to \$310 → combined input error = \$130/MWh (3.6% of total error) - Unexplained residual: \$3,800 – \$310 = \$3,490/MWh (96.4% of total error) The finding is stark: even with perfect weather and demand forecasts, the model would have predicted \$310/MWh — not \$3,800/MWh. The overwhelming majority of the error is in the unexplained residual. Investigation of the AEMO incident reports reveals that a major gas generator tripped at 4:45 PM, removing 500 MW from the supply stack and triggering scarcity pricing. This information was not available to the model at forecast time, and no feature in the model represented it.

Interpreting the Residual

The unexplained residual is not a single phenomenon — it is a catch-all for everything the model's features do not capture. In the NEM context, the major contributors to the residual are:

Source	Mechanism
Generator outages	Unplanned trips remove supply; unpredictable
Strategic rebidding	Generators rebid near dispatch, tightening supply
Network constraints	Transmission limits segment the market
Interconnector failures	Loss of capacity isolates a region
FCAS interactions	High FCAS prices cascade via co-optimisation

The uncomfortable truth about spike prediction: For the most extreme price spikes (>\$5,000/MWh), the error decomposition typically shows that 80–95% of the forecast error is in the unexplained residual — driven by events that were fundamentally unpredictable at the forecast horizon. This does not mean modelling effort is wasted: the model's ability to detect elevated-risk conditions (high demand, low wind, tight supply margins) is valuable even if it cannot predict the exact timing and magnitude of extreme events. The right framing is probabilistic: instead of trying to predict "\$5,000 at 5 PM", predict "there is a 15% chance of a price exceeding \$1,000 between 4 PM and 7 PM."

The Counterfactual: What If We Had Known?

A useful extension of the error decomposition is the **counterfactual analysis**: for each source of error, compute the dispatch regret that would have been avoided if that single source had been corrected.

Counterfactual analysis: An analytical technique that asks "what would have happened if one specific input had been different?" For error decomposition, the counterfactual replaces one forecasted input with its actual value, re-runs the price model and dispatch LP, and measures the change in revenue. This reveals the economic value of improving each upstream forecast — it answers "how much is better weather forecasting worth?"

regret_avoided(weather) = revenue(model with actual weather) – revenue(model with forecast weather)

If correcting the weather forecast would have avoided \$50,000 of regret over the backtest period, but correcting demand would have avoided \$30,000, and the unexplained residual accounts for \$400,000, the conclusion is clear: investing in better weather or demand forecasts has limited return. The economic prize is in **reducing the unexplained residual** — better supply-side modelling, real-time outage signals, or strategic rebidding indicators.

Comparing Against Mannum's Real Dispatch

The Anchor Asset

The Mannum BESS is a 100 MW / 200 MWh lithium iron phosphate (LFP) battery located in SA1. It is owned by Epic Energy and optimised by Habitat Energy, one of the most sophisticated battery optimisers in the NEM. The battery became operational in October 2025.

Mannum's **DUID** (Dispatchable Unit Identifier) in the NEM is the unique identifier used by AEMO's dispatch engine and market data systems. Its generation and consumption data are publicly available through AEMO's NEMWeb system, accessible via NEMOSIS (the same data tool used in Chapter 1).

DUID (Dispatchable Unit Identifier): A unique code assigned by AEMO to each generating unit or load in the NEM. Every scheduled generator, semi-scheduled generator, and scheduled load has a DUID. Market data — dispatch targets, generation output, bid stacks — is published at the DUID level. Battery systems typically have two DUIDs: one for the generating (discharging) unit and one for the load (charging) unit.

Pulling Mannum's Dispatch Data

Mannum's actual dispatch is available from NEMWeb via NEMOSIS, using the same `DISPATCHLOAD` table that provides generation data for all scheduled units. The key fields are:

- **INITIALMW:** The unit's actual output at the start of the dispatch interval (MW). Positive values indicate generation (discharge); negative values or the corresponding load DUID indicate consumption (charge).
- **TOTALCLEARED:** The dispatch target set by AEMO's dispatch engine (MW). This is what AEMO instructed the unit to produce; actual output may differ slightly due to ramp rates and unit dynamics.
- **AVAILABILITY:** The maximum capacity the unit offered to the market (MW). This reveals whether Habitat chose to withhold capacity.

From these fields, you can reconstruct Mannum's half-hourly dispatch profile: when it charged, when it discharged, and at what power level. Combined with the settlement price data from Chapter 1, you can compute Mannum's actual revenue for any period.

The Comparison: Course Model vs Habitat

The comparison between the course's forecast-driven dispatch and Mannum's actual dispatch is the centrepiece of this chapter. It answers the question: **how does a well-built academic model compare to a professional optimiser operating in the real market?**

The comparison is not quite apples-to-apples, and understanding the differences is as important as the numbers:

Factor	Course model	Mannum
Forecast horizon	Day-ahead	Intra-day, 5-min prices
Price data	30-min spot	5-min dispatch + FCAS
Information set	Public data only	Proprietary + real-time feeds
Dispatch frequency	Daily or 30-min	Every 5 min
Revenue streams	Energy only	Energy + FCAS
Degradation model	Simple cycle counting	Degradation-aware
Risk management	Quantile-based	Proprietary

Why the comparison matters despite these differences: The course model and Habitat are solving the same fundamental problem — buy low, sell high — with different tools. Where they agree, we gain confidence in the model. Where they disagree, we learn what we are missing. The days where Habitat significantly outperformed the course model are the most instructive: they reveal information advantages that no amount of model

tuning can replicate.

Divergence Analysis

For each day in the backtest, compute:

1. **Course model revenue:** From the Chapter 9 backtest.
2. **Mannum actual revenue:** From NEMWeb dispatch data multiplied by settlement prices.
3. **Perfect-foresight revenue:** Upper bound from the LP with actual prices.
4. **Revenue gap:** Mannum revenue minus course model revenue.

Days where the revenue gap is large and positive (Habitat outperformed) are the most interesting. Classify these days by the type of divergence:

Pattern	What happened	Lesson
Timing shift	Discharged 1–2 h earlier/later	Better intra-day information
Spike capture	Discharged during missed spike	Real-time outage detection
FCAS arbitrage	Earned from FCAS markets	Energy-only model cannot match
Capacity withhold	Reduced low-price availability	Strategic vs mechanical dispatch
SoC management	Different starting SoC	Better overnight anticipation

A day where Habitat outperformed: On 3 February 2026, the course model earned \$12,000 while Mannum earned \$48,000. Perfect-foresight revenue was \$62,000. The course model dispatched based on the day-ahead forecast, which predicted a broad afternoon peak with prices around \$150–\$300/MWh. It discharged steadily from 2 PM to 6 PM. Mannum's actual dispatch was different: it held charge through the afternoon, then discharged aggressively at 5:30 PM when prices spiked to \$2,400/MWh following a sudden interconnector constraint. Habitat appears to have detected the tightening market conditions in real time — likely from monitoring the constraint equations in AEMO's pre-dispatch engine — and adjusted dispatch accordingly. The course model could not have replicated this: it made its dispatch decision at midnight based on a day-ahead forecast. By the time the constraint emerged at 5:00 PM, the model had already discharged most of its energy. This is not a model failure — it is an information disadvantage. The lesson is that intra-day re-optimisation, as discussed in Chapter 10, would have partially closed this gap.

Figure 14.3 — Timeline comparison for a day where Habitat outperformed the course model. Top panel: actual settlement price (black), course model forecast (blue dashed). Middle panel: course model dispatch (blue bars) vs Mannum's actual dispatch (orange bars). Bottom panel: cumulative revenue comparison. The model discharged during a moderate afternoon peak; Habitat waited and captured the evening spike. The vertical grey band marks the interconnector constraint event that triggered the spike.

What Habitat Knows That the Model Does Not

The divergence analysis typically reveals several systematic information advantages:

Real-time constraint equations. AEMO publishes constraint equation binding status and marginal values in pre-dispatch and dispatch runs. These reveal network bottlenecks that can cause regional price separation. Professional optimisers monitor these in real time; the course model does not include them.

Bid-stack transparency. AEMO publishes next-day bid stacks and rebid reason strings. Sophisticated operators analyse the bid stack to estimate the supply curve and detect strategic rebidding. The course model uses lagged prices as a proxy for supply conditions, which is a weaker signal.

5-minute dispatch resolution. The NEM dispatches every 5 minutes, and the 30-minute settlement price is an average of six 5-minute dispatch prices. Professional operators optimise on 5-minute prices, capturing intra-half-hour volatility that the course model cannot see.

FCAS co-optimisation. Habitat likely co-optimises energy and FCAS, earning revenue from frequency control ancillary services. The course model earns energy arbitrage revenue only. On some days, the FCAS revenue may exceed the energy revenue, making the comparison misleading.

Proprietary meteorological and demand models. Professional operators often use private weather services and proprietary demand models that are more accurate than the public ERA5 + BOM data used in this course.

The Incident Report

Purpose and Structure

For each of the worst days by dollar impact, write a structured **incident report** — a one-page post-mortem that documents what happened, what the model predicted, what it should have done, and what could be improved.

Incident report: A structured post-mortem document for a high-regret dispatch day. The report answers four questions: (1) What happened in the market? (2) What did the model predict? (3) What would perfect foresight have done? (4) What, if anything, could have prevented the regret? Incident reports are standard practice in energy trading — they build institutional knowledge and prevent the same failure mode from recurring.

The incident report template:

INCIDENT REPORT – [Date] Region: [SA1] Regret: \$[amount] ([X]% of perfect-foresight revenue) 1. MARKET SUMMARY - What happened: [Brief description of the price event] - Peak price: \$[X]/MWh at [time] - Driver: [Demand spike / generator trip / constraint / rebid / other] 2. MODEL PREDICTION - Forecast peak: \$[X]/MWh at [time] - Forecast error at peak: \$[X]/MWh - SHAP attribution: [Top 3 features and their contributions] 3. DISPATCH COMPARISON - Model dispatch: [Charged/discharged at time X, SoC at time of spike] - Perfect foresight: [Would have discharged at time Y] - Manned actual: [Discharged at time Z, power level] 4. ERROR DECOMPOSITION - Weather-forecast error: \$[X]/MWh ([Y]%) - Demand-forecast error: \$[X]/MWh ([Y]%) - Unexplained residual: \$[X]/MWh ([Y]%) 5. LESSONS AND MITIGATIONS - Could intra-day re-optimisation have helped? [Yes/No, why] - Could better input data have helped? [Yes/No, what data] - Could a model change have helped? [Yes/No, what change] - Recommendation: [Specific action item]

Worked Example: The Five Worst Days

The following worked example takes the five worst days by regret from a Chapter 9 backtest over Q1 2026 (January–March) in SA1, and walks through the full analysis for each.

Day 1: 18 January 2026 — Generator trip during heatwave Regret: \$287,000 (72% of perfect-foresight revenue of \$398,000) **Market summary:** SA1 temperatures hit 42 degrees C. Demand surged to near-record levels. At 4:45 PM, a 500 MW gas generator tripped, removing supply during peak demand. Prices spiked from ~\$200/MWh to \$3,800/MWh within two dispatch intervals and remained above \$1,000/MWh for four half-hours. **Model prediction:** The model forecast a high-price afternoon (peak forecast \$280/MWh at 5:00 PM), correctly identifying the heatwave. However, it did not predict the generator trip or the resulting extreme spike. Forecast error at the peak interval: \$3,520/MWh. **SHAP attribution:** Temperature (+\$85), net load (+\$62), lagged price (+\$45), low wind (+\$23), hour-of-day (+\$15). All features pushed in the right direction; the magnitude was simply insufficient. **Dispatch comparison:** The model discharged from 3:00 PM to 6:00 PM at 80–100 MW, earning \$67,000. With the price shape the model predicted, this was a reasonable strategy. Perfect foresight would have concentrated discharge into the 4:30 PM–6:00 PM window, earning \$398,000. Manned discharged 100 MW from 4:30 PM to 6:30 PM, earning \$312,000 — Habitat appears to have detected the tightening conditions in real time and shifted discharge later. **Error decomposition:** Weather-forecast error: \$40/MWh (1.1%). Demand-forecast error: \$80/MWh (2.3%). Unexplained residual: \$3,400/MWh (96.6%). The residual is the generator trip. **Lessons:** No model improvement could have predicted the generator trip. However, intra-day re-optimisation with 5-minute dispatch prices would have detected the price spike within 5 minutes and adjusted dispatch. Recommendation: implement intra-day re-optimisation (Chapter 10 discussion) and monitor real-time reserve margins as a spike probability indicator.

Day 2: 7 February 2026 — Interconnector constraint during wind drought Regret: \$195,000 (65% of perfect-foresight revenue of \$300,000) **Market summary:** SA1 wind generation dropped to near zero across a 6-hour window. The Heywood interconnector hit its limit, preventing Victorian supply from flowing into SA. Prices rose to \$2,100/MWh for three consecutive half-hours. **Model prediction:** The model forecast elevated prices (peak \$180/MWh) due to low wind, but did not anticipate the interconnector constraint or the sustained duration of the spike. The probabilistic forecast's 95th percentile reached \$600/MWh — closer, but still far from reality. **SHAP attribution:** Wind generation (–\$40, i.e., low wind pushed price up), net load (+\$50), interconnector flow lag (+\$20). The wind signal was present but underweighted. **Dispatch comparison:** The model discharged 100 MW from 5:00 PM to 7:00 PM, earning \$62,000. Perfect foresight would have discharged 100 MW from 4:00 PM to 8:00 PM (the full duration of the constraint event), earning \$300,000. Manned discharged 100 MW from 4:30 PM to 7:30 PM, earning \$245,000. **Error decomposition:** Weather-forecast error: \$20/MWh (1.0%). Demand error: \$30/MWh (1.6%). Unexplained residual: \$1,870/MWh (97.4%). The residual is the interconnector constraint binding. **Lessons:** Including interconnector flow and constraint binding status as features would have improved detection. The model had a lagged interconnector flow feature, but did not have the constraint equation marginal value, which would have signalled the binding constraint hours in advance. Recommendation: add AEMO pre-dispatch constraint data as a feature.

Day 3: 22 February 2026 — Strategic rebidding by dominant generator Regret: \$142,000 (58% of perfect-foresight revenue of \$245,000) **Market summary:** Moderate demand, adequate wind. At 4:00 PM, a dominant gas generator rebid 400 MW from \$50/MWh to \$12,000/MWh — moving capacity from the bottom of the supply stack to the top. The effective supply curve shifted dramatically rightward at the current demand level. Prices jumped from \$80/MWh to \$1,800/MWh for two half-hours, then fell back as AEMO intervention and competing generators responded. **Model prediction:** The model forecast \$75/MWh — entirely normal for a mild day with adequate wind. Nothing in the feature set captured strategic rebidding intentions. **SHAP attribution:** All features were near their baseline contributions. The model saw no signal because there was no signal in the public data — the rebid occurred in real time and was not reflected in any lagged or forecast variable available to the model. **Dispatch comparison:** The model was fully charged at 4:00 PM (it had planned to discharge later in the evening based on a forecast evening peak). It did not discharge during the rebid-driven spike. Manned discharged 100 MW during both spiked half-hours, earning \$180,000 of the \$245,000 available. Habitat likely monitors the bid stack in real time and detected the rebid immediately. **Error decomposition:** Weather error: ~\$0 (weather forecast was accurate). Demand error: ~\$5/MWh. Unexplained residual: \$1,720/MWh (99.7%). This is a pure supply-side event invisible to the model. **Lessons:** Strategic rebidding is the single hardest failure mode for a model that does not ingest bid-stack data. The rebid had no precursor in public data — it was a strategic decision by a single market participant. Mitigation options: (a) include bid-stack features if available, (b) include pre-dispatch price forecasts from AEMO as features (AEMO's pre-dispatch engine sees the rebids before the settlement period), (c) accept that some events are irreducible noise and focus on risk management (Chapter 9 strategies that hold reserves for unknown spikes).

Day 4: 12 March 2026 — False alarm: model predicted spike, calm day Regret: \$38,000 (42% of perfect-foresight revenue of \$90,000) **Market summary:** A forecast heatwave did not materialise — temperatures peaked 6 degrees C below the forecast. Demand was moderate, wind was adequate, and prices remained between \$40–\$120/MWh throughout the day. A calm, unremarkable day. **Model prediction:** The model forecast peak prices of \$450/MWh at 5:00 PM, driven by the (incorrect) high-temperature forecast. The chance-constrained MPC from Chapter 9 held charge through the morning and early afternoon, waiting for an evening spike that never came. By 7 PM, when it became clear the spike was not arriving, the battery discharged at \$85/MWh — well below the \$120/MWh available at 3 PM. **SHAP attribution:** Temperature forecast (+\$140) was the dominant driver. Net load forecast (+\$60) was elevated due to the high temperature forecast. The error cascaded from weather to demand to price. **Dispatch comparison:** The model earned \$32,000. Perfect foresight would have earned \$90,000 by spreading discharge across the moderate-price afternoon. Manned earned \$72,000 — Habitat appears to have adjusted its dispatch intra-day as the temperature forecast was revised downward. **Error decomposition:** Weather-forecast error: \$250/MWh (66% of the \$380 total error at the peak interval). Demand error: \$80/MWh (21%). Unexplained residual: \$50/MWh (13%). This day's error is dominated by the weather forecast — one of the few days where

improving the weather input would have substantially reduced regret. **Lessons:** This is the prototypical false alarm case. The cost (\$38,000) is an order of magnitude lower than the worst missed spikes — confirming the asymmetry discussed earlier. Mitigation: use ensemble weather forecasts rather than a single deterministic forecast. If the ensemble shows disagreement about peak temperature, the probabilistic price forecast should widen its intervals, making the chance-constrained MPC more cautious. This is exactly the probabilistic approach from Chapter 8, applied upstream to the weather input.

Day 5: 28 March 2026 — Timing error during evening ramp **Regret:** \$31,000 (35% of perfect-foresight revenue of \$88,000) **Market summary:** Solar generation dropped off 30 minutes earlier than forecast due to cloud cover. The evening price ramp started at 4:30 PM instead of the forecast 5:00 PM. Prices reached \$380/MWh at 4:30 PM and \$520/MWh at 5:00 PM, but the model had scheduled discharge starting at 5:00 PM. **Model prediction:** The model forecast the correct magnitude (peak \$480/MWh, actual \$520/MWh) but the wrong timing (5:00 PM vs 4:30 PM). The 30-minute timing error caused the battery to miss the first half-hour of elevated prices. **SHAP attribution:** Solar generation forecast (+\$30), hour-of-day (+\$25), lagged price (+\$20). The features were appropriate but the solar generation forecast was 30 minutes late. **Dispatch comparison:** The model earned \$45,000 by discharging from 5:00 PM to 7:00 PM. Perfect foresight would have earned \$88,000 by starting discharge at 4:30 PM. Mannum earned \$74,000 — its 5-minute dispatch resolution captured the early ramp within a single dispatch interval. **Error decomposition:** Weather error (cloud cover timing): \$120/MWh (31% of peak error). Demand error: \$30/MWh (8%). Unexplained residual: \$240/MWh (62%). **Lessons:** Timing errors are a distinct failure mode from magnitude errors. A 30-minute timing error in solar generation translates directly into a 30-minute timing error in the evening ramp. Mitigation: (a) use satellite-derived nowcasting for solar, which detects cloud cover in near-real time, (b) increase dispatch frequency to 5 minutes (aligning with NEM dispatch), so the battery can respond to the actual ramp as it occurs.

Patterns Across Incident Reports

Reviewing the five worst days reveals recurring patterns:

- The unexplained residual dominates.** On four of five days, the unexplained residual (supply-side events, strategic behaviour, network constraints) accounts for more than 90% of the price forecast error. The model's inputs — weather, demand, lagged prices — are not the primary source of failure.
- Real-time information is the main advantage.** Habitat's systematic outperformance comes from operating on 5-minute data with real-time AEMO feeds, not from better day-ahead forecasting. Intra-day re-optimisation is the single highest-value improvement identified.
- The asymmetry holds.** The four missed-spike days have an average regret of \$206,000; the one false-alarm day has a regret of \$38,000. Missed spikes are 5x more costly on average.
- Generator trips and strategic rebids are unpredictable.** Two of the five worst days were caused by events that no public-data model could have predicted. Risk management (holding reserves, widening forecast intervals) is more productive than trying to predict the unpredictable.

The Forecast Error vs Decision Regret Scatter

A revealing diagnostic is to scatter-plot forecast error (MAE for the day) against decision regret (dollars lost). If forecast error and regret were perfectly correlated, all points would lie on a line. In practice, they diverge substantially:

Figure 14.4 — Scatter plot of daily forecast MAE (x-axis) vs daily decision regret in dollars (y-axis). Each dot is one day from the backtest. The weak correlation (r approximately 0.3–0.5) shows that forecast error is a poor predictor of economic impact. Days with high MAE but low regret (upper-left cluster) had large errors that did not cross decision boundaries. Days with moderate MAE but high regret (lower-right cluster) had small errors at critical moments that flipped dispatch decisions. The five labelled days are the incident reports above.

The scatter reveals four quadrants:

Quadrant	Description
Low MAE, low regret	Model worked well — accurate enough for good decisions
High MAE, low regret	Errors in the "safe zone" — prices far from decision boundaries
Low MAE, high regret	Small errors at critical decision boundaries during spikes
High MAE, high regret	Genuinely bad forecast during an important period

The existence of the high-MAE, low-regret quadrant is the key insight. Traditional forecast evaluation cannot distinguish between errors that matter and errors that do not. Only decision regret, computed through the dispatch LP, captures the economic relevance of each error.

Implications for model selection: When choosing between models, do not select the model with the lowest MAE. Select the model with the lowest regret — or equivalently, the highest capture ratio. The models may rank differently on these two criteria, and the ranking by regret is the one that determines P&L performance. Chapter 9's backtest framework computes both, but the capture ratio should be the primary selection metric.

Benchmarking Regret

Absolute and Relative Regret

Report regret in two complementary forms:

Absolute regret (\$/MWh): Total regret divided by total MWh discharged. This normalises for battery size and utilisation, making comparisons across different batteries and time periods meaningful.

$$\text{regret_per_MWh} = \text{total_regret} / \text{total_MWh_discharged}$$

Relative regret (% of perfect-foresight revenue): Total regret divided by total perfect-foresight revenue, expressed as a percentage. This is one minus the capture ratio from Chapter 9.

$$\text{regret_pct} = (\text{total_regret} / \text{total_perfect_foresight_revenue}) \cdot 100\%$$

Benchmarks for the Mannum BESS

For the Mannum BESS (100 MW / 200 MWh, SA1), the following benchmarks contextualise the results:

Strategy	Capture ratio	Regret (\$/MWh)	Regret (% PF)
Naive MPC (median)	65–75%	\$25–\$45	25–35%
Scenario MPC (Ch 9)	75–85%	\$12–\$28	15–25%
Chance-constrained (Ch 9)	78–88%	\$10–\$22	12–22%
Mannum actual (est.)	80–90%	\$8–\$18	10–20%

The Mannum estimates are derived from its NEMWeb dispatch data and contemporary settlement prices. The exact figures depend on the evaluation period; the ranges above reflect seasonal variation (summer is more volatile than winter).

The gap that matters: The difference between the course model's best strategy (chance-constrained MPC with conformal calibration) and Mannum's actual performance is approximately 5–10 percentage points of capture ratio — worth roughly \$400K–\$1.5M per year for a 100 MW battery in SA1. This gap is primarily driven by Habitat's real-time information advantage and FCAS revenue, not by a better day-ahead price forecast. Closing it requires operational infrastructure (real-time data feeds, 5-minute dispatch) rather than better modelling.

Exercises

Exercise 1: Identify the Input Whose Forecast Error Most Often Precedes a Missed Spike

From the Chapter 9 backtest, identify all missed-spike days (regret > \$20,000 and spike price > 99th percentile of training distribution). For each, compute the forecast error for each upstream input (temperature, wind speed, solar irradiance, demand).

Rank inputs by **frequency of large errors preceding missed spikes**: for each input, count days where its forecast error exceeded its own 90th percentile.

Hint: This asks which upstream forecast was most often badly wrong on missed-spike days — not which input had the largest SHAP value. SHAP measures model reliance; this measures upstream data quality failures.

Worked solution

1. Collect missed-spike days:

```
import pandas as pd
backtest = pd.read_parquet("outputs/backtest_results.parquet")
spike_threshold = backtest["actual_price"].quantile(0.99)
missed_spikes = backtest[ (backtest["regret_dollars"] > 20_000) & (backtest["actual_price_peak"] > spike_threshold) ]
```

1. Compute per-input error frequency:

```
weather_actual = pd.read_parquet("data/processed/era5_actual.parquet")
weather_forecast = pd.read_parquet("data/processed/era5_forecast.parquet")
inputs = ["temperature", "wind_speed", "solar_irradiance", "demand"]
input_errors = {}
for inp in inputs:
    errors = abs(weather_forecast[inp] - weather_actual[inp])
    threshold_90 = errors.quantile(0.90)
    large_error_days = errors.loc[missed_spikes.index] > threshold_90
    input_errors[inp] = {"count_large_errors": large_error_days.sum(), "fraction_of_missed_spikes": large_error_days.mean(), }
ranking = pd.DataFrame(input_errors).T.sort_values("count_large_errors", ascending=False)
```

1. Typical result for SA1:

```
Input Large-error days / 18 Fraction
wind_speed 12 67%
temperature 8 44%
demand 7 39%
solar_irradiance 4 22%
```

Wind speed forecast error is the most frequent precursor to missed spikes in SA1. SA1 has high wind penetration, so wind over-prediction directly under-predicts net load and price. Demand errors are also common but partially redundant with temperature errors (high temp drives high demand).

Exercise 2: Test Whether a Simple Rule on That Input Would Have Improved Dispatch

Test this rule: on days where wind-speed forecast error exceeds its 80th percentile, shift the MPC discharge quantile from $q_{0.10}$ to $q_{0.05}$ (more conservative, reserving charge for potential spikes). Compute the change in total regret.

Hint: This tests whether the signal exists before investing in a full ensemble-spread solution. One input, one threshold, one dispatch adjustment.

Worked solution

1. Flag high-wind-error days and run both backtests:

```
from grian.backtest import rolling_origin_backtest from grian.dispatch import chance_constrained_mpc wind_errors =
abs(weather_forecast["wind_speed"] - weather_actual["wind_speed"]) high_wind_error_days = wind_errors >
wind_errors.quantile(0.80) results_baseline = rolling_origin_backtest( model=model, dispatch_fn=lambda forecasts:
chance_constrained_mpc( forecasts, q_low=0.10, q_high=0.90 ), data=data, ) def adjusted_dispatch(forecasts, date): if
high_wind_error_days.loc[date]: return chance_constrained_mpc(forecasts, q_low=0.05, q_high=0.95) return
chance_constrained_mpc(forecasts, q_low=0.10, q_high=0.90) results_adjusted = rolling_origin_backtest( model=model,
dispatch_fn=adjusted_dispatch, data=data, )
```

1. Compare:

```
regret_baseline = results_baseline["regret_dollars"].sum() regret_adjusted = results_adjusted["regret_dollars"].sum()
print(f"Baseline total regret: ${regret_baseline:,.0f}") print(f"Adjusted total regret: ${regret_adjusted:,.0f}")
print(f"Relative improvement: {(regret_baseline - regret_adjusted) / regret_baseline:.1%}")
```

Typical improvement: 3-8% reduction in total regret. Diebold-Mariano test: significant at 5% level on most windows.

1. Why the improvement is modest:
2. Rule fires on only ~20% of days
3. Extra conservatism is often unnecessary (no spike materialises)
4. On some days, under-dispatch during moderate prices slightly increases regret

The signal is real but weak in isolation. A composite "spike risk" indicator combining wind-error, demand uncertainty, and supply-margin tightness would be more effective than this binary rule.

Exercise 3: Hypothesise What Additional Information Habitat Used

Pick a day where Mannum earned >2x the course model's revenue. Using NEMWeb public data, reconstruct the market context:

1. Pull AEMO pre-dispatch price forecasts (NEMSEER)
2. Check constraint equation binding status (pre-dispatch PASA)
3. Examine generator rebid data (NEMWeb bidding tables)
4. Compare regional demand vs forecast (AEMO operational demand)

Write a 300-word hypothesis naming the specific data source, the timing it became available, and how it would have changed dispatch.

Hint: Habitat operates on 5-minute intervals with real-time data; the model operates day-ahead. The question is: which piece of information, available when, would have flipped the decision?

Worked solution

Example: 7 February 2026 (interconnector constraint day).

Data reconstruction:

```
from nemseer import compile_data predispach = compile_data( start_time="2026-02-07 00:00", end_time="2026-02-08
00:00", table="PREDISPATCHPRICE", region="SA1", )
```

Pre-dispatch run Forecast peak (SA1) Signal 00:30 (midnight) \$120/MWh at 5:30 PM No spike expected (day-ahead) 06:00 \$145/MWh at 5:30 PM Wind forecast softening 10:00 \$280/MWh at 5:00 PM Wind actuals below forecast 12:30 \$650/MWh at 4:30 PM Interconnector constraint binding 14:00 \$1,200/MWh at 4:30 PM Heywood constraint confirmed 15:00 \$2,800/MWh at 4:30 PM Reserve margin near zero

Hypothesis:

What they used: AEMO pre-dispatch price forecasts and PREDISPATCH_CONSTRAINT marginal values, published every 30 minutes.

When the signal appeared: By 12:30 PM, the Heywood interconnector constraint (equation names containing "HVSC" or "HEYW") showed a non-zero marginal value, indicating SA1 was import-constrained with prices forecast above \$600/MWh. This was seven hours before the event.

How it changed dispatch: Habitat's system ingests each pre-dispatch run and re-solves the dispatch LP. When the 12:30 PM run showed elevated prices from 4:00 PM, they revised their schedule: hold charge through the afternoon, discharge at maximum rate during the constraint period. The 14:00 and 15:00 runs confirmed and amplified the signal.

Why the course model missed it: Its day-ahead forecast was made 16 hours before the event, using weather and demand forecasts that did not yet reflect the wind drought or the resulting constraint.

The minimum viable improvement: Incorporate AEMO pre-dispatch prices as a rolling feature and re-run the dispatch LP every 30 minutes as new data arrives. This is the intra-day re-optimisation from Chapter 10 — the simplest version of what Habitat does continuously.

Glossary

Term	Definition
Decision regret	Revenue gap: forecast dispatch vs perfect foresight
Asymmetric loss	Under-predicting spikes costs more than over-predicting
Regime-conditional calibration	Calibration evaluated per regime (calm vs spike)
Feature attribution	Decomposing prediction into per-feature contributions
SHAP	Shapley-value-based feature attribution method
Partial dependence	Marginal feature-prediction relationship, averaged
Counterfactual analysis	What if one specific input had been different?
Incident report	Structured post-mortem for high-regret days
DUID	AEMO unique code for each NEM unit or load
Missed spike	Actual price far exceeded forecast; missed discharge
False alarm	Forecast far exceeded actual; suboptimal dispatch
Error decomposition	Splitting error into weather, demand, and residual
Capture ratio	Fraction of perfect-foresight revenue achieved

Summary

This chapter reframes forecast evaluation around decision regret — the revenue gap between forecast-driven dispatch and perfect-foresight dispatch — rather than traditional forecast error metrics like MAE and CRPS. The core finding is that forecast error and decision regret are only weakly correlated: large errors during calm periods cost nothing because they do not flip dispatch decisions, while small errors during spike periods can cost hundreds of thousands of dollars. The asymmetry is stark: missed spikes cost 5–10x more per event than false alarms, and approximately 80% of total regret comes from 10% of trading intervals. Regime-conditional evaluation — splitting metrics and calibration diagnostics into calm and spike intervals — reveals performance differences masked by overall metrics. SHAP feature attributions and partial dependence plots identify what drove the model's predictions on bad days, but the error decomposition typically shows that 80–95% of the error on the worst days comes from the unexplained residual (generator trips, strategic rebids, network constraints) rather than from weather or demand forecast errors. Comparing the course model against Mannum's actual dispatch from NEMWeb reveals that Habitat Energy's systematic advantage comes from real-time information (5-minute dispatch prices, constraint equation binding status, bid-stack monitoring) and FCAS co-optimisation, not from a better day-ahead forecast. The incident report framework — structured post-mortems for the worst days by dollar impact — builds institutional knowledge and identifies actionable improvements: intra-day re-optimisation with pre-dispatch data is consistently the highest-value recommendation. Regret should be reported in \$/MWh and as a percentage of perfect-foresight revenue, alongside the capture ratio from Chapter 9, to give a complete picture of economic performance.

15. Live Operations and the Intra-day Loop

Why Operations Is a Separate Discipline

Everything up to this point has been built offline. The forecasting models (Chapters 6--8), the dispatch optimiser (Chapter 9), the market emulator (Chapter 12), and the decision-regret framework (Chapter 14) were all developed, tested, and tuned against historical data. They work. But a model that works on historical data and a model that works in real-time production are different things, and the gap between them is not a modelling gap --- it is an engineering and operational gap.

The Mannum BESS --- Epic Energy's 100 MW / 200 MWh lithium iron phosphate battery in SA1, optimised by Habitat Energy --- was commissioned in October 2025. It participates in the NEM's 5-minute dispatch market, submitting bid stacks to AEMO and receiving dispatch instructions every five minutes. This chapter describes what happens between "the model produces a forecast" and "the battery earns revenue," and how to keep the system healthy once it is running.

The shift from backtesting to operations: In a backtest, you have all the data before you start. In operations, data arrives incrementally, forecasts must be produced on a clock, decisions have deadlines, and the consequences of failure are measured in dollars, not error metrics. The operational loop is where forecast quality is converted into captured revenue --- or lost to latency, staleness, and unmonitored drift.

This chapter is shorter and more descriptive than the modelling chapters. It covers four topics: intra-day re-optimisation, the operational loop and its clock, monitoring and retraining, and risk controls and human oversight. The worked example runs a single volatile day through an intra-day simulation to show the revenue benefit of updating the plan as new information arrives.

Intra-day Re-optimisation

The Cost of a Stale Plan

The dispatch chapters (Chapters 5 and 9) solved for a battery schedule once per day: take tomorrow's probabilistic price forecast, run the MPC optimiser, and produce a 48-half-hour charge/discharge plan. In calm markets, this works well. Prices follow the forecast closely, the plan executes as intended, and the capture ratio is respectable.

Volatile days are different. On 24 January 2024 --- a heatwave in SA1 with temperatures above 42 degrees Celsius --- spot prices moved from \$40/MWh at 10 AM to \$6,500/MWh at 3 PM, fell to \$120/MWh at 5 PM, then spiked again to \$11,000/MWh at 7 PM. A day-ahead forecast made at 12:30 PM the previous day could not have predicted the timing or magnitude of these swings with precision. By 2 PM on the day itself, the battery's pre-committed plan was already stale: it had scheduled discharge for 5 PM based on yesterday's forecast, but the real spike was arriving two hours early.

Staleness: The degree to which a forecast or plan no longer reflects the current information set. A forecast made 18 hours ago is stale not because its model is wrong but because new data has arrived --- updated AEMO pre-dispatch forecasts, real-time demand readings, current renewable output --- that was not available when the forecast was produced. Staleness increases with the time since the last forecast update and with the rate at which market conditions are changing.

The cost of staleness is asymmetric. On calm days, updating the plan every 30 minutes versus once per day makes little difference --- the prices follow the day-ahead forecast and the plan remains approximately optimal. On volatile days, the cost of staleness can be enormous. A battery that discharges into a \$200/MWh period because its stale plan said that was the peak, while the actual peak at \$8,000/MWh arrives an hour later with the battery already empty, has missed the vast majority of the day's revenue. Volatile days contribute a disproportionate share of annual revenue (Chapter 2), so losing them to staleness has outsized consequences.

From LP Re-solve to Emulator Re-optimisation

In Chapter 5, we introduced the basic receding-horizon MPC: at each step, re-solve the dispatch LP with updated price forecasts, execute the first period's decision, and repeat. This works when the LP accurately models the market, but the NEM's dispatch process is more complex than a simple price-taker LP assumes. Interconnector constraints, generator ramping limits, and strategic bidding by other participants all affect prices in ways the LP ignores.

Chapter 12 introduced the nempy market emulator --- a model that replays the AEMO dispatch engine with modified bid stacks, allowing us to ask "what would the price be if our battery bid differently?" The emulator captures price-impact effects that the price-taker LP misses: when the Mannum BESS discharges 100 MW into a tight SA1 market, it pushes the price down; when it charges, it pulls the price up. These effects are small in liquid periods but material during the volatile periods where most revenue is earned.

Intra-day re-optimisation: The practice of updating the battery's dispatch plan multiple times within a single trading day as new information arrives. Rather than committing to a single day-ahead plan, the system re-solves the optimisation problem at regular intervals (typically every 30 minutes or every 5 minutes), incorporating updated price forecasts, current state of charge, and any new market information. Each re-optimisation replaces the remaining plan; only the next period's decision is executed.

The intra-day loop in this chapter re-optimises through the Chapter 12 emulator, not just the LP. At each re-optimisation step:

1. **Update the price forecast.** Incorporate the latest AEMO pre-dispatch forecasts, real-time demand data, and current renewable output.
2. **Set the current battery state.** Record the actual state of charge, which may differ from the plan due to AEMO dispatch instructions.

3. **Run the emulator.** For each candidate bid stack, simulate the remaining periods through the nempy emulator to estimate the resulting prices and dispatch outcomes.
4. **Solve the optimisation.** Find the bid stack that maximises expected revenue over the remaining horizon, accounting for price impact.
5. **Submit the bid.** Send the first period's bid to AEMO before gate closure.
6. **Wait for dispatch.** AEMO dispatches the battery and reports the actual price and dispatch quantity.
7. **Repeat from step 1.**

The key difference from the Chapter 9 LP approach is step 3: the emulator evaluates each candidate schedule against a simulated market, not against a fixed price forecast. This means the optimisation accounts for the battery's own effect on prices, producing more realistic (and typically more conservative) dispatch decisions. The LP and the emulator produce similar plans during low-volatility periods; during tight supply events --- heatwaves, generator outages, low wind --- the battery's dispatch is a meaningful share of the marginal supply, and the emulator's price-impact correction can shift the optimal discharge timing by one or two periods.

Re-optimisation Frequency

How often should the system re-solve? There is a trade-off between information freshness and computational cost:

Frequency	Info gain	Cost	Fit
Day-ahead only	Baseline; no updates	Negligible	Calm markets
Every 30 min	PD updates, demand revisions	Low; ~48 LP or ~10 emulator/day	Good default
Every 5 min	Max freshness; real-time signals	Moderate; 288 iter/day	Needs fast solver
Event-triggered	Re-solves on material change	Variable by volatility	Efficient; complex

For the Mannum BESS, a 30-minute re-optimisation cadence with event-triggered re-solves during high-volatility periods is a reasonable starting point. The emulator adds computational cost relative to the LP (roughly 2--5 seconds per run versus milliseconds for the LP), but even at 5-minute frequency the total computation is manageable on a single modern server.

The Operational Loop

The Market's Clock

The NEM operates on a strict temporal structure. Every decision the battery makes must respect this clock.

AEMO dispatch cycle: The NEM dispatches generation and load every 5 minutes. AEMO runs the National Electricity Market Dispatch Engine (NEMDE) at each 5-minute interval, determining the dispatch quantity and price for every registered unit. The dispatch price is the marginal price --- the cost of the next MW of supply needed to meet demand. Bids must be submitted before gate closure for each interval.

The key timestamps for a battery operator:

Event	Timing	Action
PD publication	Every 30 min; next 40 h	Price/demand forecasts per HH
5MPD	Every 5 min; next hour	Short-term price fcst (12 intervals)
Gate closure	~2 min before interval	Bid submission deadline
Dispatch	Every 5 min on clock	NEMDE sets prices, issues targets
Settlement	Post-hoc; days later	Revenue from qty × price

Gate closure: The deadline by which a market participant must submit or update their bids for a given dispatch interval. In the NEM, gate closure is approximately two minutes before the start of the dispatch interval. After gate closure, bids cannot be changed for that interval. This imposes a hard constraint on the operational loop: the forecast must be made, the optimisation must be solved, and the bid must be submitted before gate closure. Any latency in the pipeline --- data retrieval, model inference, optimisation, bid formatting --- directly reduces the time available for computation.

Data Latency as a Real Constraint

In a backtest, all data is available instantly. In production, data has latency:

- **SCADA telemetry** (real-time demand, generation, interconnector flows): typically 4--10 seconds delay from AEMO's data feeds
- **Pre-dispatch forecasts** (30-minute PD and 5-minute PD): published on schedule but require ~30 seconds to download, parse, and ingest
- **Weather observations** (BOM synoptic data): updated hourly with ~15 minutes delay
- **Weather forecasts** (ERA5 or BOM ACCESS): updated every 6--12 hours with several hours delay
- **NEMSEER pre-dispatch traces:** available with ~1 minute delay after AEMO publication

The total pipeline latency --- from "new data is published" to "updated bid is submitted to AEMO" --- is the sum of data retrieval, forecast model inference, optimisation solve time, and bid submission overhead. For the intra-day loop to be useful, this total must be less than the re-optimisation interval. At 30-minute frequency, there is ample time. At 5-minute frequency, every second counts.

Latency budget for 5-minute re-optimisation: Each 5-minute interval is 300 seconds. Gate closure is ~120 seconds before the interval starts, leaving ~180 seconds from the start of the previous interval to submit the updated bid. A typical latency budget: - Data retrieval and parsing: 30 seconds - Feature engineering: 5 seconds - Forecast model inference (GBT + QRA): 10 seconds - Emulator optimisation (10 scenario evaluations): 30 seconds - Bid formatting and submission: 5 seconds - **Total: ~80 seconds**, leaving ~100 seconds of buffer. This is comfortable on modern hardware. The bottleneck is data retrieval, not computation --- waiting for AEMO's feeds to update and downloading the latest pre-dispatch run dominates the latency budget.

The Forecast-Bid-Settle Cycle

The complete operational loop for one dispatch interval:

Figure 15.1 — The operational loop for one dispatch interval. Data arrives, the forecast updates, the emulator re-optimises, and the bid is submitted before gate closure. After dispatch, the actual price and dispatch quantity are recorded for settlement and monitoring.

1. **Ingest.** Download the latest pre-dispatch forecasts, SCADA data, and any updated weather inputs. Parse and validate.
2. **Forecast.** Run the price model (Chapter 7 GBT + Chapter 8 QRA calibration) on the updated feature vector. Produce quantile forecasts for the remaining horizon.
3. **Optimise.** Pass the updated forecast and current state of charge to the emulator-based MPC (steps 1--4 from the re-optimisation procedure above). Obtain the optimal bid stack for the next interval.
4. **Bid.** Format the bid according to AEMO's bidding interface specifications and submit before gate closure.
5. **Dispatch.** AEMO runs NEMDE, issues the dispatch target to the battery, and publishes the dispatch price.
6. **Record.** Log the forecast, bid, dispatch target, actual price, state of charge, and any deviations. These records feed the monitoring system (next section).
7. **Settle.** At the end of the trading day, compute actual revenue from dispatch quantities and settlement prices. Compare to the plan's expected revenue.

The bid is not the dispatch. A common misconception is that the battery controls its own dispatch. It does not. AEMO dispatches the battery according to its bid stack and the market's needs. If the battery bids to discharge 100 MW at \$100/MWh and the dispatch price is \$150/MWh, AEMO will dispatch the full 100 MW. But if the price is \$80/MWh, the bid is not dispatched. The battery influences its dispatch through its bid prices, but AEMO makes the final decision. This distinction matters for the emulator --- the optimisation must find the bid prices that induce the desired dispatch outcome, not just the desired quantities.

Monitoring

What to Monitor

A production forecasting and dispatch system requires continuous monitoring across four dimensions: forecast quality, calibration quality, data integrity, and decision quality.

Forecast Drift

Model drift: The gradual degradation of a model's predictive performance over time, caused by changes in the underlying data distribution. In electricity markets, drift occurs because the generation fleet evolves (coal retires, new renewables are commissioned), demand patterns shift (electrification, rooftop solar adoption), and market rules change (5-minute settlement, capacity mechanisms). A model trained on 2023 data may perform poorly on 2025 data if the market's statistical properties have shifted materially.

Monitor the following forecast metrics on a rolling basis (e.g., trailing 7-day and 30-day windows):

- **Rolling MAE and RMSE** of the point forecast (median), compared to the training-period baseline. A sustained increase indicates drift.
- **Rolling CRPS** of the full quantile forecast. CRPS captures both calibration and sharpness, so a CRPS increase may indicate either drift or calibration degradation.
- **Forecast bias** (mean error). A bias that drifts away from zero indicates systematic over- or under-prediction, often caused by a structural change in the market (e.g., a new generator commissioning).

Detecting drift in SA1: Suppose the model's 30-day rolling MAE was \$15/MWh during training and validation. If, three months into production, the rolling MAE climbs to \$22/MWh and stays there for two weeks, this is a signal of drift. Common causes in SA1: a large solar farm commissioning (pushing midday prices lower than the model expects), a gas plant retiring (removing supply and increasing afternoon prices), or a policy change affecting interconnector capacity between SA1 and VIC1.

Calibration Drift

The probabilistic forecast's calibration (Chapter 8) can degrade even when the point forecast remains accurate. Monitor:

- **PIT histogram** (probability integral transform): should be uniform. A consistent U-shape indicates underdispersion (intervals too narrow); an inverted-U indicates overdispersion.
- **Coverage rates** at key quantile levels (e.g., 10th, 50th, 90th). If the 90th percentile forecast is exceeded more than 10% of the time, the tails are underestimated.
- **Rolling calibration score** (average calibration error across quantiles). A sustained increase triggers recalibration.

Data-Quality Faults

Bad data is the most common cause of production failures, and it is entirely distinct from model drift:

- **Missing values.** AEMO feeds occasionally drop observations. The system must detect gaps and either interpolate or fall back to a simpler model.
- **Stale feeds.** A data source that stops updating (e.g., a weather API returning yesterday's data) produces forecasts based on old information without raising an obvious error.
- **Out-of-range values.** Negative demand, generation above nameplate capacity, or prices outside the NEM's market floor (−\$1,000/MWh) and cap (\$17,500/MWh) indicate data corruption.
- **Schema changes.** AEMO occasionally changes its data formats or column names. Automated ingestion that assumes a fixed schema will break silently.

The data-quality hierarchy: Fix data-quality issues before investigating model drift. Nine times out of ten, a sudden performance drop is caused by bad data, not a deteriorating model. Build data-quality checks into the ingestion pipeline --- validate ranges, check for staleness, alert on missing values --- and investigate them first when performance degrades.

Decision-Regret Drift and Capture-Ratio Drift

Chapter 14 introduced **decision regret** --- the difference between the revenue actually earned and the revenue that would have been earned with the optimal decision given the actual price. Monitoring regret over time reveals whether the dispatch system is improving or degrading.

Decision-regret drift: A sustained change in the average decision regret over time. Increasing regret indicates that the dispatch system is making worse decisions --- either because the forecast has degraded, the optimiser is misconfigured, or the market has changed in ways the system does not account for. Decision-regret drift is a more direct measure of operational quality than forecast drift alone, because it captures the downstream effect of forecast errors on revenue.

Capture-ratio drift: A sustained change in the rolling capture ratio --- the ratio of actual dispatch revenue to perfect-foresight revenue (Chapter 9). A declining capture ratio over time indicates degrading operational performance. Unlike decision regret, capture ratio normalises for market volatility: a low-volatility week naturally has lower perfect-foresight revenue, so a stable capture ratio during a calm week indicates the same operational quality as during a volatile week. The 0.50 capture ratio bar is the benchmark; sustained performance below this level warrants investigation.

Monitor both decision regret and capture ratio on rolling windows:

- **7-day rolling capture ratio.** Compare to the backtest benchmark (0.50+). A drop below 0.45 sustained for more than a week triggers investigation.
- **30-day rolling capture ratio.** The strategic view. Compare to the same period in the previous year (seasonality-adjusted).
- **Rolling mean regret per interval.** Track the average "money left on the table" per dispatch interval. Increasing regret suggests the forecast or optimiser is degrading.
- **Regret decomposition.** As in Chapter 14, decompose regret into forecast error contribution and optimiser suboptimality. This tells you whether the problem is the forecast or the dispatch logic.

Figure 15.2 — A monitoring dashboard panel for the Mannum BESS. Top: rolling 7-day capture ratio with the 0.50 benchmark line. Middle: rolling calibration score (lower is better) with a recalibration threshold. Bottom: rolling mean decision regret per interval, decomposed into forecast-error regret and optimiser regret. A spike in capture ratio (positive) on volatile days confirms the intra-day loop is working; a sustained decline in capture ratio triggers investigation.

Retraining vs Recalibration

When monitoring detects degradation, the response depends on the source:

Retraining cadence: The schedule on which a model is retrained from scratch (or fine-tuned) on updated data. Retraining is expensive --- it requires assembling a fresh training set, running hyperparameter optimisation, validating on a held-out period, and deploying the new model. A typical cadence for electricity price models is monthly or quarterly, unless drift monitoring triggers an earlier retrain.

Symptom	Cause	Response
PIT non-uniform, MAE stable	Quantile miscalibration	Recalibrate QRA/conformal
MAE rising, calibration degrading	Feature-target shift	Retrain GBT on recent data
Sudden MAE spike	Data fault (stale/missing)	Fix data pipeline
Capture ratio down, MAE stable	Optimiser or market change	Check dispatch logic

Regret biased one direction	Systematic forecast bias	Bias-correct; retrain if needed
-----------------------------	--------------------------	--

Recalibrate first, retrain second. Recalibration (updating the conformal or QRA layer from Chapter 8) is fast, cheap, and often sufficient. It corrects for distribution shift without re-learning the entire feature-target relationship. Retraining is expensive and risky --- the new model might be worse than the old one if the training data is noisy or the hyperparameter search lands in a poor region. Retrain only when recalibration fails to restore performance.

A practical retraining workflow:

1. **Weekly:** Update the conformal calibration window (Chapter 8). This is automatic and requires no human intervention.
2. **Monthly:** Refit the QRA combination layer on the most recent 90 days of data. Check that the recalibrated model's coverage rates match the target levels.
3. **Quarterly:** Retrain the GBT model from scratch on the most recent 12--18 months of data. Run the full backtest (Chapter 5) to validate before deploying.
4. **Event-triggered:** After a major market event (generator closure, new interconnector, rule change), retrain immediately regardless of the schedule.

Risk and Oversight

Position and Exposure Limits

A battery trading in the NEM is exposed to spot price risk on every dispatch interval. If the battery is discharged (short stored energy) when prices spike, it earns revenue. If it is charged (long stored energy, consuming from the grid) when prices spike, it pays a large bill. Risk controls limit the downside.

Drawdown limit: A maximum allowable loss over a specified period (a single day, a rolling week) that triggers a protective action --- typically reverting to a safe state (hold current state of charge, do not trade) until a human trader reviews the situation. Drawdown limits protect against catastrophic losses from model failures, data errors, or unprecedented market conditions. They are the last line of defence when all upstream controls fail.

Key risk parameters for the Mannum BESS:

Parameter	Value	Rationale
Max exposure/interval	\$500K	Cap single-decision loss
Daily drawdown	\$200K net loss	Hold + alert trader
Weekly drawdown	\$500K net loss	Longer-horizon cap
Max SoC deviation	40 MWh (20%)	Triggers re-optimisation
Min SoC	20 MWh (10%)	FCAS reserve

How a drawdown limit works in practice: Suppose the Mannum BESS starts the day with a plan to charge during the early morning (expected prices ~\$30/MWh) and discharge in the afternoon (expected prices ~\$150/MWh). At 8 AM, an unexpected generator trip pushes morning prices to \$3,000/MWh. The battery is charging --- buying electricity at \$3,000/MWh instead of \$30/MWh. Over two hours, this costs an extra \$500,000 versus the plan. The daily P&L drops to --\$400,000, breaching the \$200,000 drawdown limit. The system halts trading: the battery holds its current state of charge and submits bids at the market cap (ensuring it will not be dispatched to charge) and at the market floor (ensuring it will not be dispatched to discharge). The human trader is alerted. They review the situation, determine whether the morning prices were a one-off event or the start of a sustained disruption, and either resume automated trading or adjust the risk parameters.

Daily P&L Tracking

Revenue tracking is the simplest and most important operational metric. For each dispatch interval:

$\text{interval_revenue} = \text{dispatch_price} \times \text{dispatch_quantity} \times (5/60) \text{ hours}$

where dispatch_quantity is positive for discharge (earning revenue) and negative for charge (paying cost). The daily P&L is the sum across all intervals, net of any FCAS revenue and grid fees. Track this against the plan's expected P&L and against perfect-foresight revenue:

- **Plan vs actual:** Measures execution quality. Large deviations indicate forecast error or AEMO dispatch deviations from the bid.
- **Actual vs perfect foresight:** The capture ratio, computed daily. On volatile days, even a 0.40 capture ratio can represent substantial revenue; on calm days, the perfect-foresight revenue itself is small.

Human-in-the-Loop

Human-in-the-loop: An operational design in which a human trader or operator retains authority over certain decisions, typically high-stakes or unusual situations. The automated system handles routine operations (forecast, optimise, bid) but escalates to a human when conditions exceed predefined thresholds. The human provides judgement that the model cannot: awareness of events not in the data (regulatory announcements,

weather warnings, equipment issues), risk appetite adjustments, and override authority when the model's behaviour is clearly wrong.

When should a human trader override the machine?

Unprecedented events. A market event with no historical analogue (e.g., the first activation of a new market rule, a cyberattack on grid infrastructure, a sudden regulatory intervention) is outside the model's training distribution. The model will extrapolate, often poorly. A human trader who understands the event's implications can make better decisions.

Risk limit breaches. When a drawdown limit is hit, the system halts and the human decides whether to resume, adjust parameters, or remain in hold.

Known model weaknesses. If the model has a documented weakness (e.g., poor performance during interconnector outages, as identified during backtest evaluation), the human can intervene during those specific conditions.

Conflicting signals. If the forecast says prices will be low but the pre-dispatch forecast shows a supply shortfall, or if weather data suggests a heatwave that the model has not yet incorporated, the human can override the forecast or adjust the dispatch plan.

End-of-day position management. A human trader may want to ensure the battery ends the day at a specific state of charge (e.g., fully charged to prepare for expected morning discharge) regardless of what the optimiser recommends.

Automate the routine, escalate the exceptional. The operational loop should handle 95% of dispatch intervals without human intervention. The remaining 5% --- risk limit breaches, unprecedented events, known model blind spots --- should be escalated with clear information: what happened, what the model recommends, what the risk metrics show. The trader makes the call. A well-designed human-in-the-loop system makes the human's job easier, not harder, by filtering out the noise and presenting only the decisions that require judgement.

Worked Example: Intra-day Re-optimisation on a Volatile Day

Setup

We simulate the Mannum BESS (100 MW / 200 MWh, 85% round-trip efficiency) operating on 24 January 2024, a severe heatwave day in SA1. The simulation compares three strategies:

1. **Day-ahead only.** A single dispatch plan produced at 12:30 PM on 23 January, using the day-ahead price forecast. No intra-day updates.
2. **Intra-day re-optimisation.** The plan is re-solved every 30 minutes using the latest NEMSEER pre-dispatch forecasts, run through the Chapter 12 emulator.
3. **Perfect foresight.** The optimal plan computed with actual prices (the upper bound on achievable revenue).

We also note the Mannum BESS's actual dispatch and revenue for the same day as a real-world comparison.

Price Profile

The actual SA1 prices on 24 January 2024:

Time	Price (\$/MWh)	Conditions
00:00--08:00	\$25--\$45	Overnight; moderate demand
08:00--12:00	\$40--\$80	Morning ramp; solar generation increasing
12:00--14:00	\$15--\$30	Solar peak; low net demand
14:00--15:30	\$2,500--\$6,500	First spike; air conditioning load surges as temperature exceeds 42C
15:30--17:00	\$80--\$200	Brief reprieve; cloud cover temporarily reduces cooling load
17:00--19:30	\$4,000--\$11,000	Second spike; solar generation drops while cooling load remains high
19:30--24:00	\$50--\$120	Evening; temperatures fall, demand eases

Day-Ahead Plan

The day-ahead forecast, made at 12:30 PM on 23 January, correctly predicted elevated afternoon prices but underestimated their magnitude and missed the two-spike structure. It forecast a single broad peak of \$500--\$1,200/MWh from 14:00 to 19:00.

The day-ahead plan: - **Charge** 00:00--06:00 and 11:00--14:00 (cheap overnight and solar trough periods). Total charge: 200 MWh. - **Discharge** 14:00--16:00 at 100 MW (200 MWh over 2 hours, accounting for RTE losses). Battery empty by 16:00. - **Hold** for the rest of the day.

The plan discharges during the first spike (14:00--16:00) and earns revenue at the actual prices during that window. But it misses the larger second spike (17:00--19:30) entirely because the battery is empty by 16:00. The day-ahead plan discharged all its energy at the first opportunity, not knowing a bigger opportunity was coming.

Day-ahead revenue: approximately \$780,000.

Intra-day Re-optimisation

The intra-day loop re-solves every 30 minutes. Key decision points:

- 14:00 re-solve.** The 5-minute pre-dispatch now shows prices remaining elevated through 15:30. The emulator confirms that discharging at 100 MW will push prices down slightly but they remain high. The re-optimised plan continues discharging, consistent with the day-ahead plan.
- 15:30 re-solve.** Prices have dropped to \$120/MWh, but the updated pre-dispatch shows a second spike forming at 17:00--19:00 with forecast prices of \$3,000--\$7,000/MWh. The battery has 60 MWh remaining (it discharged 140 MWh during 14:00--15:30 at a controlled rate rather than maximum power). The re-optimised plan: **hold** the remaining 60 MWh and discharge at 17:00.
- 16:00 re-solve.** Pre-dispatch prices for 17:00--19:00 have increased to \$5,000--\$9,000/MWh. The plan reconfirms: hold and wait.
- 17:00 re-solve.** Prices spike to \$7,500/MWh. The emulator confirms high prices for the next 90 minutes. Discharge 60 MWh over 36 minutes at 100 MW.

The critical difference: the intra-day loop saw the second spike forming in the pre-dispatch forecasts and **reserved energy** for it. The day-ahead plan, blind to this information, had already exhausted the battery.

Intra-day revenue: approximately \$1,180,000.

Perfect Foresight

With perfect knowledge of the two-spike structure, the optimal plan: - Charge overnight and during the solar trough: 200 MWh. - Discharge 80 MWh during the first spike (14:00--15:30) at \$4,500/MWh average. - Hold during 15:30--17:00. - Discharge 120 MWh (net of RTE) during the second spike (17:00--19:00) at \$8,000/MWh average.

Perfect-foresight revenue: approximately \$1,550,000.

Comparison

Strategy	Revenue	Capture ratio
Day-ahead only	\$780,000	0.50
Intra-day re-optimisation	\$1,180,000	0.76
Perfect foresight	\$1,550,000	1.00

Figure 15.3 — Revenue comparison for the Mannum BESS on 24 January 2024 (simulated). Left: cumulative revenue over the day for day-ahead only, intra-day re-optimisation, and perfect foresight. The intra-day strategy captures the second evening spike that the day-ahead plan misses. Right: state-of-charge trajectories showing how the intra-day loop reserves energy for the second spike while the day-ahead plan exhausts the battery by 16:00.

The value of re-optimisation: On this volatile day, intra-day re-optimisation recovered an additional \$400,000 compared to the day-ahead plan --- more than half of the revenue left on the table by the stale plan. The capture ratio improved from 0.50 (at the benchmark bar) to 0.76. On calm days, the improvement is negligible. The value of re-optimisation is concentrated in precisely the days that matter most for annual revenue.

Real-World Comparison

The Mannum BESS's actual dispatch on this day (observable from AEMO's public dispatch data) provides a third reference point. Real-world performance is affected by factors not in the simulation: FCAS co-optimisation commitments, AEMO's dispatch conformance requirements, network constraints, and the operator's risk appetite. Actual battery dispatch typically falls between the day-ahead simulation and the intra-day simulation, reflecting the operator's use of real-time information tempered by practical constraints.

Exercises

Exercise 1: Re-optimisation Value vs Volatility

Task: Compare intra-day re-optimisation revenue across a calm and volatile week. Simulate the Mannum BESS with:

- Strategy A** --- Day-ahead only (one plan per day, no updates).
- Strategy B** --- 30-minute intra-day re-optimisation via the emulator.

Run both on (a) a calm SA1 winter week (June, no spikes) and (b) a volatile summer week (January, multiple spikes above \$5,000/MWh). Report weekly revenue, capture ratio, and improvement.

Worked solution

Calm week (June 2024): Prices \$20--\$120/MWh, no spikes above \$300. Perfect-foresight revenue: \$85,000.

Strategy	Weekly revenue	Capture ratio
----------	----------------	---------------

Day-ahead only	\$52,000	0.61
Intra-day re-optimisation	\$55,000	0.65
Improvement	\$3,000 (5.8%)	+0.04

Marginal gains only --- the day-ahead plan is already near-optimal when prices are smooth.

Volatile week (January 2024): Spikes above \$5,000/MWh on 3 days, above \$1,000 on 5 days. Perfect-foresight revenue: \$4,200,000.

Strategy	Weekly revenue	Capture ratio
Day-ahead only	\$2,100,000	0.50
Intra-day re-optimisation	\$3,020,000	0.72
Improvement	\$920,000 (43.8%)	+0.22

Re-optimisation benefit: Calm week → +\$3,000 (+5.8%) Volatile week → +\$920,000 (+43.8%)

Re-optimisation value scales super-linearly with volatility. This motivates event-triggered re-optimisation: re-optimize frequently when volatility indicators are elevated, conserve resources during calm periods.

Exercise 2: Drawdown Limit Design

Task: Implement a daily drawdown limit of \$150,000 for the Mannum BESS on the volatile January week from Exercise 1. Drawdown rule: if daily cumulative P&L falls below --\$150,000, halt trading for the remainder of the day.

Report:

1. Total weekly revenue with and without the limit.
2. Worst single-day P&L with and without the limit.
3. Number of days the limit was triggered.

Worked solution

Without drawdown limit:

Day	P&L (no limit)
Monday	+\$680,000
Tuesday	+\$420,000
Wednesday	--\$280,000
Thursday	+\$1,050,000
Friday	+\$390,000
Saturday	+\$510,000
Sunday	+\$250,000
Total	\$3,020,000

Wednesday's loss: unexpected price spike to \$4,000/MWh during charging (transmission constraint).

With \$150,000 drawdown limit:

Limit triggered Wednesday at ~11:00 AM. Avoids \$130,000 of additional afternoon losses but also misses a \$350,000 late-afternoon recovery.

Day	P&L (with limit)	Limit triggered?
Monday	+\$680,000	No
Tuesday	+\$420,000	No
Wednesday	--\$150,000	Yes (11:00 AM)
Thursday	+\$1,050,000	No
Friday	+\$390,000	No
Saturday	+\$510,000	No
Sunday	+\$250,000	No
Total	\$3,150,000	-

Comparison:

Metric	No limit	\$150K limit
Weekly revenue	\$3,020,000	\$3,150,000
Worst-day P&L	--\$280,000	--\$150,000

Days limit triggered	N/A	1
----------------------	-----	---

Net effect: +\$130,000 weekly revenue, worst-day loss capped at --\$150,000.

The drawdown limit's purpose is to bound downside, not maximise revenue. In this case it happened to improve both. In other scenarios it may reduce total revenue by halting before a recovery --- the trade-off is tail-risk reduction.

Drawdown limits are insurance, not optimisation. A well-set drawdown limit slightly reduces average revenue (by occasionally halting profitable trading) but substantially reduces tail risk (by capping worst-case losses). The appropriate level depends on the operator's risk appetite, the battery's annual revenue target, and the consequences of large losses (e.g., covenant breaches on project finance). For the Mannum BESS, a daily drawdown limit in the range of \$150,000--\$300,000 balances downside protection against opportunity cost.

Glossary

Term	Definition
Intra-day re-optimisation	Updating dispatch plan multiple times per day
Staleness	Plan no longer reflects current info
Gate closure	Bid deadline (~2 min before dispatch)
Model drift	Performance degrades as data distribution shifts
Retraining cadence	Schedule for retraining (monthly/quarterly)
Drawdown limit	Max loss before automated trading halts
Human-in-the-loop	Human authority over high-stakes decisions
Capture-ratio drift	Sustained decline in rolling capture ratio
Decision-regret drift	Sustained increase in actual-vs-optimal gap
AEMO dispatch cycle	5-min interval: dispatch + price setting
PD forecast	AEMO price/demand fcst; 30-min updates, 40 h
5MPD	AEMO short-term fcst; 5-min updates, 1 h
Price impact	Battery's own effect on market price
Recalibration	Update calibration layer without retraining

Summary

Live operations convert offline models into real revenue, and the gap between backtesting and production is an engineering and risk-management challenge, not a modelling one. Intra-day re-optimisation --- re-solving the dispatch plan as new pre-dispatch and 5-minute forecasts arrive, now through the Chapter 12 market emulator rather than a simple LP --- captures revenue that a stale day-ahead plan leaves on the table, with the benefit concentrated on the volatile days that dominate annual earnings. The operational loop runs on the NEM's 5-minute clock: ingest data, update the forecast, run the emulator, submit a bid before gate closure, record the outcome. Data latency is the binding constraint, not computation. Monitoring spans four dimensions --- forecast drift, calibration drift, data-quality faults, and decision-regret and capture-ratio drift --- with recalibration as the first response and full retraining reserved for confirmed structural shifts. Risk controls (drawdown limits, exposure caps, minimum state of charge) bound the downside, and a human trader retains authority over the 5% of situations that exceed the model's training distribution. For the Mannum BESS, the worked example showed that 30-minute intra-day re-optimisation lifted the capture ratio from 0.50 to 0.76 on a volatile heatwave day, recovering \$400,000 of revenue that the day-ahead plan forfeited by discharging before the largest price spike. The 0.50 capture ratio benchmark is achievable with a day-ahead plan alone; intra-day operations are what push beyond it.

16. Decision-Focused Learning

The Gap Between Accuracy and Value

Every chapter so far has measured the forecast and the dispatch as separate stages. Chapter 7 trained the forecaster on pinball loss. Chapter 9 fed the forecast into a dispatch LP. Chapter 10 evaluated the end-to-end pipeline and noted that the dispatch stage matters more than the forecast stage. This chapter closes the loop: it trains the forecaster directly on dispatch quality, so the model learns to produce the prices the LP needs, not the prices the scoring rule rewards.

The method is called **decision-focused learning** (sometimes "smart predict-then-optimize" or SPO). The idea is simple: instead of minimising forecast error, minimise the revenue gap between the forecast-driven dispatch and the dispatch you would have made with perfect foresight. That revenue gap is the **decision regret** from Chapter 14.

Decision-focused learning: A training methodology that optimises a predictive model by evaluating its predictions through the downstream decision problem they inform. Instead of measuring how close the forecast is to the truth (accuracy), it measures how much money the forecast leaves on the table compared to perfect information (decision regret). The forecast is not an end in itself — it is a means to better decisions.

Why does this matter? Because accuracy and decision quality are not the same thing, and the divergence can be large. The dispatch LP from Chapter 9 does not need the forecast to be correct everywhere. It needs the forecast to **rank the half-hours correctly** — to put the expensive intervals above the cheap ones so the battery charges in the valleys and discharges at the peaks. A forecast that misses the magnitude of a \$5,000 spike by \$2,000 but places it in the right half-hour is far more valuable than a forecast that predicts the spike's magnitude to within \$100 but places it 90 minutes too early.

The core insight: The metric that pays the desk is capture ratio, not MAE or pinball loss. A forecast trained to maximise capture ratio will allocate its representational capacity differently from one trained on accuracy — it will sacrifice precision on calm overnight periods (where dispatch decisions are trivial) to improve its discrimination during volatile afternoon and evening windows (where dispatch decisions determine revenue). Decision-focused learning is the machinery that makes this reallocation happen automatically.

The Two-Stage Baseline and Its Failure Mode

Predict, Then Optimise

The standard approach — used throughout Chapters 7 through 10 — is a two-stage pipeline:

- Predict:** Train a forecaster (GBT, LEAR, neural network) to minimise a statistical loss function — MAE, pinball loss, or CRPS — using historical price data.
- Optimise:** Feed the forecast into the dispatch LP from Chapter 9 to compute the optimal charge/discharge schedule.

The two stages are **decoupled**. The forecaster knows nothing about the dispatch problem it serves. It treats predicting a \$50 error at 3 AM (when the battery is idle) the same as predicting a \$50 error at 6 PM (when the battery is deciding whether to hold or discharge). The LP, for its part, takes whatever forecast it is given and does its best.

Two-stage (predict-then-optimize) approach: The conventional method for decision-making under uncertainty: first train a predictive model by minimising a statistical loss function, then solve an optimisation problem using the predictions as inputs. The two stages are trained and evaluated independently. The prediction stage has no knowledge of the decision stage's objective, and the decision stage has no influence over the prediction stage's training.

This decoupling is convenient — it lets you improve the forecast and the dispatch independently, test each stage in isolation, and swap components freely. But it creates a structural misalignment: the forecast is optimised for a proxy (accuracy) rather than the actual objective (revenue).

A Concrete Example of the Failure Mode

Consider the following scenario for the Mannum BESS (100 MW / 200 MWh, SA1) on a summer afternoon. The battery has 150 MWh of stored energy and needs to decide when to discharge over the next four half-hours:

Half-hour	Actual price	Forecast A (accurate)	Forecast B (less accurate)
16:00	\$180	\$175	\$120
16:30	\$160	\$165	\$110
17:00	\$450	\$200	\$280
17:30	\$800	\$220	\$500

Forecast A has lower MAE: its average absolute error is \$155, versus \$170 for Forecast B. By every standard accuracy metric, Forecast A is the better forecast.

Dispatch under Forecast A: The LP sees four periods with similar predicted prices (\$165–\$220). It spreads discharge roughly evenly, dispatching 50 MW across all four half-hours. Revenue: $50 \times 0.5 \times (\$180 + \$160 + \$450 + \$800) = \$39,750$.

Dispatch under Forecast B: The LP sees a clear price ramp — \$110, \$110, \$280, \$500. It concentrates discharge in the final two half-hours, dispatching 100 MW in each. Revenue: $100 \times 0.5 \times (\$450 + \$800) = \$62,500$.

Perfect foresight dispatch: The LP with actual prices dispatches 100 MW at 17:00 and 17:30. Revenue: $100 \times 0.5 \times (\$450 + \$800) = \$62,500$.

The less accurate Forecast B produces a dispatch that **matches perfect foresight exactly**, earning 57% more revenue than the more accurate Forecast A. Forecast B succeeds because it correctly ranks the evening periods as much more valuable than the afternoon periods, even though it gets the magnitudes wrong.

The ranking is what matters: The dispatch LP is a sorting machine. It ranks the half-hours by predicted price, discharges during the most expensive ones, and charges during the cheapest ones. If the ranking is correct, the dispatch is optimal — regardless of whether the predicted price magnitudes are right. Forecast B gets the ranking right ($17:30 > 17:00 > 16:00 > 16:30$). Forecast A gets the ranking wrong ($16:00 \approx 16:30 \approx 17:00 \approx 17:30$). The ranking error is invisible to MAE but devastating to revenue.

Figure 16.1 — Capture ratio comparison between a two-stage baseline (pinball-trained GBT) and a decision-focused model (same GBT architecture, trained on dispatch regret). Over a 90-day held-out window in SA1, the decision-focused model achieves a higher capture ratio despite comparable or slightly worse pinball loss. The improvement concentrates on volatile days where price ranking determines revenue.

Decision Regret as the Loss Function

Definition

Decision regret measures the revenue gap between the dispatch driven by a forecast and the dispatch driven by perfect foresight. It was introduced in Chapter 14 as a diagnostic; here it becomes the training objective.

Decision regret: The difference between the revenue achieved by dispatching with perfect foresight (the theoretical maximum) and the revenue achieved by dispatching on the forecast. Formally, if y is the vector of actual prices, y^* is the forecast, $z^*(y)$ is the optimal dispatch under perfect foresight, and $z^*(y^*)$ is the dispatch under the forecast, then: $\text{regret}(y^*, y) = \text{revenue}(z^*(y), y) - \text{revenue}(z^*(y^*), y)$. Regret is always non-negative (you cannot beat perfect foresight) and equals zero only when the forecast-driven dispatch matches the perfect-foresight dispatch — which happens when the forecast preserves the price ranking that determines the optimal schedule.

Several properties make regret attractive as a loss function:

It is denominated in dollars. Minimising regret is literally minimising lost revenue. There is no gap between what the loss measures and what the desk cares about.

It is asymmetric in a useful way. A forecast error during a calm period — when the battery would be idle under both the forecast and perfect foresight — incurs zero regret. A forecast error that misranks a spike incurs large regret. The loss function automatically concentrates on the intervals that matter.

It depends on the battery's specifications. A regret-trained model for the Mannum BESS (100 MW / 200 MWh, 2h duration) learns a different price-ranking strategy than one trained for a 4h battery, because different batteries have different charge/discharge capacity constraints and hence different optimal schedules.

Regret is battery-specific. A model trained to minimise regret for the Mannum BESS is tailored to its 2h duration, 90% round-trip efficiency, and 100 MW power limit. If you deploy the same forecast on a different battery (say, a 4h system), the model may underperform because it has learned rankings that are optimal for 2h dispatch windows. This is both a strength (the model is optimally adapted to its operational context) and a limitation (it is not transferable without retraining).

From Accuracy-Based Losses to Decision Regret

The standard losses from Chapter 7 treat all half-hours equally:

$$\text{MAE} = (1/T) \cdot \sum_{t=1}^T |y_t - y_t^*|$$

$$\text{Pinball}_\tau = (1/T) \cdot \sum_{t=1}^T \max(\tau \cdot (y_t - y_t^*), (\tau - 1) \cdot (y_t - y_t^*))$$

Both losses penalise errors uniformly across time. A \$100 error at 3 AM contributes the same to the loss as a \$100 error at 6 PM.

Decision regret, by contrast, weights errors by their dispatch consequences:

$$\text{regret} = \text{revenue}(z^*(y), y) - \text{revenue}(z^*(y^*), y)$$

This is not a per-period decomposition — it is a **holistic** measure that depends on the entire price trajectory and the entire dispatch schedule. An error at 3 AM contributes zero to regret if the battery is idle at 3 AM under both the forecast and perfect foresight. An error at 6 PM that causes the battery to hold when it should discharge contributes the entire missed revenue to regret.

The implication for training: a regret-minimising forecaster learns to allocate its representational capacity to the periods where the dispatch LP's decision is most sensitive to the forecast. These are typically the periods near the charge/discharge threshold — where a small change in the predicted price flips the LP's decision between "charge," "idle," and "discharge."

Where the model chooses to be wrong: A regret-trained model for the Mannum BESS might learn to systematically overpredict overnight prices (making its MAE worse) in order to improve its discrimination between evening peaks and near-peak periods (making its dispatch revenue better). This is not a failure — it is an optimal allocation of limited model capacity. The overnight overprediction costs nothing in dispatch terms because the battery is charging during those hours regardless of the exact price. The improved peak discrimination earns real revenue because it determines whether the battery discharges at \$300 or holds for \$800.

Differentiable Optimisation Layers

The Challenge: Backpropagating Through an Argmin

To train the forecaster on decision regret, we need to compute the gradient of the regret with respect to the forecast parameters. The computation graph is:

forecast parameters $\theta \rightarrow$ forecast $y^* \rightarrow$ dispatch $z^*(y^*) \rightarrow$ regret(y^*, y)

The first step ($\theta \rightarrow y^*$) is a standard neural network forward pass — differentiable by construction. The last step ($z^* \rightarrow$ regret) is arithmetic — also differentiable. The problem is the middle step: $z^*(y^*) = \text{argmin LP}(y^*)$.

The LP from Chapter 9 is an optimisation problem. Its output — the optimal dispatch schedule — is the result of an **argmin** operation. We need the gradient:

$$\partial z^* / \partial y^* = \partial / \partial y^* [\text{argmin}_z -\sum_t y_t^* \cdot z_t \text{ subject to battery constraints}]$$

This is the derivative of the optimal solution with respect to the problem's parameters (the predicted prices). If the forecast changes by a small amount, how does the optimal dispatch change?

Differentiable optimisation layer: A component in a neural network computation graph that solves a mathematical optimisation problem (e.g., a linear or quadratic program) as its forward pass, and computes the gradient of the optimal solution with respect to the problem parameters as its backward pass. This allows the optimisation problem to be embedded within a larger end-to-end differentiable pipeline, enabling gradient-based training of upstream parameters (such as forecast model weights) with respect to downstream objectives (such as dispatch revenue).

The Implicit Function Theorem

The mathematical machinery for computing $\partial z^* / \partial y^*$ is the **implicit function theorem**. The idea is that at the optimal solution, the KKT (Karush-Kuhn-Tucker) conditions of the optimisation problem are satisfied. These conditions implicitly define z^* as a function of the parameters y^* . By differentiating the KKT conditions with respect to y^* , we obtain a system of linear equations whose solution gives the desired gradient.

Implicit function theorem: A mathematical result that provides conditions under which a system of equations $F(x, y) = 0$ implicitly defines y as a differentiable function of x , and gives a formula for the derivative dy/dx . Applied to optimisation: the KKT conditions of a parameterised optimisation problem $F(z^*, y^*) = 0$ implicitly define the optimal solution z^* as a function of the parameters y^* . The implicit function theorem then gives $\partial z^* / \partial y^* = -(\partial F / \partial z^*)^{-1} \cdot (\partial F / \partial y^*)$, which is the gradient of the optimal decision with respect to the forecast.

For a convex optimisation problem with inequality constraints (like the dispatch LP), the KKT conditions are:

$$\nabla_z L(z^*, \lambda^*, y^*) = 0 \text{ (stationarity)} \quad \lambda_i^* \cdot g_i(z^*, y^*) = 0 \text{ (complementary slackness)} \quad g_i(z^*, y^*) \leq 0 \text{ (primal feasibility)} \quad \lambda_i^* \geq 0 \text{ (dual feasibility)}$$

where L is the Lagrangian, λ^* are the optimal dual variables (shadow prices from Chapter 9), and g_i are the inequality constraints (battery power limits, state-of-charge bounds, etc.).

Differentiating the stationarity and complementary slackness conditions with respect to y^* gives a linear system that can be solved for $\partial z^* / \partial y^*$. The key insight: **you do not need to re-solve the optimisation problem to compute the gradient**. You solve the LP once in the forward pass, keep the optimal primal and dual solutions, and then solve a (typically much cheaper) linear system in the backward pass.

Available Tools

Several libraries implement differentiable optimisation layers:

Library	Problems	Gradient method	Notes
cvxpylayers	Any DCP	Conic implicit diff	CVXPY interface; broadcast
qpth	QPs	KKT implicit diff	Fast; suits dispatch
OptNet	QPs in NNs	Same as qpth	PyTorch layer

cvxpylayers: A Python library that converts CVXPY convex optimisation problems into differentiable PyTorch or JAX layers. The forward pass solves the optimisation problem. The backward pass computes gradients of the optimal solution with respect to the problem parameters using the implicit function theorem applied to the KKT conditions of the problem's conic form. This enables end-to-end training of a forecaster that feeds into a convex dispatch problem.

The practical workflow is:

1. **Define the dispatch LP in CVXPY** (as done in Chapter 9), with the price forecast as a parameter (not a constant).

2. **Wrap the LP in a cvxpylayers layer.** This converts the LP into a differentiable PyTorch module.
3. **In the forward pass:** the forecaster produces a price prediction, which is fed into the cvxpylayers layer. The layer solves the LP and returns the optimal dispatch.
4. **Compute regret** from the optimal dispatch and the actual prices.
5. **Call .backward().** PyTorch's autograd calls the cvxpylayers backward pass, which uses implicit differentiation to compute $\partial z^*/\partial y^*$, and chains this with the forecaster's gradient to get $\partial \text{regret}/\partial \theta$.
6. **Update θ** with a standard optimiser (Adam, SGD).

Analogy — training through a sorting machine: Imagine you are training someone to predict the order of finish in a horse race, and the scoring rule depends on how much money you make from betting on their predicted finish order. Standard training teaches them to predict each horse's speed accurately (accuracy-based loss). Decision-focused training instead says: "I don't care how fast you predict each horse to be — I care whether your predicted finish order matches the actual finish order in the positions where I placed my bets." The implicit function theorem is the mathematical machinery that tells the predictor: "If you shifted your prediction for horse 3 up by one second, the predicted finish order would change in this specific way, and that would affect the betting outcome by this much."

The Vanishing-Gradient Problem

Why Linear Objectives Make Gradients Vanish

The dispatch LP from Chapter 9 is a **linear** program: the objective is linear in both the prices and the dispatch decisions:

$$\text{maximise } \sum_t y_t^* \cdot (\text{discharge}_t - \text{charge}_t) \cdot \Delta t$$

subject to battery constraints (power limits, state-of-charge bounds, efficiency).

Because the objective is linear and the constraints are linear, the feasible region is a **polyhedron** (a multi-dimensional polygon), and the optimal solution always occurs at a **vertex** of this polyhedron. As the price forecast y^* varies continuously, the optimal dispatch $z^*(y^*)$ does not vary continuously — it **jumps** from one vertex to another.

Piecewise-constant solution: The optimal solution of a linear program is a piecewise-constant function of its parameters. As the parameters (prices) change smoothly, the optimal solution remains at the same vertex of the feasible polyhedron until a critical threshold is crossed, at which point it jumps discontinuously to a different vertex. Between jumps, the gradient $\partial z^*/\partial y^*$ is exactly zero — the dispatch does not change in response to small forecast perturbations.

This is devastating for gradient-based training. Consider a single half-hour period t . The LP's decision for period t is typically:

- **Discharge at full power** if y_t^* is above some threshold (determined by the other prices and the battery constraints)
- **Charge at full power** if y_t^* is below some threshold
- **Idle** if y_t^* is between the two thresholds

For most values of y_t^* , a small perturbation does not change the LP's decision — the battery was already discharging (or charging, or idle) and continues to do so. The gradient $\partial z_t^*/\partial y_t^*$ is exactly zero. The gradient is non-zero only at the exact threshold prices where the LP's decision switches — a set of measure zero.

The vanishing-gradient problem for linear objectives: Because the dispatch LP has a linear objective, the optimal dispatch is a piecewise-constant function of the forecast. Gradients are zero almost everywhere and undefined at the jump points. Standard gradient-based training (backpropagation through the LP) produces zero gradients in almost every training step, making learning impossible. This is not a numerical issue — it is a fundamental property of linear programming.

Figure 16.2 — Left: the optimal dispatch as a function of the predicted price for a single half-hour, under a linear objective (LP). The dispatch is piecewise-constant — the gradient is zero everywhere except at the two thresholds (charge/idle and idle/discharge). Right: adding a small quadratic regulariser turns the LP into a QP, making the dispatch a smooth function of the predicted price. The gradient is now non-zero everywhere, providing a useful training signal for backpropagation.

Fix 1: Quadratic Regularisation

The first fix is mechanical: add a small quadratic penalty to the LP's objective, converting it into a **quadratic program** (QP). Instead of:

$$\text{maximise } \sum_t y_t^* \cdot (\text{discharge}_t - \text{charge}_t) \cdot \Delta t$$

solve:

$$\text{maximise } \sum_t y_t^* \cdot (\text{discharge}_t - \text{charge}_t) \cdot \Delta t - (\gamma/2) \cdot \sum_t (\text{discharge}_t^2 + \text{charge}_t^2)$$

The added term $-(\gamma/2) \cdot \sum_t (\text{discharge}_t^2 + \text{charge}_t^2)$ is a **quadratic regulariser** that penalises extreme dispatch decisions. The parameter $\gamma > 0$ controls the strength of the regularisation.

Quadratic regularisation: The addition of a small quadratic penalty term to the objective function of a linear program, converting it into a quadratic program. For the dispatch problem, the penalty $-(\gamma/2) \cdot \sum_t (\text{discharge}_t^2 + \text{charge}_t^2)$ discourages the optimiser from concentrating all dispatch in a few periods. The penalty is not physically motivated — its purpose is purely computational: to make the optimal solution a smooth, differentiable function of the forecast, thereby providing non-zero gradients for training.

Why does this work? A quadratic objective on a polyhedral feasible set has a **unique** optimal solution that varies **smoothly** with the problem parameters. The optimal dispatch is no longer piecewise-constant — it transitions gradually between charge, idle, and discharge as the predicted price changes. This smoothness means the gradient $\partial z^*/\partial y^*$ is non-zero almost everywhere, providing a useful training signal.

Mathematically, the KKT conditions of the QP include:

$$y_t^* - \gamma \cdot \text{discharge}_t + \mu_t = 0 \text{ (for discharge)} \quad -y_t^* - \gamma \cdot \text{charge}_t + \nu_t = 0 \text{ (for charge)}$$

where μ_t and ν_t are dual variables. Differentiating with respect to y_t^* gives:

$$\partial \text{discharge}_t / \partial y_t^* = 1/\gamma \text{ (when the constraint is not active)}$$

The gradient is $1/\gamma$ — non-zero, and inversely proportional to the regularisation strength. Larger γ gives smaller gradients (more regularisation smooths the solution more, so it responds less to price perturbations). Smaller γ gives larger gradients but recovers the LP's piecewise-constant behaviour in the limit.

The trade-off is fundamental:

γ	Gradients	Dispatch	Training
Very small ($\rightarrow 0$)	Large, noisy	Near-optimal	Unstable
Moderate	Moderate, stable	Slightly suboptimal	Smooth convergence
Very large	Small, stable	Significantly suboptimal	Slow; distorted

The regularisation trade-off: Quadratic regularisation introduces a bias-variance trade-off in the optimisation layer. Too little regularisation gives vanishing or unstable gradients (high variance). Too much regularisation distorts the dispatch away from the LP's optimal solution (high bias). In practice, γ is tuned on a validation set by measuring the decision-focused model's capture ratio — the same metric that ultimately matters. Typical values for the Mannum BESS dispatch problem are $\gamma \in [0.01, 1.0]$, with the best performance often around $\gamma = 0.1$.

In practice, the QP regulariser has a second benefit: it resolves the LP's **degeneracy**. The dispatch LP often has multiple optimal solutions — for example, when two half-hours have the same predicted price, the LP is indifferent between discharging in either one. Degeneracy makes the LP's solution non-unique and its gradient undefined. The quadratic regulariser breaks the tie by preferring the solution with smaller dispatch magnitudes, guaranteeing a unique solution and a well-defined gradient.

Fix 2: The SPO+ Surrogate Loss

The second fix is more elegant: instead of modifying the optimisation problem (adding regularisation to the LP), modify the **loss function** to have useful gradients even though the underlying LP is unmodified.

The **SPO+ loss** (Smart Predict-then-Optimise plus), introduced by Elmachet and Grigas (2022), is a convex surrogate for decision regret that has non-zero gradients even when the LP's optimal solution is piecewise-constant.

SPO+ loss: A convex surrogate loss function for decision regret in predict-then-optimize problems with linear objectives. Given a forecast y^* , the actual prices y , and the optimal decision under actual prices $z^*(y)$, the SPO+ loss is: $L_{\text{SPO+}}(y^*, y) = \max_z [(2y^* - y)^T \cdot z] - 2(y^*)^T \cdot z^*(y^*) + y^T \cdot z^*(y)$. The key property: the gradient of the SPO+ loss with respect to the forecast y^* is $2 \cdot (z^*_{\text{SPO}} - z^*(y^*))$, where z^*_{SPO} is the optimal dispatch under the modified cost vector $(2y^* - y)$. This gradient is non-zero whenever the forecast-driven dispatch differs from the SPO dispatch, providing a consistent training signal.

The intuition behind SPO+ is as follows. Decision regret itself is hard to differentiate because it depends on $z^*(y^*)$, which is piecewise-constant. The SPO+ loss constructs a **tight upper bound** on regret that is convex in the forecast y^* . Because it is convex, it has well-defined subgradients everywhere, and gradient descent on SPO+ drives down the actual regret.

The construction works by creating a **surrogate cost vector** $(2y^* - y)$ that amplifies the forecast's errors. If the forecast overpredicts a price ($y_t^* > y_t$), the surrogate cost $2y_t^* - y_t$ is even larger, exaggerating the overprediction. If the forecast underpredicts ($y_t^* < y_t$), the surrogate cost is smaller (possibly negative), emphasising the underprediction. The LP is then solved with this surrogate cost vector instead of the original forecast. Because the surrogate cost exaggerates forecast errors, the resulting dispatch z^*_{SPO} differs from $z^*(y^*)$ whenever the forecast is imperfect, and the difference provides the gradient direction.

SPO+ in action: Suppose the actual price for 17:30 is \$800, and the forecast is $y_{17:30}^* = \$300$ (a significant underprediction). The surrogate cost is $2 \times \$300 - \$800 = -\$200$. The LP under the surrogate cost thinks 17:30 is a negative-price period and avoids discharging there. But the LP under the original forecast thinks 17:30 is a moderately expensive period and does discharge there. The gradient $2 \cdot (z^*_{\text{SPO}} - z^*(y^*))$ points in the direction of "increase the predicted price for 17:30" — which is exactly the correction needed to improve the dispatch.

Comparing the two fixes:

Property	QP regularisation	SPO+ loss
Modifies LP?	Yes (quadratic penalty)	No
Dispatch quality	Slightly suboptimal	Optimal
Gradients via	KKT implicit diff	Surrogate LP
Hyperparameter	γ	None
Theory	Heuristic	Convex, consistent

Complexity	Needs qpth solver	Standard LP solver
------------	-------------------	--------------------

Practical recommendation: For battery dispatch, QP regularisation is the simpler starting point — the dispatch LP from Chapter 9 can be converted to a QP with a single additional term, and qpth or cvxpylayers handles the differentiation. SPO+ is theoretically cleaner (no dispatch distortion, no hyperparameter) but requires a more involved training loop because the SPO+ gradient is computed outside the standard autograd framework. Both methods produce decision-focused models that significantly outperform the two-stage baseline on capture ratio.

The Training Loop

Computation Graph

The decision-focused training loop connects four components into a single end-to-end differentiable pipeline:

Forecaster (parameterised by θ): Takes features x_t (weather, demand, lags, calendar variables — all the features from Chapters 3–7) and produces a price forecast $y^* = f_\theta(x)$.

Differentiable dispatch layer: Takes the forecast y^* and solves the dispatch QP (or LP, if using SPO+) to produce the optimal schedule $z^* = \arg\min \text{QP}(y^*)$.

Regret computation: Takes the dispatch z^* , the actual prices y , and the perfect-foresight dispatch $z^*(y)$ to compute the regret loss $L = \text{revenue}(z^*(y), y) - \text{revenue}(z^*, y)$.

Gradient computation and parameter update: Backpropagates $\partial L / \partial \theta$ through all three components and updates the forecaster parameters θ .

Figure 16.3 — The computation graph of decision-focused training. Features x flow into the forecaster f_θ , producing a price forecast y^* . The forecast enters a differentiable dispatch layer (QP), which outputs the optimal schedule z^* . The schedule is evaluated against actual prices y to compute regret. The gradient of regret with respect to the forecaster parameters θ flows backward through the entire graph: from the regret, through the dispatch layer (via implicit differentiation), and into the forecaster.

Step-by-Step Training Procedure

Initialisation: - Pre-train the forecaster using pinball loss (Chapter 7–8) to provide a reasonable starting point. Starting from a random initialisation with the decision-focused loss often fails because the initial forecasts are so poor that the dispatch layer produces degenerate schedules. - Set the QP regularisation parameter γ (if using QP regularisation) or prepare the SPO+ gradient computation (if using SPO+). - Compute the perfect-foresight dispatch $z^*(y)$ for all training days. This is done once and cached, since it depends only on the actual prices and the battery specifications.

Training loop (for each epoch):

```
For each training day d: 1. Forward pass through forecaster:  $y^* = f_\theta(x_d)$  # predicted prices for day d
2. Forward pass through dispatch layer:  $z^*(y^*) = \text{solve\_QP}(y^*, \text{battery\_specs})$  # optimal dispatch under forecast
3. Compute regret:  $L_d = \text{revenue}(z^*(y_d), y_d) - \text{revenue}(z^*(y^*), y_d)$ 
4. Backward pass:  $\partial L_d / \partial \theta = \partial L_d / \partial z^* \cdot \partial z^* / \partial y^* \cdot \partial y^* / \partial \theta$  #  $\uparrow$  arithmetic  $\uparrow$  implicit diff  $\uparrow$  autograd
5. Update parameters:  $\theta \leftarrow \theta - \eta \cdot \partial L_d / \partial \theta$ 
```

Pre-training: Initialising the forecaster by training it on a standard statistical loss (pinball loss, MAE) before switching to the decision-focused loss. Pre-training provides a warm start — a reasonable forecast that the decision-focused loss can refine. Without pre-training, the initial forecasts may be so poor that the dispatch layer always produces the same degenerate schedule (e.g., fully charge or fully idle), giving zero or near-zero gradients and preventing learning.

Practical Considerations

Batch size and training cost. Each training step requires solving a QP (or two LPs, if using SPO+). For a 48-period dispatch problem, each QP takes approximately 5–20 ms on a CPU. A training set of 365 days (one year) with 10 epochs costs $365 \times 10 \times 20 \text{ ms} \approx 73$ seconds of QP solve time. This is manageable on a laptop but much slower than training the same forecaster on pinball loss (which requires no optimisation solves). Batch sizes of 16–32 days balance gradient quality against computation time.

Validation and early stopping. Validate on capture ratio, not on the training loss. The QP regulariser introduces a gap between the training loss (regret under the regularised QP) and the deployment metric (capture ratio under the unregularised LP). Monitor capture ratio on a held-out validation set and stop training when it plateaus.

The rolling-origin backtest from Chapter 5. Decision-focused training uses the same rolling-origin protocol as every other model in the course. The training set expands as the window rolls forward, and the embargo prevents leakage. The only difference is that the training loss is regret rather than pinball loss.

Learning rate. The gradient $\partial z^* / \partial y^*$ scales as $1/\gamma$ (for QP regularisation), which can produce very large gradients when γ is small. Use a small learning rate ($\eta \approx 1e-4$ to $1e-3$) and gradient clipping to prevent instability.

Training timeline for the Mannum BESS: Using a two-year training window (FY2024–FY2025) with 90-day held-out validation, a small GBT-based forecaster (128 leaf nodes, 100 iterations) wrapped in a differentiable dispatch QP with $\gamma = 0.1$: - Pre-training (pinball loss): 2 minutes - Decision-focused fine-tuning (10 epochs): 8 minutes - Validation (capture ratio on 90-day held-out): 30 seconds - Total wall time: under 11 minutes on a MacBook M2 This is fast enough for daily retraining, which the rolling-origin protocol requires.

Connection to the Methods Hierarchy

Where Decision-Focused Learning Sits

Chapter 12 introduced a methods hierarchy for battery dispatch, ranging from simple heuristics to sophisticated learning-based approaches. Decision-focused learning occupies a specific position in this hierarchy, and understanding that position prevents two common misunderstandings.

The methods hierarchy (from Chapter 12): 1. **Convex optimisation** (LP, QP, stochastic programming) — the core dispatch engine. Solves the dispatch problem given a price forecast. This is what Chapters 5 and 9 built. 2. **Decision-focused learning** — wraps the convex core to train the forecaster. It uses convex optimisation as a subroutine; it does not replace it. This is what the current chapter develops. 3. **Reinforcement learning and genetic search** — for the price-maker emulator from Chapter 12, where the battery's dispatch affects market prices and the optimisation problem is no longer convex. Not used here because the Mannum BESS is a price taker (100 MW in a market with ~30 GW of capacity). Each level uses the one below it. Decision-focused learning calls the convex dispatch solver on every forward pass. The price-maker emulator calls the convex solver within a simulation loop. The hierarchy is not a ranking — each level is appropriate for a different problem.

Misunderstanding 1: "Decision-focused learning replaces the dispatch LP." No. The dispatch LP (or QP) is solved on every forward pass of training and on every dispatch decision in deployment. Decision-focused learning adds a training signal; it does not change the dispatch mechanism. In deployment, the trained forecaster produces a price prediction, and the standard LP from Chapter 9 computes the dispatch — exactly as before. The only difference is that the forecaster has been trained with a different loss function.

Misunderstanding 2: "Decision-focused learning is reinforcement learning." No. Reinforcement learning (RL) learns a **policy** — a direct mapping from state to action — by interacting with an environment and observing rewards. Decision-focused learning learns a **forecast** — a mapping from features to predicted prices — and still uses an explicit optimiser (the LP) to convert the forecast into a dispatch decision. The distinction matters: the LP provides structure, constraints, and guarantees that RL must learn from scratch. RL is appropriate when the optimisation problem is non-convex or the environment is too complex to model explicitly (as in the price-maker emulator from Chapter 12). For price-taker dispatch, the LP is both faster and more reliable.

Price taker vs. price maker: A price taker is a market participant whose actions do not affect market prices — the participant is too small relative to the market. A price maker is large enough that its dispatch decisions influence the clearing price. The Mannum BESS (100 MW in a ~30 GW market) is a price taker: its charge/discharge decisions do not move the SA1 price. Price-taker dispatch is a convex optimisation problem (the LP from Chapter 9). Price-maker dispatch is non-convex because the prices depend on the dispatch, creating a feedback loop that requires the emulator from Chapter 12.

Decision-Focused Learning and FCAS

The FCAS-aware dispatch from Chapter 11 co-optimises energy arbitrage and frequency control ancillary services. Decision-focused learning extends naturally to this setting: the dispatch layer becomes an FCAS-aware QP (rather than a pure-arbitrage QP), and the regret is defined as the gap in total revenue (arbitrage + FCAS) between the forecast-driven dispatch and perfect foresight.

The key subtlety: the FCAS-aware dispatch has more decision variables (energy dispatch and FCAS capacity in each period) and more constraints (minimum FCAS enablement, contingency reserves). This makes the QP larger and the implicit differentiation more expensive, but the conceptual framework is identical. The forecaster learns to predict prices (both energy and FCAS prices) in a way that maximises total dispatch revenue, automatically learning the trade-offs between energy arbitrage and FCAS provision.

Worked Example: Decision-Focused Training for the Mannum BESS

Setup

We train two models with identical architectures and compare their capture ratios:

Model A (two-stage baseline): A gradient-boosted tree (GBT) forecaster trained on pinball loss at quantile $\tau = 0.5$ (median), using the QRA combination from Chapter 8. The forecast is fed into the standard dispatch LP from Chapter 9. This is the baseline from the capstone (Chapter 10).

Model B (decision-focused): The same GBT architecture, pre-trained on pinball loss, then fine-tuned using decision regret with a quadratically regularised dispatch layer ($\gamma = 0.1$).

Battery specifications (Mannum BESS): - Power: 100 MW - Energy: 200 MWh (2h duration) - Round-trip efficiency: 90% (LFP chemistry) - Cycle limit: none enforced (LFP degradation is minimal at 1–2 cycles per day) - Region: SA1

Data: - Training: 1 July 2024 – 30 June 2025 (365 days) - Validation: 1 July 2025 – 30 September 2025 (92 days) - Test: 1 October 2025 – 31 December 2025 (92 days, coinciding with the first operational quarter of the Mannum BESS)

Results

Metric	Model A (two-stage)	Model B (decision-focused)
Pinball loss ($\tau = 0.5$)	\$18.2/MWh	\$19.8/MWh

MAE	\$24.1/MWh	\$26.3/MWh
Capture ratio (test)	0.68	0.74
Annual arbitrage revenue (extrapolated)	\$8.2M	\$8.9M
Revenue improvement	—	+\$700K/year

The headline result: The decision-focused model has **worse** accuracy (higher MAE and pinball loss) but **better** economic performance (higher capture ratio). It earns an estimated \$700K more per year for the Mannum BESS — a 9% increase in arbitrage revenue — by learning to forecast prices in a way that helps the dispatch LP, even at the cost of traditional forecast quality.

Where the Improvement Comes From

Examining the two models' forecasts reveals a systematic pattern. The decision-focused model produces forecasts that are:

Sharper at the peaks. During afternoon and evening ramps (15:00–20:00), the decision-focused model predicts larger price spreads between consecutive half-hours. This helps the LP time its discharge precisely, concentrating output in the most expensive half-hours rather than spreading it across the ramp.

Flatter during calm periods. During overnight hours (22:00–06:00) and mild midday periods, the decision-focused model's forecasts are less accurate — it predicts nearly constant prices, even when actual prices have small fluctuations. This is rational: the battery is charging during these hours regardless, and predicting the exact charging price does not affect the dispatch decision.

More aggressive on spike detection. When a spike is likely, the decision-focused model overpredicts its magnitude relative to surrounding hours. The overprediction is inaccurate (worsening MAE) but useful: it ensures the LP reserves enough capacity to discharge fully during the spike.

Figure 16.4 — Scatter plot of daily MAE (x-axis, lower is more accurate) versus daily capture ratio (y-axis, higher is more profitable) for the two models over the 92-day test period. Blue dots: two-stage model. Orange dots: decision-focused model. The two clouds overlap in accuracy but the decision-focused model's cloud sits higher in capture ratio. On volatile days (large dots), the separation is most pronounced — the decision-focused model captures more spike revenue despite comparable or worse accuracy.

Interpretation

The decision-focused model learns an implicit **attention mechanism**: it allocates its forecast accuracy to the periods where accuracy matters for dispatch. This is not a modification to the model architecture — the GBT has the same number of trees, leaves, and features in both cases. The difference is entirely in the loss function used during training, which reshapes what the model considers "important" to predict.

This is the practical realisation of the principle stated in Chapter 10: optimising the pipeline end-to-end can improve the final output (revenue) even when individual stages (forecast accuracy) appear to get worse.

Benchmark: Decision-Focused vs. Two-Stage on the Held-Out Period

Experimental Protocol

Following the honest benchmarking rules from Chapter 5, we evaluate both models on the fixed held-out test period (Q4 2025) using the rolling-origin backtest with the same embargo, features, and battery specifications. The only difference is the training loss: pinball loss (two-stage) vs. decision regret (decision-focused).

Results by Day Type

Day type	N days	Two-stage CR	Decision-focused CR	Improvement
Low volatility (IQR < \$50)	38	0.72	0.73	+1 pp
Medium volatility (\$50 ≤ IQR < \$200)	35	0.67	0.72	+5 pp
High volatility (IQR ≥ \$200)	19	0.61	0.76	+15 pp
All days	92	0.68	0.74	+6 pp

The improvement concentrates on volatile days — exactly the days where the most revenue is at stake. On low-volatility days, both models perform similarly because the dispatch decision is easy (charge overnight, discharge during a predictable afternoon peak). On high-volatility days, the price ranking is uncertain, and the decision-focused model's advantage in ranking accuracy translates directly into capture ratio.

Statistical Significance

A paired Diebold-Mariano test on daily regret (the same test used in Chapter 10) rejects the null hypothesis of equal capture ratios at $p < 0.01$. The improvement is not due to a few lucky days — the decision-focused model produces lower regret on 67 of the 92 test days (73% win rate).

Comparison with AEMO Pre-Dispatch

For context, a naive MPC using AEMO pre-dispatch forecasts (the public benchmark from Chapter 5) achieves a capture ratio of 0.58 on the same test period. The two-stage model improves by 10 percentage points (0.68), and the decision-focused model improves by a further 6 percentage points (0.74). In dollar terms for the Mannum BESS, the progression is:

Forecast	CR	Estimated annual revenue	Improvement over AEMO
AEMO pre-dispatch	0.58	\$7.0M	—
Two-stage GBT (pinball)	0.68	\$8.2M	+\$1.2M
Decision-focused GBT (regret)	0.74	\$8.9M	+\$1.9M

The capture ratio is the metric that pays. The progression from AEMO pre-dispatch (0.58) to two-stage GBT (0.68) to decision-focused GBT (0.74) represents a 16 percentage-point improvement in capture ratio, worth approximately \$1.9M per year in additional arbitrage revenue for the Mannum BESS. Each percentage point of capture ratio is worth roughly \$120K per year — the same order of magnitude as the estimate from Chapter 9.

Exercises

Exercise 1: The Regularisation Trade-Off

Problem: Evaluate the decision-focused model at five values of γ : {0.001, 0.01, 0.1, 1.0, 10.0} using the Mannum BESS specifications.

1. Compute the average gradient magnitude $|\partial L / \partial y^*|$ across the training set.
2. Compute the capture ratio on the validation set.
3. Compute the dispatch suboptimality (gap between QP and LP dispatch on the same forecast, averaged over validation).
4. Plot all three quantities vs. γ on a log scale. Identify the sweet spot.

Worked solution

γ	Avg gradient magnitude	Capture ratio (val)	Dispatch suboptimality (%)
0.001	0.02	0.66	0.1%
0.01	0.15	0.71	0.3%
0.1	0.85	0.74	1.2%
1.0	2.1	0.72	4.8%
10.0	3.8	0.64	18.5%

Sweet spot: $\gamma = 0.1 \rightarrow$ CR 0.74, suboptimality only 1.2%

The capture ratio is non-monotone — it peaks where the gradient is strong enough to learn but dispatch distortion remains tolerable. Gradient magnitude and suboptimality both increase monotonically with γ .

Key insight: γ is a computational parameter controlling landscape smoothness, not a statistical regulariser. Its optimal value depends on battery size (larger batteries tolerate more γ) and price volatility (more volatile prices tolerate larger γ because correct rankings are more distinct).

Exercise 2: Where the Model Chooses to Be Wrong

Problem: Compare forecast errors of the decision-focused (DF) and two-stage models, period by period.

1. For each half-hour of the day, compute average absolute error for both models over the test period.
2. For each half-hour, compute average absolute regret contribution. (Approximate by re-solving the LP with actual price substituted at one period at a time and measuring regret change.)
3. Identify periods where the DF model is more/less accurate. Is there a pattern?

Worked solution

DF model more accurate (lower MAE): 15:00-20:00 (peak/ramp periods)

DF model less accurate (higher MAE): 00:00-06:00 (overnight), 10:00-14:00 (midday trough)

Similar accuracy: 06:00-10:00, 20:00-00:00

Regret contribution by period:

Period	MAE (2-stg)	MAE (DF)	Regret (2-stg)	Regret (DF)
00–06	\$12.3	\$16.8	\$0	\$0
06–10	\$18.7	\$18.2	\$120	\$95
10–14	\$22.4	\$28.1	\$85	\$90

14–18	\$31.5	\$25.8	\$2,400	\$1,100
18–22	\$28.9	\$24.3	\$1,800	\$850
22–00	\$15.6	\$17.2	\$180	\$150

Peak period (14:00–18:00): DF saves ~\$1,300/day in regret Over 92-day test period: this concentrated improvement accounts for most of the CR difference

The DF model reallocates accuracy from dispatch-insensitive periods (overnight charging, midday trough) to dispatch-sensitive periods (afternoon-evening peak). Higher overall MAE is not a concern if errors concentrate where the battery is idle.

Exercise 3: SPO+ vs. QP Regularisation

Problem: Implement the SPO+ training loop for the Mannum BESS dispatch problem and compare with QP regularisation.

1. For each training epoch, compute both SPO+ loss and QP-regularised regret. Do they track each other?
2. Compare final capture ratios on the test set.
3. Compare training wall times.

Worked solution

The SPO+ loop differs in two ways:

1. **Forward pass:** Solve the standard LP (no regularisation) — exact optimal dispatch, no distortion.
2. **Gradient computation:** Compute explicitly without KKT differentiation:
3. Surrogate cost: $c_SPO = 2y^* - y$
4. Solve LP with surrogate costs: $z^*_SPO = \operatorname{argmin}_z (-c_SPO^T z)$ subject to battery constraints
5. Gradient: $dL_SPO+/dy^* = 2 * (z^*_SPO - z^*(y^*))$

Requires only two LP solves and a subtraction — no matrix inversions.

Metric	QP regularisation ($\gamma = 0.1$)	SPO+
Capture ratio (test)	0.74	0.75
Pinball loss (test)	\$19.8/MWh	\$20.1/MWh
Training wall time (10 epochs)	8 min	6 min
Hyperparameters to tune	γ , learning rate	Learning rate only

SPO+ advantage: +0.01 CR, 25% faster training, one fewer hyperparameter

The main advantage of SPO+ is practical: one fewer hyperparameter and the deployment LP is identical to the training LP. The two losses track each other loosely during training (SPO+ is an upper bound on regret). Speed advantage grows with problem size (longer horizons, FCAS co-optimisation).

Glossary

Term	Definition
Decision-focused learning	Train forecaster via downstream decision quality
SPO	Smart predict-then-optimize (Elmachtoub & Grigas)
SPO+ loss	Convex surrogate for decision regret
Decision regret	Revenue gap vs perfect-foresight dispatch
Diff. optimisation layer	Solves optimisation fwd; implicit diff bwd
Implicit differentiation	Gradients of optimal solution via KKT diff
Argmin differentiation	Gradient of argmin of parameterised program
Quadratic regularisation	Quadratic penalty → smooth, differentiable dispatch
KKT conditions	Optimality conditions for constrained convex programs
Piecewise-constant soln	LP dispatch jumps at thresholds; zero grad elsewhere
Pre-training	Init on standard loss before decision-focused loss
cvxpylayers	Differentiable CVXPY layers (PyTorch/JAX)
qpth	Differentiable QP solver for PyTorch
Price taker	Participant whose actions don't affect prices

Capture ratio	Actual revenue / perfect-foresight revenue
---------------	--

Summary

Decision-focused learning closes the gap between what the forecaster optimises (accuracy) and what the trading desk is paid for (capture ratio). The two-stage predict-then-optimize approach — used throughout Chapters 7–10 — decouples the forecaster from the dispatch problem, training the model to minimise pinball loss or MAE without knowledge of how the forecast will be used. Decision-focused learning reconnects the two stages by training the forecaster directly on decision regret: the revenue gap between the forecast-driven dispatch and the perfect-foresight dispatch. The technical challenge is backpropagating through the dispatch LP, which requires computing the gradient of an argmin. For linear programs, this gradient is zero almost everywhere because the optimal dispatch is a piecewise-constant function of the forecast. Two fixes restore gradient signal: quadratic regularisation converts the LP into a QP with smooth, differentiable solutions, while the SPO+ surrogate loss of Elmachtoub and Grigas provides a convex upper bound on regret with non-zero gradients everywhere. Both fixes are implemented in standard differentiable optimisation libraries (cvxpylayers, qpth). The training loop pre-trains the forecaster on pinball loss for a warm start, then fine-tunes on decision regret through the differentiable dispatch layer. For the Mannum BESS (100 MW / 200 MWh, SA1), the decision-focused model achieves a capture ratio of 0.74 versus 0.68 for the two-stage baseline — a 6 percentage-point improvement worth approximately \$700K per year in additional arbitrage revenue — despite having slightly worse MAE and pinball loss. The improvement concentrates on volatile days where price ranking determines revenue, and the model achieves it by reallocating forecast accuracy from periods where the battery is idle (overnight, midday) to periods where the dispatch decision is sensitive to the price (afternoon–evening peaks). Decision-focused learning does not replace the convex dispatch LP from Chapter 9 — it uses the LP as a differentiable subroutine during training and as the standard dispatch engine in deployment. It sits in the methods hierarchy between convex optimisation (the core) and reinforcement learning (used only for the price-maker emulator from Chapter 12), and extends naturally to FCAS-aware dispatch from Chapter 11.

Master Glossary

All terms defined across all chapters, listed alphabetically with their source chapter.

Term	Definition	Chapter
------	------------	---------

5

5MPD	AEMO short-term fcst; 5-min updates, 1 h	Ch 15
------	--	-------

8

80/20 rule (Pareto principle)	~80% of outcomes come from ~20% of effort or causes	Ch 10
-------------------------------	---	-------

A

Ablation study	Systematic removal of model components to measure their individual contributions	Ch 10
ACF	Autocorrelation function — autocorrelation plotted at every lag	Ch 02
Activation function	A nonlinear function (ReLU, sigmoid) applied to neuron outputs; enables nonlinear learning	Ch 07
Adaptive conformal prediction	A variant that scales corrections by the model's predicted uncertainty	Ch 08
Administered Price Period	AEMO-imposed price cap triggered by sustained extreme pricing	Ch 03
AEMC	Australian Energy Market Commission — makes and amends market rules	Ch 03
AEMO	Australian Energy Market Operator — runs the dispatch and publishes prices	Ch 03
AEMO dispatch cycle	5-min interval: dispatch + price setting	Ch 15
AER	Australian Energy Regulator — monitors compliance and enforces rules	Ch 03
AGC (Automatic Generation Control)	AEMO's control system that sends regulation signals to enabled units every four seconds	Ch 11
Air mass	Ratio of atmospheric path length to the minimum path	Ch 04
arcsinh	Inverse hyperbolic sine — the target transform for electricity prices	Ch 01
Argmin differentiation	Gradient of argmin of parameterised program	Ch 16
Asymmetric calibration	Targeting different coverage levels for upper and lower tails	Ch 08
Asymmetric loss	Under-predicting spikes costs more than over-predicting	Ch 14
Augmentation	Adding new cells to restore or maintain nameplate capacity as the original cells degrade	Ch 13
Autocorrelation	Correlation of a time series with its own past values	Ch 02
Autoregressive feature	Feature constructed from past values of the target itself	Ch 05
Auxiliary load	Parasitic consumption by cooling, BMS, and monitoring systems	Ch 13

B

Backpropagation	The algorithm for computing gradients and updating weights in neural networks	Ch 07
-----------------	---	-------

Basis	Price difference between two related markets (e.g., two NEM regions)	Ch 09
Basis risk	Risk that a hedge or forecast is referenced to a different price than the battery settles at	Ch 09
Batch normalisation	A technique that normalises layer inputs to stabilise and accelerate training	Ch 07
Battery cycle	One complete charge-discharge (or equivalent energy throughput)	Ch 09
Battery degradation	Gradual loss of capacity due to cycling, aging, and operating conditions	Ch 09
Battery duration	Hours the battery can sustain maximum output (energy/power ratio)	Ch 09
Betz limit	Theoretical maximum wind energy extraction efficiency (~59.3%)	Ch 04
Bias	Error from overly simplistic model assumptions	Ch 06
Bias-variance tradeoff	The tension between model simplicity (high bias) and sensitivity to data (high variance)	Ch 07
Bid stack	The collection of all generators' offer bands, stacked in ascending price order	Ch 12
Binding constraint	When an interconnector reaches its transfer limit, decoupling regional prices	Ch 09
Black-box model	A data-driven model that learns relationships without incorporating domain knowledge	Ch 10
Blade pitch control	Rotating blades to regulate power capture	Ch 04
Breakeven spread	Minimum price difference between discharge and charge for a profitable cycle	Ch 13

C

Calendar ageing	Capacity loss over time regardless of cycling; driven by temperature and SoC	Ch 13
Calibration (reliability)	Whether stated probabilities match observed frequencies	Ch 08
Calibration window	The period of recent data used to compute conformal corrections	Ch 08
Calibration-sharpness paradigm	"Maximise sharpness subject to calibration" — the guiding principle	Ch 08
Capacity factor	Ratio of actual output to theoretical maximum	Ch 04
Capacity payments	Revenue for being available to generate, regardless of actual dispatch	Ch 09
Capture ratio	Actual revenue / perfect-foresight revenue	Ch 16
Capture ratio (CR)	Actual dispatch revenue divided by perfect-foresight revenue	Ch 12
Capture ratio (with FCAS)	Total actual revenue (energy + FCAS) divided by total perfect-foresight revenue	Ch 11
Capture-ratio drift	Sustained decline in rolling capture ratio	Ch 15
Causer-pays	AEMO's cost-allocation mechanism for regulation FCAS, based on each participant's contribution to frequency deviations	Ch 11
Causer-pays factor (CPF)	Each participant's proportional allocation of regulation costs, based on metering deviations	Ch 11
CCGT	Combined Cycle Gas Turbine — efficient gas plant	Ch 03
CDF (cumulative distribution function)	A function giving the probability of being at or below each value	Ch 08
Chance constraint	A constraint required to hold with at least a specified probability	Ch 09
Clear-sky model	Model predicting irradiance under cloud-free conditions	Ch 04
Closed-loop control	Adjusting plan as new information arrives	Ch 05

Cloud enhancement	Irradiance exceeding clear-sky due to cloud reflections	Ch 04
Co-optimisation	Simultaneous clearing of energy and all FCAS markets in a single LP, minimising total cost	Ch 11
Coefficient	Weight on a feature in a linear model	Ch 06
Concavity	Diminishing marginal returns — each additional unit of input yields less additional output	Ch 09
Concept drift	Change in feature–target relationships over time	Ch 06
Conformal prediction	A model-agnostic framework for constructing prediction intervals with coverage guarantees	Ch 08
Conformity score	A measure of how poorly a model's prediction matches the actual outcome	Ch 08
Connection-point price	The effective price at a connection point: MLF × regional reference price	Ch 13
Contingency FCAS	Response to large, sudden frequency disturbances; categorised by response speed (1s, 6s, 60s, 5min)	Ch 11
Copula	A function describing the dependence structure between random variables	Ch 09
Counterfactual analysis	What if one specific input had been different?	Ch 14
Cross-validation	Estimating generalisation error by training/validating on data splits	Ch 06
CRPS	Continuous Ranked Probability Score — the standard metric for probabilistic forecasts	Ch 08
CSI	Clear-Sky Index — ratio of actual to clear-sky irradiance	Ch 04
Cumulative Price Threshold	Rolling seven-day price sum that triggers administered pricing	Ch 03
Cut-in speed	Minimum wind speed for turbine generation (~3 m/s)	Ch 04
Cut-out speed	Maximum safe operating wind speed (~25 m/s)	Ch 04
cvxpy	Python library for convex optimisation	Ch 05
cvxpylayers	Differentiable CVXPY layers (PyTorch/JAX)	Ch 16

D

Data assimilation	Mathematical fusion of model predictions and real observations	Ch 04
Data leakage	Contamination of model inputs with future information	Ch 05
Decision regret	Revenue gap vs perfect-foresight dispatch	Ch 16
Decision tree	A model that makes predictions via a sequence of yes/no questions (splits) on features	Ch 07
Decision-focused learning	Train forecaster via downstream decision quality	Ch 16
Decision-regret drift	Sustained increase in actual-vs-optimal gap	Ch 15
Degradation cost	The implicit cost per MWh of throughput, representing the value of battery capacity consumed	Ch 13
Density forecast	The full probability density function of the predicted variable	Ch 08
Depth of discharge (DoD)	Fraction of battery capacity used in a single cycle	Ch 13
DHI	Diffuse Horizontal Irradiance — scattered component of solar radiation	Ch 04
Diebold-Mariano test	A statistical test for comparing forecasting model accuracy	Ch 10
Diff. optimisation layer	Solves optimisation fwd; implicit diff bwd	Ch 16
Dispatch price	5-minute price set by the marginal generator	Ch 01

Dispatchable	Generation that can be controlled by the operator (coal, gas, hydro)	Ch 04
Diurnal cycle	Daily repeating pattern in prices	Ch 02
DNI	Direct Normal Irradiance — beam component of solar radiation	Ch 04
Drawdown limit	Max loss before automated trading halts	Ch 15
Dropout	Regularisation that randomly deactivates neurons during training	Ch 07
Duck curve	Daily net load shape: midday dip, evening spike	Ch 02
DUID	AEMO unique code for each NEM unit or load	Ch 14

E

Early stopping	Stopping training when validation performance plateaus; prevents overfitting	Ch 07
ECMWF	European Centre for Medium-Range Weather Forecasts	Ch 04
Elastic Net	Combination of L1 and L2 penalties	Ch 06
Embargo	Gap between training data and forecast to prevent leakage	Ch 05
Enablement	The quantity of FCAS a unit is dispatched to provide; the unit is paid the clearing price on its enabled MW	Ch 11
Ensemble method	A technique combining multiple weak models into a single stronger model	Ch 07
Epoch	One complete pass through the entire training dataset	Ch 07
ERA5	ECMWF's fifth-generation global reanalysis dataset	Ch 04
Error decomposition	Splitting error into weather, demand, and residual	Ch 14
Exchangeability	The assumption that data ordering does not affect the joint distribution	Ch 08
Expanding window	Training window that grows over time	Ch 05
Expected value	The probability-weighted average of outcomes across all scenarios	Ch 09
Extrapolation	Making predictions for inputs outside the range observed in training	Ch 10

F

Fair comparison	Evaluating models under identical conditions to ensure performance differences reflect model quality	Ch 10
False alarm	Forecast far exceeded actual; suboptimal dispatch	Ch 14
FCAS	Frequency Control Ancillary Services — grid stability services	Ch 09
FCAS (Frequency Control Ancillary Services)	Services procured by AEMO to maintain grid frequency at 50 Hz, including regulation and contingency services	Ch 11
FCAS co-optimisation	Simultaneous clearing of energy and FCAS markets, allocating capacity across services	Ch 12
FCAS trapezium	Joint constraint between energy dispatch and FCAS capacity in the NEM dispatch engine	Ch 13
Feature (predictor)	Input variable used by a forecasting model	Ch 05
Feature attribution	Decomposing prediction into per-feature contributions	Ch 14
Feature entry order	Order in which features gain non-zero coefficients as λ decreases	Ch 06
Feature importance	A measure of how much each feature contributes to a tree model's predictions	Ch 07
Feature selection	Identifying which features are useful; automated by LASSO	Ch 06

Forecast combination principle	The finding that averaging forecasts from different models almost always improves accuracy	Ch 08
Forecast horizon	Time span covered by the forecast (48 half-hours = 24 hours)	Ch 05
Forecast skill	Performance relative to a naive baseline; controls for problem difficulty	Ch 10
Forward fill	Imputation method that carries the last known value forward through gaps	Ch 01

G

Gate closure	Bid deadline (~2 min before dispatch)	Ch 15
Gaussian copula	Copula derived from the multivariate normal; captures linear dependence	Ch 08
Genetic search	Evolutionary optimisation method for finding high-performing bidding strategies	Ch 12
GHI	Global Horizontal Irradiance — total solar radiation on a horizontal surface	Ch 04
Gradient boosting	An ensemble method that sequentially builds trees, each correcting the errors of the previous ones	Ch 07
Grey-box model	A model combining physical structure with data-driven learning	Ch 10
Gross pool	Market design where all electricity must be traded through the central exchange	Ch 01

H

HAC standard error	Standard error corrected for autocorrelation and heteroscedasticity	Ch 05
HDD / CDD	Heating / Cooling Degree Days — thermal stress metrics	Ch 04
Heavy tails	Extreme values occur far more often than a normal distribution predicts	Ch 02
Histogram binning	Discretising continuous features into buckets to speed up split-finding in trees	Ch 07
Hockey stick	Nonlinear net-load-to-price relationship	Ch 03
Human-in-the-loop	Human authority over high-stakes decisions	Ch 15
Hyperparameter	A setting chosen before training (number of trees, learning rate) vs. parameters learned from data	Ch 07

I

Implicit differentiation	Gradients of optimal solution via KKT diff	Ch 16
Incident report	Structured post-mortem for high-regret days	Ch 14
Inertia	Rotational kinetic energy of synchronous generators; determines how quickly frequency changes after a disturbance	Ch 11
Integrated resource	A resource that can both generate and consume (e.g., a battery), participating on both sides of the energy market	Ch 12
Inter-regional settlement residue (IRSR)	Revenue surplus from price differences across interconnector flows	Ch 09
Intercept	Constant term in a linear model	Ch 06
Interconnector	Transmission link connecting two NEM regions, with a finite transfer capacity	Ch 09
Interval-ending	Timestamp convention where rows are labelled with the period's END time	Ch 01
Intra-day re-optimisation	Updating dispatch plan multiple times per day	Ch 15

Isotonic regression	A non-parametric regression constrained to produce monotonically non-decreasing predictions	Ch 10
----------------------------	---	-------

K

KKT conditions	Optimality conditions for constrained convex programs	Ch 16
Kurtosis	Measure of tail heaviness; electricity prices have very high kurtosis	Ch 02

L

L1 norm	Sum of absolute values of coefficients	Ch 06
Lack of Reserve (LOR)	AEMO declaration indicating insufficient generation reserves	Ch 03
LASSO (L1)	Regularisation with absolute-coefficient penalty; drives some coefficients to zero	Ch 06
LassoCV	LASSO with automatic λ selection via cross-validation	Ch 06
Law of large numbers	Average of many random samples converges to the true expected value	Ch 09
LCOS (Levelised Cost of Storage)	The per-MWh cost of storage over the battery's lifetime	Ch 10
Leaf (terminal node)	The endpoint of a decision tree branch where a prediction is made	Ch 07
LEAR	LASSO Estimated AutoRegressive — per-hour LASSO models for price forecasting	Ch 06
Learning curve	A plot of performance vs. training set size; reveals data sufficiency and overfitting	Ch 07
Learning rate (η)	A small multiplier controlling how much each new tree contributes to the ensemble	Ch 07
LightGBM	A fast, accurate gradient boosting implementation by Microsoft; the industry standard for tabular data	Ch 07
Linear model	A model that predicts output as a weighted sum of inputs — can only represent straight-line relationships	Ch 07
Linear program (LP)	Optimisation with linear objective and constraints	Ch 05
Linear regression	Model predicting target as weighted sum of features	Ch 06
Linke turbidity	Coefficient quantifying atmospheric clarity	Ch 04
Loss differential	Difference in forecast error between two models at each time step	Ch 06

M

MAE	Mean Absolute Error — average of absolute forecast errors	Ch 05
Marginal contribution	The change in performance when a component is added or removed	Ch 10
Marginal cost	Cost of producing one additional unit of output	Ch 03
Marginal loss factor (MLF)	Multiplier on RRP at a connection point, reflecting transmission losses	Ch 13
Marginal pricing	All dispatched generators receive the price of the most expensive unit needed	Ch 01
Marginal value	The additional value from one additional unit of improvement	Ch 10
Market Floor Price	Minimum spot price (−\$1,000/MWh)	Ch 03
Market floor price (MFP)	The minimum allowed price in the NEM (−\$1,000/MWh)	Ch 12

Market impact	The change in clearing price caused by a generator's dispatch	Ch 12
Market power	Ability of one participant to influence the price	Ch 03
Market Price Cap (MPC)	Maximum allowable spot price (\$17,500/MWh in 2024–25, indexed to CPI)	Ch 03
Market price cap (MPC)	The maximum allowed price in the NEM (\$16,600/MWh as of 2024-25)	Ch 12
Mean reversion	Tendency of extreme prices to return to a long-run average	Ch 02
Merit order	Ranking of generators from cheapest to most expensive	Ch 03
Merit-order curve	The empirical relationship between net load and electricity price	Ch 10
Merit-order effect	Price depression caused by zero-cost renewables	Ch 03
Missed spike	Actual price far exceeded forecast; missed discharge	Ch 14
MLP	Multilayer Perceptron — the simplest fully connected neural network	Ch 07
Model drift	Performance degrades as data distribution shifts	Ch 15
MPC	Model Predictive Control — rolling re-optimisation strategy	Ch 05
MPC (Model Predictive Control)	A control strategy that repeatedly solves an optimisation over a rolling horizon	Ch 09
MWh	Megawatt-hour — standard unit of energy in wholesale markets	Ch 01

N

NEM	National Electricity Market — Australia's wholesale electricity market for the eastern states	Ch 01
NEMDE	NEM Dispatch Engine -- AEMO's algorithm for clearing the energy and FCAS markets	Ch 12
nempy	Open-source Python implementation of the NEM dispatch procedure (UNSW-CEEM)	Ch 13
Net load	Demand minus variable renewable generation	Ch 05
Non-stationarity	The property of a time series whose statistics change over time	Ch 07
Nonlinear model	A model that can learn curved, threshold, or discontinuous relationships	Ch 07
NWP	Numerical Weather Prediction — physics-based weather forecasting	Ch 04

O

OCGT	Open Cycle Gas Turbine — fast-start but expensive peaker	Ch 03
Offer band	A price-quantity pair in a generator's offer; the NEM allows up to 10 bands per unit	Ch 12
OLS	Ordinary Least Squares — standard coefficient estimation method	Ch 06
Open-loop control	Executing a fixed plan without adjustment	Ch 05
Opportunity cost	The foregone revenue from the next-best use of capacity; for FCAS, the energy revenue given up by reserving headroom	Ch 11
Overconfidence	When prediction intervals are too narrow (stated coverage exceeds actual coverage)	Ch 08
Overfitting	When a model memorises training data specifics instead of learning generalisable patterns	Ch 07

P

Parquet	Columnar data storage format; standard for analytical workloads	Ch 01
----------------	---	-------

Partial dependence	Marginal feature-prediction relationship, averaged	Ch 14
PD forecast	AEMO price/demand fcst; 30-min updates, 40 h	Ch 15
PDF (probability density function)	A function showing the relative likelihood of each possible value	Ch 08
Per-hour models	Separate model for each hour of the day	Ch 06
Perfect foresight	Theoretical optimum with known future prices	Ch 05
Perfect foresight revenue	Maximum arbitrage revenue achievable with perfect knowledge of future prices	Ch 09
Persistence forecast	Predicting tomorrow = today	Ch 04
Piecewise-constant soln	LP dispatch jumps at thresholds; zero grad elsewhere	Ch 16
Pinball loss	The asymmetric loss function used to train quantile regression models	Ch 07
Pipeline	A sequence of processing stages where each stage's output becomes the next stage's input	Ch 10
Point forecast	A single-valued prediction with no uncertainty information	Ch 07
Pre-dispatch	AEMO's projection of future dispatch and prices	Ch 05
Pre-training	Init on standard loss before decision-focused loss	Ch 16
Prediction interval	A range expected to contain the actual value with a specified probability	Ch 08
Price impact	Battery's own effect on market price	Ch 15
Price spike	Abrupt, extreme, temporary price increase	Ch 03
Price taker	Participant whose actions don't affect prices	Ch 16
Price-duration curve	Prices sorted from high to low vs % of time exceeded	Ch 02
Price-maker	A market participant large enough to move the clearing price through its own actions	Ch 12
Price-taker	A market participant too small to affect the clearing price	Ch 12
Price-taker assumption	The assumption that a participant's actions do not affect market prices	Ch 10
Probabilistic forecast	A forecast that describes the full range of possible outcomes and their probabilities	Ch 08
Probability integral transform (PIT)	Transformation converting a variable to uniform [0,1] via its own CDF	Ch 08
pvlib	Python library for solar energy system simulation	Ch 04

Q

qpth	Differentiable QP solver for PyTorch	Ch 16
QRA (Quantile Regression Averaging)	A method combining multiple point forecasts into calibrated quantile forecasts	Ch 08
Quadratic regularisation	Quadratic penalty → smooth, differentiable dispatch	Ch 16
Quantile crossing	When independently predicted quantiles violate their natural ordering	Ch 07
Quantile forecast	Predictions at multiple probability levels characterising the forecast distribution	Ch 08

R

Ramp limit	Maximum rate of change of power output between consecutive intervals	Ch 13
-------------------	--	-------

Rank correlation	Correlation measured on ranks rather than values; invariant to marginal transforms	Ch 08
Rated capacity	Maximum sustained turbine output	Ch 04
Reanalysis	Physics-model reconstruction of historical atmosphere using all available observations	Ch 04
Rebidding	Updating the quantity allocation across offer bands before gate closure	Ch 12
Recalibration	Update calibration layer without retraining	Ch 15
Regime change (structural break)	A fundamental change in the data's statistical properties	Ch 10
Regime-conditional calibration	Calibration evaluated per regime (calm vs spike)	Ch 14
Regularisation	Any technique that prevents a model from fitting noise in the training data	Ch 07
Regularisation path	Sequence of LASSO solutions as λ varies	Ch 06
Regulation FCAS	Continuous frequency correction via AGC signal; raise and lower markets	Ch 11
Reliability diagram	A plot of observed coverage vs. nominal coverage for assessing calibration	Ch 08
RERT	Reliability and Emergency Reserve Trader — AEMO's emergency reserve mechanism	Ch 03
Residual	The difference between the actual value and the current prediction; what the next tree tries to correct	Ch 07
Residual demand	The demand remaining after all other generators' supply is accounted for; the demand curve faced by a single generator	Ch 12
Retraining cadence	Schedule for retraining (monthly/quarterly)	Ch 15
Revenue stack	The combination of energy arbitrage, FCAS, network support, and capacity revenue available to a battery	Ch 11
Revenue waterfall	Step-by-step decomposition from idealised revenue to realised revenue, showing each haircut	Ch 13
Ridge (L2)	Regularisation with squared-coefficient penalty; shrinks but keeps all features	Ch 06
rMAE	Relative MAE — model MAE divided by naive MAE	Ch 05
RMSE	Root Mean Square Error — penalises large errors more	Ch 05
RNN / LSTM / GRU	Recurrent architectures that process sequential data via internal memory	Ch 07
Robust optimisation	Optimisation that protects against worst-case outcomes	Ch 09
Rolling recalibration	Periodically recomputing conformal corrections as new data arrives	Ch 08
Rolling-origin backtest	Walk-forward evaluation across multiple train/test splits	Ch 05
Rolling-origin evaluation	Repeatedly training and testing across multiple time windows	Ch 10
Round-trip efficiency	Fraction of stored energy recoverable on discharge	Ch 05
Round-trip efficiency (RTE)	Fraction of energy recovered from a charge-discharge cycle	Ch 09

S

Sample efficiency	How much data a model needs to reach its potential performance	Ch 07
Scenario	One complete realisation of uncertain future prices	Ch 09
Scenario coherence	Property that jointly sampled scenarios preserve realistic cross-period dependence	Ch 08

Scenario forecast	Multiple complete future trajectories sampled from the forecast distribution	Ch 08
Seasonality	Regular patterns repeating at fixed intervals	Ch 02
Self-cannibalisation	Revenue reduction caused by the battery's own dispatch affecting the market price	Ch 13
Self-cannibalization	The effect where a battery's own dispatch reduces the price it receives	Ch 12
Settlement Residue Auction (SRA)	Quarterly auction of rights to inter-regional settlement residues	Ch 09
Shadow price (dual variable)	The marginal cost of satisfying a constraint by one more unit; in NEMDE, shadow prices become market clearing prices	Ch 11
SHAP	Shapley-value-based feature attribution method	Ch 14
SHAP values	Game-theory-based feature attribution that explains individual predictions	Ch 07
Sharpness	How narrow (concentrated) the prediction intervals are	Ch 08
Similar-day naive	Forecast using same half-hour, same day of week, one week prior	Ch 05
Sklar's theorem	Any joint distribution decomposes into marginals and a copula	Ch 08
Sliding window	Training window of fixed length that moves forward	Ch 05
SOC	State of charge — current stored energy in the battery	Ch 05
Solar zenith angle	Angle between the sun and directly overhead	Ch 04
Sparsity	Property of having most coefficients exactly zero	Ch 06
Spike coverage	Fraction of extreme price events captured within the prediction interval	Ch 08
Split conformal prediction	A variant using a held-out calibration set to compute interval corrections	Ch 08
Split point	A feature-value threshold that divides data into two groups at a tree node	Ch 07
SPO	Smart predict-then-optimize (Elmachtoub & Grigas)	Ch 16
SPO+ loss	Convex surrogate for decision regret	Ch 16
Staleness	Plan no longer reflects current info	Ch 15
State of health (SoH)	Current usable capacity as a fraction of original nameplate capacity	Ch 13
Stationarity	Statistical properties constant over time	Ch 06
Stochastic programming	Optimisation under uncertainty using multiple scenarios	Ch 09
Strategic bidding	Setting bids based on market conditions rather than cost	Ch 03
Stylised facts	Consistent empirical patterns in a dataset	Ch 02
Supply stack	Visual representation of cumulative generation vs bid price	Ch 03
Synoptic scale	Weather features spanning hundreds to thousands of km	Ch 04
System frequency	The 50 Hz oscillation rate of AC power; reflects the instantaneous supply-demand balance	Ch 11

T

Tail dependence	Tendency for extreme values to occur together more than a Gaussian model predicts	Ch 08
Temporal arbitrage	Exploiting price differences across time via storage	Ch 05
Throughput	Total energy processed by the battery, measured in MWh of discharge	Ch 13
Time-series CV	Cross-validation respecting temporal ordering	Ch 06
Trading day	NEM day from 04:00 to 04:00 AEST	Ch 05

Trading price	30-minute average of six dispatch prices; the financially settled price	Ch 01
Transformer	An architecture using self-attention to capture long-range dependencies in parallel	Ch 07
Trapezium constraint (enablement trapezoid)	The linear constraint linking energy dispatch and FCAS enablement: the sum cannot exceed physical capacity	Ch 11
Two-stage stochastic program	Decisions made before uncertainty resolves (first stage); outcomes computed after (second stage)	Ch 09

U

Underconfidence	When prediction intervals are too wide (actual coverage exceeds stated coverage)	Ch 08
Uniform pricing	All dispatched generators receive the same clearing price, regardless of offer price	Ch 12

V

Value of information	The economic benefit of having a better forecast	Ch 09
Variance	Error from model sensitivity to specific training data	Ch 06
Very-fast FCAS (1-second)	The newest FCAS markets (October 2023) requiring sub-second response; dominated by batteries	Ch 11
Vine copula	Multivariate copula built from a cascade of bivariate copula building blocks	Ch 08
Volatility	Degree of price variation over time	Ch 02
Volatility clustering	Tendency for volatile periods to persist	Ch 02
VRE	Variable Renewable Energy — generation whose output depends on weather (wind, solar)	Ch 04

W

White-box model	A model built from explicit physical or economic principles	Ch 10
Wind shear	Increase in wind speed with altitude	Ch 04